



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

CÁLCULO DE COMPILADORES CORRECTOS: DEL
RÉGIMEN ESTRICTO AL PEREZOSO

T E S I S

QUE PARA OBTENER EL TÍTULO DE:

LICENCIADO EN CIENCIAS DE LA COMPUTACIÓN

P R E S E N T A :

ERICK DANIEL ARROYO MARTÍNEZ

TUTOR

M.C.I.C. MANUEL SOTO ROMERO



CIUDAD UNIVERSITARIA, CDMX, 2025

«It's the questions we can't answer that teach us the most. They teach us how to think. If you give a man an answer, all he gains is a little fact. But give him a question and he'll look for his own answers.»

Patrick Rothfuss

Índice general

1. Introducción	3
1.1. Planteamiento del problema	3
1.1.1. Descripción del problema específico	4
1.1.2. Importancia de abordar el problema	5
1.1.3. Alcance y limitaciones del estudio	5
1.2. Objetivos	7
1.2.1. Objetivo principal	7
1.2.2. Objetivos secundarios	7
2. Marco teórico	9
2.1. Introducción a los compiladores y su importancia	9
2.2. Lenguajes de alto nivel	10
2.3. Compiladores: definición y estructura	12
2.4. Sintaxis de lenguajes de programación	17
2.4.1. Sintaxis concreta	19
2.4.2. Sintaxis abstracta	27
2.5. Semántica de lenguajes de programación	35
2.5.1. Semántica operacional	39
2.5.2. Cálculo λ puro	48
2.5.3. Ambientes de evaluación	57
2.5.4. Cerraduras de función	61
2.5.5. Regímenes de evaluación	63
2.5.6. Puntos estrictos	70
2.5.7. Recursión	72
2.6. Máquinas abstractas	80
2.6.1. Máquinas virtuales	81
2.7. Razonamiento ecuacional	84
2.7.1. Ejemplos de razonamiento en <i>HASKELL: or y reverse</i>	85
2.7.2. Ejemplo en el <i>Cálculo λ: snd</i>	86
2.8. Inducción estructural	87
2.8.1. Inducción estructural en <i>HASKELL</i>	88
2.9. Inducción sobre derivaciones	90
2.10. Método de <i>Cálculo de Compiladores Correctos</i>	92
2.10.1. Síntesis y refinamiento del método	93

2.10.2. Metodología general	94
2.10.3. Aplicación práctica del método a MINIB	95
3. Caso de estudio 1: λANIS	105
3.1. Formalización de la sintaxis léxica	106
3.1.1. Sintaxis libre de contexto	107
3.2. Formalización de la sintaxis abstracta	109
3.2.1. Representación de la sintaxis abstracta	114
3.3. Formalización de la semántica operacional	116
3.4. Introducción al método	122
3.4.1. Representación en HASKELL	122
3.4.2. Especificación	125
3.4.3. Cálculo	128
3.4.4. Resultados	142
3.5. Reflexión	145
4. Caso de estudio 2: IRIS	151
4.1. Formalización de la semántica operacional	151
4.2. Introducción al método	165
4.2.1. Representación en HASKELL	165
4.2.2. Especificación	174
4.2.3. Cálculo	176
4.2.4. Resultados	188
4.3. Reflexión	193
5. Análisis comparativo	201
6. Incorporación de la recursión	205
7. Conclusiones	217
Apéndice A. Material suplementario	227
Bibliografía	227

1 Introducción

A lo largo del desarrollo de la computación, la necesidad de traducir ideas humanas a instrucciones comprensibles para una máquina ha impulsado la creación de lenguajes que permiten describir procesos de forma clara, precisa y estructurada. Sin embargo, entre la formulación de una idea y su ejecución en una computadora existe un proceso intermedio fundamental: la traducción automática de programas en ese lenguaje de alto nivel a una forma que la máquina pueda interpretar. Este proceso, conocido como compilación, no solo debe ser funcional, sino también confiable, de modo que el resultado final preserve el comportamiento previsto por quien escribe el programa.

Lograr este tipo de traducción precisa se vuelve más desafiante a medida que los lenguajes se vuelven más expresivos y complejos. Existen distintos estilos en los que un lenguaje puede decidir cómo y cuándo ejecutar sus instrucciones, y algunos permiten incluso aplazar decisiones hasta que son estrictamente necesarias. Además, ciertas construcciones como la posibilidad de definir construcciones que se definen en términos de sí mismas (algo muy común y útil, conocido como *recursión*) añaden nuevos niveles de dificultad. Estas características modernas exigen enfoques más sólidos para asegurarse de que la transformación del programa no introduzca errores ni altere lo que originalmente se quiso expresar.

Este trabajo se propone explorar una manera sistemática de construir herramientas que realicen esta traducción de forma garantizada, es decir, que se pueda demostrar que funcionan correctamente desde su concepción. A través de este enfoque, se busca ampliar las posibilidades de diseñar lenguajes de programación modernos sin renunciar a la seguridad y confiabilidad que requiere cualquier sistema que dependa de ellos. En los apartados siguientes se delimita con mayor precisión el problema que motiva esta investigación, así como los objetivos, alcances y limitaciones del estudio.

1.1. Planteamiento del problema

La construcción correcta de compiladores es uno de los pilares fundamentales en la teoría de lenguajes de programación, pues garantiza que el comportamiento de un programa se preserve desde el lenguaje fuente hasta su representación en un lenguaje destino. En este trabajo se plantea abordar la adopción y extensión del método de *Cálculo de Compiladores Correctos*, propuesto por Graham Hutton y Patrick Bahr [1], aplicándolo a lenguajes de programación que emplean estrategias de paso de parámetros como *Call-by-Value* y *Call-by-Name*, bajo regímenes de evaluación ansioso (estricto) o perezoso (no estricto) respectivamente, incorporando además el manejo de funciones recursivas (véase las secciones 2.5.5 y 2.5.7).

Este enfoque responde a la necesidad de garantizar que la transformación realizada por un compilador preserve el significado del programa fuente, incluso en escenarios que involucren comportamientos complejos como evaluación diferida. La *verificación formal*¹ de esta propiedad consiste en demostrar, mediante métodos rigurosos, que la ejecución del programa traducido produce los mismos efectos observables que su versión original, lo cual requiere un marco preciso de especificación y prueba [3].

1.1.1. Descripción del problema específico

A diferencia de los trabajos previos, que han modificado el núcleo del *Cálculo de Compiladores Correctos* en cuanto a su formulación para adaptarlo a distintos contextos, en esta investigación se adopta el enfoque original presentado por los autores [1], preservando su estructura conceptual y técnica. Esta decisión responde a la necesidad de mantener la claridad y sistematicidad que caracterizan al método, especialmente al explorar su aplicabilidad a lenguajes con características que no han sido abordadas previamente bajo este marco. El problema central se manifiesta en la necesidad de garantizar que los compiladores sean correctos por construcción, independientemente del régimen de evaluación del lenguaje estudiado. Esto incluye tanto el régimen ansioso como el perezoso, los cuales presentan desafíos particulares en la preservación del significado del programa fuente tras la compilación. Este problema se puede desglosar en los siguientes aspectos:

- En primer lugar, se requieren formalizaciones del lenguaje fuente que estén estructuradas dentro de un marco riguroso capaz de sustentar las propiedades fundamentales del método, tales como la aplicabilidad y la escalabilidad. Estas formalizaciones deben permitir razonar de manera precisa sobre la preservación del significado del programa tras su transformación al lenguaje destino. La claridad y precisión de estas definiciones son esenciales para garantizar la correcta integración del método en lenguajes con estructuras más complejas.
- La adaptación del enfoque original a escenarios que integran tanto regímenes de evaluación ansiosos como perezosos impone retos específicos. En particular, el tratamiento adecuado de los puntos estrictos (véase la sección 2.5.6) en contextos no estrictos representa un desafío central. Estos mecanismos, indispensables para forzar la evaluación en ciertos contextos dentro del régimen perezoso, exigen una formalización cuidadosa en la semántica² del lenguaje. Es necesario evaluar cómo dicha formalización impacta la aplicabilidad y escalabilidad del método, así como su integración en el proceso de cálculo del compilador. Aunque puntos estrictos y evaluación perezosa están estrechamente vinculados desde un punto de vista operativo, en este trabajo se abordan como conceptos diferenciados con el fin de hacer explícita su correlación. Esta distinción permite un análisis más detallado de su interacción y facilita su incorporación controlada dentro del marco del *Cálculo de Compiladores Correctos*.

¹La *verificación formal* consiste en el uso de métodos matemáticos para demostrar la corrección de un programa respecto a una especificación dada [2].

²A partir de este punto, utilizaremos el término *semántica* de manera informal para referirnos al significado y comportamiento observable de los programas en distintos lenguajes. La formalización rigurosa de este concepto puede consultarse en la sección 2.5.

- Finalmente, la verificación rigurosa de la corrección semántica se realiza como parte inherente del propio método, lo cual permite prescindir de fases externas o posteriores de verificación. En este enfoque, la validez del compilador no se basa únicamente en comprobaciones empíricas posteriores, sino que se garantiza formalmente durante el proceso de diseño. No obstante, la validación práctica, entendida como la comprobación experimental del comportamiento del compilador, puede complementarse mediante pruebas unitarias que aseguren que la implementación respeta las propiedades especificadas en el diseño formal.

1.1.2. Importancia de abordar el problema

Abordar este problema resulta fundamental por varias razones:

1. **Confiabilidad del software:** La capacidad de un compilador para preservar el significado del programa fuente en su versión traducida es fundamental para garantizar que el software desarrollado se comporte de acuerdo con su especificación original. Un compilador que respeta esta propiedad contribuye directamente a la confiabilidad y seguridad del software, al reducir la probabilidad de defectos que puedan derivar en fallos durante su ejecución.
2. **Avance en la investigación académica:** Al abordar elementos que han sido poco tratados dentro del marco original del cálculo, como la recursividad y el régimen perezoso, esta investigación aporta al estudio del método en escenarios más representativos de lenguajes reales. Esto no solo enriquece el cuerpo teórico del método, sino que también permite evaluar su viabilidad en aplicaciones prácticas, ampliando sus alcances y estableciendo una base para trabajos futuros.
3. **Formalización y razonamiento ecuacional:** La utilización de técnicas formales, como la inducción estructural y el razonamiento ecuacional, en la derivación de compiladores correctos, permite establecer de forma rigurosa que el significado del programa fuente se preserva en el lenguaje destino (véase las secciones 2.7 y 2.8). Esto reduce la necesidad de validaciones empíricas posteriores, que si bien útiles, no ofrecen el mismo nivel de garantía formal. Así, la corrección del compilador queda incorporada desde su diseño, dentro de un marco sistemático y confiable.

1.1.3. Alcance y limitaciones del estudio

El presente estudio se enfoca en extender el análisis del método de *Cálculo de Compiladores Correctos* hacia lenguajes con características que no han sido abordadas en trabajos previos bajo el enfoque original. En particular, se propone su aplicación en dos escenarios diferenciados, definidos a través del diseño de dos lenguajes específicos:

- **Caso 1:** Se propone el lenguaje ALANIS, desarrollado en este trabajo, que incorpora un régimen de evaluación ansioso y soporta operaciones aritméticas, manejo de enteros y booleanos, variables, pares ordenados, proyecciones, estructuras condicionales y funciones anónimas mediante abstracciones lambda (véase la sección 2.5.2). Además,

permite la definición de funciones recursivas, y estrategia de paso de parámetros por valor (*Call-By-Value*).

- **Caso 2:** Se introduce el lenguaje IRIS, el cual se extiende sintácticamente de ALANIS, pero adopta un régimen de evaluación perezoso. En este contexto, se incorpora el uso de puntos estrictos como mecanismo propio del régimen perezoso para forzar evaluaciones cuando es necesario. Este lenguaje también permite la definición de funciones recursivas, y utiliza la estrategia de paso de parámetros por nombre (*Call-By-Name*). Su análisis presenta retos adicionales en términos de formalización debido a la evaluación bajo demanda y al manejo explícito de los puntos estrictos.

Si bien el estudio aborda tanto aspectos fundamentales como avanzados del método, se reconocen algunas limitaciones específicas:

1. **Complejidad de la formalización:** La incorporación de funciones recursivas y puntos estrictos dentro del régimen perezoso aumenta significativamente la complejidad de la formalización del lenguaje. Esto se debe a que la introducción de dichos mecanismos implica extender el comportamiento del lenguaje con reglas que gestionen la evaluación parcial y condicional de expresiones. Por este motivo, se acota el análisis a patrones de uso frecuentes, dejando fuera estructuras adicionales como ciclos explícitos o secuencias de ejecución, cuya inclusión requeriría su propia formalización y, en muchos casos, derivarían en expresiones equivalentes a las ya contempladas.
2. **Validación empírica:** La validación empírica de los compiladores generados se lleva a cabo en el lenguaje HASKELL, dado que su orientación funcional permite una implementación natural de la formalización y resultados del método manteniendo coherencia con el enfoque original propuesto por sus autores. Aunque otros lenguajes funcionales podrían utilizarse, HASKELL ofrece las herramientas necesarias para expresar tanto las definiciones algebraicas como el comportamiento de forma directa.
3. **Generalización de resultados:** Este estudio se enfoca en lenguajes que incorporan un conjunto esencial de construcciones a partir del cual es posible derivar otras expresiones más complejas. Esta elección permite trabajar con una base representativa pero manejable en términos de complejidad. Si bien no se pretende abarcar todas las posibles variantes del diseño de lenguajes, los resultados obtenidos han sido desarrollados con la intención de ser lo suficientemente generales como para servir como fundamento en el análisis de lenguajes más expresivos. En este sentido, se establece un punto de partida para futuras investigaciones orientadas a extender estos resultados hacia contextos con estructuras más elaboradas, sin perder la alineación con el enfoque original del método.

Este primer acercamiento permite ilustrar la pertinencia y los desafíos involucrados en el estudio del *Cálculo de Compiladores Correctos* aplicado a lenguajes que presentan características no abordadas previamente bajo este marco. Es importante precisar que la noción de extensión adoptada en este trabajo no implica una modificación del método propuesto por los autores, sino que se refiere a la ampliación del conjunto de lenguajes objeto de estudio, manteniendo intactos los principios fundamentales del enfoque original.

1.2. Objetivos

1.2.1. Objetivo principal

Analizar y validar la aplicabilidad y escalabilidad del método de *Cálculo de Compiladores Correctos*, propuesto por Bahr y Hutton, a través de su aplicación directa a lenguajes de programación que integran regímenes de evaluación estrictos y perezosos, junto con funciones recursivas. El propósito es determinar si la formulación original del método puede aplicarse a estos lenguajes sin requerir modificaciones estructurales, en contraste con los ajustes propuestos en trabajos previos. Con ello, se busca evidenciar su capacidad para adaptarse a contextos más complejos de manera sistemática.

1.2.2. Objetivos secundarios

- Especificar rigurosamente el comportamiento de los lenguajes propuestos mediante un conjunto de reglas que definan su dinámica de evaluación, tanto bajo un régimen ansioso como perezoso, considerando construcciones fundamentales como expresiones aritméticas, booleanas, variables, condicionales, pares ordenados, definiciones de funciones (lambda) y aplicaciones.
- Incorporar y modelar explícitamente los puntos estrictos en el contexto de evaluación perezosa, analizando su papel en la conservación del significado de los programas y en el proceso de derivación del compilador. Aunque están estrechamente vinculados con el régimen no estricto, se estudian como conceptos diferenciados para permitir una integración clara y controlada que facilite su formalización dentro del marco del método.
- Extender ambos lenguajes para soportar funciones recursivas, identificando los retos que plantea su integración en términos de formalización y derivación de los componentes del compilador.
- Derivar, de manera sistemática, los compiladores correctos y sus representaciones ejecutables, asegurando que la transformación preserve rigurosamente las propiedades de significado entre el lenguaje fuente y su representación en lenguaje destino.
- Corroborar empíricamente la validez del proceso de compilación y ejecución mediante una implementación funcional en HASKELL, complementada con pruebas que demuestran la correspondencia entre el comportamiento del programa fuente y su ejecución tras la traducción; dichos recursos están disponibles para su revisión en el material suplementario (véase Apéndice A).

Este conjunto de objetivos apunta a fortalecer la comprensión del método de *Cálculo de Compiladores Correctos*, trasladándolo a escenarios con mayor complejidad formal y expresiva. Así, se busca ofrecer evidencia tanto teórica como práctica sobre su aplicabilidad a lenguajes de programación con capacidades avanzadas y útiles en la computación actual.

2 Marco teórico

Este capítulo presenta los fundamentos conceptuales y técnicos que sustentan el desarrollo del presente trabajo. Se abordan temas fundamentales relacionados con lenguajes de programación, semántica formal, técnicas de compilación y métodos de verificación, organizados de forma progresiva para facilitar su comprensión. Algunos conceptos son tratados con mayor detalle por su relevancia directa con los objetivos del estudio (como el *Cálculo λ* , los regímenes de evaluación o el método de *Cálculo de Compiladores Correctos*) mientras que otros se presentan de forma más general con el fin de ofrecer un panorama amplio y contextualizado del problema abordado. Esta revisión teórica permite establecer un marco sólido que guía tanto el diseño formal como la implementación práctica de los lenguajes y compiladores propuestos.

2.1. Introducción a los compiladores y su importancia

El diseño de un compilador, un programa que traduce programas escritos en un lenguaje de alto nivel a otros semánticamente equivalentes en un lenguaje más próximo al hardware¹, como el código máquina, está influido por múltiples áreas de las Ciencias de la Computación. Los compiladores permiten programar utilizando lenguajes que privilegian la legibilidad, la abstracción y la expresividad², evitando que el programador deba enfrentarse directamente a la complejidad de bajo nivel asociada al hardware. Esta complejidad no se refiere únicamente al manejo físico de dispositivos, sino a los detalles técnicos de representación, control y optimización del flujo de ejecución, como la administración de registros, el uso eficiente de memoria o las técnicas específicas de arquitecturas modernas como segmentación, paralelismo o unidades funcionales múltiples, que resultan arduas y propensas a errores cuando se programan directamente. Así, los compiladores no solo automatizan la traducción, sino que también asumen gran parte del trabajo técnico requerido para producir código eficiente en entornos cada vez más complejos.

El estudio, desarrollo y verificación de compiladores es fundamental para la evolución del software³ y hardware moderno, ya que incide directamente en propiedades esenciales como

¹El término hardware hace referencia a los componentes físicos y tangibles de un sistema informático, como el procesador, la memoria, el disco duro, la unidad de control, entre otros.

²En el contexto de lenguajes de programación, la expresividad se refiere a la capacidad de un lenguaje para representar ideas, conceptos y algoritmos de forma clara, concisa y natural, permitiendo resolver problemas con construcciones mínimas y efectivas [4].

³El término software hace referencia al conjunto de programas, procedimientos, reglas, documentación y datos asociados que forman parte de las operaciones de un sistema de computación [5].

la escalabilidad, la modularidad y la eficiencia. La construcción de un compilador implica abordar desde la especificación formal de la sintaxis y semántica de un lenguaje de programación, hasta la generación de código, posiblemente optimizado, que pueda ejecutarse correctamente en el hardware. En este sentido, se recurre a técnicas provenientes de la Teoría de Autómatas, algoritmos de análisis sintáctico, diseño de estructuras de datos y estrategias de optimización.

Además, la modularización en el desarrollo de compiladores no solo facilita su mantenimiento y evolución, sino que también permite la reutilización de componentes y la adaptación a nuevos lenguajes o arquitecturas. Además, la verificación formal del proceso de compilación es esencial para garantizar que las transformaciones realizadas durante la compilación conserven la semántica original del programa, aportando así mayor confiabilidad y seguridad al software generado [6].

En el contexto del presente trabajo, estos conceptos cobran especial relevancia, ya que el planteamiento del problema y los objetivos se fundamentan en la necesidad de desarrollar compiladores que sean correctos, eficientes y capaces de adaptarse a las demandas actuales del desarrollo de software. Así, se pretende no solo profundizar en el conocimiento teórico del diseño de compiladores, sino también aportar herramientas y metodologías prácticas que contribuyan a mejorar la calidad y la robustez en la implementación de sistemas de software.

2.2. Lenguajes de alto nivel

En las primeras etapas del desarrollo de sistemas computacionales, los lenguajes de programación evolucionaron con el objetivo de facilitar la interacción entre los humanos y las máquinas. Inicialmente, las instrucciones debían configurarse directamente en el hardware mediante medios físicos o códigos de máquina, lo que representaba una tarea laboriosa, propensa a errores y difícil de mantener. Posteriormente, surgieron los lenguajes ensambladores, que introdujeron mnemónicos⁴ para representar instrucciones de máquina, permitiendo una representación algo más comprensible aunque aún estrechamente ligada a la arquitectura del procesador.

Sin embargo, conforme los sistemas crecieron en complejidad y se diversificaron sus aplicaciones, fue necesario un nivel de abstracción mayor que permitiera expresar ideas algorítmicas de forma más cercana al razonamiento humano que a la lógica del hardware. Esto impulsó el desarrollo de los llamados lenguajes de *alto nivel*⁵, como FORTRAN, COBOL y LISP en la segunda mitad de los años cincuenta. Según Aho, Lam, Sethi y Ullman (2008), estos lenguajes fueron diseñados con la intención de ofrecer notaciones más expresivas y adecuadas a distintos dominios, como el cálculo numérico, el procesamiento de datos de negocios y la

⁴En el contexto de lenguajes de programación, los *mnemónicos* son palabras o abreviaciones simbólicas utilizadas para representar instrucciones de bajo nivel de forma más comprensible para los humanos. Por ejemplo, en lenguaje ensamblador, el mnemónico *ADD* puede representar una instrucción de suma. Aunque los lenguajes de alto nivel se alejan del nivel de instrucciones máquina, todavía emplean construcciones que pueden considerarse mnemónicas en tanto ayudan al programador a recordar o identificar fácilmente una operación, como *if*, *while* o *return*.

⁵En este contexto, el término “nivel” alude al grado de abstracción con respecto al hardware. Un lenguaje de alto nivel permite al programador centrarse en la lógica y estructura del problema, en lugar de en los detalles específicos del funcionamiento de la máquina.

computación simbólica, respectivamente. Su éxito marcó un punto de inflexión en la historia de la programación y sentó las bases para el desarrollo posterior de muchos otros lenguajes orientados a facilitar una programación más natural, robusta y mantenible [7].

Los lenguajes de alto nivel se distinguen de los lenguajes de bajo nivel por su capacidad de expresar conceptos y estructuras de manera más cercana al razonamiento humano y a los dominios específicos de los problemas a resolver. Mientras que los lenguajes de bajo nivel operan en términos de instrucciones y registros específicos del hardware, los lenguajes de alto nivel permiten a los programadores concentrarse en la lógica del problema sin preocuparse por detalles de implementación a nivel de máquina.

Este tipo de evolución ha sido decisiva en el desarrollo de la computación, ya que ha permitido la construcción de sistemas más complejos y robustos sin la carga de manejar directamente el hardware subyacente. Como resultado, los lenguajes de alto nivel han facilitado la programación, haciendo que sea más accesible y eficiente para un mayor número de personas.

El uso de lenguajes de alto nivel ofrece diversas ventajas, muchas de las cuales han transformado la industria del software y la investigación en computación [8]:

1. **Mayor abstracción y expresividad.** Los lenguajes de alto nivel permiten a los programadores describir problemas en términos más cercanos a su contexto original, sin necesidad de traducirlos a estructuras de bajo nivel.
2. **Facilidad de desarrollo y mantenimiento.** Los programas escritos en lenguajes de alto nivel son más fáciles de leer, depurar y mantener, lo que resulta en una reducción de costos en el ciclo de vida del software.
3. **Portabilidad.** Al estar menos vinculados a una arquitectura específica, los programas pueden ejecutarse en múltiples plataformas con cambios mínimos o nulos.
4. **Optimización y seguridad.** Los compiladores y entornos de ejecución pueden realizar análisis estáticos y dinámicos para optimizar el código y prevenir errores comunes (véase la sección 2.5).
5. **Soporte para estilos modernos.** Los lenguajes de alto nivel han evolucionado para incluir características como programación orientada a objetos, programación funcional y concurrencia, facilitando la creación de software más modular y escalable.

A pesar de las ventajas que ofrecen los lenguajes de alto nivel, en ciertos escenarios los lenguajes de bajo nivel siguen siendo necesarios, especialmente cuando se requiere un control preciso sobre los recursos del hardware o en sistemas con restricciones estrictas de rendimiento. No obstante, para la mayoría de las aplicaciones, el uso de lenguajes de alto nivel permite un desarrollo más eficiente y accesible.

Para que un programa escrito en un lenguaje de alto nivel pueda ejecutarse sobre una máquina física, es necesario traducirlo a una representación que el hardware pueda interpretar. Esta transformación se realiza mediante compiladores, los cuales convierten el código fuente en código máquina. Dicho código puede ser objeto de procesos de optimización, dependiendo de las decisiones de diseño y del contexto en el que se ejecute el programa. Diversos autores

destacan la importancia central de este proceso dentro de la ingeniería de software y la arquitectura de sistemas [6, 8].

Si bien en esta sección no se aborda una revisión detallada del concepto de compilador, conviene subrayar su papel fundamental en la implementación de lenguajes de alto nivel. El desarrollo de compiladores correctos constituye una línea de estudio esencial en Ciencias de la Computación. En particular, el método conocido como *Cálculo de Compiladores Correctos*, propuesto por Bahr y Hutton [1], ofrece un enfoque formal orientado a garantizar que la traducción de programas preserve su semántica original. Este método cobra especial relevancia en contextos donde se requiere optimizar el proceso de compilación sin comprometer la corrección del código generado.

En el marco de esta tesis, el análisis de compiladores correctos resulta especialmente significativo, dado que se busca preservar con rigor la semántica de los programas escritos en lenguajes de alto nivel. Un compilador se considera correcto cuando asegura que el comportamiento del programa traducido, al ejecutarse sobre una arquitectura concreta, es coherente con el comportamiento que tendría el programa original bajo la semántica del lenguaje fuente. La implementación precisa de estos lenguajes es fundamental para garantizar tanto la confiabilidad del software como su desempeño en entornos diversos.

2.3. Compiladores: definición y estructura

Como hemos visto, la visión tradicional de un compilador es tomar un programa escrito en algún lenguaje *fuentes* de alto nivel como entrada y traducirlo en un programa equivalente en otro lenguaje (el lenguaje *destino*); véase en la figura 2.1.

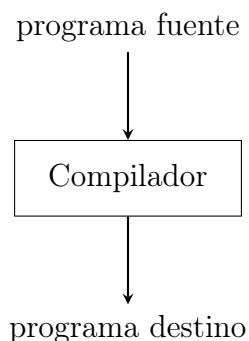


Figura 2.1: Un compilador. Tomado de [7].

El compilador puede detectar diversos tipos de errores en el programa fuente, particularmente aquellos relacionados con su estructura, sin necesidad de ejecutar el código. Este tipo de análisis se enmarca dentro de lo que se denomina *semántica estática* del lenguaje (véase la sección 2.5), la cual define reglas que pueden verificarse en tiempo de compilación. Entre estas reglas se incluyen la *estructura sintáctica válida*, como la correcta anidación de expresiones o el cierre de paréntesis, el *tipificado estático*, y el *alcance léxico* de las variables. A través del análisis semántico, el compilador también puede diagnosticar errores como el uso de identificadores no declarados, la aplicación de operaciones entre tipos incompatibles, o la invocación de funciones con un número o tipo incorrecto de argumentos. Para ello, se

apoya en estructuras como la *tabla de símbolos*⁶, que asocia cada identificador con su tipo, ámbito y otras propiedades relevantes del programa.

Si alguna de estas reglas es violada, el compilador genera un diagnóstico que detiene la traducción antes de la ejecución, ayudando así a evitar fallos en tiempo de ejecución. Sin embargo, existen aspectos del comportamiento del programa que no pueden verificarse en tiempo de compilación. Estos corresponden a la *semántica dinámica* del lenguaje e incluyen, por ejemplo, errores que solo pueden detectarse al ejecutar el programa, como accesos fuera de los límites de un arreglo, desreferenciación de apuntadores nulos o el uso de variables que no han sido inicializadas (véase la sección 2.5). Aunque algunos compiladores insertan verificaciones en tiempo de ejecución para detectar estos problemas, otros simplemente no los manejan, delegando la responsabilidad al programador o al entorno de ejecución. Por otro lado, los intérpretes constituyen otro tipo de procesadores de lenguajes. A diferencia del compilador, que traduce el programa completo a un lenguaje destino, el intérprete ejecuta directamente el código fuente instrucción por instrucción. En este sentido, el intérprete actúa como el lugar de control en tiempo de ejecución, implementando una especie de *máquina virtual* (véase la sección 2.6.1) cuyo lenguaje "nativo" es el lenguaje de alto nivel. Esta modalidad de ejecución permite ofrecer diagnósticos de error más precisos, ya que el contexto de ejecución real está disponible para generar mensajes detallados al ocurrir un fallo.

Además, los intérpretes son especialmente útiles para implementar lenguajes en los que ciertas decisiones, como el tamaño de estructuras de datos, el tipo de las variables o incluso la asociación de nombres a valores, pueden depender de la entrada en tiempo de ejecución, véase la figura 2.2. Este tipo de flexibilidad, conocida como *enlace tardío*, es difícil o costosa de lograr con compilación tradicional. Algunos lenguajes, como LISP, PROLOG y varios lenguajes de *scripting*⁷ como PYTHON o RUBY, explotan esta capacidad al permitir que un programa modifique su propia estructura o ejecute fragmentos generados dinámicamente durante su ejecución [6]. Además, existen procesadores de lenguaje que combinan la compi-

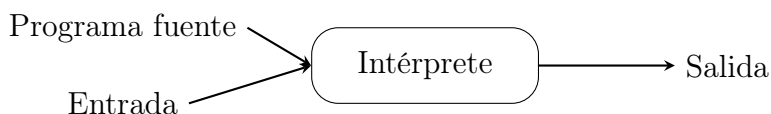


Figura 2.2: Un intérprete. Tomado de [7].

lación y la interpretación, cómo se muestra en la figura 2.3. Un compilador híbrido combina estas características, de modo que el código fuente se convierte primero en un formato intermedio, que no es directamente ejecutable por el hardware pero sí interpretable por un entorno específico, como una máquina virtual. Posteriormente, este programa intermedio se

⁶La *tabla de símbolos* es una estructura de datos utilizada por los compiladores para registrar y acceder a información relacionada con los identificadores del programa, como su nombre, tipo de dato, posición en memoria y otros atributos semánticos. Es construida de manera incremental durante las fases de *análisis* y es consultada durante la *síntesis* del código [7].

⁷En este contexto, el término *scripting* hace referencia a un estilo de programación que se basa en lenguajes diseñados para automatizar tareas dentro de un entorno ya existente (como un sistema operativo o una aplicación), sin necesidad de compilación previa. Los lenguajes de *scripting* suelen ejecutarse mediante intérpretes y permiten un desarrollo ágil y flexible [9].

ejecuta mediante interpretación o mediante una traducción adicional a código de máquina en tiempo de ejecución.

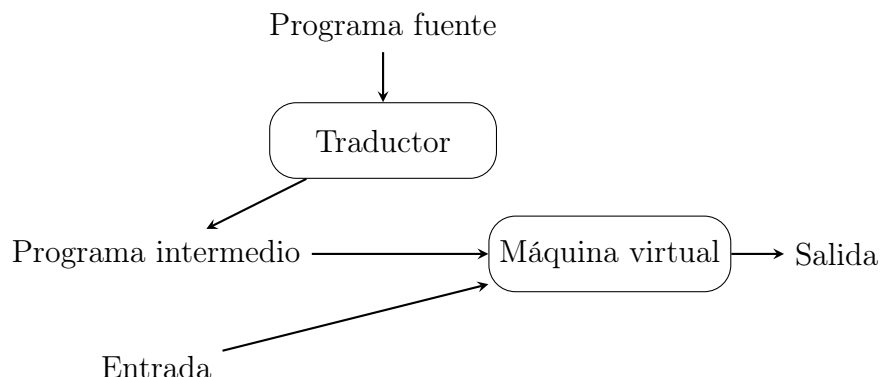


Figura 2.3: Combinación de compilación e interpretación. Tomado de [7].

Un ejemplo paradigmático de este enfoque es el lenguaje JAVA, cuyo diseño se basó desde el inicio en la generación de un código intermedio portátil conocido como *bytecode*. Este código no se ejecuta directamente en el hardware, sino que es interpretado o compilado en tiempo de ejecución por la *Java Virtual Machine* (JVM). Esta arquitectura permite que los programas escritos en JAVA puedan distribuirse de forma independiente de la plataforma, lo que fue esencial para su adopción en entornos heterogéneos como la web. Las primeras implementaciones de JAVA utilizaban exclusivamente interpretación de *bytecode*, pero con el tiempo se incorporó la técnica de compilación “justo a tiempo” (*just-in-time*, o JIT), la cual traduce el *bytecode* a lenguaje máquina justo antes de su ejecución, mejorando significativamente el rendimiento. Este modelo híbrido ha sido adoptado también por otros lenguajes como C#, cuyo compilador produce código en *Common Intermediate Language* (CIL), también sujeto a traducción *JIT* antes de su ejecución.

Para la creación de un programa ejecutable, además del compilador, pueden intervenir otros programas que facilitan el proceso de traducción y organización del código, véase la figura 2.4. En muchos casos, el código fuente se divide en múltiples módulos almacenados en archivos separados, lo que requiere herramientas adicionales para ensamblar y gestionar estos fragmentos antes de obtener un ejecutable final.

Uno de estos programas es el preprocesador, que se encarga de realizar transformaciones iniciales en el código fuente, como la inclusión de archivos externos y la expansión de *macros*, que son fragmentos de código reutilizables.

Posteriormente, el ensamblador convierte el código ensamblador en instrucciones de máquina relocizables, que aún requieren ajustes antes de su ejecución. En proyectos extensos, estos archivos objeto se generan por separado y deben integrarse mediante un enlazador, el cual combina múltiples archivos objeto y bibliotecas externas, resolviendo referencias entre ellos y asignando direcciones de memoria [8].

Estructura general de un compilador

Hasta este punto hemos exhibido el proceso de compilación como la traducción de un programa fuente en un programa destino semánticamente equivalente, es decir, un progra-

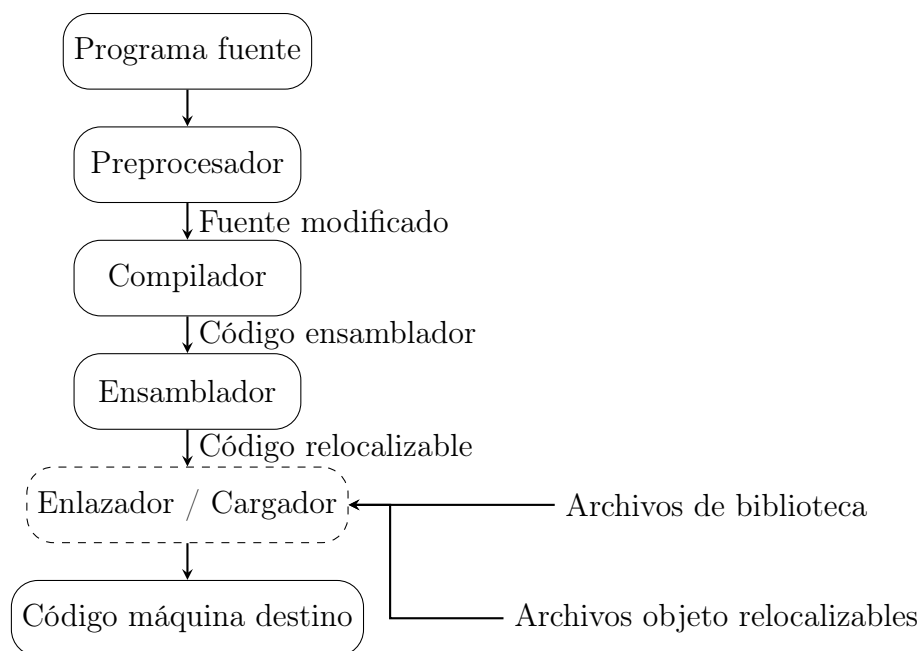


Figura 2.4: Un sistema de procesamiento de lenguaje. Tomado de [7].

ma que conserva el mismo comportamiento observado desde el punto de vista del usuario: produce los mismos resultados y efectos observables para las mismas entradas, independientemente de su representación a nivel de máquina o del entorno de ejecución. Sin embargo, aún no hemos descrito la estructura latente de estos poderosos programas. En concreto, este proceso de traducción se divide en dos componentes principales: el *análisis* (o *front-end*), que analiza el código fuente, y la *síntesis* (o *back-end*), que genera el código destino.

Por un lado, el *análisis* se centra en las primeras fases del compilador; véase la figura 2.5, específicamente se encarga de las siguientes tareas:

- **Análisis léxico.** Representa la fase inicial dentro del proceso de compilación. Su propósito fundamental es recorrer el código fuente carácter por carácter, con el fin de agruparlos en unidades mínimas con significado conocidas como *lexemas*⁸. A partir de cada lexema, se genera un *token*⁹, que es posteriormente enviado al analizador sintáctico, el cual lo utiliza para construir la estructura gramatical del programa.

Durante esta fase, el analizador léxico también interactúa con la tabla de símbolos. En particular, cuando se identifica un lexema como un posible identificador, este debe ser registrado en dicha tabla. Además, la información almacenada en ella puede ser consultada para decidir con mayor precisión qué *token* se debe generar.

⁸Un *lexema* es una secuencia de caracteres de entrada que conforman un solo *token* y que posee significado léxico. En el contexto de compiladores, los lexemas son las unidades mínimas reconocidas por el analizador léxico a partir de las cuales se generan los *tokens* [7]. En lingüística, un lexema es la unidad mínima con significado léxico, independientemente de su forma gramatical, como indica la Real Academia Española.

⁹Un *token* es una unidad de información léxica compuesta por un nombre simbólico y un valor atributo. El nombre-*token* representa la categoría del lexema (como identificador, palabra reservada, número, etc.), y el valor-atributo puede ser un puntero a una entrada en la tabla de símbolos o una representación literal del lexema [7].

- **Análisis sintáctico.** También conocido como *parsing*, constituye la segunda fase del proceso de compilación. Su función principal es recibir la secuencia de *tokens* generada por el analizador léxico y examinarla con base en las reglas gramaticales del lenguaje de programación.

A partir de esta secuencia, el analizador sintáctico construye una representación jerárquica que refleja la estructura lógica del programa: el *árbol sintáctico*. En este árbol, cada nodo interno representa una construcción del lenguaje (como una operación, una sentencia o una expresión), y sus nodos hijos corresponden a los componentes o argumentos de dicha construcción.

El árbol sintáctico resultante actúa como una representación intermedia del programa, que facilita las fases posteriores del compilador, como el análisis semántico y la generación de código. Esta etapa garantiza que el código fuente respete la sintaxis formal del lenguaje y permite detectar errores estructurales en el programa de forma temprana y precisa.

- **Análisis semántico.** Constituye la tercera fase del proceso de compilación. Su objetivo es garantizar que el programa fuente sea semánticamente correcto, es decir, que las construcciones del lenguaje utilizadas respeten las reglas de significado definidas por el lenguaje de programación.

Para ello, el analizador semántico toma como entrada el árbol sintáctico generado previamente y accede a la información almacenada en la tabla de símbolos. Con estos elementos, realiza comprobaciones que aseguran, por ejemplo, que las variables hayan sido declaradas antes de usarse, que los tipos de datos sean compatibles en las operaciones, y que las funciones sean invocadas con los parámetros correctos.

Una de las tareas fundamentales de esta fase es la *verificación de tipos*. Mediante la cual se comprueba que los operandos de cada operador sean del tipo adecuado. Esta verificación previene errores que podrían comprometer la ejecución del programa.

Además de detectar inconsistencias semánticas, esta etapa también se encarga de enriquecer la información semántica del árbol sintáctico o de actualizar la tabla de símbolos, con el fin de facilitar las fases siguientes del compilador, especialmente la generación de código intermedio.

Por otro lado, la *síntesis* se ocupa de las siguientes fases:

- **Optimización de código.** Esta fase, aunque opcional, resulta altamente beneficiosa dentro del proceso de compilación. Su propósito es transformar el código intermedio en una versión más eficiente, sin alterar la semántica del programa original. Se considera independiente de la máquina cuando se aplica sin asumir detalles específicos del hardware.

Entre sus objetivos comunes se encuentran la reducción del tiempo de ejecución, el uso eficiente de memoria y la disminución del tamaño del binario. Para ello, se emplean técnicas como la eliminación de código muerto, la propagación de constantes y la simplificación de expresiones algebraicas. Estas optimizaciones pueden aplicarse de

forma local (dentro de un bloque básico¹⁰) o global, considerando el análisis del flujo de datos entre bloques.

Según Aho et al. [7], incluso optimizaciones simples pueden mejorar considerablemente el rendimiento sin comprometer la velocidad de compilación. Aunque no garantizan una solución óptima en términos absolutos, permiten obtener código sustancialmente más eficiente que una traducción directa y literal del código fuente.

- **Generación de código.** Es la fase del compilador encargada de transformar la representación intermedia del programa en un código equivalente en el lenguaje destino, comúnmente código máquina o ensamblador. Esta fase marca la transición del análisis abstracto del programa hacia una versión ejecutable por el hardware del sistema.

Durante esta etapa, el compilador asigna recursos físicos concretos, como registros o direcciones de memoria, a las variables utilizadas en el programa. Luego, traduce cada instrucción de la representación intermedia a una o más instrucciones del lenguaje destino, asegurando que conserven el mismo significado y efecto operacional.

Uno de los aspectos más importantes de esta fase es la *asignación eficiente de registros*, ya que el número de registros disponibles en la mayoría de las arquitecturas es limitado. Una buena estrategia de asignación puede influir significativamente en el rendimiento del programa final, reduciendo el uso de memoria y mejorando la velocidad de ejecución.

Además, algunos compiladores incluyen fases adicionales, como la *gestión de memoria* y la *traducción a múltiples arquitecturas*, dependiendo de los requisitos del sistema de ejecución [6, 7].

2.4. Sintaxis de lenguajes de programación

La sintaxis de un lenguaje de programación define las reglas y estructuras que determinan el formato adecuado para los programas escritos en ese lenguaje. Es una de las piezas más importantes del proceso de compilación, ya que organiza y establece cómo deben estructurarse los programas para que sean reconocidos y procesados correctamente por el compilador.

Para especificar formalmente la sintaxis de un lenguaje de programación, es común emplear *gramáticas libres de contexto*, escritas mediante la notación BNF¹¹, que permiten describir de forma precisa la estructura jerárquica de las expresiones válidas del lenguaje. A fin de mejorar la legibilidad y expresividad, es frecuente recurrir a una variante extendida de esta notación, conocida como EBNF (*Extended BNF*), que introduce operadores adicionales (como opcionales, repeticiones o agrupamientos) para representar las producciones de expresiones de manera más concisa, elegante y cercana a la estructura lógica del lenguaje [7]. Esta notación no solo reduce la cantidad de reglas necesarias, sino que también facilita su interpretación tanto para humanos como para herramientas de análisis sintáctico.

Estas gramáticas cumplen un rol central no solo en la descripción formal de lenguajes, sino también en las fases de análisis y traducción durante el proceso de compilación. Para ilustrar

¹⁰Un *bloque básico* es una secuencia de instrucciones consecutivas del programa que se ejecutan linealmente, es decir, sin bifurcaciones excepto al final, y sin puntos de entrada intermedios [7].

¹¹Por sus siglas en inglés *Backus-Naur Form*.

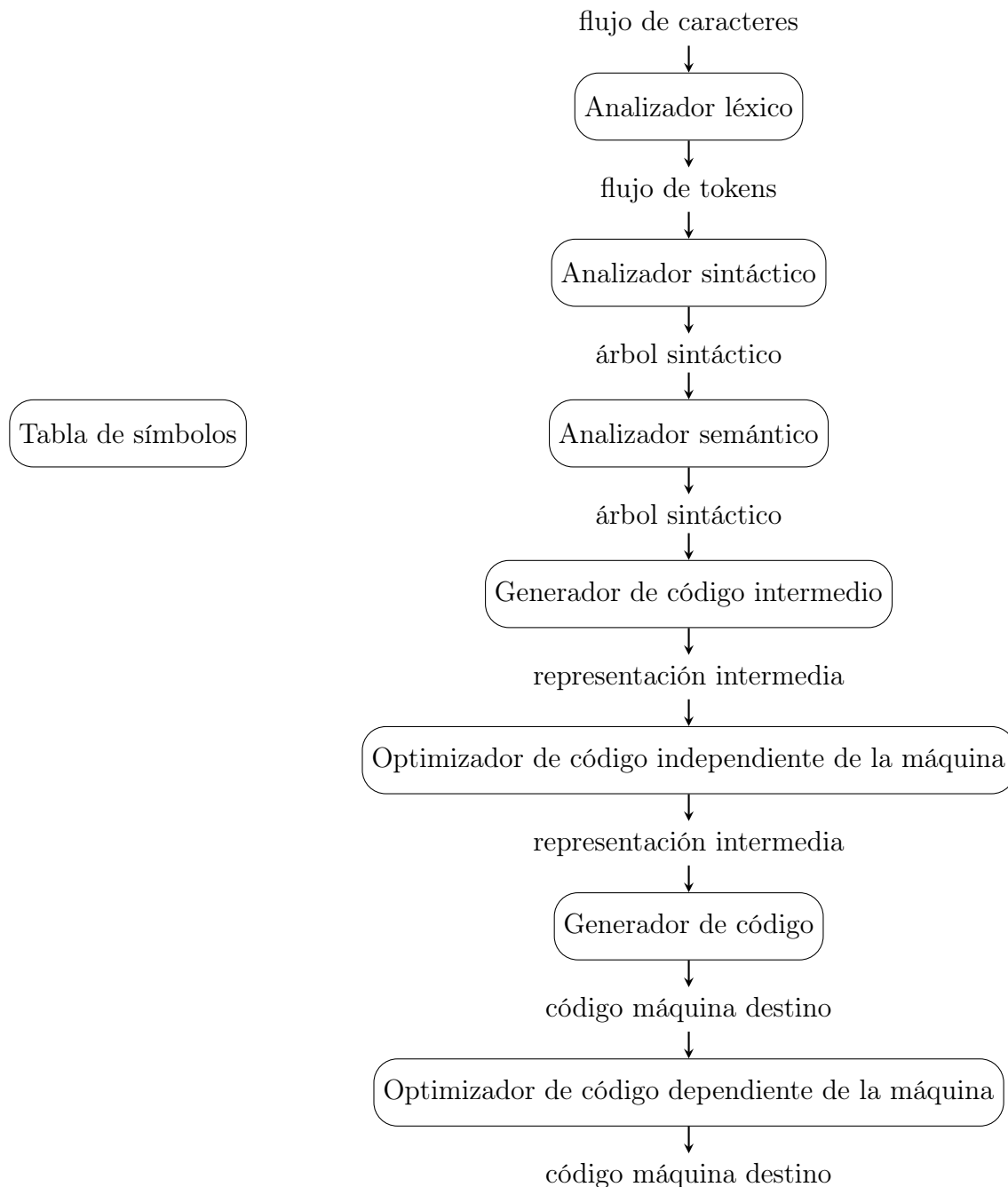


Figura 2.5: Fases de un compilador. Tomado de [7].

estos conceptos, empleamos un lenguaje simple y didáctico, denominado MINIB. Este mini lenguaje modela un subconjunto de la lógica clásica proposicional, compuesto únicamente por constantes booleanas y operadores lógicos, sin variables ni estructuras complejas. Su simplicidad lo convierte en un candidato ideal para exponer, de manera clara y controlada, las distintas etapas del proceso de formalización, como la definición de gramáticas, derivaciones sintácticas y la aplicación del método de *Cálculo de Compiladores Correctos*. En este trabajo, presentamos la gramática de MINIB utilizando notación BNF, lo cual permitirá ejemplificar

su representación estructurada y sentar las bases para las transformaciones posteriores.

2.4.1. Sintaxis concreta

La *sintaxis concreta* de un lenguaje de programación establece la forma externa y detallada en la que deben escribirse los programas, incluyendo todos los elementos visibles como palabras reservadas, operadores, delimitadores y la disposición específica de estos símbolos. Desde la perspectiva de la teoría de lenguajes formales, esta sintaxis se define mediante una gramática formal que delimita de manera precisa las secuencias válidas de símbolos que conforman los programas escritos en el lenguaje.

Esta especificación suele organizarse en dos niveles jerárquicos: por un lado, la *sintaxis léxica*, que describe cómo se forman los componentes básicos del lenguaje (como identificadores, literales y operadores) a partir de caracteres individuales; y por otro lado, la *sintaxis estructural* o *gramática libre de contexto*, que indica cómo se combinan esos componentes léxicos para formar construcciones sintácticas más complejas como expresiones, instrucciones o bloques de código [7].

Sintaxis léxica

Una vez identificado el papel fundamental del análisis léxico en el proceso de compilación, como responsable de segmentar el código fuente en unidades mínimas con significado conocidas como *tokens*, es necesario formalizar la estructura de dichos componentes mediante la *sintaxis léxica*. Los *tokens* incluyen elementos como palabras reservadas, identificadores, literales, operadores y delimitadores, y constituyen las piezas básicas sobre las que se construye la estructura sintáctica del programa.

Desde un enfoque formal, la sintaxis léxica se describe utilizando *expresiones regulares*, las cuales permiten definir patrones para secuencias válidas de caracteres. Estas expresiones se pueden traducir directamente a *autómatas finitos*, lo que facilita la construcción automática de analizadores léxicos eficientes y correctos [7].

El uso de expresiones regulares para esta tarea responde a razones tanto teóricas como prácticas [7]. En particular:

1. Las reglas léxicas suelen ser generalmente simples y no requieren la expresividad de las gramáticas libres de contexto.
2. Permiten modularizar claramente el compilador, separando el análisis léxico del análisis sintáctico posterior.
3. Proveen una notación compacta, expresiva y fácil de mantener para describir clases comunes de *tokens*.

Esta elección de modelado no es arbitraria, sino que responde a la naturaleza de los patrones léxicos que se encuentran comúnmente en los lenguajes de programación. Si bien no tienen la capacidad de representar construcciones anidadas complejas, como paréntesis balanceados o estructuras condicionales del tipo *if-then-else*, son notablemente eficaces para los patrones necesarios en la definición de *tokens*. En ese sentido, su uso resulta no solo práctico,

sino también suficiente para la mayoría de los requerimientos del análisis léxico. Por estas razones, las expresiones regulares se consideran una herramienta central en la especificación de la sintaxis léxica [7].

En términos formales, la definición léxica de un lenguaje parte de los siguientes componentes [10]:

- **Alfabeto** (Σ): Es el conjunto finito de símbolos permitidos en el lenguaje, incluyendo letras, dígitos y caracteres especiales.
- **Tokens**: Son cadenas de caracteres formadas a partir del alfabeto, que corresponden a categorías sintácticas básicas. Cada tipo de *token* se define mediante una expresión regular que describe su patrón.

Definición 2.4.1: Expresión regular

Una expresión regular E sobre un alfabeto Σ se define de manera recursiva de la siguiente forma:

- ε (la cadena vacía) es una expresión regular.
- Para cada símbolo $a \in \Sigma$, a es una expresión regular que representa la cadena que contiene únicamente el símbolo a .
- Si E_1 y E_2 son expresiones regulares, entonces $E_1 E_2$ (concatenación) también es una expresión regular.
- Si E_1 y E_2 son expresiones regulares, entonces $E_1 + E_2$ (unión) es una expresión regular.
- Si E es una expresión regular, entonces E^* (cerradura de Kleene) es una expresión regular, la cual representa cero o más repeticiones de E .

[10]

En cuanto al orden de evaluación, el operador unario $*$ tiene la mayor precedencia, seguido por la concatenación, y finalmente la unión $+$. Todos ellos se asocian hacia la izquierda, lo que permite omitir paréntesis en muchos casos sin ambigüedad. Estas convenciones simplifican tanto la escritura como el procesamiento de expresiones regulares en analizadores léxicos [7].

A continuación, presentamos la especificación léxica del lenguaje MINIB, definiendo formalmente su alfabeto y los principales tipos de *tokens* utilizados.

Ejemplo 2.4.1: Sintaxis léxica de MINIB

Definimos la sintaxis léxica de nuestro lenguaje MINIB, comenzando con especificar el alfabeto del lenguaje, es decir, el conjunto de símbolos que pueden aparecer en las expresiones. En esta versión, solo se consideran constantes booleanas y operadores lógicos, junto con el uso de paréntesis para agrupar expresiones. El alfabeto de MINIB queda especificado por:

$$\Sigma = \{True, False, (,), \neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$$

Nota: Usamos símbolos *Unicode* para los operadores lógicos con el fin de representar directamente su forma simbólica habitual. En implementaciones prácticas, estos podrían reemplazarse por palabras reservadas como *not*, *and*, *or*, *=>*, *<=>*, etc., si se desea una codificación *ASCII*.

Ahora, los *tokens* de nuestro lenguaje corresponden a las unidades léxicas que podemos formar a partir del alfabeto. En esta versión, consideramos los siguientes tipos de *tokens* y las expresiones regulares que definen cada uno de estos:

- Constantes Booleanas

$$True + False$$

En este caso, la expresión representa la unión de dos posibles valores que pueden ser utilizados como constantes booleanas en el lenguaje. La operación de unión aquí implica que cualquiera de las dos cadenas, *True* o *False*, es una constante válida. Esto significa que el lenguaje acepta como entrada tanto el valor *True* como *False*, pero no cualquier otra combinación de caracteres. Cabe destacar que la secuencia debe coincidir exactamente con estas cadenas literales, respetando mayúsculas y minúsculas, ya que el lenguaje es sensible a la distinción entre ellas.

- Paréntesis

$$(+)$$

Se utilizan para agrupar subexpresiones lógicas. Al igual que en lenguajes de notación matemática o lógica simbólica, permiten definir la precedencia de los operadores.

- Operadores Lógicos

- **Negación** ($\neg$): Representa el operador unario de negación lógica. Aparece antes de una subexpresión.
- **Conjunción** ($\wedge$): Representa el operador binario lógico *y*.
- **Disyunción** ($\vee$): Representa el operador binario lógico *ó*.
- **Implicación** ($\rightarrow$): Operador binario que representa la implicación lógica.
- **Equivalencia** ($\leftrightarrow$): Operador binario que representa equivalencia lógica.

Con los *tokens* definidos en MINIB, podemos construir expresiones lógicas válidas, como

las siguientes:

$$\begin{aligned} & (True \wedge False) \\ & \neg(True \vee False) \\ & (True \rightarrow False) \\ & (True \leftrightarrow \neg False) \end{aligned}$$

Gramáticas libres de contexto

Una gramática libre de contexto (GLC) es un formalismo utilizado ampliamente para describir la estructura sintáctica de los lenguajes de programación. Este tipo de gramática es particularmente adecuada para modelar la forma en que se construyen expresiones y sentencias, y constituye la base del análisis sintáctico realizado por los compiladores [6].

Definición 2.4.2: Gramática libre de contexto

Formalmente, una GLC se define como una tupla $G = (N, \Sigma, P, S)$, donde:

- N es un conjunto finito de símbolos no terminales. Estos representan categorías sintácticas intermedias y permiten construir la jerarquía estructural del lenguaje.
- Σ es un conjunto finito de símbolos terminales, que corresponden a los *tokens* generados por el análisis léxico.
- P es un conjunto finito de producciones de la forma $A \rightarrow \alpha$, donde $A \in N$ y $\alpha \in (N \cup \Sigma)^*$.
- $S \in N$ es el símbolo inicial, desde el cual se derivan todas las cadenas válidas del lenguaje.

[10]

Cada uno de estos componentes tiene un papel específico en la descripción de la sintaxis:

1. **Terminales:** Representan los elementos básicos del lenguaje que no pueden ser descompuestos más allá dentro de la gramática. En el contexto de MINIB, los terminales incluyen las constantes booleanas *True* y *False*, así como los operadores lógicos y los paréntesis.
2. **No terminales:** Son variables sintácticas que permiten definir la estructura y combinaciones posibles entre los distintos elementos del lenguaje. En nuestra gramática, los no terminales principales serán *expr* (expresiones), *binop* (operadores binarios) y *cons* (constantes).
3. **Símbolo inicial:** Es el punto de partida para la generación de expresiones válidas en el lenguaje. En MINIB, el símbolo inicial será *expr*, ya que nos interesa definir la estructura de las expresiones lógicas.

4. **Producciones:** Son las reglas que permiten derivar expresiones complejas a partir de elementos más simples. Estas reglas modelan cómo se combinan operadores, constantes y subexpresiones de forma recursiva para construir la sintaxis del lenguaje.

Para ilustrar la construcción de la gramática de nuestro lenguaje MINIB, que permite la representación de operaciones lógicas como la negación ($\neg$), disyunción ($\vee$), conjunción ($\wedge$), implicación ($\rightarrow$) y equivalencia ($\leftrightarrow$), comenzamos con la siguiente definición:

Ejemplo 2.4.2: Sintaxis libre de contexto de MINIB

$$\begin{aligned} \text{expr} &\rightarrow \text{cons} \\ \text{expr} &\rightarrow (\neg \text{expr}) \\ \text{expr} &\rightarrow (\text{expr} \text{ binop } \text{expr}) \\ \text{binop} &\rightarrow \wedge \\ \text{binop} &\rightarrow \vee \\ \text{binop} &\rightarrow \rightarrow \\ \text{binop} &\rightarrow \leftrightarrow \\ \text{cons} &\rightarrow \text{True} \\ \text{cons} &\rightarrow \text{False} \end{aligned}$$

Esta gramática captura de manera precisa las construcciones válidas del lenguaje:

- El no terminal *cons* define los valores lógicos básicos, *True* y *False*.
- El no terminal *binop* especifica los operadores binarios disponibles, que permiten combinar expresiones lógicas.
- El no terminal *expr* describe las expresiones lógicas como tales, que pueden ser:
 - una constante booleana,
 - la negación de otra expresión,
 - o bien una expresión compuesta mediante un operador binario.

Para facilitar la comprensión estructural y mejorar la presentación formal del lenguaje MINIB, resulta útil reformular su gramática desde una notación BNF clásica hacia una notación extendida (EBNF). Esta transformación no modifica el poder expresivo de la gramática, pero sí aporta ventajas importantes: permite una especificación más concisa, reduce la cantidad de reglas redundantes y hace explícitas construcciones comunes como secuencias opcionales o repetidas, lo cual mejora tanto la legibilidad como la mantenibilidad de la gramática. Además, esta forma extendida es ampliamente utilizada en herramientas modernas de análisis sintáctico y compiladores, lo que facilita la comparación con otros lenguajes formales y la implementación práctica de procesadores del lenguaje [11].

Notación BNF

La notación BNF es una metalengua sintáctica formal utilizada para describir la gramática de lenguajes de programación y otros lenguajes formales. Su uso se consolidó tras ser adoptada para la especificación del lenguaje ALGOL 60 [12], y desde entonces ha sido ampliamente utilizada en la literatura y en especificaciones técnicas. Aunque no fue establecida inicialmente por un organismo oficial, su aceptación generalizada la ha convertido en una convención comúnmente adoptada para expresar estructuras gramaticales de manera precisa¹².

Una de las principales fortalezas de BNF es su capacidad para expresar reglas de producción de manera clara y estructurada. Esto facilita no solo el análisis automático de los lenguajes, sino también su diseño, implementación y documentación. Sin embargo, a pesar de su amplio uso, existen diversas variantes de esta notación, lo que ha dificultado su comprensión universal y ha limitado el reconocimiento de sus ventajas.

En su forma básica, una regla en BNF se representa mediante una producción con el símbolo $::=$ y puede incluir alternativas mediante el operador $|$. Las variables (o símbolos no terminales) suelen escribirse entre signos de mayor y menor ($< >$), mientras que los símbolos terminales (como palabras reservadas u operadores) se escriben tal cual aparecen en el lenguaje.

Por ejemplo, en BNF una regla puede escribirse así:

$$< expr > ::= < const > \mid (< expr > < binop > < expr >)$$

Esta notación resulta especialmente útil para definir estructuras como expresiones, instrucciones, listas de parámetros, entre otros componentes en los lenguajes de programación.

Gramática en BNF de MINIB. Presentamos la gramática de nuestro lenguaje MINIB utilizando la notación clásica BNF. Esta definición formal especifica cómo se construyen las expresiones lógicas válidas a partir de valores constantes y operadores lógicos. La estructura refleja directamente la jerarquía sintáctica del lenguaje, haciendo explícitas las reglas que gobiernan su correcta formación.

¹²Este tipo de convenciones ampliamente utilizadas sin haber sido impuestas por una autoridad formal se conocen como *estándares de facto* [13].

Ejemplo 2.4.3: GLC de MINIB en BNF

$$\begin{aligned}
 \langle expr \rangle &::= cons \\
 &| (\neg \langle expr \rangle) \\
 &| (\langle expr \rangle \langle binop \rangle \langle expr \rangle) \\
 \langle binop \rangle &::= \wedge \\
 &| \vee \\
 &| \rightarrow \\
 &| \leftrightarrow \\
 \langle cons \rangle &::= True \\
 &| False
 \end{aligned}$$

Esta representación hace evidente que una expresión puede ser simplemente un valor constante, la negación de otra expresión, o la combinación binaria de dos subexpresiones a través de un operador lógico. Aunque clara y precisa, esta forma BNF puede llegar a ser redundante o poco legible en casos más complejos. Por ello, se suele recurrir a una notación más compacta: la EBNF.

Gramática en EBNF de MINIB: Como se mencionó, la notación EBNF es una extensión ampliamente aceptada de la notación BNF clásica. Fue desarrollada con el propósito de ofrecer una forma más concisa, legible y expresiva de describir la sintaxis de lenguajes formales, incorporando mecanismos para representar repeticiones, secuencias opcionales y agrupaciones [12]. Estas capacidades permiten expresar reglas sintácticas complejas de forma más clara, mejorando tanto la comprensión como la implementación de analizadores sintácticos. Formalmente, la notación EBNF introduce los siguientes elementos adicionales:

- **Repetición:** Se expresa mediante llaves $\{\}$, indicando que una secuencia puede repetirse cero o más veces.
- **Opcionalidad:** Se representa con corchetes $[]$, permitiendo indicar partes opcionales de una producción.
- **Agrupación:** Se realiza mediante paréntesis $()$, útiles para controlar la precedencia o delimitar alternativas.
- **Terminales:** Se suelen escribir entre comillas para distinguirlos claramente de los no terminales.
- **Alternativas:** Se siguen expresando con el operador $|$, heredado de BNF.

Sin embargo, al analizar la gramática del lenguaje MINIB presentada anteriormente, observamos que ninguna de estas construcciones adicionales es necesaria. La gramática se compone únicamente de combinaciones secuenciales y alternativas simples, sin necesidad de representar repeticiones, elementos opcionales ni agrupaciones complejas. Por tanto, al transformarla a EBNF, su estructura permanece idéntica.

Esta coincidencia no implica una limitación, sino que evidencia que la definición actual de MINIB es suficientemente simple como para que la notación BNF ya proporcione una representación clara y completa. En consecuencia, la forma EBNF de nuestra gramática resulta ser estructuralmente equivalente a la forma BNF previamente presentada.

A partir de las definiciones formales desarrolladas hasta este punto (la sintaxis léxica, donde se especifica el alfabeto y los *tokens* mediante expresiones regulares, y la sintaxis libre de contexto, que determina cómo se combinan dichos *tokens* para formar expresiones válidas) estamos en condiciones de establecer la sintaxis concreta del lenguaje MINIB.

Definición 2.4.3: Sintaxis concreta

Formalmente, la sintaxis concreta de un lenguaje de programación se define como una tupla (L, G) , donde:

- L es la definición léxica, representada por un conjunto de expresiones regulares R , que describen los *tokens* válidos del lenguaje.
- $G = (N, \Sigma, P, S)$ es la gramática libre de contexto que define la estructura jerárquica de las expresiones del lenguaje.

[10]

En este contexto, la sintaxis concreta puede entenderse como el conjunto de todas las secuencias de caracteres que son reconocidas como programas válidos conforme a las reglas gramaticales definidas. Esta representación incluye los detalles específicos de escritura, como paréntesis, operadores explícitos y palabras reservadas, que permiten construir de manera precisa las estructuras del lenguaje en su forma textual.

Con esta base, es posible avanzar hacia un nivel más abstracto de especificación: la *sintaxis abstracta*. En esta etapa, se eliminan los elementos puramente sintácticos que no afectan el significado semántico del programa, con el fin de modelar únicamente su estructura lógica esencial. Esta representación permite facilitar procesos internos del compilador, como el análisis semántico y la generación de código intermedio.

Árboles de sintaxis concreta

Un *árbol de sintaxis concreta* (ASC) muestra, en forma gráfica, la manera en que el símbolo inicial de una gramática deriva una cadena en el lenguaje. Si el no terminal A tiene una producción $A \rightarrow XYZ$, entonces un ASC podría tener un nodo interior etiquetado como A , con tres hijos, de izquierda a derecha, etiquetados como X , Y y Z .

De manera formal, dada una gramática libre de contexto, un ASC de acuerdo con la gramática es un árbol que satisface las siguientes propiedades:

1. La raíz se etiqueta con el símbolo inicial.
2. Cada hoja se etiqueta con un terminal, o con ε (cadena vacía).
3. Cada nodo interior se etiqueta con un no terminal.

4. Si A es el no terminal que etiqueta a un nodo interior, y $X_1, X_2, \dots, X_n$ son las etiquetas de sus hijos (de izquierda a derecha), entonces debe existir una producción

$$A \rightarrow X_1 X_2 \cdots X_n,$$

en la gramática [7].

Podemos representar un programa mediante un árbol al que llamamos ASC. Este árbol incluye todos los detalles del código fuente, tales como paréntesis y operadores, reflejando fielmente la derivación completa conforme a la gramática.

A modo de ejemplo, consideremos la siguiente expresión en MINIB:

$$(\neg(True \wedge False))$$

Esta expresión se deriva usando la gramática presentada en 2.4.3. El ASC 2.6 ilustra la derivación de la expresión, mostrando de forma explícita cada paréntesis, operador y constante:

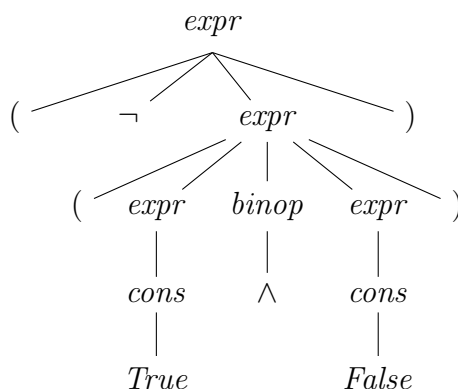


Figura 2.6: Árbol de sintaxis concreta para $(\neg(True \wedge False))$.

En este árbol se observa que:

- La raíz está etiquetada como *expr*, derivando la estructura completa.
- Se incluyen todos los elementos sintácticos, como los paréntesis, el operador de negación ($\neg$) y el operador de conjunción ($\wedge$).
- Cada nodo refleja una regla aplicada de la gramática, de forma que la estructura del árbol corresponde a la derivación concreta de la expresión.

2.4.2. Sintaxis abstracta

A diferencia de la sintaxis concreta, que describe cómo deben escribirse las construcciones del lenguaje incluyendo paréntesis, operadores escritos explícitamente o reglas estrictas de agrupación, la *sintaxis abstracta* modela la estructura semántica esencial de dichas construcciones, omitiendo los detalles superficiales que no afectan su significado computacional.

En particular, la sintaxis abstracta omite elementos considerados como *azúcar sintáctica*, es

decir, formas alternativas y más legibles de expresar una misma construcción. Estas formas, aunque distintas a nivel superficial, son tratadas de manera equivalente por el compilador o el intérprete [11]. No obstante, es importante aclarar que la sintaxis abstracta no elimina directamente la azúcar sintáctica, sino que se construye a partir de representaciones que ya han sido transformadas en etapas previas del compilador, a saber, el análisis léxico. Así, expresiones diferentes en la sintaxis concreta pueden ser equivalentes a nivel de sintaxis abstracta, lo que simplifica el análisis semántico posterior.

Formalizar la sintaxis abstracta de un lenguaje como MINIB no solo mejora la claridad conceptual del mismo, sino que además proporciona una base sólida para el desarrollo de analizadores, optimizadores y traductores. Esta representación estandarizada permite a las herramientas del compilador operar de forma más eficiente, robusta y extensible, al mismo tiempo que contribuye a un ecosistema de desarrollo más cohesionado¹³.

En el diseño de lenguajes de programación, la sintaxis abstracta representa un nivel de formalización que se enfoca exclusivamente en la estructura lógica y semántica del código fuente, dejando de lado los detalles específicos y superficiales de la notación concreta. Esta forma de representación simplificada es fundamental durante el proceso de compilación e interpretación, ya que permite trabajar con una versión estructurada del programa que captura su jerarquía interna y sus operaciones.

Árboles de sintaxis abstracta

A diferencia del árbol de sintaxis concreta, que refleja la estructura superficial del código fuente, el *árbol de sintaxis abstracta* (ASA¹⁴) ofrece una representación centrada en la estructura lógica del programa, tal como la concibe la persona programadora.

En un ASA se omiten los elementos puramente sintácticos que no afectan el significado computacional, conservando únicamente la jerarquía semánticamente relevante de las construcciones [?]. Esta simplificación reduce la complejidad estructural y resulta útil en fases posteriores del compilador, como la interpretación, el análisis semántico y la generación de código intermedio [7].

Estructura de los ASA. Los ASA se constituyen a partir de un conjunto reducido de construcciones fundamentales, lo que facilita su análisis y procesamiento. Este enfoque presupone una fase previa de transformación sintáctica, en la cual se reescriben expresiones complejas en términos más básicos, un proceso conocido como *desazucaramiento*. Así, se refuerza la distinción entre sintaxis concreta y abstracta, y se favorece un diseño modular y comprensible del compilador.

¹³En el ámbito de la computación, la *cohesión* describe el grado de relación entre los elementos internos de un módulo o componente. Se considera que un sistema es altamente cohesionado cuando sus partes (como funciones, variables o clases) están organizadas para cumplir una única responsabilidad claramente definida. Este principio promueve el diseño de software más mantenible, comprensible y reutilizable [10].

¹⁴En inglés, se conoce comúnmente como *Abstract Syntax Tree* (AST).

Definición 2.4.4: Árbol de sintaxis abstracta

Un árbol de sintaxis abstracta es un árbol ordenado $A = (N, E, r)$, donde:

- N es un conjunto finito de nodos etiquetados que representan construcciones del lenguaje; las hojas corresponden a valores constantes.
- $E \subseteq N \times N$ es el conjunto de aristas dirigidas entre nodos.
- $r \in N$ es la raíz del árbol.

[10]

Por ejemplo, la expresión lógica $(\neg(True \wedge False))$ se representa mediante el ASA mostrado en la figura 2.7, el cual preserva únicamente la jerarquía semántica de las operaciones, omitiendo elementos sintácticos superficiales.

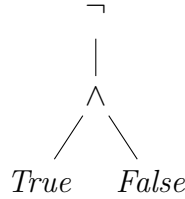


Figura 2.7: Árbol de sintaxis abstracta para $(\neg(True \wedge False))$.

Representación por reglas. Podemos definir formalmente los ASA de MINIB mediante reglas de inferencia que nos permiten construir árboles válidos a partir de expresiones más simples. A cada constructo sintáctico del lenguaje le asignamos una etiqueta única, y describimos cómo se construyen sus árboles a partir de subárboles [10].

Definimos la relación *e* ASA, la cual denota que *e* es un árbol de sintaxis abstracta válido. A continuación, describimos cada una de las construcciones básicas del lenguaje:

Constantes booleanas

Definimos el conjunto de valores booleanos como $\mathbb{B} = \{0, 1\}$, donde cada elemento numérico se asocia a una constante lógica: *1* representa *True* y *0* representa *False*. Estos valores se modelan como nodos hoja en los árboles de sintaxis abstracta, dado que, al ser atómicos, no presentan subestructuras internas¹⁵.

$$\frac{b \in \mathbb{B}}{\text{Boolean}(b) \text{ ASA}} \text{ ATOM}$$

En este sentido, tanto *True* como *False* son ASA por sí mismos. No tienen hijos y se consideran nodos terminales.

¹⁵En lenguajes como C y C++, aunque se distingue el tipo *bool*, los valores booleanos se implementan tradicionalmente como números, interpretando el cero como *false* y cualquier valor distinto de cero como *true* [14].

Negación

Una expresión negada se representa con un nodo etiquetado como *Not*, que tiene un único hijo correspondiente a la subexpresión que se niega.

$$\frac{e \text{ ASA}}{\text{Not}(e) \text{ ASA}} \text{ NOT}$$

La construcción *Not*(*e*) es un ASA si *e* también lo es.

Conjunción

Una conjunción se representa con un nodo etiquetado como *And*, con dos hijos que corresponden a las dos expresiones que se conjugan.

$$\frac{\frac{e_1 \text{ ASA}}{\text{And}(e_1, e_2) \text{ ASA}} \quad \frac{e_2 \text{ ASA}}{\text{And}(e_1, e_2) \text{ ASA}}}{\text{And}(e_1, e_2) \text{ ASA}} \text{ AND}$$

Si tanto *e*₁ como *e*₂ son ASA, entonces su conjunción también lo es.

Disyunción

Análogamente a la conjunción, una disyunción se representa con el nodo *Or*, también con dos subárboles como hijos.

$$\frac{\frac{e_1 \text{ ASA}}{\text{Or}(e_1, e_2) \text{ ASA}} \quad \frac{e_2 \text{ ASA}}{\text{Or}(e_1, e_2) \text{ ASA}}}{\text{Or}(e_1, e_2) \text{ ASA}} \text{ OR}$$

La construcción *Or*(*e*₁, *e*₂) es un ASA válido si ambos *e*₁ y *e*₂ lo son.

Estas reglas permiten construir de forma inductiva cualquier árbol de sintaxis abstracta válido en MINIB, partiendo desde las constantes booleanas y aplicando operadores lógicos sobre subestructuras válidas. En la siguiente sección, veremos cómo transformar un árbol de sintaxis concreta en su correspondiente árbol de sintaxis abstracta siguiendo estas construcciones.

Transformación de la sintaxis concreta a la sintaxis abstracta

La transformación de un ASC a un ASA constituye una etapa esencial del análisis sintáctico. Este proceso elimina paréntesis, palabras reservadas y otros símbolos del lenguaje, conservando únicamente la estructura lógica subyacente, relevante para la interpretación semántica [7].

Dicha depuración estructural no solo simplifica la representación del programa, sino que también refuerza la separación entre sintaxis concreta y semántica, facilitando una arquitectura de compilador más clara, modular y mantenible. Formalmente, esta transformación puede expresarse como una función de traducción:

$$\varphi : S \rightarrow \text{ASA}$$

la cual puede descomponerse en dos etapas fundamentales:

1. **Análisis léxico:** La función *lexer* transforma una cadena de caracteres del alfabeto Σ en una secuencia de *tokens*:

$$lexer : \Sigma^* \rightarrow [Token]$$

2. **Análisis sintáctico:** A partir de dicha secuencia, la función *parser* construye el árbol de sintaxis abstracta:

$$parser : [Token] \rightarrow ASA$$

La composición de estas funciones permite transformar directamente un programa fuente en su representación estructural:

$$parser \circ lexer : \Sigma^* \rightarrow ASA$$

En el caso del lenguaje MINIB, el ASA se ha diseñado a partir de un conjunto mínimo de construcciones lógicas: las constantes *True* y *False*, la negación ($\neg$), la conjunción ($\wedge$) y la disyunción ($\vee$). Esta selección ofrece una base expresiva suficiente, ya que otras construcciones más complejas, como la implicación o la equivalencia, pueden ser reescritas como azúcar sintáctica a partir de estas primitivas [15].

Así, la transformación desde la sintaxis concreta hacia la sintaxis abstracta no solo elimina construcciones derivadas o redundantes, sino que también permite un modelo semántico más claro y operativo, facilitando las etapas posteriores del compilador [10].

Ejemplo

Consideremos la expresión lógica concreta:

$$(True \wedge (\neg False))$$

Su correspondiente árbol de sintaxis concreta incluiría todos los detalles gramaticales, como los paréntesis, el operador binario $\wedge$ y el operador unario $\neg$. No obstante, al transformarlo en un árbol de sintaxis abstracta, eliminamos los elementos puramente sintácticos y nos centramos en la estructura semántica de la expresión. El resultado sería el siguiente árbol de sintaxis abstracta:

$$And(Boolean(True), Not(Boolean(False)))$$

La representación visual del ASA correspondiente se muestra en la figura 2.8. En ella, se observa que la operación principal es una conjunción (*And*) cuyos operandos son la constante booleana *Boolean(True)* y la negación de *Boolean(False)*. Esta estructura refleja únicamente la jerarquía semántica esencial de la expresión, omitiendo elementos como paréntesis o el orden textual original, los cuales son relevantes únicamente en la sintaxis concreta.

Este proceso de transformación permite trabajar con una representación lógica y estructural del programa, mucho más adecuada para su análisis y manipulación [10].

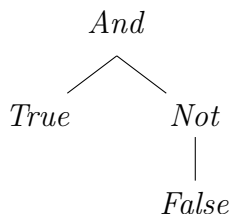


Figura 2.8: Árbol de sintaxis abstracta para $And(True, Not(False))$

Azúcar sintáctica

En el diseño de lenguajes de programación, se denomina *azúcar sintáctica* (del inglés *syntactic sugar*) a aquellas construcciones sintácticas que, si bien no amplían las capacidades fundamentales del lenguaje, se introducen con el objetivo de mejorar su legibilidad, expresividad o conveniencia. El término fue acuñado por Peter Landin en 1964, aludiendo a que tales construcciones “endulzan” el lenguaje para hacerlo más cómodo de utilizar, sin alterar su sustancia computacional.

Desde una perspectiva formal, se considera azúcar sintáctica a cualquier construcción que puede ser traducida, durante la compilación, a otras construcciones más fundamentales del lenguaje. Esta traducción se conoce como *desazucarización* o *desugaring*, y consiste en una transformación sistemática desde la sintaxis abstracta de la superficie del lenguaje (aquella que ve la persona programadora) hacia un subconjunto más reducido, denominado *lenguaje núcleo* o *core*. Así, la azúcar sintáctica divide conceptualmente el lenguaje en dos niveles: una interfaz rica y expresiva (superficie), y un núcleo mínimo y esencial (core) [16]. Esta fase de transformación (considerada una etapa de compilación) se implementa típicamente mediante reglas que operan sobre árboles de sintaxis abstracta, mapeando construcciones superficiales a representaciones semánticamente equivalentes en el núcleo del lenguaje. Por ejemplo, un ciclo *for-each* en JAVA puede desazucarizarse en un ciclo *for* convencional con un iterador explícito: ambas formas tienen el mismo comportamiento semántico, aunque difieren en su estructura sintáctica [17].

La existencia de una fase de desazucarización conlleva ventajas prácticas y teóricas: por un lado, reduce la complejidad del núcleo semántico del lenguaje, facilitando tareas como la generación de herramientas o la verificación formal de propiedades (por ejemplo, pruebas de solidez de tipos); por otro lado, permite a las personas programadoras expresar ideas con una sintaxis más cercana a su intención original [18].

No obstante, el uso de azúcar sintáctica debe justificarse cuidadosamente. Idealmente, solo debería utilizarse para definir abstracciones que no pueden expresarse directamente mediante mecanismos ya presentes en el núcleo del lenguaje, como funciones, clases o definiciones de datos. En particular, el diseño de nuevos constructos de enlace (*binding*) o la manipulación de nombres de variables, aspectos no triviales de abstraer en muchos lenguajes, representa un caso legítimo para introducir azúcar sintáctica [16].

Hasta este punto, se ha definido una transformación desde los árboles de sintaxis concreta hacia árboles de sintaxis abstracta que abarca únicamente las construcciones fundamentales del lenguaje MINIB: las constantes lógicas *True* y *False*, la negación ($\neg$), la conjunción ($\wedge$) y la disyunción ($\vee$). Sin embargo, construcciones como la implicación lógica ($\rightarrow$) o la

equivalencia ($\leftrightarrow$) aún no han sido formalmente tratadas en dicha transformación, lo cual impide que esta pueda considerarse una función *total*¹⁶.

Desde la lógica proposicional clásica, es bien sabido que tanto la implicación como la equivalencia pueden expresarse mediante combinaciones de los operadores básicos. Por tanto, su inclusión en la sintaxis concreta del lenguaje responde a fines pragmáticos, como mejorar la legibilidad o reducir errores, sin aumentar su poder expresivo. En consecuencia, estas construcciones se consideran *azúcar sintáctica* y pueden ser eliminadas mediante una transformación estructural sin pérdida semántica.

Formalmente, dicha transformación corresponde a una etapa posterior al análisis sintáctico [20], donde se trabaja sobre representaciones estructuradas de los programas, es decir, sobre árboles de sintaxis abstracta. Siguiendo la notación introducida por Inala [17], definimos dos representaciones abstractas distintas para el lenguaje MINIB:

- *srcASA*: el árbol de sintaxis abstracta del lenguaje fuente, que puede contener construcciones azucaradas como *Implies* (implicación) o *Equiv* (equivalencia lógica).
- *dstASA*: el árbol desazucarado, en el cual todas las construcciones se han reescrito utilizando únicamente los operadores fundamentales del núcleo lógico: *Not*, *And*, y *Or*.

A fin de formalizar la relación *e srcASA*, introducimos reglas de inferencia análogas a las utilizadas para los ASA ya definidos. Estas reglas permiten modelar de forma inductiva la estructura de los árboles abstractos con azúcar sintáctica.

Las nuevas reglas para las construcciones azucaradas son las siguientes:

$$\begin{array}{c}
 \frac{b \in \mathbb{B}}{Boolean_{scr}(b) \ srcASA} \text{ATOM}_{scr} \qquad \frac{e \ srcASA}{Not_{scr}(e) \ srcASA} \text{NOT}_{scr} \\
 \\
 \frac{e_1 \ srcASA \quad e_2 \ srcASA}{And_{scr}(e_1, e_2) \ srcASA} \text{AND}_{scr} \qquad \frac{e_1 \ srcASA \quad e_2 \ srcASA}{Or_{scr}(e_1, e_2) \ srcASA} \text{OR}_{scr} \\
 \\
 \frac{e_1 \ srcASA \quad e_2 \ srcASA}{Implies_{scr}(e_1, e_2) \ srcASA} \text{IMPLIES}_{scr} \qquad \frac{e_1 \ srcASA \quad e_2 \ srcASA}{Equiv_{scr}(e_1, e_2) \ srcASA} \text{EQUIV}_{scr}
 \end{array}$$

Así, definimos *dstASA* simplemente como un alias para los árboles de sintaxis abstracta contruidos mediante esta relación ASA, que no incluye construcciones azucaradas. Esta distinción terminológica permite explicitar el resultado de la eliminación del azúcar sintáctica y evita ambigüedades en la formalización semántica y en las transformaciones del compilador. La función de traducción entre estas representaciones se denomina:

$$desugar : srcASA \rightarrow dstASA$$

La función *desugar* se define de forma inductiva sobre la estructura de los árboles etiquetados con las formas de *srcASA*, y produce árboles de tipo *dstASA* (es decir, árboles ASA).

¹⁶Una función total está definida para todas las entradas posibles en su dominio, lo cual garantiza la completitud y previsibilidad del comportamiento del programa [19].

Su dominio son expresiones construidas con las etiquetas $Boolean_{src}$, Not_{src} , And_{src} , Or_{src} , $Implies_{src}$ y $Equiv_{src}$, y su codominio son expresiones formadas únicamente por las etiquetas de la relación ASA.

$$desugar(e) = \begin{cases} Boolean(b) & \text{si } e = Boolean_{src}(b), \\ Not(desugar(e')) & \text{si } e = Not_{src}(e'), \\ And(desugar(e_1), desugar(e_2)) & \text{si } e = And_{src}(e_1, e_2), \\ Or(desugar(e_1), desugar(e_2)) & \text{si } e = Or_{src}(e_1, e_2), \\ Or(Not(desugar(e_1)), desugar(e_2)) & \text{si } e = Implies_{src}(e_1, e_2), \\ Or(And(desugar(e_1), desugar(e_2)), \\ And(Not(desugar(e_1)), Not(desugar(e_2)))) & \text{si } e = Equiv_{src}(e_1, e_2). \end{cases} \quad (2.1)$$

Cada cláusula aplica recursivamente la transformación sobre las subfórmulas de la expresión, garantizando que, sin importar la profundidad de la fórmula original, el resultado estará completamente libre de construcciones derivadas. Esto asegura que toda fórmula en MINIB puede ser transformada a una forma básica que solo incluye operadores para los cuales la sintaxis abstracta está previamente definida mediante reglas y etiquetas bajo la relación ASA.

Ejemplo. Sea la fórmula de entrada en sintaxis concreta:

$$(True \rightarrow (False \leftrightarrow (\neg True)))$$

Su representación como árbol en $srcASA$ es la siguiente:

$$Implies_{src}(Boolean_{src}(True), Equiv_{src}(Boolean_{src}(False), Not_{src}(Boolean_{src}(True))))$$

Aplicamos la función $desugar$ paso a paso sobre esta estructura:

$$\begin{aligned} & desugar(Implies_{src}(Boolean_{src}(True), Equiv_{src}(Boolean_{src}(False), Not_{src}(Boolean_{src}(True)))) \\ &= Or(Not(desugar(Boolean_{src}(True))), desugar(Equiv_{src}(\dots))) \\ &= Or(Not(Boolean(True)), Or(And(Boolean(False), Not(Boolean(True))), \\ &\quad And(Not(Boolean(False)), Not(Not(Boolean(True)))))) \end{aligned}$$

El resultado es un árbol etiquetado únicamente con las construcciones válidas de la relación ASA ($dstASA$), es decir, pertenece a la sintaxis abstracta desazucarada ASA.

Así, el análisis sintáctico produce primero un $srcASA$, que contiene todas las construcciones utilizadas por la persona programadora. Posteriormente, la función $desugar$ transforma este árbol en un $dstASA$ equivalente pero expresado únicamente con las formas esenciales del lenguaje. Este enfoque separa claramente el diseño orientado a la persona programadora (con una sintaxis rica y expresiva) del núcleo semántico formal (más reducido y manejable).

Con esta incorporación, redefinimos el proceso de transformación del programa fuente como una composición de tres etapas:

$$\varphi = \text{desugar} \circ \text{parser} \circ \text{lexer} : S \rightarrow \text{dstASA}$$

Donde:

- *lexer*: Toma una cadena de entrada $S \in \Sigma^*$ y produce una secuencia de tokens:

$$\text{lexer} : \Sigma^* \rightarrow [\text{Token}]$$

- *parser*: Construye el árbol de sintaxis abstracta del lenguaje fuente:

$$\text{parser} : [\text{Token}] \rightarrow \text{srcASA}$$

- *desugar*: Traduce el ASA fuente a su forma núcleo eliminando el azúcar sintáctica:

$$\text{desugar} : \text{srcASA} \rightarrow \text{dstASA}$$

Esta arquitectura modular no solo permite un razonamiento más claro sobre la semántica del lenguaje MINIB, sino que también facilita la implementación de compiladores y verificadores, al reducir la complejidad de las representaciones internas y delegar la riqueza sintáctica a fases tempranas del análisis.

2.5. Semántica de lenguajes de programación

Hasta ahora hemos construido una caracterización formal del lenguaje MINIB, partiendo de su sintaxis concreta y llegando a su representación abstracta mediante ASA. Sin embargo, conocer únicamente la estructura de un programa no basta para comprender su significado. A continuación, abordaremos un aspecto fundamental en el estudio de los lenguajes de programación: su *semántica*.

En términos generales, la semántica de un lenguaje de programación describe el significado de los programas escritos en él, es decir, especifica qué efectos produce su ejecución. Aunque resulta relativamente accesible explicar de manera informal lo que hace un programa simple, por ejemplo, indicar que una instrucción imprime un valor en pantalla o que una función retorna un número, proporcionar una definición precisa y rigurosa de ese comportamiento implica enfrentar una complejidad mayor. Esta dificultad proviene de la necesidad de modelar formalmente todos los aspectos de ejecución del programa, incluyendo su ambiente, sus efectos secundarios y su evolución paso a paso, algo que excede con mucho la especificación de su sintaxis [7].

Mientras que la sintaxis define las estructuras válidas del lenguaje, es decir, qué cadenas de símbolos forman programas correctos desde el punto de vista gramatical, la semántica permite determinar tanto los efectos de ejecutar esos programas como su validez a nivel de significado. Por ejemplo, la expresión “*hola*” + 3 puede ser sintácticamente válida en ciertos lenguajes (si las reglas permiten la suma entre una cadena y un número), pero semánticamente incorrecta si dicha operación carece de sentido en el modelo de ejecución. Este tipo

de situaciones ilustra cómo una expresión aceptada por la gramática puede carecer de un significado válido, subrayando la necesidad de una especificación semántica clara que complemente las reglas sintácticas [7].

Dentro del estudio de la semántica, se suelen distinguir dos niveles principales: la *semántica estática* y la *semántica dinámica*. La primera se refiere a propiedades del programa que pueden determinarse sin ejecutarlo, como la verificación de tipos, mientras que la segunda describe el comportamiento del programa durante su ejecución, incluyendo la evaluación de expresiones y la evolución del estado del sistema. Cada una permite responder preguntas distintas sobre los programas, y ambas son esenciales para comprender, razonar y garantizar el correcto funcionamiento de un lenguaje de programación [6].

Semántica estática

La *semántica estática* se refiere a aquellas propiedades semánticas que pueden determinarse sin ejecutar el programa, es decir, durante la fase de compilación o análisis previo a la ejecución. Estas propiedades permiten detectar errores y garantizar ciertas condiciones sobre el comportamiento del programa antes de su ejecución [21].

Entre los aspectos que abarca la semántica estática se encuentran las reglas de *tipificado*¹⁷, así como el análisis del *alcance léxico*, que determina en qué partes del programa una variable puede ser referenciada válidamente. Este análisis del alcance se basa en estructuras auxiliares conocidas como *ambientes de evaluación*¹⁸, las cuales también desempeñan un papel central en la semántica dinámica [6].

Por ejemplo, el tipo de una expresión nos indica qué operaciones son permitidas sobre ella (como sumar números o concatenar cadenas), mientras que el alcance define los contextos en los que una variable tiene significado. Así, la semántica estática establece restricciones que permiten prevenir errores antes de la ejecución, como intentar realizar operaciones aritméticas sobre valores booleanos, acceder a variables no declaradas o invocar funciones con un número incorrecto de argumentos.

En muchos lenguajes, estas reglas se formalizan mediante un sistema de tipos, que agrupa valores y expresiones según sus propiedades semánticas compartidas. Aunque las reglas sintácticas determinan si una secuencia de símbolos constituye una construcción válida del lenguaje, es la semántica estática la que permite verificar si dicha construcción es coherente en cuanto a tipos, alcance y contexto dentro del programa [21].

Semántica dinámica

Por otro lado, la *semántica dinámica* establece la relación entre las construcciones sintácticas del lenguaje y un modelo computacional que describe su comportamiento en tiempo de ejecución. Esta dimensión semántica se centra en la ejecución efectiva de los programas, especificando cómo se llevan a cabo los cálculos y qué efectos producen sobre el estado de

¹⁷Un *tipo* es, en términos generales, una categoría de datos que define qué valores puede tomar una variable, qué operaciones se pueden realizar sobre ella y cómo se representa internamente en el sistema [22].

¹⁸Un *ambiente de evaluación* es una estructura que mantiene un registro de las asociaciones entre identificadores y valores en un momento dado durante la ejecución de un programa. Permite modelar el contexto en el que se evalúan las expresiones, especialmente en presencia de funciones, ámbitos léxicos y sustitución de variables [20].

un sistema. En otras palabras, responde preguntas como qué valor resulta de evaluar una expresión o qué cambios introduce una instrucción en el ambiente de ejecución. La semántica dinámica no puede ser verificada completamente en tiempo de compilación, por lo que sus reglas suelen aplicarse en tiempo de ejecución, incluyendo aspectos como la validez de accesos a memoria, el control de errores como la división por cero o el desbordamiento aritmético y la garantía de que las variables han sido inicializadas antes de su uso [6, 21].

En esta línea, se han desarrollado distintos enfoques formales para describir la semántica de los lenguajes de programación, entre los cuales destacan los siguientes:

- **Semántica operacional.** Define el comportamiento dinámico de un lenguaje de programación a través de una *máquina abstracta* (véase la sección 2.6) que ejecuta los términos del lenguaje. En este contexto, el estado de ejecución se representa mediante las propias expresiones del lenguaje, y su evolución se modela mediante reglas de transición [19].

Existen dos estilos predominantes para formalizar este tipo de semántica. El primero es el enfoque de *paso pequeño* (*small-step semantics*) o también conocido como *semántica operacional estructural* (*Structural Operational Semantics*, SOS), introducido por Plotkin [23]. En este estilo, la ejecución de un programa se descompone en una secuencia detallada de pasos, cada uno representando una transición elemental que transforma una expresión en otra. Esta granularidad permite razonar con precisión sobre cada etapa intermedia de la evaluación y es especialmente útil para estudiar comportamientos no deterministas, concurrencia y errores en tiempo de ejecución.

El segundo enfoque es el de *paso grande* (*big-step semantics*), también llamado *semántica natural* (*natural semantics*), propuesto por Kahn en 1987 [19]. Este estilo describe cómo una expresión completa se evalúa directamente hasta su resultado final, omitiendo los pasos intermedios. Su carácter relacional lo hace particularmente adecuado para especificar la evaluación total de programas e interpretar correctamente su significado en términos de entrada y salida. Además, este enfoque facilita la implementación de intérpretes al establecer reglas claras que asocian expresiones con sus respectivos valores finales.

- **Semántica denotativa.** Este enfoque define el significado de un programa al asociar cada expresión del lenguaje con un objeto matemático, como un número, una función o una estructura más compleja, que captura su comportamiento de manera abstracta. Esta técnica, conocida como *semántica denotativa*, fue desarrollada originalmente por Christopher Strachey y formalizada matemáticamente por Dana Scott mediante el uso de órdenes parciales completos, funciones continuas y puntos fijos mínimos [3].

El proceso de definir una semántica denotativa consiste en establecer una colección de *dominios semánticos* apropiados y una *función de interpretación* que asigne a cada construcción del lenguaje un elemento del dominio correspondiente [19]. Este nivel de abstracción permite prescindir de los detalles operativos de la evaluación, centrando la atención en las propiedades esenciales del lenguaje.

Una de sus principales ventajas radica en que facilita el razonamiento formal sobre los programas: permite demostrar, por ejemplo, la equivalencia entre dos programas o verificar que un programa cumple con una especificación dada. Además, cuando el modelo

es adecuadamente construido, posibilita la obtención de leyes algebraicas y teoremas que serían difíciles de derivar utilizando únicamente métodos operacionales [23].

- **Semántica axiomática.** A diferencia de los enfoques operacional o denotativa, la *semántica axiomática* (*axiomatic semantics*) no define el significado de un programa en términos de su ejecución o su interpretación matemática, sino a través de las propiedades que pueden probarse sobre él. En este marco, el significado de una construcción del lenguaje se establece mediante reglas lógicas (axiomas o reglas de inferencia) que permiten derivar teoremas sobre su comportamiento [19].

Este enfoque fue introducido por Floyd y Hoare, y se consolidó con el desarrollo de la llamada *lógica de Hoare* (*Hoare logic*), una metodología basada en triples de la forma $\{P\} C \{Q\}$, donde P y Q son aserciones lógicas sobre el estado del programa antes y después de ejecutar la instrucción C [3]. Su objetivo principal es el de verificar la corrección de los programas desde el inicio, empleando invariantes y razonamiento deductivo.

Una de las características distintivas de la semántica axiomática es que las propias reglas de inferencia constituyen la definición semántica del lenguaje: “el significado de un término es aquello que puede demostrarse sobre él” [19]. Esta perspectiva ha sido particularmente útil en el desarrollo de sistemas de verificación formal y herramientas automáticas de análisis estático, al permitir una representación directa de los requisitos funcionales y de seguridad que debe satisfacer un programa.

Si bien esta aproximación aporta elegancia y claridad en ciertos dominios, por ejemplo, en lenguajes imperativos simples como los programas *while*, su aplicabilidad general está limitada por la incompletitud inherente de los sistemas axiomáticos, como lo sugiere el teorema de incompletitud de Gödel [3]. No obstante, su integración con enfoques operacional y denotativa ha dado lugar a marcos semánticos unificados que aprovechan los beneficios complementarios de cada técnica, permitiendo razonamientos más robustos sobre el comportamiento de los programas [19, 23].

A través de la semántica operacional, podremos establecer un marco formal riguroso que especifique de manera precisa cómo se evalúan las expresiones y cómo evolucionan los programas en MINIB. Este enfoque no solo resulta apropiado por su claridad estructural y cercanía con la implementación, sino que se alinea directamente con la propuesta metodológica de Bahr y Hutton [1], quienes en su desarrollo inicial del lenguaje limitan deliberadamente el análisis semántico al nivel operacional como vía principal para razonar sobre la corrección y el comportamiento del lenguaje.

El uso de enfoques alternativos, como la semántica denotativa o axiomática, no sería idóneo en este contexto. La semántica denotativa, al requerir una abstracción matemática de alto nivel (por ejemplo, funciones continuas o dominios ordenados) busca capturar el significado de los programas en términos de objetos matemáticos, lo cual introduce un nivel de complejidad que sobrepasa los objetivos operativos y pedagógicos del presente trabajo. Por su parte, la semántica axiomática se orienta principalmente a la verificación de propiedades lógicas mediante invariantes y reglas de inferencia, lo cual resulta más adecuado para la especificación de contratos formales que para describir mecanismos de evaluación [19, 21].

En cambio, la semántica operacional proporciona un marco sintácticamente guiado y suficientemente expresivo para modelar paso a paso el comportamiento de los programas y facilita la implementación directa de intérpretes o prototipos del lenguaje.

2.5.1. Semántica operacional

Dado lo anterior, conviene destacar la relevancia metodológica de la semántica operacional como marco formal para especificar y analizar el comportamiento dinámico de los programas. Este enfoque no solo describe la evolución de una expresión mediante reglas de inferencia, sino que además ofrece una estructura que facilita el razonamiento matemático y la implementación directa de intérpretes y herramientas de análisis.

En particular, su formulación a través de un sistema de reglas de transición (ya sea en el estilo *small-step* o *big-step*) permite establecer relaciones precisas entre una expresión y sus formas evaluadas, lo cual resulta esencial tanto para la verificación formal de propiedades del lenguaje como para su aplicación práctica en compiladores y entornos de ejecución [3, 19, 23]. Además, la semántica operacional aporta una ventaja sustancial frente a otros enfoques, como la semántica denotativa o axiomática, al mantener una correspondencia explícita entre la sintaxis y la dinámica de ejecución. Esta propiedad estructural refuerza la claridad y modularidad en la definición de las reglas semánticas, promoviendo una formalización más accesible y adaptable a distintas extensiones del lenguaje¹⁹.

En este trabajo, dicho enfoque será fundamental para desarrollar la semántica del lenguaje en estudio, guiando la definición de sus reglas evaluativas y proporcionando un marco formal que respalda su implementación y análisis.

Sistema de transición

Para describir formalmente el comportamiento dinámico de los programas, es decir, cómo se ejecutan y evolucionan paso a paso, diseñamos un *sistema de transición*. Esta noción proporciona una base común para modelar distintos aspectos de la ejecución de programas de lenguajes de programación, desde la evaluación de expresiones hasta la ejecución de comandos. En este marco, el comportamiento se describe mediante transiciones entre configuraciones (o estados) del sistema, definidas de forma inductiva mediante reglas.

¹⁹Plotkin introdujo el enfoque estructural precisamente para preservar la correspondencia entre la estructura sintáctica y la semántica de ejecución, lo cual ha demostrado ser una base sólida para el análisis formal de lenguajes, incluso en presencia de concurrencia o efectos secundarios [23].

Definición 2.5.1: Sistema de transición

Un *sistema de transición* es una tupla (E, δ, I, F) , donde:

- E es un conjunto (posiblemente infinito) de *estados*.
- $\delta \subseteq E \times E$ es una *relación de transición*, que describe cómo un estado puede evolucionar hacia otro.
- $I \subseteq E$ es el conjunto de *estados iniciales*, desde los cuales puede comenzar la ejecución del sistema.
- F es un conjunto (posiblemente disjunto de E) de *estados finales*, que representan los puntos de término de la ejecución.

[10, 21]

En el contexto de los lenguajes de programación, los elementos de esta definición se interpretan del siguiente modo:

- **Estados:** Cada estado representa una configuración del programa durante su ejecución. Esto puede incluir la posición actual del programa (como el punto de control en el código), el contenido de la memoria, el ambiente de evaluación, el estado de la pila de ejecución, entre otros aspectos relevantes.
- **Estados iniciales:** Son las configuraciones posibles desde las cuales puede comenzar la ejecución de un programa. Generalmente representan el programa justo antes de su ejecución principal, con las variables inicializadas y sin efectos secundarios.
- **Estados finales:** Representan las configuraciones en las que la ejecución del programa ha terminado. Estas pueden corresponder a finalizaciones exitosas, a errores (como excepciones o fallos de ejecución), o a puntos de retorno dentro de funciones o ciclos.
- **Relación de transición (δ):** Describe cómo evoluciona el programa paso a paso. Esta relación se especifica mediante un conjunto de reglas operacionales que definen cómo una instrucción afecta al estado del programa. Por ejemplo, una regla puede describir cómo una asignación modifica el valor de una variable en el ambiente actual.

Este enfoque inductivo y basado en reglas nos permite formalizar el comportamiento de los programas de manera modular y extensible. Sin embargo, debido a que las reglas operacionales dependen directamente de la sintaxis del lenguaje, esta técnica puede dificultar la comparación entre programas escritos en lenguajes distintos, ya que las estructuras sintácticas difieren entre ellas [21].

Sistema de transición de paso grande para MINIB

Definimos ahora un sistema de transición que modela la semántica natural del lenguaje MINIB. Este sistema describe cómo se evalúan las expresiones booleanas del lenguaje a valores finales.

Ejemplo 2.5.1: Sistema de transición *big-step* para MINIB

El sistema de transición para MINIB se define como la tupla $(E, \Rightarrow, I, F)$, donde:

- $E = \{e \mid e \in \text{ASA}\}$ es el conjunto de estados del sistema. Cada estado corresponde a una expresión bien formada del lenguaje MINIB en sintaxis abstracta.
- $I = E$, es decir, cualquier expresión bien formada puede considerarse como estado inicial.
- $F = \{\text{Boolean}_V(\text{True}), \text{Boolean}_V(\text{False})\}$, el conjunto de estados finales, que representan los únicos valores posibles al evaluar una expresión booleana.
- $\Rightarrow \subseteq I \times F$ es la relación de transición, que relaciona una expresión con su valor booleano resultante.

La relación de transición se escribe como:

$$e \Rightarrow v$$

y se interpreta como “la expresión e se evalúa al valor v ”.

A continuación, definimos las reglas de inferencia que describen esta relación. Es importante señalar que los operadores utilizados para denotar la evaluación de las expresiones (negación ($\neg$), conjunción ($\wedge$) y disyunción ($\vee$)) provienen directamente de la lógica proposicional clásica. Por tanto, no dependen de la implementación o sintaxis de ningún lenguaje de programación específico, lo que permite mantener una descripción formal, general y puramente semántica de MINIB.

Constantes booleanas. Las expresiones constantes se evalúan a sí mismas:

$$\frac{}{\text{Boolean}(\text{True}) \Rightarrow \text{Boolean}_V(\text{True})}$$

$$\frac{}{\text{Boolean}(\text{False}) \Rightarrow \text{Boolean}_V(\text{False})}$$

Negación. Una expresión de la forma $\text{Not}(e)$ se evalúa negando el resultado de evaluar e :

$$\frac{e \Rightarrow \text{Boolean}_V(v)}{\text{Not}(e) \Rightarrow \text{Boolean}_V(\neg v)}$$

donde $v \in \mathbb{B}$.

Conjunción. Esta expresión se evalúa como la conjunción de los valores de sus subexpresiones:

$$\frac{e_1 \Rightarrow \text{Boolean}_V(v_1) \quad e_2 \Rightarrow \text{Boolean}_V(v_2)}{\text{And}(e_1, e_2) \Rightarrow \text{Boolean}_V(v_1 \wedge v_2)}$$

Disyunción. Se evalúa como la disyunción de los valores de sus subexpresiones:

$$\frac{e_1 \Rightarrow \text{Boolean}_V(v_1) \quad e_2 \Rightarrow \text{Boolean}_V(v_2)}{\text{Or}(e_1, e_2) \Rightarrow \text{Boolean}_V(v_1 \vee v_2)}$$

Cada una de estas reglas refleja la semántica natural del lenguaje: evalúan las expresiones de manera composicional, basándose en los resultados de evaluar sus componentes.

Ejemplo 2.5.2: Derivación de expresiones de MINIB

Sintaxis concreta: $(\neg \text{True})$

Sintaxis abstracta: $\text{Not}(\text{Boolean}(\text{True}))$

Evaluación:

$$\frac{\text{Boolean}(\text{True}) \Rightarrow \text{Boolean}_V(\text{True})}{\text{Not}(\text{Boolean}(\text{True})) \Rightarrow \text{Boolean}_V(\text{False})}$$

Sintaxis concreta: $((\neg(\text{True} \wedge \text{False})) \vee (\text{False} \vee \text{True}))$

Sintaxis abstracta:

$\text{Or}(\text{Not}(\text{And}(\text{Boolean}(\text{True}), \text{Boolean}(\text{False}))), \text{Or}(\text{Boolean}(\text{False}), \text{Boolean}(\text{True})))$

Evaluación:

$$\frac{\frac{\frac{\text{True} \Rightarrow \text{True} \quad \text{False} \Rightarrow \text{False}}{\text{And}(\text{True}, \text{False}) \Rightarrow \text{False}}}{\text{Not}(\text{And}(\text{True}, \text{False})) \Rightarrow \text{True}} \quad \frac{\text{False} \Rightarrow \text{False} \quad \text{True} \Rightarrow \text{True}}{\text{Or}(\text{False}, \text{True}) \Rightarrow \text{True}}}{\text{Or}(\text{Not}(\text{And}(\text{True}, \text{False})), \text{Or}(\text{False}, \text{True})) \Rightarrow \text{True}}$$

Nota: En el último ejemplo omitimos el constructo del ASA (Boolean) y el constructo Boolean_V de F para volver más legible el ejemplo.

Sistema de transición de paso pequeño para MINIB

Para modelar la evaluación paso a paso de las expresiones en MINIB, introducimos una relación de transición elemental, denotada por:

$$e_1 \rightarrow e_2,$$

la cual distingue a la semántica de paso pequeño de la semántica natural. Mientras que esta última relaciona directamente una expresión con su resultado final, la semántica de paso pequeño especifica cómo una expresión e_1 se transforma en otra e_2 a través de una única reducción, capturando explícitamente cada etapa intermedia del proceso de evaluación [21].

Ejemplo 2.5.3: Sistema de transición *small-step* para MINIB

El sistema de transición se define sobre la misma tupla $(E, \rightarrow, I, F)$ previamente establecida para la semántica de paso grande, con los siguientes componentes:

- E sigue siendo el conjunto de expresiones bien formadas del lenguaje MINIB en su sintaxis abstracta.
- $I = E$, cualquier expresión válida puede ser considerada como estado inicial.
- $F = \{Boolean(True), Boolean(False)\}$, representa el conjunto de estados finales. Cabe destacar que, en este sistema de reducción, la notación de los valores booleanos se simplifica omitiendo la envoltura de constructo explícito ($Boolean_v$), ya que el énfasis está puesto en el comportamiento sintáctico de las expresiones y no en su representación interna como valores evaluados. Esta notación más ligera permite una presentación más uniforme en la inferencia estructural.
- La relación de transición $\rightarrow$ se define mediante reglas de inferencia estructural, las cuales determinan cómo una expresión puede reducirse en función de su forma sintáctica [23]. En este sentido, $\rightarrow$ relaciona expresiones con otras expresiones (que pueden ser intermedias o finales), modelando el proceso de evaluación como una secuencia de transiciones paso a paso.

A diferencia de la evaluación total en la semántica natural, aquí el foco está en describir detalladamente el mecanismo de evaluación, permitiendo observar con precisión el orden, control y propagación del cómputo a lo largo de la ejecución de un programa.

Constantes booleanas. Las expresiones constantes se evalúan a sí mismas:

$$\frac{}{Boolean(True) \rightarrow Boolean(True)}$$

$$\frac{}{Boolean(False) \rightarrow Boolean(False)}$$

Negación. El comportamiento de la negación se divide en dos reglas para los siguientes casos respectivamente.

- **Caso 1:** El argumento de la negación no ha sido reducido a un valor. El siguiente paso consiste en reducir la subexpresión e .

$$\frac{e \rightarrow e'}{Not(e) \rightarrow Not(e')}$$

- **Caso 2:** Si el argumento de la negación ha sido reducido a un valor, entonces el paso siguiente es negar el valor.

$$\frac{e \rightarrow Boolean(v)}{Not(e) \rightarrow Boolean(\neg v)}$$

donde $v \in \mathbb{B}$.

Conjunción. El comportamiento de la conjunción se divide en tres casos:

- **Caso 1:** Ninguno de los argumentos de la conjunción ha sido reducido a un valor. El siguiente paso consiste en reducir el lado izquierdo.

$$\frac{e_1 \rightarrow e'_1}{And(e_1, e_2) \rightarrow And(e'_1, e_2)}$$

Por ejemplo, si tenemos $And(And(Boolean(v_1), Boolean(v_2)), Boolean(v_3))$, un paso de evaluación corresponde a evaluar $And(Boolean(v_1), Boolean(v_2))$.

- **Caso 2:** El argumento izquierdo de la conjunción ha sido reducido a un valor pero el derecho no. El siguiente paso consiste en reducir el argumento derecho.

$$\frac{e_2 \rightarrow e'_2}{And(Boolean(v_1), e_2) \rightarrow And(Boolean(v_1), e'_2)}$$

Por ejemplo, si tenemos $And(Boolean(True), Or(Boolean(False), Boolean(True)))$, el lado izquierdo ya es un valor, por lo que procedemos a reducir el lado derecho.

- **Caso 3:** Los dos argumentos de la conjunción han sido reducidos a valores. El siguiente paso es aplicar la conjunción entre los valores.

$$\frac{}{And(Boolean(v_1), Boolean(v_2)) \rightarrow Boolean(v_1 \wedge v_2)}$$

Disyunción. El comportamiento de la disyunción es análogo al de la conjunción.

- **Caso 1:** Si ninguno de los argumentos de la disyunción ha sido reducido a un valor, entonces el siguiente paso consiste en reducir el lado izquierdo.

$$\frac{e_1 \rightarrow e'_1}{Or(e_1, e_2) \rightarrow Or(e'_1, e_2)}$$

- **Caso 2:** El argumento izquierdo de la disyunción ha sido reducido a un valor pero el derecho no. El siguiente paso consiste en reducir el argumento derecho.

$$\frac{e_2 \rightarrow e'_2}{Or(Boolean(v_1), e_2) \rightarrow Or(Boolean(v_1), e'_2)}$$

- **Caso 3:** Los dos argumentos de la disyunción han sido reducidos a valores, entonces el siguiente paso es aplicar la disyunción entre los valores.

$$\frac{}{And(Boolean(v_1), Boolean(v_2)) \rightarrow Boolean(v_1 \vee v_2)}$$

Ejemplo 2.5.4: Derivación de expresiones de MINIB**Sintaxis concreta:** $(\neg \text{True})$ **Sintaxis abstracta:** $\text{Not}(\text{Boolean}(\text{True}))$ **Evaluación:**

$$\begin{aligned}
& \text{Not}(\text{Boolean}(\text{True})) \\
& \rightarrow \text{Not}(\text{Boolean}(\text{True})) \\
& \rightarrow \text{Boolean}(\text{False})
\end{aligned}$$

Sintaxis concreta: $((\neg(\text{True} \wedge \text{False})) \vee \text{True})$ **Sintaxis abstracta:** $\text{Or}(\text{Not}(\text{And}(\text{Boolean}(\text{True}), \text{Boolean}(\text{False}))), \text{Boolean}(\text{True}))$ **Evaluación:**

$$\begin{aligned}
& \text{Or}(\text{Not}(\text{And}(\text{Boolean}(\text{True}), \text{Boolean}(\text{False}))), \text{Boolean}(\text{True})) \\
& \rightarrow \text{Or}(\text{Not}(\text{Boolean}(\text{False})), \text{Boolean}(\text{True})) \\
& \rightarrow \text{Or}(\text{Boolean}(\text{True}), \text{Boolean}(\text{True})) \\
& \rightarrow \text{Boolean}(\text{True})
\end{aligned}$$

Equivalencia entre la semántica natural y la estructural

Hemos definido dos estilos complementarios de semántica operacional para el lenguaje MINIB. Ambos describen el proceso de evaluación de expresiones, aunque lo hacen desde enfoques distintos: mientras que la *semántica natural* relaciona directamente una expresión con su valor final, omitiendo los pasos intermedios, la *semántica estructural* detalla cómo una expresión evoluciona a través de reducciones sucesivas, paso a paso [19].

A pesar de estas diferencias de granularidad, ambas semánticas coinciden en el resultado final que producen. El estilo estructural permite analizar con precisión el orden de evaluación, los posibles errores en tiempo de ejecución y las interacciones entre subexpresiones, mientras que el estilo natural es útil para verificar la corrección global de programas. No obstante, es esencial establecer formalmente que, a pesar de estas diferencias metodológicas, ambas semánticas son *equivalentes* en términos del valor que asignan a una expresión bien formada. Para capturar esta equivalencia, introducimos la *cerradura reflexiva y transitiva* de la relación de reducción elemental $\rightarrow$, denotada por $\rightarrow^*$, que modela la evaluación completa en semántica de paso pequeño.

Definición 2.5.2: Cerradura transitiva y reflexiva de $\rightarrow$

Se denota la cerradura transitiva y reflexiva de $\rightarrow$ por $\rightarrow^*$ y se define con las siguientes dos reglas:

$$\frac{}{e \rightarrow^* e}$$

$$\frac{e_1 \rightarrow e_2 \quad e_2 \rightarrow^* e_3}{e_1 \rightarrow^* e_3}$$

[21]

La relación $\rightarrow^*$ permite comparar de forma directa el resultado de la evaluación paso a paso con la evaluación directa de la semántica natural. Este puente conceptual permite enunciar el teorema 2.5.1 de equivalencia semántica:

Teorema 2.5.1: Equivalencia entre semántica natural y estructural

Para cualquier expresión e de MINIB y cualquier estado final f , se cumple que:

$$e \Rightarrow f \quad \text{si y sólo si} \quad e \rightarrow^* f.$$

[10]

Este resultado establece que ambos estilos semánticos determinan el mismo valor final para una expresión. En otras palabras, si una expresión e se evalúa a un valor f mediante la semántica natural ($\Rightarrow$), entonces existe una secuencia finita de pasos de reducción ($\rightarrow$) que lleva desde e hasta f ; y recíprocamente, si $e \rightarrow^* f$, entonces $e \Rightarrow f$.

La demostración de este teorema se estructura típicamente en dos direcciones:

- Para la implicación directa ($e \Rightarrow f \longrightarrow e \rightarrow^* f$), se procede por inducción estructural sobre la derivación natural, mostrando que cada evaluación completa puede descomponerse en una secuencia de reducciones elementales.
- Para la implicación inversa ($e \rightarrow^* f \longrightarrow e \Rightarrow f$), se usa inducción sobre la longitud de la cadena de reducciones, reconstruyendo la derivación natural a partir del valor final alcanzado.

Este tipo de resultados ha sido ampliamente estudiado en la literatura de semánticas formales [19, 21], y garantiza la coherencia interna del marco teórico: elegir un estilo u otro depende de los objetivos específicos del análisis. En el caso de MINIB, esta equivalencia nos permite utilizar indistintamente ambos estilos según convenga para la verificación, ejecución o implementación del lenguaje.

Determinismo

Una de las ventajas fundamentales de contar con una semántica formal para un lenguaje de programación es que permite demostrar propiedades fundamentales de su comportamiento.

to. Entre ellas, el *determinismo* ocupa un lugar central. Esta propiedad establece que, dada una expresión e , si existen dos posibles reducciones en un paso $e \rightarrow e_1$ y $e \rightarrow e_2$, entonces necesariamente $e_1 = e_2$ [24]. En otras palabras, no existe ambigüedad en la forma de evaluar una expresión: su evolución está completamente determinada por su estructura sintáctica actual. Este tipo de garantías es especialmente útil cuando se utiliza la semántica estructural, que detalla explícitamente cada paso de evaluación [19].

Proposición 2.5.1: Determinismo de MINIB

Si $e \rightarrow e_1$ y $e \rightarrow e_2$, entonces $e_1 = e_2$.

Esta propiedad puede demostrarse mediante *inducción estructural* sobre las reglas semánticas del lenguaje, asegurando que para cada forma de expresión, las reglas de evaluación prescriben como máximo una única transición posible. El resultado es una semántica donde cada entrada produce a lo sumo una salida, lo que facilita tanto el análisis como la implementación.

Además de ser determinista, el lenguaje MINIB también es *total*²⁰. En consecuencia, toda expresión del lenguaje termina su evaluación sin necesidad de imponer restricciones adicionales.

Esta característica es relevante porque permite trabajar con una noción de corrección total: cualquier transformación que preserve la semántica lo hace sobre expresiones cuya evaluación siempre converge. No obstante, esta situación contrasta con la de otros lenguajes abordados en este trabajo, en los que la terminación no está garantizada. En tales casos, se adopta un compromiso pragmático habitual en la literatura: asumir la terminación de la evaluación de expresiones para el *razonamiento formal*. Este enfoque permite razonar bajo una noción de *corrección parcial*, aplicable únicamente a aquellas entradas cuya evaluación concluye. Aunque esta restricción puede parecer fuerte, es una práctica común y razonable cuando el objetivo es preservar el significado semántico de los programas mediante transformaciones sistemáticas [1].

Cabe destacar que en lenguajes con efectos secundarios (como asignación mutable o concurrencia) el determinismo puede perderse, y con ello también se complica el estudio de la terminación. En estos contextos, propiedades como la *confluencia* o la *propiedad diamante* ofrecen garantías más débiles, pero suficientes para ciertos razonamientos [19].

Por el contrario, en *lenguajes puros*²¹, deterministas y totales como MINIB, la unicidad del camino de evaluación y la garantía de terminación nos permiten concentrarnos directamente en el objetivo central de este trabajo: asegurar que las transformaciones y análisis sobre los

²⁰Un lenguaje total es aquel en el que todas las funciones están definidas para cualquier posible entrada, es decir, no existen expresiones cuyo resultado sea indefinido o que puedan provocar una evaluación no terminante. Formalmente, un lenguaje es total si para toda expresión e , su evaluación siempre produce un resultado en un número finito de pasos [4].

²¹Un lenguaje *puro* es aquel en el que las computaciones no producen efectos secundarios: su única consecuencia es el resultado que devuelven. Esto implica la ausencia de operaciones como entrada/salida (I/O), asignaciones mutables o manipulación explícita del estado. En este modelo, los programas están compuestos únicamente por expresiones, y no por comandos, destacando el uso de funciones de orden superior y recursión. La evaluación de las expresiones se basa en reglas de reducción simbólica, lo que permite una semántica sencilla, homogénea y predecible. Ejemplos clásicos de lenguajes puramente funcionales son HASKELL y MIRANDA [22].

programas conserven su semántica, sin necesidad de excluir explícitamente los casos de no terminación.

2.5.2. Cálculo λ puro

El *Cálculo λ* fue introducido por Alonzo Church en los años treinta como parte de un ambicioso proyecto fundacional de las matemáticas, cuyo objetivo era proporcionar una base formal rigurosa para la lógica y la teoría de funciones [25]. Este enfoque pretendía establecer un sistema axiomático autosuficiente para formalizar todo el conocimiento matemático. Sin embargo, se demostró que la versión completa de dicho sistema era inconsistente, es decir, que dentro de él se podían derivar contradicciones lógicas, como lo evidenciaron Kleene y Rosser en 1936. Esta inconsistencia surgía al considerar el sistema en su totalidad, incluyendo componentes autorreferenciales.

A pesar de ello, el subconjunto del sistema enfocado exclusivamente en la manipulación de funciones resultó ser no solo consistente, sino también sorprendentemente expresivo desde el punto de vista computacional. Este núcleo, conocido hoy como *Cálculo λ puro*, se consolidó como un modelo fundamental para el estudio de las funciones computables [25]. De hecho, Kleene y Rosser demostraron que todas las funciones recursivas pueden representarse dentro de este sistema, mientras que Turing estableció la equivalencia entre las funciones representables en el *Cálculo λ* y aquellas computables por una máquina de Turing [25].

Así, el *Cálculo λ* se posiciona como un modelo alternativo, e igualmente robusto, al de la máquina de Turing, lo cual cimienta su relevancia en la teoría de la computación. Su carácter formal, basado exclusivamente en funciones, lo convierte además en el fundamento teórico de los lenguajes de programación funcionales. En particular, su versión no tipificada o “*type-free*” permite que cualquier expresión pueda aplicarse a cualquier otra, favoreciendo una flexibilidad sintáctica sin igual.

En este trabajo abordamos lenguajes de programación que permiten la declaración y evaluación de funciones anónimas, funciones recursivas y aplicaciones de función. Por ello, consideramos pertinente introducir el marco teórico del *Cálculo λ* como base formal que proporciona el contexto adecuado para comprender tanto los conceptos fundamentales de la programación funcional como las estrategias de paso de parámetros a funciones que exploraremos más adelante. Nos alineamos así con el enfoque propuesto por Bahr y Hutton [1], quienes trabajan con una variante del *Cálculo λ* con paso de parámetros por valor (*Call-by-Value*) y sugieren el estudio de otras estrategias de evaluación como *Call-by-Name* y *Call-by-Need*.

Formalmente, el *Cálculo λ puro* se basa en tres constructos fundamentales: variables, funciones anónimas (abstracciones λ) y las aplicaciones de función. La computación se describe mediante la transformación sucesiva de términos, siguiendo reglas de reducción que simplifican expresiones hasta alcanzar una forma normal. Debido a su simplicidad sintáctica y riqueza expresiva, este formalismo no solo ha servido como modelo teórico para caracterizar lo computable, sino que también constituye un pilar esencial en la semántica formal de los lenguajes de programación funcionales.

Sintaxis

El *Cálculo λ* se denomina como un *sistema formal*²² minimalista que modela funciones y su aplicación utilizando únicamente reglas sintácticas. Este sistema se fundamenta en una gramática elemental compuesta por variables, abstracciones y aplicaciones [25].

Definición 2.5.3: Variables

Las variables son elementos atómicos que denotan identificadores utilizados como argumentos o nombres dentro de funciones. Consideramos un conjunto infinito de variables, usualmente representadas por letras minúsculas del alfabeto:

$$a \mid b \mid c \mid \dots \mid x \mid y \mid z \mid \dots$$

[27]

Definición 2.5.4: Términos λ

La clase de expresiones del *Cálculo λ* , llamadas *términos λ* y denotadas por Λ , se define inductivamente como la menor clase que satisface:

1. Si x es una variable, entonces $x \in \Lambda$.
2. Si $M \in \Lambda$ y x es una variable, entonces $\lambda x. M \in \Lambda$.
3. Si $M, N \in \Lambda$, entonces $(MN) \in \Lambda$.

[27]

Estas reglas constituyen la base de la gramática del *Cálculo λ* , donde los términos pueden pensarse como expresiones, las abstracciones $\lambda x. M$ como funciones anónimas, y las aplicaciones MN como la ejecución de una función sobre un argumento. Como se aclara en [27], el término x por sí solo no tiene un significado definido, mientras que $(x \ y)$ representa la aplicación de x a y , sin que ambos estén necesariamente definidos. En cambio, la abstracción $\lambda x. x$ representa la función identidad, ya que devuelve su argumento sin modificarlo.

Algunos ejemplos de términos bien formados son:

- x : una variable.
- $(x \ y)$: aplicación de x a y .
- $(\lambda x. (x \ y))$: función que aplica su argumento a y .
- $(\lambda f. (f \ f))$: función que se aplica a sí misma.
- $(\lambda x. (\lambda y. (x \ y)))$: función que devuelve otra función.

²²El *Cálculo λ* es un sistema formal en el sentido de que está compuesto por un conjunto finito de símbolos, reglas sintácticas bien definidas para formar expresiones (términos), y un conjunto de reglas semánticas para manipularlas, particularmente las reglas de reducción [26].

Estas construcciones ilustran cómo, a partir de una sintaxis extremadamente concisa, es posible definir funciones, aplicar argumentos, e incluso expresar recursión y composición de funciones dentro del marco del *Cálculo λ* .

Abstracción y aplicación

Las dos construcciones fundamentales del *Cálculo λ* son la *abstracción* y la *aplicación*, representadas directamente en la segunda y tercera cláusula de la definición 2.5.4, respectivamente:

- La *abstracción*, denotada por expresiones de la forma $\lambda x.M$, permite la definición de funciones anónimas. En esta construcción, el término $\lambda x.M$ representa una función que toma como parámetro una variable x y retorna el término M , en el cual dicha variable puede aparecer libre o no.
- La *aplicación*, expresada como (MN) , describe el proceso de aplicar una función M a un argumento N . En el marco del *Cálculo λ* , tanto M como N son términos bien formados, lo que posibilita la composición funcional y la construcción de funciones de orden superior. La evaluación de una aplicación se basa en la sustitución del argumento en el cuerpo de la función, proceso que se formaliza en la sección 2.5.2.

Con fines expositivos, y para facilitar la comprensión inicial, utilizaremos en esta sección una notación extendida que incorpora constructos ajenos al *Cálculo λ* puro, tales como constantes numéricas y operadores aritméticos. Aunque estos elementos no forman parte del formalismo original, su inclusión constituye un recurso didáctico común en textos pedagógicos como [27], ya que permite establecer una conexión más directa con lenguajes funcionales reales como HASKELL, en los que expresiones como $((\backslash x \rightarrow x + 2)1)$ son válidas y operativas.

Además, como se detalla en la sección 2.5.7, esta notación puede interpretarse como una forma abreviada para expresar términos más complejos mediante macros, lo que mejora la legibilidad sin comprometer el rigor teórico. En particular, el uso de los numerales de Church —cuya definición formal se aborda en la sección 2.5.3— permite modelar números naturales y operaciones sobre ellos dentro del marco del *Cálculo λ* puro.

Por ejemplo, de manera intuitiva, una función que suma 2 a su argumento puede escribirse como:

$$\lambda x. x + 2$$

y su aplicación al número 1 se expresa como:

$$(\lambda x. x + 2) 1 = 3$$

Estas expresiones son útiles para transmitir la idea general de la abstracción funcional, aunque, en un desarrollo completamente formal, deberían formularse únicamente con las construcciones básicas del *Cálculo λ* . En ese contexto, tanto la función de suma como sus operandos deben representarse mediante funciones lambda anidadas, como ocurre con los numerales de Church y los combinadores correspondientes, lo que da lugar a expresiones más extensas y menos accesibles a simple vista.

Por ello, utilizamos esta notación extendida de forma limitada y con fines ilustrativos. A partir de los siguientes apartados retomaremos exclusivamente la sintaxis formal del *Cálculo* λ . Más adelante, en la sección 2.5.3, reintroduciremos esta notación una vez establecido el soporte teórico necesario, incluyendo los numerales de Church y otras definiciones auxiliares, con el fin de desarrollar ejemplos más cercanos a entornos de evaluación reales y a la práctica de lenguajes funcionales.

Forma *currificada*

Aunque el *Cálculo* λ define únicamente funciones de un solo argumento, es posible representar funciones de múltiples argumentos mediante una técnica conocida como *currificación*²³. Esta técnica consiste en construir funciones a través de una sucesión de abstracciones y aplicaciones unarias, de modo que una función de varios parámetros se represente como una secuencia de funciones que toman un argumento a la vez.

Por ejemplo, una función binaria puede representarse de la siguiente manera:

$$\lambda x. \lambda y. f \ x \ y$$

y aplicarse secuencialmente como:

$$((\lambda x. \lambda y. f \ x \ y) \ a) \ b$$

obteniéndose $f \ a \ b$ tras aplicar la β -reducción en dos ocasiones (véase la sección 2.5.2). Este enfoque no reduce el poder expresivo del sistema, ya que toda función de varios argumentos puede transformarse en una función que los recibe uno a uno [27].

En la práctica, esta forma funcional se ve favorecida por convenciones sintácticas ampliamente aceptadas que eliminan la necesidad de paréntesis innecesarios. De acuerdo con [28], se asume que la abstracción lambda asocia a la derecha:

$$\lambda x \ y \ z. M \equiv \lambda x. (\lambda y. (\lambda z. M))$$

y que la aplicación de funciones asocia a la izquierda:

$$F \ a \ b \equiv ((F \ a) \ b)$$

Gracias a estas convenciones, es posible expresar de forma compacta funciones de múltiples argumentos como:

$$(\lambda x_1 \ \dots \ x_n. f \ x_1 \ \dots \ x_n) \ x_1 \ \dots \ x_n = f \ x_1 \ \dots \ x_n$$

donde el lado izquierdo representa una secuencia de aplicaciones sucesivas, y el lado derecho corresponde a la notación tradicional para funciones de n argumentos.

Esta notación, que puede considerarse una versión más legible de la forma currificada, se alinea con la práctica habitual en lenguajes funcionales como HASKELL, donde toda función es, en esencia, una función unaria que retorna otra función cuando se requieren argumentos

²³Nombrada en honor al matemático Haskell B. Curry, aunque su formulación original fue propuesta por Moses Schönfinkel.

adicionales. Así, expresiones como $(\lambda x y \rightarrow f x y) a b$ en HASKELL constituyen una manifestación directa del principio de curificación.

Como se discutirá más adelante, este enfoque es esencial para modelar funciones de múltiples parámetros dentro de un sistema formal en el que, por construcción, todas las funciones son unarias.

Variables libres y ligadas

En el *Cálculo λ* , las variables que aparecen en una expresión pueden clasificarse en dos tipos fundamentales: *ligadas* y *libres*. Esta distinción es esencial para comprender el funcionamiento de la abstracción, la aplicación y la sustitución en este formalismo.

Una variable se considera *ligada* cuando aparece como *parámetro formal*²⁴ en una abstracción; es decir, cuando su ocurrencia se encuentra bajo el alcance de un operador λ [26]. Por ejemplo, en el término $\lambda x.xy$, la variable x es un parámetro formal ligado por el operador λ , mientras que y es libre, ya que no está asociada a ninguna abstracción en dicha expresión. Formalmente, se definen dos funciones inductivas sobre los términos lambda para determinar los conjuntos de variables ligadas y libres:

■ **Variables ligadas** (BV^{25}):

$$\begin{aligned} BV(x) &= \emptyset \\ BV(\lambda x.M) &= BV(M) \cup x \\ BV(M N) &= BV(M) \cup BV(N) \end{aligned}$$

■ **Variables libres** (FV^{26}):

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.M) &= FV(M) - \{x\} \\ FV(M N) &= FV(M) \cup FV(N) \end{aligned}$$

Un término se denomina *combinador* cuando no tiene variables libres, es decir, cuando $FV(M) = \emptyset$ [29].

Además, como parte de una práctica habitual (llamada *higiene de variables*) se suele asumir que los nombres de las variables ligadas son distintos a los de las variables libres en una expresión. Esto se puede lograr fácilmente mediante un renombramiento de variables ligadas, sin alterar el significado de la función. Por ejemplo, las siguientes expresiones son consideradas equivalentes:

$$(\lambda x.x) a = a = (\lambda y.y) a$$

Esta equivalencia se denomina α -*equivalencia*, y establece que dos términos lambda que solo difieren en los nombres de sus variables ligadas representan el mismo comportamiento [25, 28].

²⁴Un parámetro formal en una función lambda es una variable que actúa como un marcador genérico en la abstracción, sustituido por un valor concreto durante la aplicación. Por ejemplo, en $\lambda x.M$, x es el parámetro formal [20].

²⁵BV proviene del inglés *Bound Variables*.

²⁶FV proviene del inglés *Free Variables*.

Definición 2.5.5: α -equivalencia

Dos términos λ se consideran α -*equivalentes* si difieren solo en los nombres de sus variables ligadas. Es decir, $M \equiv_\alpha N$ si uno puede obtenerse del otro renombrando consistentemente las variables ligadas. Esta equivalencia refleja el hecho de que los nombres de las variables ligadas no afectan el comportamiento computacional del término.

Por ejemplo:

$$(\lambda x. x) z \equiv_\alpha (\lambda y. y) z$$

ya que solo se ha renombrado la variable ligada x por y . En cambio:

$$(\lambda x. x) z \not\equiv_\alpha (\lambda z. y) z$$

pues el cuerpo de la función ha cambiado su estructura y significado.

[29]

Sustitución

La *sustitución* es una operación fundamental en el *Cálculo λ* , denotada como $M[x := N]$, y se interpreta como: “en el término M , reemplazar todas las ocurrencias libres de la variable x por el término N ”. Esta operación debe tratarse con cuidado, ya que no se aplica sobre ocurrencias ligadas de la variable.

Por ejemplo, en la expresión $yx(\lambda x. x)$, al sustituir x por N se obtiene:

$$yx(\lambda x. x)[x := N] = yN(\lambda x. x)$$

Aquí, la variable x ligada por la abstracción λx permanece inalterada, ya que la sustitución solo afecta a las ocurrencias libres de x en el término original [25].

Un aspecto crítico a considerar es la *captura de variables*, que ocurre cuando una variable libre del término de sustitución queda, tras la sustitución, bajo el alcance de una abstracción, convirtiéndose en una variable ligada. Esto puede cambiar el significado del término resultante.

Considérese el siguiente ejemplo:

$$(\lambda f. f x)[x := f y] = \lambda f. f(f y)$$

En este caso, la variable f , que era libre en el término de sustitución $f y$, queda capturada por la abstracción λf , alterando el comportamiento esperado.

Para evitar esta situación, se recurre al *renombramiento de variables ligadas*, asegurando que los nombres de las variables ligadas no entren en conflicto con las variables libres del término de sustitución. Por ejemplo:

$$(\lambda g. g x)[x := f y] = \lambda g. g(f y)$$

Aquí se evita la captura porque la variable ligada ha sido renombrada previamente, preservando la semántica original [25].

Definición 2.5.6: Sustitución

Sean $M, N \in \Lambda$ y x, y, z variables. La operación de sustitución, denotada por $M[x := N]$, se define inductivamente de la siguiente manera:

$$\begin{aligned}
 x[x := N] &= N \\
 y[x := N] &= y \quad \text{si } x \neq y \\
 (\lambda x. P)[x := N] &= \lambda x. P \\
 (\lambda y. P)[x := N] &= \lambda y. (P[x := N]) \quad \text{si } x \notin FV(P) \text{ o } y \notin FV(N) \\
 (\lambda y. P)[x := N] &= \lambda z. (P[y := z])[x := N] \quad \text{si } x \in FV(P) \text{ y } y \in FV(N), \\
 &\quad z \text{ es una variable nueva} \\
 (PQ)[x := N] &= (P[x := N])(Q[x := N])
 \end{aligned}$$

[25, 28]

La operación de sustitución formal definida anteriormente puede parecer técnica a primera vista, pero cada una de sus reglas captura un caso particular del comportamiento esperado al reemplazar una variable por un término. A continuación, se explica la intuición detrás de cada una de estas reglas:

- $x[x := N] = N$
Este caso corresponde al objetivo directo de la sustitución: cuando el término es exactamente la variable que se desea reemplazar, se sustituye por el término dado N .
- $y[x := N] = y$ con $x \neq y$
Si el término es una variable distinta de x , no se ve afectado por la sustitución. No hay nada que modificar, ya que la variable objetivo no aparece.
- $(PQ)[x := N] = (P[x := N])(Q[x := N])$
En una aplicación, tanto el lado izquierdo (P) como el derecho (Q) pueden contener ocurrencias libres de x . Por lo tanto, la sustitución debe aplicarse recursivamente en ambos subtérminos.
- $(\lambda x. P)[x := N] = \lambda x. P$
Cuando una abstracción introduce a x como parámetro formal, las ocurrencias de x en el cuerpo P están ligadas por dicha abstracción. En este contexto, la sustitución no debe afectar a esas ocurrencias, ya que forman parte del alcance local de la definición.
- $(\lambda y. P)[x := N] = \lambda y. (P[x := N])$ con $x \neq y$, $x \notin FV(P)$ o $y \notin FV(N)$
Aquí, y es un parámetro formal distinto de x , y no hay riesgo de que la sustitución cause una captura de variables. Es seguro aplicar la sustitución dentro de P , sin necesidad de modificar el nombre del parámetro ligado.
- $(\lambda y. P)[x := N] = \lambda z. (P[y := z])[x := N]$ si $x \in FV(P)$ y $y \in FV(N)$, con z nueva variable

En este caso, sustituir directamente podría provocar la captura de la variable libre y del término N , ya que y es también el parámetro formal de la abstracción. Para evitar este problema, se realiza primero un renombramiento del parámetro y por una nueva variable z , obteniendo un término α -equivalente. Luego se aplica la sustitución con seguridad. Este paso es necesario únicamente cuando existe una posible colisión entre una variable ligada en P y una variable libre en N .

El último caso en la definición de sustitución se considera correcto siempre que se evite la captura de variables. Esta situación se previene eligiendo cuidadosamente los nombres de las variables ligadas, conforme a la convención de higiene de variables previamente mencionada.

Lema 2.5.1: Sustitución

Sean $M, N, L \in \Lambda$, y sean x, y variables con $x \neq y$ y $x \notin FV(L)$. Entonces se cumple:

$$M[x := N][y := L] \equiv_{\alpha} M[y := L][x := N[y := L]]$$

[25]

Este resultado, conocido como el *lema de sustitución*, garantiza la coherencia al aplicar sustituciones sucesivas. En particular, establece que es posible reordenar sustituciones, siempre que se respeten ciertas condiciones de no interferencia entre las variables. Este principio será fundamental en razonamientos posteriores sobre evaluación y transformación de términos.

β -reducción

En el *Cálculo λ* , las funciones se modelan como expresiones que toman un único argumento. Para aplicar una función a un valor, se debe alcanzar una forma específica: $(\lambda x. M) N$, donde una abstracción $\lambda x. M$ se aplica a un argumento N [29].

A este tipo de expresiones se les denomina *redex* (expresiones reducibles) y constituyen el punto de partida para llevar a cabo la β -reducción, una operación central en la dinámica de evaluación del *Cálculo λ* . Esta operación consiste en aplicar efectivamente una función a su argumento mediante la sustitución del parámetro formal x por el argumento N en el cuerpo M de la abstracción.

Definición 2.5.7: β -reducción

Una expresión del tipo $(\lambda x. M) N$ se reduce mediante β -reducción de la siguiente forma:

$$(\lambda x. M) N \xrightarrow{\beta} M[x := N]$$

donde $M[x := N]$ denota la sustitución de todas las ocurrencias libres de x en M por N . [29]

El objetivo de la β -reducción es simplificar una expresión hasta alcanzar, si es posible, una *forma normal*, es decir, una expresión en la que ya no existen más *redex* que puedan ser reducidos. A continuación, se muestran algunos ejemplos básicos:

$$\begin{aligned}
(\lambda x.x) y &\xrightarrow{\beta} y \\
(\lambda x.x) (\lambda x.x) &\xrightarrow{\beta} \lambda x.x \\
(\lambda x.x) (\lambda z.z) &\xrightarrow{\beta} \lambda z.z
\end{aligned}$$

Sin embargo, no todas las expresiones en el Cálculo λ admiten una forma normal. Existen términos que nunca dejan de reducirse, generando una secuencia infinita de pasos de reducción. A este fenómeno se le conoce como *divergencia*²⁷. En tales casos, la evaluación no finaliza y el término no puede ser interpretado como un valor [29].

Combinadores de punto fijo

En matemáticas, se dice que un elemento x es un *punto fijo* de una función f si satisface la igualdad:

$$x = f(x).$$

En el contexto del Cálculo λ , es posible construir, dentro del propio sistema, términos que reciben una función como argumento y devuelven otro término que actúa como su punto fijo. Estos términos reciben el nombre de *combinadores de punto fijo*. Dado un término F , un combinador de este tipo produce un término X tal que $F X = X$.

Este mecanismo permite definir funciones recursivas sin necesidad de referirse a ellas por nombre, lo que resulta especialmente útil en lenguajes puramente funcionales y en el marco teórico de los lenguajes formales, donde la autorreferencia explícita no está permitida.

Teorema 2.5.2: Teorema del punto fijo

1. Para todo término $F \in \Lambda$, existe un término $X \in \Lambda$ tal que $F X = X$.
2. Existen términos cerrados, denominados *combinadores de punto fijo*, que satisfacen:

$$\forall F \in \Lambda \quad F(\text{fix } F) = \text{fix } F.$$

[25]

Un ejemplo clásico de combinador de punto fijo es el *combinador* Y , definido de la siguiente manera:

Definición 2.5.8: Combinador de punto fijo Y

$$Y = \lambda f. (\lambda x. f(x x))(\lambda x. f(x x))$$

²⁷Un ejemplo clásico de divergencia es el combinador Ω , definido como $\Omega := (\lambda x.x x)(\lambda x.x x)$. Este término se reduce a sí mismo infinitamente:

$$\Omega \xrightarrow{\beta} (\lambda x.x x)(\lambda x.x x) = \Omega \xrightarrow{\beta} \Omega \xrightarrow{\beta} \dots$$

Por tanto, Ω no tiene forma normal [29].

Este término tiene la propiedad de que, para toda función F , el término YF es un punto fijo de F ; es decir, cumple que $F(YF) = YF$. Esta construcción, puramente sintáctica, demuestra que es posible expresar recursión sin nombres explícitos dentro del *Cálculo λ* [29].

El combinador Y resulta particularmente útil en contextos de evaluación *perezosa*, como el de HASKELL, donde los términos no se evalúan hasta que su valor es requerido. En estos entornos, la expresión YF puede evaluarse sin necesidad de expandir inmediatamente los términos autoreferentes, lo que permite que la recursión basada en Y produzca un valor definido. Este comportamiento se alinea con la semántica de evaluación *por demanda* o *lazy evaluation* característica de ciertos lenguajes funcionales, donde las aplicaciones no son evaluadas hasta que su resultado es estrictamente necesario.

En contraste, bajo un régimen de evaluación ansioso, la evaluación inmediata de los argumentos provoca una divergencia directa, ya que YF entra en un ciclo infinito de reducciones. Esto se debe a que en un entorno *Call-by-Value* —donde los argumentos deben evaluarse antes de ser aplicados— la autorreferencia directa impide alcanzar una forma normal, lo que genera una falla en la ejecución de la recursión. Tal comportamiento se observa en lenguajes como ISWIM, donde la evaluación estricta impide que el combinador Y funcione correctamente [27].

Para mitigar este problema, se introduce una variante del combinador adaptada al estilo *Call-by-Value*, conocida como el *combinador Z* , o combinador de punto fijo de Plotkin [19]:

Definición 2.5.9: Combinador de punto fijo Z

$$Z = \lambda f. (\lambda x. f (\lambda y. (x x) y)) (\lambda x. f (\lambda y. (x x) y))$$

El combinador Z evita la evaluación prematura del argumento autoreferente encapsulándolo en una función anónima, lo cual permite alcanzar un punto fijo sin incurrir en divergencia inmediata. Esta propiedad lo hace adecuado para regímenes de evaluación estricta, donde se requiere controlar cuidadosamente la expansión de los términos.

Ambos combinadores ilustran la notable capacidad expresiva del *Cálculo λ* , al permitir la definición de funciones recursivas sin necesidad de nombres. Esta propiedad resulta especialmente valiosa en entornos donde se busca preservar la pureza funcional y la anonimidad de los términos. En este sentido, los combinadores de punto fijo no solo representan un resultado teórico profundo, como establece el teorema 2.5.2, sino también una herramienta práctica para expresar recursión de manera controlada y coherente con los principios del paradigma funcional [25].

2.5.3. Ambientes de evaluación

Para comprender cómo se gestionan las variables y su significado durante la ejecución de un programa, es necesario introducir el concepto de *ambiente de evaluación*. Este constituye una herramienta esencial para describir la asociación entre nombres (identificadores) y los valores que estos representan en distintos momentos de la ejecución. Más aún, los ambientes permiten formalizar el *alcance* de una variable, es decir, las regiones del programa donde un identificador está correctamente definido.

Formalmente, un ambiente puede concebirse como una función parcial finita que asigna a

cada identificador un valor o una *referencia abstracta*, sin necesidad de involucrar directamente conceptos de memoria física ni estructuras como el *heap*. Esta abstracción, ampliamente utilizada en semánticas operacionales, permite modelar de forma precisa cómo un lenguaje interpreta las variables a lo largo del tiempo, conservando su estructura léxica y estática [19].

El concepto de ambiente de evaluación es especialmente relevante en la construcción de *máquinas abstractas* (véase la sección 2.6.1), como las máquinas SECD, CEK o Krivine, desarrolladas para interpretar el *Cálculo λ* de manera eficiente [28]. Estas máquinas surgieron como respuesta a los altos costos computacionales asociados a la sustitución repetida de variables en la evaluación directa de términos λ . De hecho, la operación de sustituir un término N por una variable x en todas sus ocurrencias dentro de un cuerpo M , como ocurre en la evaluación de $(\lambda x. M) N$, puede alcanzar una complejidad de orden $O(n^2)$ ²⁸. Esta cota hace referencia al tamaño del cuerpo M , ya que cada ocurrencia de x podría requerir duplicar una copia de N , y si M contiene n ocurrencias de x , el tamaño total del término resultante tras la sustitución puede crecer cuadráticamente. Esta observación motivó el uso de ambientes como estrategia de optimización, al permitir la evaluación mediante *referencias compartidas* en lugar de duplicación sintáctica.

Una operación esencial en este contexto es la función de *búsqueda* (*lookup*²⁹), la cual consulta el ambiente para determinar el valor (o ubicación) asociado a un identificador dado. Esta función modela el proceso mediante el cual el programa resuelve el significado de una variable durante su evaluación [32].

En lugar de realizar sustituciones explícitas, las máquinas abstractas gestionan los *bindings*³⁰ dinámicamente a través de un ambiente, implementado típicamente como una estructura tipo pila (*stack*), que almacena asociaciones entre identificadores y sus valores (o representaciones intermedias). Cada vez que se aplica una función se agrega un nuevo *binding* al ambiente. Posteriormente, cuando se necesita resolver el valor de una variable, se realiza una operación de búsqueda.

Esta estrategia evita la duplicación de código y mejora la eficiencia al reducir la evaluación a operaciones de búsqueda, que pueden alcanzar una complejidad promedio de $O(1)$ en ambientes implementados como tablas *hash*, o $O(n)$ en estructuras lineales, dependiendo del estilo de programación y de la estructura del ambiente utilizada [6].

Desde el punto de vista semántico, esta separación entre *ambiente* y *estado* es fundamental:

- El *ambiente* (*environment*) modela el alcance léxico de los identificadores, es decir, la asociación de nombres con valores o referencias que surge de la estructura estática del programa.
- El *estado* (*store*) modela el contenido mutable (si lo hubiera) asociado a dichas referencias, útil principalmente en lenguajes con efectos de estado.

²⁸La notación O (*Big O*) describe el comportamiento asintótico de una función, estimando la complejidad en el peor de los casos en términos del tiempo o espacio requerido [30].

²⁹La función *lookup* se encarga de localizar una clave (nombre de variable) dentro de una base de datos (ambiente), devolviendo el registro asociado si existe, o *error* en caso contrario [31].

³⁰Un *binding* es una asociación entre un nombre y un valor (o ubicación de memoria) en un ambiente. Este mecanismo es característico del alcance estático, donde las asociaciones se determinan en tiempo de compilación, permitiendo que las funciones retornen otras funciones y evitando la necesidad de reescritura del código fuente [20].

Esta distinción permite extender el modelo a lenguajes imperativos o funcionales con efectos secundarios, sin comprometer la claridad semántica de la evaluación.

En la evaluación de una aplicación de función del tipo $(\lambda x. M) N$, la interpretación con ambientes consiste en extender el ambiente actual con el *binding* $[x \leftarrow N]$ y evaluar M en este nuevo contexto. Este modelo admite dos formas de resolución de identificadores [7]:

- *Alcance estático (léxico)*: La búsqueda se basa en la estructura del programa, ignorando el flujo de ejecución. Este es el modelo usado en la mayoría de los lenguajes funcionales y permite razonar localmente sobre el significado de los identificadores.
- *Alcance dinámico*: La búsqueda se realiza desde el tope de la pila de ambientes, es decir, se toma el *binding* más reciente sin importar su posición estática, de esta forma, una misma variable puede adquirir diferentes valores dependiendo de la trayectoria de ejecución. Esta estrategia, aunque menos común, aparece en ciertos lenguajes de *scripting* o en modelos experimentales de evaluación.

El uso de ambientes en lugar de sustitución directa no solo mejora el rendimiento, sino que también proporciona una base sólida para la definición formal de semánticas operacionales. En nuestro trabajo, este enfoque es especialmente importante para la implementación y análisis de los lenguajes objeto de estudio, ya que tanto sus intérpretes como sus reglas de evaluación dependen directamente de esta estructura. La formalización precisa de los ambientes permite modelar la ejecución de programas de manera composicional, facilitando la demostración de propiedades como la corrección semántica y la equivalencia entre diferentes representaciones de programas.

Evaluación mediante ambientes

Retomando la notación introducida en la sección 2.5.2 por legibilidad, adoptamos el *Cálculo λ* como lenguaje base para formalizar la evaluación mediante ambientes.

Considérese el siguiente ejemplo:

$$(\lambda x. (\lambda f. (\lambda x. f \ 3) \ 10) (\lambda y. x + y)) \ 5 \quad (2.2)$$

La evaluación de este término implica los siguientes pasos con la gestión del ambiente mediante una pila:

1. Evaluamos la aplicación externa:

$$(\lambda x. M) \ 5, \quad \text{donde } M = (\lambda f. (\lambda x. f \ 3) \ 10) (\lambda y. x + y)$$

Se introduce el *binding* $[x \leftarrow 5]$.

Pila actual: $\left[\begin{array}{l} x \leftarrow 5 \end{array} \right]$

2. Evaluamos la aplicación:

$$(\lambda f. (\lambda x. f\ 3)\ 10)(\lambda y. x + y)$$

La abstracción $\lambda f. N$ se aplica a $f = \lambda y. x + y$, definida en el ambiente actual con $x = 5$.

$$\text{Pila actual: } \left[\begin{array}{l} f \leftarrow \lambda y. x + y \\ x \leftarrow 5 \end{array} \right]$$

3. Evaluamos el cuerpo:

$$(\lambda x. f\ 3)\ 10$$

Se introduce un nuevo binding $[x \leftarrow 10]$, que oculta temporalmente el valor anterior de x .

$$\text{Pila actual: } \left[\begin{array}{l} x \leftarrow 10 \\ f \leftarrow \lambda y. x + y \\ x \leftarrow 5 \end{array} \right]$$

4. Evaluamos la aplicación:

$$f\ 3$$

Se introduce el *binding* $[y \leftarrow 3]$ al aplicar $f = \lambda y. x + y$.

$$\text{Pila actual: } \left[\begin{array}{l} y \leftarrow 3 \\ x \leftarrow 10 \\ f \leftarrow \lambda y. x + y \\ x \leftarrow 5 \end{array} \right]$$

Evaluación bajo alcance estático

En esta política, las variables libres se resuelven en el ambiente en el que una función es definida, no donde es aplicada. En nuestro ejemplo, la función $f = \lambda y. x + y$ se define cuando el *binding* activo para x era $[x \leftarrow 5]$, correspondiente a la primera aplicación de función.

Al aplicar f a 3, se introduce el *binding* $[y \leftarrow 3]$, pero la variable x sigue resolviéndose en su ambiente léxico original, es decir, $x = 5$. El *binding* $[x \leftarrow 10]$, introducido en un paso posterior, no afecta esta evaluación.

Por tanto, la expresión se evalúa como:

$$x + y = 5 + 3 = \boxed{8}$$

Alcance dinámico

Bajo esta política, las variables libres se resuelven en el ambiente de llamada, es decir, según el estado actual de la pila en el momento de la aplicación. En nuestro ejemplo, cuando se aplica $f = \lambda y. x + y$ a 3, el *binding* activo más reciente para x es $[x \leftarrow 10]$, introducido

por la aplicación $(\lambda x. f\ 3)\ 10$.

Así, al evaluar el cuerpo de f , el intérprete encuentra:

$$x = 10, \quad y = 3$$

lo que produce el resultado:

$$x + y = 10 + 3 = \boxed{13}$$

La implementación de la evaluación mediante ambientes, estructurados como una pila de *bindings*, permite manejar con claridad los distintos contextos de variable, evitando sustituciones repetitivas y costosas. Esta técnica resulta fundamental para diferenciar entre políticas de alcance léxico y dinámico, ya que hace explícito el ambiente en el que se resuelven las variables.

El ejemplo desarrollado muestra cómo el orden de introducción de los *bindings* y el momento de evaluación afectan el resultado final. Bajo alcance estático, se respeta el ambiente en el que las funciones fueron definidas originalmente, mientras que en alcance dinámico se utiliza el ambiente en el momento de ejecución.

Comprender esta diferencia es esencial tanto para diseñar lenguajes como para construir intérpretes.

2.5.4. Cerraduras de función

Desde la perspectiva del *Cálculo λ* , las funciones no sólo pueden recibir argumentos, sino que también pueden devolver otras funciones como resultado. Esta característica las clasifica dentro de la *taxonomía de orden superior*, lo que significa que las funciones son tratadas como valores de primera clase: pueden ser pasadas como parámetros, almacenadas en estructuras de datos y, en particular, retornadas por otras funciones [33].

Este comportamiento introduce una cuestión esencial en la semántica de los lenguajes funcionales: ¿qué significa que una función sea un valor?, ¿qué información debe contener una función para poder ser aplicada correctamente en contextos donde fue separada de su ambiente original?

Sabemos que para aplicar una función correctamente se requiere conocer tanto su parámetro formal como su cuerpo. Sin embargo, si el cuerpo contiene referencias a variables libres, esto es, variables no definidas localmente dentro de la función, entonces se necesita algo más: el *ambiente léxico* en el que fue definida la función. Aquí es donde surge la noción de *cerradura de función* o *clausura*.

Definición 2.5.10: Cerradura de función

Una cerradura o *closure* es una tupla (p, c, ε) , donde:

- p es el parámetro formal de la función,
- c es el cuerpo de la abstracción, y
- ε es el *ambiente de evaluación* en el momento en que la función fue creada.

[33]

Este mecanismo es fundamental para que las funciones de orden superior conserven acceso a las variables libres del ambiente donde fueron definidas, aún cuando se invoquen en un contexto diferente. En otras palabras, las cerraduras permiten que una función “recuerde” el ambiente léxico de su definición, garantizando que la evaluación posterior de su cuerpo respete el alcance estático del lenguaje [34].

Construcción de cerraduras

Consideremos el término 2.2 y describamos el proceso de construcción de clausuras asociado a su evaluación.

- **Paso 1:** Término $\lambda x.(\lambda f.(\lambda x.f\ 3)\ 10)(\lambda y.x + y)$
Se crea la cerradura inicial, sin ambiente externo:

$$C_1 = (x, (\lambda f.(\lambda x.f\ 3)\ 10)(\lambda y.x + y), [])$$

- **Paso 2:** Aplicar C_1 al valor 5.
Extendemos el ambiente:

$$\varepsilon_1 = [x \leftarrow 5]$$

Evaluamos el cuerpo con ese ambiente:

$$(\lambda f.(\lambda x.f\ 3)\ 10)(\lambda y.x + y)$$

- **Paso 3:** Creamos la cerradura para $\lambda y.x + y$
Esta función hace referencia libre a x , que está en ε_1 . Por lo tanto:

$$C_2 = (y, x + y, [x \leftarrow 5])$$

- **Paso 4:** Aplicar $\lambda f.(\lambda x.f\ 3)\ 10$ a C_2
Se genera la cerradura:

$$C_3 = (f, (\lambda x.f\ 3)\ 10, [f \leftarrow C_2])$$

- **Paso 5:** Evaluar $(\lambda x.f\ 3)\ 10$ en $\varepsilon_2 = [f \leftarrow C_2]$
Creamos la cerradura:

$$C_4 = (x, f\ 3, [f \leftarrow C_2])$$

Aplicamos C_4 al valor 10 en $\varepsilon_3 = [x \leftarrow 10, f \leftarrow C_2]$

- **Paso 6:** Evaluar $f\ 3$ en ε_3

Recordamos que $f = C_2 = (y, x + y, [x \leftarrow 5])$, evaluamos $f\ 3$ como aplicar la función C_2 a 3:

Extendemos el ambiente de C_2 :

$$\varepsilon_4 = [y \leftarrow 3, x \leftarrow 5]$$

Evaluamos el cuerpo $x + y$ en ε_4 :

$$x + y = 5 + 3 = \boxed{8}$$

La construcción de cerraduras hace evidente cómo el cálculo con ambientes logra preservar el alcance estático de las variables. En el ejemplo anterior, observamos que la variable libre x dentro de la función $\lambda y.x + y$ no es resuelta dinámicamente al valor de x más reciente en la pila de llamadas, sino que se conserva desde el ambiente en el cual la función fue definida, es decir, aquel en el que $x = 5$. Esto es posible porque la cerradura captura no sólo el cuerpo de la función, sino también el ambiente activo en ese momento, asegurando que cualquier referencia a variables externas se mantenga anclada a su contexto léxico original. De esta manera, el sistema de evaluación con ambientes refleja el comportamiento del alcance estático, haciendo explícita la separación entre el momento de definición y el momento de ejecución de una función.

2.5.5. Regímenes de evaluación

En los lenguajes de programación funcionales, el *régimen de evaluación* define cuándo y cómo se computan los valores de las expresiones. Este conjunto de reglas determina si las subexpresiones, como los argumentos de una función, deben evaluarse antes de aplicar la función, o únicamente cuando su valor sea necesario. Esta decisión impacta tanto en el comportamiento del lenguaje como en la eficiencia de ejecución de los programas.

Existen dos regímenes principales de evaluación: la evaluación *ansiosa* (*eager*) y la evaluación *perezosa* (*lazy*). En el régimen ansioso, las expresiones se evalúan tan pronto como se vinculan a una variable. Es decir, los argumentos de una función se computan antes de que la función sea aplicada. Este enfoque es predecible y eficiente en ciertos contextos, pero puede llevar a la evaluación innecesaria de valores que nunca serán utilizados [3].

Por otro lado, la evaluación perezosa difiere en posponer la evaluación de las expresiones hasta que su resultado sea estrictamente necesario. Este régimen se puede implementar mediante la estrategia de paso de parámetros conocida como *Call-by-Name*, la cual tiene como alternativa optimizada el enfoque de *Call-by-Need*. En esta variante, cada expresión se evalúa como máximo una vez, gracias a técnicas de *compartición* de resultados. Esta estrategia posibilita, entre otras cosas, la construcción de estructuras infinitas y la definición de funciones potencialmente no terminantes, sin que ello implique una evaluación inmediata.

La elección entre un régimen ansioso o perezoso no solo afecta el orden de ejecución, sino que también influye en la semántica del lenguaje, la forma en que se definen las funciones recursivas y la gestión de los recursos computacionales [3]. Como se señala en la literatura, los

lenguajes con evaluación ansiosa, como STANDARD ML, tienen un comportamiento predecible y eficiente para muchas tareas, mientras que los lenguajes con evaluación perezosa, como HASKELL, permiten una mayor flexibilidad y eficiencia en tareas que involucran estructuras grandes o infinitas [20].

La adopción de uno u otro régimen de evaluación también impacta en la forma de tratar definiciones recursivas. En el caso de la evaluación ansiosa, las definiciones recursivas pueden causar problemas de evaluación inmediata, mientras que en el régimen perezoso, la recursión se maneja de manera más controlada y eficiente, utilizando evaluaciones diferidas.

Evaluación ansiosa

El régimen de evaluación ansioso se caracteriza por evaluar los argumentos de una función antes de que esta sea aplicada, sin importar si dichos argumentos se utilizarán realmente en el cuerpo de la función. Este comportamiento está vinculado con la estrategia de paso de parámetros por valor *Call-by-Value*, en la que los parámetros se evalúan estrictamente antes de ser pasados a la función [19]. En este contexto, el paso de parámetros no depende de si los argumentos serán utilizados en la computación del cuerpo de la función, lo que hace que este régimen sea considerado *estricto*.

Formalmente, bajo la evaluación ansiosa, cuando se aplica una función de la forma $(\lambda x.M) N$, el argumento N se evalúa completamente a un valor V , antes de que se realice la sustitución de x por V en el cuerpo M de la función:

$$(\lambda x.M) N \xrightarrow{\text{CBV}} M[x := V] \quad \text{donde } N \text{ se evalúa a } V$$

Es importante destacar que, aunque el argumento N pueda no ser utilizado dentro del cuerpo de la función, su evaluación se realiza necesariamente.

Como ejemplo de este régimen, considere el término:

$$(\lambda x.5) ((\lambda y.y) (\lambda z.z))$$

Bajo la estrategia de paso de parámetros *Call-by-Value*, la subexpresión $((\lambda y.y) (\lambda z.z))$ se evalúa primero a $\lambda z.z$, y luego se aplica la función externa, que simplemente retorna 5, sin hacer uso del argumento x :

$$(\lambda x.5) ((\lambda y.y) (\lambda z.z)) \xrightarrow{\text{CBV}} (\lambda x.5) (\lambda z.z) \xrightarrow{\text{CBV}} 5$$

Este régimen de evaluación es ampliamente adoptado debido a su previsibilidad y eficiencia, especialmente en entornos donde los efectos secundarios deben ser controlados y se busca optimizar el uso de recursos de ejecución [35].

Es relevante diferenciar esta estrategia de paso de parámetros asociada al régimen de evaluación ansiosa de otras estrategias, como el *Call-by-Name*, que corresponde a una evaluación perezosa o no estricta. En este último, los argumentos de la función no se evalúan hasta que realmente se necesiten en el cuerpo de la función, lo que introduce una estrategia de evaluación más flexible y menos exigente en cuanto a los recursos [3].

Evaluación perezosa

El régimen de evaluación perezoso agrupa un conjunto de estrategias que, a diferencia de la evaluación ansiosa, posponen la evaluación de una expresión hasta que su valor resulta estrictamente necesario. Este enfoque, también conocido como *evaluación no estricta* (*non-strict*), permite suspender la computación de argumentos y expresiones, lo que aporta una mayor flexibilidad semántica y eficiencia en contextos donde ciertos resultados pueden no ser requeridos en absoluto [34].

Las estrategias más representativas de este régimen son *Call-by-Name* y *Call-by-Need*. En *Call-by-Name*, los argumentos se sustituyen sin evaluarse en el cuerpo de la función, provocando que una misma expresión pueda evaluarse múltiples veces si es referenciada repetidamente. Para mitigar esta redundancia computacional, la estrategia *Call-by-Need* introduce la compartición de resultados: una expresión se evalúa como máximo una vez, y su resultado se reutiliza en las sucesivas referencias [34, 35].

Dicha compartición se implementa mediante estructuras conocidas como *thunks*, que encapsulan una computación suspendida y permiten diferir su evaluación. Cuando un *thunk* es forzado, es decir, cuando se requiere su valor, este se evalúa y se actualiza con el resultado, garantizando que futuras referencias simplemente reutilicen dicho valor, evitando así recomputaciones innecesarias [34]. La implementación y comportamiento de los *thunks* se aborda con mayor detalle en la sección 2.5.5.

La evaluación perezosa no se limita al paso de argumentos en funciones, sino que también se extiende al tratamiento de estructuras de datos. Sus componentes pueden representarse como *thunks*, permitiendo construir estructuras potencialmente infinitas o parcialmente evaluadas. Este enfoque resulta especialmente eficaz en el manejo de definiciones recursivas generales, ya que pospone las llamadas recursivas hasta que su evaluación sea indispensable, lo cual habilita una semántica operacional coherente para expresiones autoreferenciales [36].

Además, la evaluación perezosa está estrechamente relacionada con el segundo teorema de Church-Rosser, el cual establece que, si una forma normal existe, puede ser alcanzada mediante estrategias no estrictas. En este sentido, evitar la evaluación prematura de los argumentos permite conservar la posibilidad de llegar a dicha forma normal, en concordancia con el comportamiento perezoso [37].

Evaluación bajo *Call-by-Name*

Analicemos paso a paso cómo la evaluación con ambientes y la estrategia de paso *Call-by-Name*, a partir del siguiente término:

$$\lambda x.(\lambda y.(\lambda x.y) (3 + 3)) (2 + x)) (2 + 2) \quad (2.3)$$

■ Paso 1: Construcción del ambiente

Comenzamos la evaluación aplicando el término externo. Como estamos bajo *Call-by-Name*, no evaluamos los argumentos, sino que almacenamos la asociación como una entrada en el ambiente:

$$[x \leftarrow (2 + 2)]$$

Ahora evaluamos el cuerpo:

$$(\lambda y.(\lambda x.y) (3 + 3)) (2 + x)$$

Agregamos una nueva entrada al ambiente (de nuevo, sin evaluar):

$$\left[\begin{array}{l} y \leftarrow (2 + x) \\ x \leftarrow (2 + 2) \end{array} \right]$$

Ahora llegamos a la aplicación:

$$(\lambda x.y) (3 + 3)$$

Añadimos otra entrada al ambiente:

$$\left[\begin{array}{l} x \leftarrow (3 + 3) \\ y \leftarrow (2 + x) \\ x \leftarrow (2 + 2) \end{array} \right]$$

■ Paso 2: Evaluación del cuerpo de la abstracción más interna

El cuerpo de la abstracción más interna es simplemente el identificador y , por lo que buscamos su valor en el ambiente. Encontramos:

$$[y \leftarrow (2 + x)]$$

Sustituimos entonces y por $(2 + x)$, y evaluamos esta nueva expresión dentro del ambiente actual.

Paso 3: Evaluación de la expresión $(2 + x)$

Para evaluar esta expresión, necesitamos el valor de x . Según el ambiente, el *binding* más reciente es:

$$[x \leftarrow (3 + 3)]$$

Por lo tanto, sustituimos:

$$2 + x \Rightarrow 2 + (3 + 3) = \boxed{8}$$

Este ejemplo ilustra claramente cómo, al evaluar funciones dentro de un ambiente usando la estrategia de paso de parámetros *Call-by-Name* junto con un único ambiente de evaluación global, se introduce de manera implícita un *alcance dinámico*. En otras palabras, las expresiones son evaluadas en el ambiente activo en el momento de su uso, no necesariamente en el ambiente léxicamente correspondiente a su definición. Esto se evidencia en el uso de la variable x dentro de la expresión ligada a y , la cual no se resuelve conforme al ambiente original donde y fue definida, sino respecto al *binding* más reciente de x , que corresponde al

cuerpo más interno.

Este fenómeno es una consecuencia directa de no capturar ni preservar el ambiente léxico donde cada expresión fue definida, lo que resulta en un comportamiento contrario al deseado en lenguajes con alcance estático.

Thunks

En el régimen de evaluación perezoso, los *thunks* constituyen el mecanismo operativo esencial que permite retrasar la evaluación de una expresión sin perder el contexto léxico en el que ésta fue definida. Específicamente, un *thunk* encapsula una computación suspendida junto con el ambiente léxico correspondiente, garantizando que cuando finalmente se evalúe (esto es, cuando se *fuera* su ejecución), dicha evaluación se produzca de forma correcta conforme al alcance estático del lenguaje.

Históricamente, el término *thunk* se originó en el contexto del paso de parámetros por nombre en ALGOL 60, cuando un grupo de trabajo observó que la evaluación diferida podía entenderse como una expresión que ya había sido “pensada” (“*think about*”) durante la compilación, aunque su cálculo se pospusiera para el momento de ejecución [31]. Esta idea fue formalizada posteriormente por Landin, quien aplicó evaluación diferida a la implementación de flujos, y más adelante por Friedman y Wise, quienes integraron esta técnica en LISP, haciendo que operadores como *cons* difirieran automáticamente la evaluación de sus argumentos, lo que dio lugar a listas con comportamiento perezoso por defecto.

La implementación de *thunks* es fundamental para estrategias como *Call-by-Name* y su optimización *Call-by-Need*. Mientras que *Call-by-Name* simplemente sustituye los argumentos sin evaluarlos, lo cual puede inducir evaluaciones redundantes cuando una misma expresión se reutiliza, *Call-by-Need* utiliza *thunks* con memoización para evaluar una expresión solo una vez, almacenando el resultado para usos futuros [34]. Esta optimización reduce drásticamente la duplicación de trabajo, incrementando la eficiencia computacional sin comprometer la semántica del programa.

El proceso de forzar un *thunk* ocurre únicamente cuando su valor es estrictamente necesario: por ejemplo, al utilizarse como argumento de una operación primitiva, en condiciones booleanas o como operador en una aplicación de función [31]. Esta evaluación controlada requiere que el *thunk* contenga toda la información necesaria para reconstruir la computación original: la expresión diferida y su entorno de evaluación. De este modo, se evita que la evaluación ocurra en contextos inapropiados o más recientes, como podría suceder en una implementación puramente dinámica.

Diversos modelos han sido propuestos para manejar el forzado y la actualización de *thunks*, destacando dos enfoques principales. El modelo de celda (*cell model*) introduce un indicador de estado en cada *closure*; al forzar el *thunk*, si no ha sido evaluado, se computa su valor y se almacena junto con una marca de evaluación, permitiendo reutilizar el resultado en futuras invocaciones. Por su parte, el modelo autoactualizable (*self-updating model*), adoptado por la máquina STG, transfiere la lógica de actualización al código del propio *thunk*, eliminando verificaciones externas y posibilitando optimizaciones más agresivas cuando se sabe que solo se evaluará una vez [34].

Aunque conceptualmente un *thunk* puede verse como una función sin parámetros que encapsula una computación suspendida, esta representación resulta ineficiente sin mecanismos

de memoización, ya que implica repetir el cálculo en cada invocación [34].

Definición 2.5.11: *Thunk*

Un *thunk* se define formalmente como una tupla (e, ε) , donde:

- e denota la expresión cuya evaluación se difiere, y
- ε representa el ambiente léxico capturado en la construcción del *thunk*.

[31]

Evaluación bajo *Call-by-Name* y *thunks*

Consideremos el término 2.3 para su evaluación:

▪ **Paso 1: Construcción del ambiente con *thunk***

Aplicamos la función más externa, asociando el argumento $(2 + 2)$ como *thunk* en el ambiente:

$$[x \leftarrow ((2 + 2), \varepsilon_0)]$$

donde ε_0 es el ambiente vacío inicial. Procedemos con el cuerpo de la aplicación:

$$(\lambda y. (\lambda x. y) (3 + 3)) (2 + x)$$

En esta expresión, al aplicar la función con parámetro y , diferimos $(2 + x)$, pero capturamos el ambiente actual ε_1 que ya contiene $[x \leftarrow ((2 + 2), \varepsilon_0)]$:

$$\left[\begin{array}{l} y \leftarrow ((2 + x), \varepsilon_1) \\ x \leftarrow ((2 + 2), \varepsilon_0) \end{array} \right]$$

Pasamos ahora a la siguiente aplicación:

$$(\lambda x. y) (3 + 3)$$

Una vez más, el argumento se encapsula como *thunk* junto con el ambiente actual ε_2 :

$$\left[\begin{array}{l} x \leftarrow ((3 + 3), \varepsilon_2) \\ y \leftarrow ((2 + x), \varepsilon_1) \\ x \leftarrow ((2 + 2), \varepsilon_0) \end{array} \right]$$

▪ **Paso 2: Evaluación del cuerpo de la abstracción más interna**

El cuerpo de la función más interna es simplemente el identificador y , por lo que buscamos su valor en el ambiente:

$$[y \leftarrow ((2 + x), \varepsilon_1)]$$

Forzamos este *thunk* evaluando la expresión $2 + x$ en el ambiente ε_1 , que nos lleva a buscar la definición de x en dicho ambiente:

$$[x \leftarrow ((2 + 2), \varepsilon_0)]$$

Por lo tanto, para evaluar $2 + x$, forzamos el *thunk* asociado a x , que da:

$$x = (2 + 2) = 4$$

Sustituimos y evaluamos:

$$2 + x = 2 + 4 = \boxed{6}$$

La evaluación realizada mediante *thunks* permite preservar el alcance léxico incluso bajo un régimen de evaluación perezoso. Esta técnica garantiza que cada expresión se evalúe dentro del ambiente donde fue originalmente asociada, asegurando así una correcta interpretación estática del contexto de las variables libres [35].

Sin embargo, a diferencia de un modelo estrictamente evaluado, esta técnica introduce una dinámica distinta: los valores no se computan inmediatamente, sino que su cálculo se difiere hasta que sea estrictamente necesario. En este sentido, un *thunk* representa una computación suspendida, junto con el ambiente léxico donde dicha computación fue pospuesta. Su evaluación se desencadena únicamente cuando su resultado es requerido de manera estricta, lo cual ocurre en lo que se conoce como un *punto estricto*; es decir, una posición del programa donde el valor de una expresión debe conocerse de inmediato (por ejemplo, como argumento de una operación primitiva, una condición booleana o una aplicación de función). El proceso de activar la evaluación de un *thunk* se formaliza mediante la relación auxiliar *force*, la cual fuerza su evaluación usando el ambiente capturado en el momento de su creación.

Aunque conceptualmente un *thunk* puede representarse como una función sin parámetros que encapsula una computación diferida, esta representación no resulta eficiente en ausencia de mecanismos de memoización, ya que conduciría a evaluaciones redundantes cada vez que se requiera su valor [34].

Closures y thunks: similitudes y diferencias. Aunque tanto los *closures* como los *thunks* permiten capturar y preservar el ambiente léxico, es decir, el contexto de evaluación estática, su propósito operativo dentro de los lenguajes de programación es diferente. Los *closures* representan funciones junto con el ambiente en el que fueron definidas, y son esenciales para implementar correctamente funciones de orden superior en cualquier régimen de evaluación, ya sea estricto o perezoso. En cambio, los *thunks* son estructuras propias del régimen de evaluación perezoso, cuya finalidad es posponer la evaluación de una expresión (no necesariamente una función), junto con el ambiente en el que esta fue suspendida. Esta

distinción más detallada y ajustada entre *thunks* y *closures* se expone formalmente en la sección 4.1.

2.5.6. Puntos estrictos

Como hemos mencionado, en un lenguaje con evaluación perezosa, una expresión no se computa de inmediato, sino hasta que su valor es indispensable para continuar la ejecución del programa. El lugar donde ocurre esta necesidad ineludible se conoce como *punto estricto*.

Los puntos estrictos son fundamentales para el avance de la evaluación: si no se computa el valor requerido en estas posiciones, el sistema no puede realizar nuevas transiciones en la evaluación, lo que provoca el estancamiento del cómputo. Es decir, los puntos estrictos marcan los lugares donde forzosamente deben resolverse los *thunks* para que la ejecución pueda progresar.

Típicamente, estos puntos se manifiestan en operadores primitivos que requieren los valores concretos de sus argumentos; por ejemplo, en una operación aritmética como la suma, ambos operandos deben evaluarse antes de efectuar el cálculo. En contraste, otras construcciones (como la definición de funciones o la creación de estructuras de datos) no son estrictas, ya que sus componentes pueden diferirse sin afectar la semántica del programa [31]. En este sentido, el diferimiento se articula mediante los *thunks*: las expresiones pendientes de evaluación permanecen suspendidas hasta que se alcanza un punto estricto. Esta capacidad de postergar el cómputo sin perder el contexto léxico original es central para la eficiencia, la modularidad y la expresividad de lenguajes funcionales no estrictos como HASKELL [35].

Formalización funcional de *delay* y *force*

Para modelar formalmente la evaluación perezosa, introducimos dos operaciones fundamentales: *delay*, que permite encapsular un cómputo, y *force*, que permite forzar su evaluación bajo demanda. Estas operaciones se construyen sobre la noción de *thunk*, estructura que representa una expresión pendiente de evaluación junto con el ambiente léxico en el que fue creada [31].

Antes de formalizar estas operaciones, es importante precisar qué entendemos por *valor*. En este contexto, un *valor* es una entidad que no requiere evaluación adicional: puede tratarse de constantes, cerraduras de función o estructuras de datos completamente evaluadas. Por el contrario, una expresión que aún puede ser evaluada, por ejemplo, una aplicación o una operación aritmética, no se considera un valor.

Siguiendo este criterio, definimos las operaciones de evaluación diferida y forzada de la siguiente manera:

- **Evaluación diferida (*delay*):** encapsula una expresión junto con el ambiente en el que fue escrita, construyendo un *thunk*. Formalmente, dado un término e y un ambiente ε , definimos:

$$\text{delay} : (\text{Expr} \times \text{Env}) \rightarrow \text{Thunk}, \quad \text{delay}(e, \varepsilon) = (e, \varepsilon)$$

- **Forzado de evaluación (*force*):** desencapsula el *thunk* y evalúa su expresión en el ambiente capturado. El resultado de esta evaluación puede ser un valor o bien otro

think; en este último caso, se continúa forzando recursivamente hasta obtener un valor en sentido estricto:

$$force : Thunk \rightarrow Value, \quad force((e, \varepsilon)) = \begin{cases} v, & \text{si } e \Rightarrow_{\varepsilon} v \text{ y } v \notin Thunk \\ force(v), & \text{si } e \Rightarrow_{\varepsilon} v \in Thunk \end{cases}$$

Una implementación práctica y común de estas operaciones consiste en representar *delay* como una abstracción lambda sin argumentos, es decir, convertir una expresión en una función que puede ser invocada más adelante. En este enfoque, *force* simplemente aplica dicha función para obtener su resultado [31]. Esta técnica es ampliamente utilizada por su simplicidad y porque refleja directamente el principio de suspender la evaluación. Recordando que, cuando los *thunks* se reutilizan múltiples veces, por ejemplo, en estructuras como flujos infinitos, es común aplicar estrategias de memoización para evitar reevaluaciones [38].

Adicionalmente, formulaciones más abstractas de estas operaciones, como las propuestas por Asperti y Curien, utilizan herramientas categóricas como *co-mónadas*³¹ para describir los comportamientos de *delay* y *force* de manera más estructurada [38].

Definición 2.5.12: Punto estricto

Un *punto estricto* en un lenguaje de programación perezoso es una posición sintáctica dentro de una expresión donde la semántica del lenguaje exige la evaluación completa de una subexpresión a su valor (si este existe) antes de continuar con la computación global. En estos puntos, la evaluación perezosa se ve forzada a reducir la expresión subyacente para poder seguir adelante. [10]

En estos puntos, los *thunks* se fuerzan de manera automática para obtener el valor requerido. Algunos ejemplos característicos incluyen:

- Las posiciones de los operandos en operadores primitivos, como $+$, $-$ o $\times$, así como en operadores lógicos como *and*, *or* y *not*, donde el valor debe conocerse para aplicar la operación.
- El predicado en una expresión condicional (*if* o *cond*), ya que se necesita determinar su valor booleano antes de decidir qué rama ejecutar.
- El operador en una aplicación de función, como en $(f \ e)$. Aunque la intuición podría sugerir que el punto estricto es el argumento e , lo que realmente debe evaluarse primero es f , para verificar que representa efectivamente una función aplicable. Por ejemplo, en la expresión $(3 \ 5)$, intentar aplicar un número como función constituye un error; por eso, el sistema debe forzar f para asegurar su naturaleza funcional antes de proceder a aplicar el argumento.

³¹Una comonada captura una computación que consume contexto, lo que es útil para representar flujos de datos, gramáticas atributivas y otros constructos sensibles al contexto [39].

Memoización de *thunks*

Aunque no profundizaremos en la implementación detallada de la memoización, es importante incluirla en el marco de estudio por su papel esencial como optimización en la evaluación perezosa. La *memoización* es una técnica que evita la redundancia computacional almacenando los resultados de una función (o de la evaluación de un *thunk*) para reutilizarlos en futuras invocaciones. Esta técnica permite mejorar significativamente la eficiencia, especialmente en programas con alto grado de recursión o estructuras perezosas como flujos (*streams*) [31, 38].

Formalmente, cuando un *thunk* es forzado por primera vez, su resultado se guarda, y futuras aplicaciones de *force* sobre ese mismo *thunk* devuelven el valor almacenado sin reevaluación:

$$Thunk(e, \varphi) \rightsquigarrow Val(v) \quad \text{cuando } force(Thunk(e, \varphi)) = v$$

Este comportamiento corresponde al paso de parámetros por necesidad (*Call-by-Need*), que puede verse como una versión memoizada del paso de parámetros por nombre (*Call-by-Name*) [40]. Como señala Harper, esta estrategia mantiene la semántica no estricta al tiempo que reduce el costo computacional de reevaluar expresiones [41].

Desde una perspectiva más general, la memoización comparte principios con la programación dinámica, al reutilizar soluciones de subproblemas previamente computados. Sin embargo, a diferencia de esta última, la memoización se aplica típicamente a funciones donde los subproblemas no son independientes y pueden compartir sub-subproblemas, lo cual requiere un almacenamiento explícito de resultados ya evaluados [42].

En el contexto de la programación funcional pura, donde no existen efectos secundarios, la implementación de memoización es particularmente adecuada. Las funciones puramente funcionales (aquellas cuyo resultado depende únicamente de sus argumentos) son candidatas ideales para ser memoizadas, ya que permiten utilizar sus argumentos como llaves de búsqueda sin introducir inconsistencias. No obstante, esto impone restricciones: funciones con efectos secundarios o dependientes del estado global no deben ser memoizadas, pues podrían comprometer la corrección del programa. Además, en lenguajes puramente funcionales, no se dispone de estructuras de almacenamiento mutable como en los lenguajes imperativos, lo que exige enfoques alternativos para construir tablas de memoización, como el uso de árboles persistentes o estructuras inmutables eficientes [42].

En las secciones siguientes, nos centraremos en el modelado formal de la evaluación perezosa y la implementación de intérpretes. Aunque no abordaremos explícitamente la implementación concreta de la memoización, reconocemos su papel central como optimización imprescindible para hacer viable la evaluación perezosa en sistemas reales.

2.5.7. Recursión

La recursión es un mecanismo esencial para la definición de funciones, especialmente en los lenguajes de programación funcional. Permite expresar cálculos repetitivos o estructuralmente anidados mediante funciones que se aplican a sí mismas (directa o indirectamente) sin necesidad de estructuras de control explícitas como bucles. Esta capacidad es especialmente útil para manipular estructuras inductivas como listas o árboles, de manera concisa y declarativa.

Desde el punto de vista computacional, recursión e iteración son equivalentes en cuanto a su poder expresivo: toda computación que puede realizarse mediante un bucle puede expresarse recursivamente, y viceversa. La presencia de al menos uno de estos mecanismos es suficiente para extender el poder computacional de un lenguaje más allá del de los autómatas finitos [6]. En lenguajes imperativos suele preferirse la iteración debido a su compatibilidad con efectos secundarios, mientras que en lenguajes funcionales, donde dichos efectos se evitan o controlan, la recursión es una alternativa más natural y congruente con su modelo semántico [41].

Una función recursiva opera realizando aplicaciones sucesivas a sí misma con diferentes argumentos, hasta alcanzar un *caso base* que actúa como cláusula de escape. Para garantizar la terminación de este proceso, es esencial que las llamadas recursivas se aproximen al caso base, ya sea mediante reducción estructural de los argumentos o mediante una medida decreciente. La validez de este comportamiento puede demostrarse formalmente mediante inducción estructural o inducción bien fundada (véase la sección 2.8). Este tipo de razonamiento inductivo (donde se asume que la función se comporta correctamente en los casos más simples para probar que también lo hace en casos generales) es fundamental en la verificación formal de programas [41].

En algunos contextos, la recursión puede optimizarse a través de una transformación conocida como *recursión de cola*. Una función recursiva es de cola cuando su llamada recursiva es la última operación que se ejecuta antes de retornar el resultado. Este patrón permite al compilador reutilizar el marco de pila actual en lugar de crear uno nuevo, lo que reduce el consumo de memoria. No obstante, no todas las funciones recursivas pueden reformularse en esta forma, y forzar dicha transformación puede comprometer la claridad o el estilo declarativo del código [6]. Por ello, aunque muchos compiladores modernos detectan y optimizan automáticamente este patrón, su aplicabilidad sigue siendo limitada.

Formalmente, en el *Cálculo λ* , la recursión no está disponible de forma directa, ya que las funciones no poseen nombres explícitos. Por lo tanto, no es posible invocar una función por su nombre dentro de su propio cuerpo como se hace en lenguajes como Haskell. En lugar de ello, la autorreferencia se logra mediante *autoaplicación*, usando combinadores de punto fijo, como el combinador *Y*. Este combinador permite definir expresiones que, al aplicarse a una función, producen un comportamiento recursivo sin requerir una referencia nominal. Esta técnica es fundamental para dotar al *Cálculo λ* de la capacidad para representar todas las funciones computables [3].

Recursión de punto fijo

Como se introdujo previamente en la sección 2.5.2, el concepto de *punto fijo* en el *Cálculo λ* permite representar funciones recursivas en un sistema que, por definición, no incluye mecanismos de asignación de nombres ni definiciones autoreferenciales. Formalmente, un punto fijo de una función F es un valor X tal que $F X = X$ (véase el teorema 2.5.2). Esta propiedad es esencial para modelar la recursión dentro del *Cálculo λ* puro.

Los operadores de punto fijo, como los combinadores *Y* y *Z*, permiten construir tales valores X para una función arbitraria F . Aplicar un operador de punto fijo a una función produce una instancia autoreferencial que satisface la ecuación característica de los puntos

fijos, es decir:

$$(fix\ F) = F\ (fix\ F).$$

Este mecanismo hace posible la definición de funciones recursivas sin depender de identificadores explícitos.

Sin embargo, el comportamiento operacional de estos combinadores está profundamente influenciado por el régimen de evaluación adoptado. La convergencia³² o divergencia de un término resultante depende críticamente del régimen de evaluación adoptado, ya sea ansioso con estrategia de paso de parámetros por valor *Call-by-Value* o perezoso con paso de parámetros por nombre *Call-by-Name*. Esta diferencia afecta tanto la posibilidad de terminación de las definiciones recursivas como la elección del combinador de punto fijo adecuado. Esta distinción resulta fundamental para la correcta interpretación y ejecución de definiciones recursivas en distintos lenguajes funcionales [3].

Operador Y

En el contexto del *Cálculo* λ , el combinador de punto fijo Y (véase la definición 2.5.8) constituye la forma canónica para definir funciones recursivas bajo evaluación no estricta. Este combinador, conocido como el operador de punto fijo por *Call-by-Name*, explota las características de evaluación diferida propias de este régimen para evitar la expansión inmediata de la estructura autoreferencial. En un entorno perezoso, las aplicaciones de la forma $x\ x$ dentro del cuerpo de Y no se evalúan hasta que su valor es estrictamente requerido, lo cual permite que la función recursiva pueda acceder a su propia definición sin incurrir en divergencia prematura.

En consecuencia, el operador Y resulta funcional y seguro en contextos perezosos, siempre que el cuerpo funcional F contenga un caso base alcanzable. No obstante, este mismo operador es inadecuado en evaluación estricta: bajo una estrategia *Call-by-Value*, cualquier intento de evaluar $Y\ F$ conduce a una expansión infinita, ya que se fuerza la evaluación de las aplicaciones recursivas inmediatamente. Por tanto, en este ambiente, Y no puede ser utilizado para definir funciones recursivas sin caer en divergencia [3].

Operador Z

Para habilitar la recursión en contextos de evaluación estricta, su definición, atribuida a Plotkin, está cuidadosamente diseñada para evitar la evaluación prematura de las llamadas recursivas (véase la definición 2.5.9). La esencia de este operador radica en la protección de las aplicaciones recursivas mediante una abstracción adicional $(\lambda y. \dots)$. Esta encapsulación impide que el evaluador reduzca inmediatamente la expresión autoreferencial, permitiendo así que el término converja, siempre que F esté correctamente definida.

Este operador, comúnmente denominado Z , también es conocido como Y_v , donde el subíndice v alude a *value*, haciendo referencia a su adaptación para contextos con paso de parámetros por valor (*Call-by-Value*). En este régimen, el combinador clásico Y conduce a

³²Un término λ converge si su evaluación, de acuerdo con la semántica operacional del régimen correspondiente, alcanza una *forma normal*, es decir, un término al que no se le puede aplicar ninguna otra β -reducción [25].

la divergencia, ya que intenta evaluar de forma inmediata la autoaplicación (xx) , lo que provoca una secuencia infinita de reducciones. En contraste, Y_v encapsula dicha autoaplicación dentro de una capa adicional de abstracción, retrasando su evaluación y evitando así la divergencia [43].

De este modo, el operador Z o Y_v representa la alternativa adecuada para entornos con evaluación estricta, como en muchos lenguajes funcionales modernos, incluidos ML y PCF. Cabe señalar que, si bien estos combinadores pueden derivarse a partir de principios fundamentales del *Cálculo λ* , tales construcciones suelen ser técnicamente complejas y no se abordan aquí. En cambio, nos centramos en su comportamiento semántico y en su utilidad práctica para expresar recursión en distintos modelos de evaluación [19].

Recursión en acción

Habiendo establecido el fundamento teórico de los combinadores de punto fijo y su relación con las estrategias de paso de parámetros, aquí exploramos un ejemplo concreto que ilustra su aplicación práctica: la definición recursiva de la función que suma los primeros n números naturales en el *Cálculo λ* (véase la sección 2.5.2).

Codificando booleanos. Para facilitar la escritura de expresiones complejas y evitar una notación excesivamente cargada, introducimos a continuación una serie de abreviaciones o *macros*³³, utilizadas para representar los valores booleanos básicos y la estructura condicional en el *Cálculo λ* :

$$\begin{aligned} true &\doteq \lambda x.\lambda y. x \\ false &\doteq \lambda x.\lambda y. y \\ if &\doteq \lambda v.\lambda t.\lambda f. v \ t \ f \end{aligned}$$

Estas macros permiten expresar condicionales de forma más legible y directa, lo cual resulta particularmente útil al definir funciones con comportamiento distinto según condiciones, como en estructuras recursivas con casos base [27].

Para justificar su utilidad, mostramos que estas definiciones satisfacen el comportamiento esperado de un condicional. Específicamente, deseamos que la expresión condicional devuelva el primer argumento cuando la condición sea *true*, y el segundo cuando sea *false*. Esta propiedad refleja la semántica funcional deseada: que la ejecución condicional seleccione efectivamente una de las dos ramas según el valor booleano evaluado.

Verificamos este comportamiento mediante reducciones sucesivas. Para cualesquiera términos M y N , se cumple:

$$if \ true \ M \ N \ = \ M$$

Demostramos esta equivalencia expandiendo paso a paso las definiciones involucradas:

³³La notación $\doteq$ indica la definición de una abreviación sintáctica o *macro*, es decir, un atajo notacional que representa un término λ completo [27].

$$\begin{aligned}
\text{if } \text{true } M \ N &\doteq (\lambda v. \lambda t. \lambda f. v \ t \ f) (\lambda x. \lambda y. x) \ M \ N \\
&\xrightarrow{\beta} (\lambda t. \lambda f. (\lambda x. \lambda y. x) \ t \ f) \ M \ N \\
&\xrightarrow{\beta} (\lambda f. (\lambda x. \lambda y. x) \ M \ f) \ N \\
&\xrightarrow{\beta} (\lambda x. \lambda y. x) \ M \ N \\
&\xrightarrow{\beta} (\lambda y. M) \ N \\
&\xrightarrow{\beta} M
\end{aligned}$$

De manera análoga, el comportamiento esperado se verifica cuando el valor booleano es *false*:

$$\text{if } \text{false } M \ N = N$$

Estas secuencias de reducción confirman que las macros propuestas reproducen correctamente la semántica operativa de una estructura condicional, permitiendo su implementación íntegramente dentro del *Cálculo* λ , sin necesidad de incorporar primitivas externas.

Codificación de pares. El *Cálculo* λ puro no cuenta con construcciones primitivas para representar estructuras de datos como los pares ordenados. No obstante, al tratarse de un sistema expresivo basado exclusivamente en funciones, es posible codificar pares a partir de aplicaciones de función adecuadamente estructuradas. Para ello, introducimos tres macros fundamentales:

- (M, N) : notación para un par cuyos elementos son M y N ,
- *fst*: función que recupera el primer componente de un par,
- *snd*: función que recupera el segundo componente.

Estas macros deben satisfacer las siguientes propiedades semánticas:

$$\begin{aligned}
\text{fst}(M, N) &= M \\
\text{snd}(M, N) &= N
\end{aligned}$$

Proponemos la siguiente codificación funcional para representar esta estructura:

$$\begin{aligned}
(M, N) &\doteq \lambda s. s \ M \ N \\
\text{mkpair} &\doteq \lambda x. \lambda y. \lambda s. s \ x \ y \\
\text{fst} &\doteq \lambda p. p \ \text{true} \\
\text{snd} &\doteq \lambda p. p \ \text{false}
\end{aligned}$$

El constructo de pares (M, N) es una función que, al ser aplicada a un término booleano, delega en dicho término la elección de uno de sus dos elementos internos. Esto motiva las definiciones de *fst* y *snd*, las cuales simplemente aplican el par recibido como argumento a *true* o *false*, respectivamente, para recuperar el valor deseado.

Veamos una reducción ejemplar que ilustra el funcionamiento de *fst*:

$$\begin{aligned}
 fst(M, N) &\doteq (\lambda p. p \ true) (\lambda s. s \ M \ N) \\
 &\xrightarrow{\beta} (\lambda s. s \ M \ N) \ true \\
 &\xrightarrow{\beta} true \ M \ N \\
 &\doteq (\lambda x. \lambda y. x) \ M \ N \\
 &\xrightarrow{\beta} (\lambda y. M) \ N \\
 &\xrightarrow{\beta} M
 \end{aligned}$$

El caso análogo para *snd* sigue una secuencia de reducciones idéntica, sustituyendo *false* en lugar de *true* y obteniendo como resultado *N*.

Esta representación funcional de pares se alinea con el enfoque canónico de codificación de estructuras de datos en *Cálculo λ* , y resulta plenamente operativa en presencia de los booleanos definidos previamente [27].

Codificación de números naturales. La idea central consiste en representar un número natural *n* como una función de dos argumentos *f* y *x*, que aplica *f* a *x* exactamente *n* veces. Esta interpretación funcional permite capturar el significado de la iteración dentro del propio formalismo λ . La codificación más difundida es la propuesta por Alonzo Church, y a los términos que la siguen se les conoce como *numerales de Church*.

Establecemos las siguientes macros:

$$\begin{aligned}
 zero &\doteq \lambda f. \lambda x. x \\
 one &\doteq \lambda f. \lambda x. f \ x \\
 two &\doteq \lambda f. \lambda x. f \ (f \ x) \\
 three &\doteq \lambda f. \lambda x. f \ (f \ (f \ x)) \\
 &\vdots
 \end{aligned}$$

Sucesor. Con esta representación, la función sucesor, que incrementa un numeral en una unidad, se define como:

$$succ \doteq \lambda n. \lambda f. \lambda x. f \ (n \ f \ x)$$

Esta función construye un nuevo numeral aplicando *f* una vez más sobre el resultado de aplicar *n f x*, es decir, emula la operación $n + 1$.

Suma. La adición puede entonces definirse como la iteración de la función *succ* exactamente *m* veces sobre *n*, lo cual se expresa elegantemente como:

$$add \doteq \lambda n. \lambda m. m \ succ \ n$$

Prueba de cero. Una propiedad útil en esta representación es la capacidad de distinguir el cero funcionalmente. Para ello, se define una función *iszero* que recibe un numeral y retorna *true* si se trata de cero, y *false* en caso contrario. La esencia está en aprovechar el comportamiento iterativo del numeral:

$$iszero \doteq \lambda n. n (\lambda x. false) true$$

Aquí, si $n = 0$, la función $(\lambda x. false)$ no será aplicada y se conservará el valor *true*; de lo contrario, se aplicará al menos una vez, resultando en *false*.

Predecesor. Definir la operación inversa a la sucesión, es decir, la función *pred* que calcula el predecesor de un numeral, requiere un tratamiento más sofisticado. Como la aplicación de funciones en *Cálculo λ* no es invertible en general, necesitamos simular la eliminación de la primera iteración.

Para ello, se recurre a una estructura de pares que lleva información sobre si ya se ha aplicado la función. La estrategia consiste en:

- Inicializar un par $(true, x)$, donde *true* actúa como bandera de “salto” para omitir la primera aplicación de *f*.
- Usar una versión modificada de *f*, llamada *wrap*, que consulta la bandera y aplica *f* sólo si la bandera es *false*.

Definimos las siguientes macros:

$$\begin{aligned} wrap &\doteq \lambda f. \lambda p. (false, if (fst p) (snd p) (f (snd p))) \\ pred &\doteq \lambda n. \lambda f. \lambda x. snd (n (wrap f) (true, x)) \end{aligned}$$

Así, al aplicar n veces la función *wrap f*, se “salta” la primera iteración y se recupera el resultado correspondiente a $n - 1$ aplicaciones de *f* sobre *x*.

Es importante destacar que *zero* y *false* comparten la misma codificación: ambas corresponden al término $\lambda f. \lambda x. x$. Este solapamiento implica que ningún término puede distinguir funcionalmente entre el valor cero y el booleano *false*, por lo que es fundamental mantener el contexto semántico adecuado para evitar errores en las construcciones [27].

Suma de los primeros n números naturales

A partir de los elementos ya introducidos (booleanos, pares, numerales de Church, y funciones auxiliares como *succ*, *pred*, *iszero*, y *add*) es posible definir funciones recursivas aritméticas. Un ejemplo clásico es la suma de los primeros n naturales, es decir:

$$sum\ n = 0 + 1 + 2 + 3 + \cdots + n$$

Formalmente, esta función se define de manera recursiva como:

$$sum(n) = \begin{cases} 0 & \text{si } n = 0 \\ n + sum(n - 1) & \text{en otro caso} \end{cases}$$

En notación de términos λ , empleando los macros previamente definidos y el combinador Y para la recursión, esta función se codifica como sigue:

$$sum \doteq Y (\lambda f. \lambda n. iszero\ n\ zero\ (add\ n\ (f\ (pred\ n))))$$

Este término captura la definición matemática: si n es cero, devuelve cero; si no, suma n con la suma recursiva de su predecesor. Procedamos a realizar la evaluación paso a paso del término $sum\ zero$, bajo evaluación perezosa y estrategia de paso de parámetros *Call-by-Name*:

$$\begin{aligned} sum\ zero &\doteq Y(\lambda f. \lambda n. iszero\ n\ zero(add\ n(f(pred\ n))))zero \\ &\doteq (\lambda f. (\lambda x. f(xx))(\lambda x. f(xx)))(\lambda f. \lambda n. iszero\ n\ zero(add\ n(f(pred\ n))))zero \\ \text{Sea } \Delta &\doteq (\lambda f. \lambda n. iszero\ n\ zero(add\ n(f(pred\ n)))) \\ &\xrightarrow{\beta} (\lambda x. \Delta(xx))(\lambda x. \Delta(xx))zero \\ &\xrightarrow{\beta} \Delta((\lambda x. \Delta(xx))(\lambda x. \Delta(xx)))zero \\ &\doteq (\lambda f. \lambda n. iszero\ n\ zero(add\ n(f(pred\ n))))((\lambda x. \Delta(xx))(\lambda x. \Delta(xx)))zero \\ &\xrightarrow{\beta} (\lambda n. iszero\ n\ zero(add\ n((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ n))))zero \\ &\xrightarrow{\beta} iszero\ zero\ zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\doteq (\lambda n. n(\lambda x. false)true)zero\ zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\xrightarrow{\beta} zero(\lambda x. false)true\ zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\doteq (\lambda f. \lambda x. x)(\lambda x. false)true\ zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\xrightarrow{\beta} (\lambda x. x)true\ zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\xrightarrow{\beta} true\ zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\xrightarrow{\beta} (\lambda x. \lambda y. x)zero(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\xrightarrow{\beta} (\lambda y. zero)(add\ zero((\lambda x. \Delta(xx))(\lambda x. \Delta(xx))(pred\ zero))) \\ &\xrightarrow{\beta} zero \\ &\doteq \lambda f. \lambda x. x \end{aligned}$$

Para completar nuestro análisis, conviene observar cómo se comporta el mismo ejemplo bajo un régimen de evaluación *ansioso* y estrategia *Call-by-Value*. En este modelo, los argumentos deben evaluarse por completo antes de ser aplicados, lo que vuelve inadecuado el uso directo del combinador Y , ya que su autoaplicación conduce inmediatamente a una expansión infinita.

En su lugar, empleamos el operador Z , una variante del operador de punto fijo diseñada específicamente para este tipo de evaluación. Su definición, recordemos, difiere ligeramente para evitar la expansión inmediata de la autoreferencia, garantizando así que el término se mantenga en una forma controlada hasta que se requiera su evaluación efectiva.

Aunque no desarrollamos nuevamente la evaluación completa, el ejemplo de la suma de los primeros números naturales aplicado al número cero bajo este régimen resulta análogo al

caso perezoso, siempre que se respeten los principios de la estrategia *Call-by-Value*. Es decir, el operador Z diferirá en su estructura interna, pero la lógica recursiva y el comportamiento observable de la función siguen siendo los mismos, convergiendo finalmente al resultado esperado: *zero*.

2.6. Máquinas abstractas

Las máquinas abstractas constituyen un modelo teórico de computación, diseñado para describir de manera precisa y estructurada cómo se procesan los cálculos en un sistema. De manera general, una máquina abstracta está compuesta por un conjunto de estados, incluyendo estados iniciales y finales, y un conjunto de reglas que permiten transitar de un estado a otro. Esta formalización constituye una herramienta fundamental en computación teórica, ya que permite especificar formalmente el comportamiento dinámico de los lenguajes de programación, así como abordar problemas clásicos de la teoría de la computabilidad y de la complejidad. Un ejemplo paradigmático es la máquina de Turing, propuesta por Alan Turing en 1936 para abordar el célebre *Entscheidungsproblem* (Problema de Decisión).

Aunque su propósito principal es ofrecer una abstracción idealizada del proceso de cómputo, las máquinas abstractas mantienen una estrecha relación conceptual con las *máquinas virtuales*. Mientras que las primeras son modelos puramente teóricos, las segundas pueden verse como implementaciones concretas de aquellas, adaptadas a las restricciones del entorno de ejecución. Las máquinas abstractas operan bajo suposiciones ideales, como memoria infinita o ejecución atómica, mientras que las máquinas virtuales deben considerar las limitaciones prácticas del hardware. Esta distinción, aunque sutil, es fundamental para comprender la forma en que una especificación formal puede derivar en una implementación ejecutable. Como se ha demostrado en diversos trabajos, una semántica definida formalmente puede ser utilizada para generar código que se ejecuta sobre una máquina, garantizando que el resultado obtenido coincida con el significado esperado del programa original [24].

Tanto las máquinas abstractas como las virtuales ofrecen una abstracción del hardware físico y de los sistemas operativos. Se caracterizan comúnmente por los tipos de datos que manejan de forma nativa, por su lenguaje de instrucciones y, en ocasiones, por su modelo de memoria. En el diseño de máquinas abstractas orientadas al estudio de lenguajes de programación, resulta importante adoptar una semántica de paso pequeño, ya que, a diferencia de algunos modelos teóricos que permiten el no determinismo, en los entornos computacionales reales la ejecución debe ser determinista y el orden de evaluación perfectamente definido.

En el marco de esta tesis, las máquinas abstractas cumplen un papel central, ya que permiten representar el comportamiento dinámico de los lenguajes objeto de estudio mediante reglas de evaluación sobre entornos. Estas descripciones, similares en estructura a las formalizaciones presentadas para el lenguaje MINIB en la sección 2.5.1, constituyen el punto de partida para la aplicación del método de *Cálculo de Compiladores Correctos* que permite derivar una implementación concreta [1]. Una máquina virtual, que reproduce la semántica establecida. En esencia, una máquina abstracta describe no solo los valores y estructuras de datos que pueden ser representados durante la computación, sino también el conjunto de reglas que gobiernan la evolución del sistema desde su estado inicial hasta alcanzar un estado final mediante una sucesión de transformaciones bien definidas. De este modo, constituyen un marco

riguroso para comprender, especificar y analizar los procesos de cómputo, destacándose como un componente central tanto en la teoría como en la práctica de los lenguajes formales y la programación [21].

2.6.1. Máquinas virtuales

Las máquinas virtuales constituyen una realización concreta del concepto de máquina abstracta, cuyo propósito es proporcionar un entorno de ejecución independiente sobre una plataforma física. En términos generales, una máquina virtual es una capa de software que emula una arquitectura computacional específica, permitiendo que los programas se comporten como si se ejecutaran en una máquina autónoma. Esta emulación se lleva a cabo mediante la interpretación o traducción de instrucciones definidas por la máquina virtual a operaciones concretas sobre los recursos físicos del sistema subyacente (véase la figura 2.9).

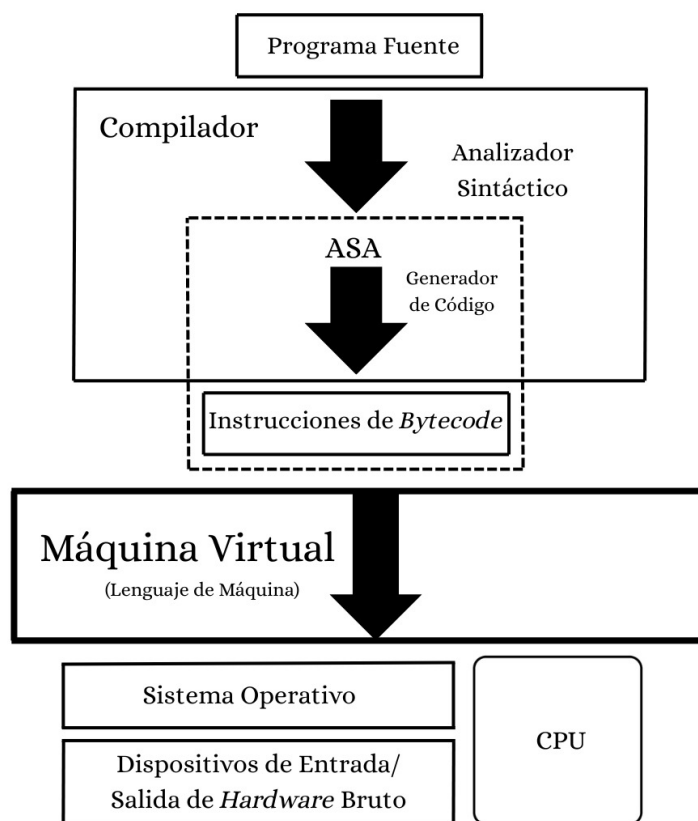


Figura 2.9: Implementación de una máquina virtual. Tomado de [11].

Formalmente, el comportamiento de una máquina virtual puede describirse mediante *configuraciones*, que representan el estado del sistema en un instante determinado. Estas configuraciones incluyen, entre otros elementos, el contenido de la memoria virtual, los registros y el contador de programa. La ejecución se modela como una sucesión de transiciones entre configuraciones, definidas por reglas que describen con precisión la interpretación de

cada instrucción. Este enfoque constituye una aplicación directa de los principios de la semántica operacional, donde cada transición captura el significado formal de una instrucción.

Para que la virtualización funcione correctamente, debe existir un mapeo adecuado entre los componentes virtuales (como la memoria, los registros y el conjunto de instrucciones) y los recursos físicos del sistema anfitrión. Esta tarea recae en el software de virtualización, cuya responsabilidad es garantizar que la ejecución del código sea coherente y segura sobre la infraestructura física disponible [6].

Entre los modelos más utilizados para implementar máquinas virtuales se encuentran las arquitecturas basadas en pila. En este tipo de arquitectura, los operandos no se especifican explícitamente en las instrucciones, sino que se gestionan a través de una pila de evaluación. Esto simplifica tanto el diseño del intérprete como la generación de código durante la compilación, lo que ha llevado a su amplia adopción debido a su estructura sencilla y eficiente manejo de valores temporales.

No obstante, estudios comparativos han mostrado que las arquitecturas basadas en registros pueden ofrecer ventajas en ciertos contextos, especialmente desde el punto de vista de la optimización. Por ejemplo, traducir el *bytecode* de una máquina de pila, como la utilizada por la JVM, a una representación basada en registros puede reducir el número total de instrucciones ejecutadas, aunque con un incremento en los accesos a memoria. Aun así, en entornos donde la velocidad de ejecución de instrucciones prevalece sobre el costo del acceso a memoria, esta transformación puede mejorar significativamente el rendimiento [44, 45].

La JVM es uno de los ejemplos más representativos del enfoque de máquina virtual. Diseñada para ejecutar programas escritos en el formato intermedio *bytecode*, la JVM actúa como una capa de abstracción entre el lenguaje JAVA y la arquitectura física subyacente. El compilador *javac* convierte el código fuente en archivos *.class*, que contienen instrucciones que son interpretadas por la JVM [11]. Esta separación entre la compilación y la ejecución, junto con la independencia del hardware, ha convertido a la JVM en un componente esencial para la portabilidad del lenguaje.

Lenguajes como PYTHON o LUA también utilizan máquinas virtuales en sus intérpretes. Si bien existen alternativas más sofisticadas, como los compiladores JIT, estas suelen implicar mayores costos de implementación y presentan limitaciones en contextos donde los recursos son restringidos o donde existen políticas estrictas de seguridad, como en dispositivos móviles. Por estas razones, muchas implementaciones continúan optando por arquitecturas interpretadas, priorizando la simplicidad y la portabilidad sobre la optimización agresiva [46].

En este contexto, el enfoque adoptado en este trabajo (propuesto por Bahr y Hutton) permite derivar sistemáticamente máquinas virtuales, en particular de tipo pila, a partir de la semántica del lenguaje fuente. Esta metodología no solo aporta una base teórica rigurosa para el diseño de máquinas virtuales, sino que además orienta la implementación correcta de intérpretes, como se desarrolla en la sección 2.10.

Configuración de Pila

En el diseño de máquinas virtuales para lenguajes funcionales, la pila constituye el mecanismo central para organizar y gestionar la ejecución paso a paso. Su función principal es controlar la evaluación secuencial de expresiones y mantener los elementos necesarios durante las transiciones, como resultados intermedios, argumentos de funciones y direcciones

de retorno. Esta estructura, definida formalmente como una secuencia ordenada de valores, permite modelar con claridad el flujo de ejecución mediante operaciones elementales como *apilar* (*push*) y *desapilar* (*pop*).

Más allá del almacenamiento de datos temporales, la pila también interactúa estrechamente con otras estructuras fundamentales del entorno de ejecución, en particular con los ambientes de evaluación. De este modo, la configuración de la pila no actúa de forma aislada, sino que colabora activamente en la representación del estado computacional completo, donde cada marco de pila puede incluir una referencia a su entorno léxico. Esta combinación permite reconstruir de forma precisa el contexto de ejecución en cada etapa del programa. En lenguajes funcionales, este enfoque cobra particular relevancia. Dado que la computación se expresa predominantemente como aplicación de funciones, la pila de control se convierte en el principal recurso para sostener el estado de evaluación. A diferencia de los lenguajes imperativos, donde el manejo explícito del *heap* es frecuente y abierto al programador, los lenguajes funcionales suelen restringir o encapsular su uso, favoreciendo modelos en los que el *heap* se administra automáticamente y permanece oculto en gran medida [19].

Incluso en contextos donde se permite cierto grado de evaluación diferida o efectos controlados, como en HASKELL, que emplea técnicas como la evaluación perezosa y el uso de mónadas para encapsular efectos secundarios, la pila conserva un papel central como estructura de soporte del modelo operacional, entendido como la descripción paso a paso de la ejecución de un programa a través de transiciones entre configuraciones del sistema. Aunque la evaluación diferida introduce mecanismos que pueden reducir la acumulación inmediata de marcos en la pila, esta sigue siendo esencial para gestionar la evaluación de expresiones y el control de flujo. Sin embargo, dicha evaluación puede ocasionar problemas como los *space leaks*, donde se mantienen en memoria referencias a datos que ya no son necesarios, lo que incrementa el uso del *heap*. Para abordar estos problemas, se han desarrollado técnicas de perfilamiento y estrategias específicas de optimización, como la evaluación estricta selectiva mediante la función *seq*, que fuerza la evaluación de expresiones antes de que sean almacenadas en estructuras de datos, reduciendo así el consumo innecesario de memoria [47].

La ventaja de centrar el modelo de ejecución en la pila, en lugar de delegar la complejidad al manejo del *heap*, radica en que se facilita tanto el razonamiento formal como la implementación práctica. Las transiciones entre configuraciones se describen de manera precisa y predecible, utilizando únicamente operaciones sobre la pila y actualizaciones de ambiente, lo que simplifica el razonamiento formal sobre la semántica de lenguajes de programación [6,11].

Concepto de configuración en la máquina virtual de Bahr y Hutton

Dentro del enfoque de Bahr y Hutton, el término *configuración* adquiere un significado técnico específico que va más allá de su uso coloquial. Formalmente, una configuración representa la encapsulación completa del contexto de ejecución de un programa dentro de la máquina virtual. No se trata simplemente de almacenar una pila de valores o un estado de memoria, sino de agrupar todos los componentes relevantes que permiten modelar el avance de la ejecución paso a paso, de acuerdo con la semántica formal del lenguaje.

Este enfoque se articula estrechamente con la semántica operacional estructural (véase la sección 2.5.1), en la que la ejecución de un programa se modela como una sucesión de pasos entre configuraciones. Cada paso representa una transición individual guiada por re-

glas de inferencia que especifican cómo un fragmento de programa evoluciona a partir de su contexto actual. Así, las configuraciones no solo contienen información operativa (como la pila o el estado de memoria), sino también todos los elementos necesarios para evaluar correctamente la expresión en curso, incluyendo ambientes de evaluación, un posible estado mutable, y cualquier otro componente relevante del contexto semántico.

Formalmente, una configuración se define como una tupla que agrupa estas estructuras de datos y refleja el entorno de evaluación vigente. La pila de evaluación, los ambientes de variables para la resolución de nombres, y, cuando se requiere, el estado mutable o componentes auxiliares, forman parte de esta representación unificada. Su objetivo no es meramente simular una ejecución mecánica, sino capturar con precisión la semántica formal del lenguaje y permitir una evaluación rigurosa paso a paso.

En el método propuesto por Bahr y Hutton, esta noción de configuración se vuelve un instrumento esencial para la derivación sistemática de máquinas virtuales. Su diseño modular permite incorporar extensiones semánticas (como memoria mutable o nuevos mecanismos de control) mediante la simple adición o ajuste de componentes dentro de la configuración, sin alterar la estructura esencial de las reglas de transición. Esto refuerza la idea de que una máquina virtual definida de este modo no solo ejecuta programas, sino que figura directamente su semántica operacional [1].

2.7. Razonamiento ecuacional

El razonamiento ecuacional es una herramienta fundamental en la programación funcional, particularmente adecuada para lenguajes como HASKELL y formalismos como el *Cálculo* λ . Su enfoque se centra en aplicar reglas de reescritura basadas en leyes ecuacionales, definiciones de funciones y equivalencias semánticas para derivar, simplificar o verificar propiedades de expresiones con el objetivo de establecer equivalencias semánticas, usualmente en forma de igualdades. Esta metodología permite derivar propiedades de los programas, optimizarlos o verificar su corrección sin necesidad de ejecutarlos [48].

El razonamiento ecuacional en programación funcional consiste en transformar expresiones mediante la aplicación sistemática de leyes ecuacionales, igualdades condicionales que permiten sustituir un patrón de código por otro operacionalmente equivalente, hasta alcanzar una forma deseada o simplificada. Cada paso de reescritura se basa en definiciones de funciones, reglas de evaluación del lenguaje o principios semánticos bien establecidos.

En HASKELL, la presencia de funciones de orden superior y definiciones recursivas hace que el estilo algebraico de razonamiento ecuacional resulte natural y directo. De manera análoga, en el *Cálculo* λ puro nuestra herramienta fundamental son las reglas de β - y η -reducción³⁴, que permiten reescribir términos para demostrar propiedades de programas [3].

³⁴La η -reducción formaliza la idea de que una función que recibe un argumento y acto seguido lo pasa a otra función es equivalente a dicha función. Formalmente,

$$\lambda x. g x \xrightarrow{\eta} g \quad \text{si } x \notin \text{FV}(g).$$

Por ejemplo, $\lambda y. (\text{map } f) y \xrightarrow{\eta} \text{map } f$ [27].

Una técnica fundamental es la *simplificación*, donde un término o fórmula se reemplaza por otro que es lógicamente equivalente pero “menor” según algún orden bien fundado. Puesto que ningún orden bien fundado admite cadenas infinitas, este proceso de simplificación no puede prolongarse indefinidamente³⁵. Esto resulta esencial tanto para la demostración de corrección de programas como para la optimización de código y la verificación automática de teoremas [50].

2.7.1. Ejemplos de razonamiento en HASKELL: *or* y *reverse*

El razonamiento ecuacional en el contexto de HASKELL consiste en demostrar propiedades de programas mediante la transformación paso a paso de expresiones, aplicando definiciones de funciones y leyes conocidas. Cada paso mantiene la equivalencia semántica, lo cual garantiza que las modificaciones preservan el comportamiento del programa. Este método resulta particularmente natural en programación funcional debido a la ausencia de efectos secundarios y a la transparencia referencial de las expresiones.

Como ilustración inicial, consideremos la función lógica *or*, que devuelve *True* si al menos uno de sus argumentos es *True*, y *False* en caso contrario:

Listing 2.1: Definición de la función lógica *or* en HASKELL

```
or :: Bool -> Bool -> Bool
or False False = False
or - -         = True
```

Probaremos que la función *or* es idempotente, es decir, que para cualquier valor booleano *b*, se cumple que:

$$\text{or } b \ b = b$$

Procedemos mediante razonamiento por casos, analizando los posibles valores de *b*:

■ **Caso 1:** *b = False*

$$\text{or } \text{False } \text{False} = \text{False} \quad \text{aplicando la definición de } \text{or}$$

■ **Caso 2:** *b = True*

$$\text{or } \text{True } \text{True} = \text{True} \quad \text{aplicando la definición de } \text{or}$$

En ambos casos, la propiedad se cumple. Por tanto, hemos demostrado que *or* es idempotente.

Ahora retomamos un ejemplo clásico de razonamiento ecuacional en HASKELL, presentado en [51], que demuestra cómo analizar la definición de una función mediante la expansión paso a paso de su comportamiento.

Consideremos la definición estándar de la función *reverse*, que invierte los elementos de una lista:

³⁵La terminación de la simplificación se garantiza porque cada reducción disminuye el término respecto a un orden sin cadenas infinitas, asegurando que no existan secuencias de reescritura infinitas [49].

Listing 2.2: Definición de la función *reverse* en HASKELL

```
reverse :: [a] -> [a]
reverse [] = []
reverse (x:xs) = reverse xs ++ [x]
```

Utilizando esta definición, podemos demostrar que *reverse* aplicada a una lista con un único elemento produce la misma lista. Formalmente, queremos mostrar que:

$$\text{reverse } [x] = [x]$$

Procedemos a razonar ecuacionalmente:

$$\begin{aligned} \text{reverse } [x] &= \text{reverse } (x : []) && \text{notación de listas} \\ &= \text{reverse } [] ++ [x] && \text{aplicando la definición de reverse} \\ &= [] ++ [x] && \text{aplicando la definición de reverse} \\ &= [x] && \text{aplicando la definición de } (++) \end{aligned}$$

Así, concluimos que toda aparición de una expresión de la forma *reverse* $[x]$ puede ser reemplazada directamente por $[x]$ sin alterar el significado del programa, mejorando además su eficiencia al evitar evaluaciones innecesarias.

2.7.2. Ejemplo en el Cálculo λ : *snd*

En el contexto del Cálculo λ puro, el razonamiento ecuacional permite establecer equivalencias entre términos a partir de sus definiciones y las reglas de reducción.

Recordemos las definiciones de algunas macros:

$$\begin{aligned} (M, N) &\doteq \lambda s. s \ M \ N \\ \text{false} &\doteq \lambda x. \lambda y. y \\ \text{snd} &\doteq \lambda p. p \ \text{false} \end{aligned}$$

Queremos demostrar la siguiente propiedad:

$$\text{snd}((M, N)) = N$$

Similar al razonamiento de *fst* presentado en la sección 2.5.7, procedamos a desarrollar el razonamiento paso a paso:

$$\begin{aligned} \text{snd}((M, N)) &\doteq (\lambda p. p \ \text{false}) ((M, N)) && \text{definición de } \text{snd} \\ &\xrightarrow{\beta} (M, N) \ \text{false} \\ &\doteq (\lambda s. s \ M \ N) \ \text{false} && \text{definición de par } (M, N) \\ &\xrightarrow{\beta} \text{false} \ M \ N \\ &\doteq (\lambda x. \lambda y. x) \ M \ N && \text{definición de } \text{false} \\ &\xrightarrow{\beta} (\lambda y. N) \ M \\ &\xrightarrow{\beta} N \end{aligned}$$

Así, concluimos que aplicar *snd* a un par devuelve su primer componente, conforme a la intención de su definición.

Este estilo de razonamiento refleja la importancia de entender no solo las definiciones básicas, sino también cómo se encadenan las aplicaciones en el *Cálculo* λ para garantizar la corrección formal de propiedades más complejas [27].

2.8. Inducción estructural

El razonamiento ecuacional proporciona una base sólida para demostrar propiedades sobre estructuras definidas recursivamente, como las que se encuentran en programas funcionales o en el *Cálculo* λ . Sin embargo, para establecer propiedades más generales y robustas, es necesario introducir técnicas de demostración más poderosas, entre las cuales destaca la *inducción estructural*.

La inducción, en sus diversas formas, constituye una familia de métodos de prueba fundamentales en la teoría de programas. La forma más clásica es la *inducción matemática*, utilizada para demostrar propiedades sobre los números naturales. Este principio se basa en la construcción inductiva de los naturales: partiendo de 0 y generando sucesores de manera finita, similar a los numerales de Church presentados en la sección 2.5.7. De manera formal, para probar una propiedad $P(n)$ para todo número natural n , basta con demostrar:

- Base: $P(0)$ es verdadera.
- Paso inductivo: si $P(m)$ es verdadera, entonces también lo es $P(m + 1)$ para todo $m \in \mathbb{N}$.

Esta estructura de prueba refleja la forma en que los números naturales son generados: si podemos establecer la propiedad en el caso base y asegurar su preservación a través del paso de un número al siguiente, hemos demostrado su validez universal.

Ahora bien, cuando trabajamos con lenguajes de programación o estructuras de datos más complejas (como listas, árboles, o expresiones sintácticas), usar la sucesión de un número a otro no basta. En estos casos, la técnica adecuada es la *inducción estructural*.

La inducción estructural permite demostrar propiedades sobre objetos construidos recursivamente, mostrando que si una propiedad es válida para los componentes más simples de una estructura y se preserva al construir estructuras más complejas a partir de ellas, entonces es válida para toda la estructura.

En otras palabras, al igual que la inducción matemática, se basa en que las construcciones finitas no generan cadenas infinitas descendentes³⁶, lo que garantiza que el proceso de prueba siempre termina.

Ambas formas de inducción, la matemática y la estructural, son casos particulares de un principio más general: la *inducción bien fundada*. Este tipo de inducción se apoya en la idea

³⁶Una *cadena infinita descendente* en un conjunto ordenado es una sucesión infinita de elementos $x_0, x_1, x_2, \dots$ tal que $x_0 > x_1 > x_2 > \dots$. En un conjunto bien fundado, como los números naturales con el orden usual, no existen tales cadenas, lo que garantiza que los procesos de simplificación o de aplicación de la inducción deben terminar en algún punto [3].

de que todo conjunto bien fundado ³⁷ carece de cadenas estrictamente decrecientes infinitas, lo cual permite demostrar propiedades de cualquier elemento a partir de sus predecesores. En este contexto, el *razonamiento ecuacional* complementado con inducción estructural se convierte en una herramienta poderosa para demostrar la corrección de programas, asegurando que las propiedades deseadas se mantienen a lo largo de la construcción recursiva de los datos y las computaciones [3].

2.8.1. Inducción estructural en HASKELL

El principio fundamental de la inducción estructural establece que para probar que una propiedad P se cumple para todos los elementos de una estructura, es suficiente mostrar:

- Base: P se cumple en los casos básicos o elementos atómicos.
- Paso inductivo: Asumiendo que P se cumple para los componentes inmediatos (hipótesis inductiva), entonces también debe cumplirse para la estructura completa.
- Estas dos cláusulas son suficientes para definir el conjunto de los números naturales.

Una instancia clásica en HASKELL es la definición de los números naturales de forma inductiva [51]:

Listing 2.3: Definición inductiva del tipo de dato *Nat* (Números Naturales) en HASKELL

```
data Nat = Zero | Succ Nat
```

Donde:

- *Zero* representa el número 0.
- *Succ* n representa el sucesor de un número natural n , es decir, $n + 1$.

Como ejemplo de aplicación de la inducción estructural, consideremos la función de suma sobre naturales:

Listing 2.4: Función *add* para la suma de números naturales en HASKELL

```
add :: Nat -> Nat -> Nat
add Zero m = m
add (Succ n) m = Succ (add n m)
```

Queremos probar la propiedad:

$$add\ n\ Zero = n \quad \text{para todo natural } n$$

Procedemos mediante inducción sobre n :

³⁷Un *conjunto bien fundado* es un conjunto dotado de una relación de orden tal que no existen cadenas infinitas descendentes. Esto implica que cualquier proceso de descenso, como una secuencia de simplificaciones o un razonamiento inductivo, debe necesariamente terminar. Un ejemplo clásico de conjunto bien fundado es el conjunto de los números naturales con el orden usual de menor a mayor [3].

Base: Mostrar que la propiedad se cumple para $n = \text{Zero}$:

$$\text{add } \text{Zero } \text{Zero} = \text{Zero} \quad \text{por la definición de } \text{add}$$

Paso inductivo: Suponemos como hipótesis inductiva que:

$$\text{add } n \text{ Zero} = n$$

y debemos demostrar que:

$$\text{add } (\text{Succ } n) \text{ Zero} = \text{Succ } n$$

Desarrollo:

$$\begin{aligned} \text{add } (\text{Succ } n) \text{ Zero} &= \text{Succ}(\text{add } n \text{ Zero}) && \text{por definición de } \text{add} \\ &= \text{Succ}(n) && \text{por hipótesis inductiva} \end{aligned}$$

Lo que prueba el caso inductivo. $\square$ Ahora consideremos una función que duplica un número natural:

Listing 2.5: Función *double* para duplicar números naturales en HASKELL

```
double :: Nat -> Nat
double Zero = Zero
double (Succ n) = Succ (Succ (double n))
```

Queremos probar que:

$$\text{add } n \text{ } n = \text{double } n \quad \text{para todo natural } n$$

Base: Para $n = \text{Zero}$:

$$\begin{aligned} \text{add } \text{Zero } \text{Zero} &= \text{Zero} && \text{por definición de } \text{add} \\ \text{double } \text{Zero} &= \text{Zero} && \text{por definición de } \text{double} \end{aligned}$$

La propiedad se cumple.

Paso inductivo: Supongamos que:

$$\text{add } n \text{ } n = \text{double } n$$

y demostremos que:

$$\text{add } (\text{Succ } n) (\text{Succ } n) = \text{double } (\text{Succ } n)$$

Desarrollo:

$$\begin{aligned} \text{add } (\text{Succ } n) (\text{Succ } n) &= \text{Succ}(\text{add } n (\text{Succ } n)) && \text{por definición de } \text{add} \\ &= \text{Succ}(\text{Succ}(\text{add } n \text{ } n)) && \text{por definición de } \text{add} \\ &= \text{Succ}(\text{Succ}(\text{double } n)) && \text{por hipótesis inductiva} \\ &= \text{double}(\text{Succ } n) && \text{por definición de } \text{double} \end{aligned}$$

$\square$

2.9. Inducción sobre derivaciones

En el contexto de la verificación de propiedades de programas, la inducción estructural es una herramienta fundamental. Sin embargo, cuando el objetivo es demostrar propiedades relativas a sistemas de reglas (como la semántica operacional de lenguajes de programación) la simple inducción estructural sobre la forma de los términos resulta insuficiente. En estos casos, es más adecuado aplicar un principio más general: la *inducción sobre derivaciones*, también conocida como *inducción por reglas* [3].

Este estilo de razonamiento, que tiene un papel central en el enfoque desarrollado por Graham Hutton y Patrick Bahr en el artículo que guía nuestro estudio, consiste en realizar la inducción no sobre los objetos directamente, sino sobre las *derivaciones* que establecen sus propiedades, construidas conforme a un conjunto de reglas de inferencia específicas.

Formalización de las derivaciones

Formalmente, las derivaciones se organizan a partir de un conjunto de reglas de la forma:

$$\frac{}{x}$$

o bien

$$\frac{x_1 \quad \cdots \quad x_n}{x}$$

donde:

- Una regla con premisas vacías corresponde a un *axioma*, es decir, una verdad básica aceptada sin necesidad de justificación previa.
- Una regla con premisas no vacías describe cómo, a partir de ciertos supuestos $(x_1, \dots, x_n)$, se puede inferir una conclusión x .

Cada derivación puede visualizarse como un *árbol de prueba*: un árbol finito en el que cada nodo corresponde a la conclusión de una instancia de regla, y cuyas ramas representan las subderivaciones de las premisas asociadas.

Para formalizar este concepto, definimos un conjunto de *instancias de reglas*, donde cada instancia se describe como un par (X/y) , con X un conjunto finito de premisas y y una conclusión. En notación más familiar:

$$\bar{y} \quad \text{cuando} \quad X = \emptyset,$$

$$\frac{x_1, \dots, x_n}{y} \quad \text{cuando} \quad X = \{x_1, \dots, x_n\}.$$

El conjunto de todos los elementos que admiten una derivación conforme a las reglas dadas se denota I_R , y constituye el *mínimo conjunto R -cerrado*. Decimos que un conjunto Q es *R -cerrado* si cumple que, siempre que las premisas de una instancia de regla pertenezcan a Q , también su conclusión pertenece a Q [3].

Principio general de inducción por reglas

El principio general de la inducción por reglas establece que, para probar que una propiedad P se cumple en todos los elementos del conjunto I_R , basta con verificar que para cada instancia de regla (X/y) :

$$(\forall x \in X. P(x)) \implies P(y).$$

En otras palabras:

- Para los axiomas (donde $X = \emptyset$), simplemente se debe mostrar que $P(y)$ es verdadera.
- Para las reglas generales (donde X no es vacío), se debe demostrar que si la propiedad P es válida para todas las premisas $x_1, \dots, x_n$, entonces también lo es para la conclusión y .

Esta forma de inducción refleja la naturaleza constructiva de las derivaciones: cada nueva conclusión está justificada por derivaciones ya conocidas, asegurando que ninguna inferencia sea injustificada [3].

Inducción por Reglas en Semánticas Operacionales

La inducción por reglas resulta ser una herramienta particularmente adecuada para demostrar propiedades de semánticas operacionales definidas a través de reglas de inferencia. Como era de esperarse, en el estudio de semánticas presentadas mediante reglas, la aplicación del principio de inducción por reglas adopta una forma ligeramente diferente en la superficie, pero esencialmente equivalente a la descrita.

En este caso, en lugar de realizar una inducción sobre la estructura sintáctica de los programas, la inducción se efectúa sobre la *estructura de las derivaciones mismas*. Cada derivación de un juicio operacional (por ejemplo, de la forma $c \Rightarrow v$ en MINIB) se construye aplicando repetidamente reglas de inferencia. Así, la prueba de una propiedad se realiza demostrando que:

- La propiedad se cumple para los juicios derivados directamente mediante axiomas (caso base).
- La propiedad es preservada por las reglas: si la propiedad se cumple para los juicios en las premisas, entonces también se cumple para el juicio en la conclusión (paso inductivo).

De manera natural, este tipo de razonamiento garantiza que las propiedades de interés son *invariantes* respecto al proceso mismo de derivación de los juicios: cada paso lógico mantiene la validez de la propiedad [3].

Interpretación conceptual

Desde un punto de vista conceptual, la inducción por reglas se basa en dos principios fundamentales:

1. **Construcción finita:** Todas las derivaciones son finitas, es decir, no existen cadenas infinitas de aplicación de reglas descendentes³⁸.
2. **Preservación de propiedades:** Las propiedades se transmiten de las premisas a las conclusiones a través de las reglas.

En consecuencia, si cada instancia de aplicación de una regla respeta la propiedad deseada, toda derivación completa (sin importar su tamaño o complejidad) también respetará dicha propiedad. De este modo, la inducción por reglas no solo proporciona un esquema de prueba

robusto y sistemático, sino que también se exhibe como el marco conceptual natural para razonar sobre semánticas operacionales y otros sistemas definidos mediante reglas, tal como proponen Hutton y Bahr en su método.

2.10. Método de *Cálculo de Compiladores Correctos*

El método propuesto por Bahr y Hutton ofrece un enfoque sistemático para derivar compiladores correctos a partir de la semántica operacional de un lenguaje fuente. El objetivo central es transformar la semántica formal en un compilador que traduzca programas fuente hacia un lenguaje objetivo de más bajo nivel, junto con una máquina virtual capaz de ejecutar los programas compilados. Esta transformación garantiza que tanto el compilador como la máquina virtual sean correctos por construcción, eliminando la necesidad de posteriores pruebas de verificación.

El procedimiento se fundamenta en técnicas de razonamiento ecuacional y cálculo inductivo, permitiendo que la derivación se realice de manera accesible y rigurosa, sin necesidad de recurrir a herramientas conceptuales avanzadas como *continuaciones* o *defuncionalización*. A partir de una función de evaluación que especifica la semántica del lenguaje fuente, se formula una *especificación de corrección*, y mediante un proceso constructivo basado en inducción estructural, se calculan simultáneamente las definiciones del compilador y de la máquina virtual correspondiente.

Originalmente, esta metodología se orientó al cálculo de compiladores para máquinas virtuales basadas en pila, y demostró ser efectiva para una amplia gama de características de lenguajes, tales como excepciones, manejo de estado mutable, variantes del *Cálculo λ* , estructuras de control como bucles, no determinismo e interrupciones [52]. Posteriormente, el enfoque fue extendido para abarcar máquinas basadas en registros [53], lenguajes con tipos dependientes [52], semánticas no terminantes [54] y lenguajes concurrentes [55], consolidando su aplicabilidad a diferentes paradigmas de programación. Además, permitió reformular la verificación seminal de compiladores propuesta por McCarthy y Painter como un proceso de cálculo de compiladores, integrando de forma natural la verificación dentro del propio método de construcción [1].

Aunque los cálculos y ejemplos concretos se expresan en el lenguaje HASKELL, el enfoque se basa únicamente en conceptos esenciales como tipos recursivos, funciones recursivas y pruebas inductivas. Esta simplicidad metodológica, combinada con su generalidad, sitúa al

³⁸La ausencia de cadenas infinitas descendentes garantiza que todo proceso de construcción mediante reglas alcanza eventualmente un “piso” de axiomas [3].

método de Bahr y Hutton como una herramienta poderosa para el desarrollo riguroso y sistemático de compiladores correctos en diversos contextos y niveles de abstracción.

2.10.1. Síntesis y refinamiento del método

El método de Bahr y Hutton no surgió de manera inmediata en su forma actual, sino que fue resultado de un proceso de síntesis y refinamiento gradual. Inicialmente, el enfoque se estructuró en una secuencia de transformaciones sistemáticas a partir de la semántica de alto nivel del lenguaje fuente [1]:

1. Definir una función de evaluación de manera composicional;
2. Calcular una versión generalizada que utiliza una pila explícita;
3. Calcular una versión aún más generalizada que emplea continuaciones;
4. Defuncionalizar para obtener el compilador y la máquina virtual.

A partir de esta metodología escalonada, no solo se derivaron de forma sistemática los componentes principales: el compilador, el lenguaje destino y la máquina virtual; sino que también se establecieron las ecuaciones fundamentales que garantizan la corrección del proceso de compilación. Estas especificaciones de corrección son las siguientes:

$$exec (comp\ x)\ s = eval\ x\ :\ s \quad (2.4)$$

$$exec (comp'\ x\ c)\ s = exec\ c\ (eval\ x\ :\ s) \quad (2.5)$$

La primera ecuación establece que ejecutar el código compilado de una expresión sobre una pila dada produce el mismo resultado que evaluar la expresión y apilar su resultado. La segunda especifica que la ejecución del código compilado de una expresión, seguida de una continuación de código c , es equivalente a evaluar primero la expresión, apilar el valor y luego continuar ejecutando c [1].

Una característica importante es que el código generado no se presenta como una mera secuencia de instrucciones tradicionales, sino que adopta una representación en notación CPS³⁹. Aunque comparten el nombre, esta representación difiere de las semánticas en CPS, ya que aquí las continuaciones son estructuras simbólicas y no funciones de primer orden. Observando el proceso completo, se identificó que los pasos 2 y 3 corresponden a dos generalizaciones sucesivas del mismo principio fundamental: controlar explícitamente el flujo de evaluación. Asimismo, dado que las continuaciones introducidas en el paso 3 son eliminadas de inmediato en el paso 4 mediante defuncionalización, se vislumbró la posibilidad de simplificar el método.

³⁹El estilo de paso de continuaciones (*Continuation-Passing Style*, CPS) es una forma de escribir programas en la que las funciones, en lugar de devolver resultados directamente, reciben una función adicional (la continuación) que especifica los pasos a seguir con el resultado. Esto hace que el flujo de control y de datos sea completamente explícito, facilita transformaciones del programa como la eliminación de recursión de cola, y se relaciona estrechamente con el *Cálculo λ* [56].

En este sentido, es posible combinar directamente los pasos 2, 3 y 4 en una única fase, utilizando las especificaciones de corrección como guía para calcular simultáneamente el compilador y la máquina virtual. La esencia para esta simplificación radica en considerar los tipos involucrados: partiendo de la sintaxis (*Expr*), su semántica (*eval*), y el modelo de pila (*Stack*), se busca calcular el tipo de código (*Code*), las funciones de compilación (*comp* y *comp'*), y la función de ejecución (*exec*), de modo que satisfagan las especificaciones de corrección⁴⁰ establecidas.

Este proceso se desarrolla mediante una inducción estructural sobre las expresiones del lenguaje. Para cada caso de la sintaxis, se busca reescribir la forma $exec(comp' x c)$ hasta alcanzar una expresión del tipo $exec c' s$, donde c' representa el código resultante de compilar x continuado con c . La necesidad de nuevos constructores para el tipo *Code*, junto con sus respectivas interpretaciones en la función *exec*, emerge de manera natural a lo largo de este cálculo [1].

2.10.2. Metodología general

El refinamiento del método de Bahr y Hutton condujo a una metodología general más estructurada y directa para calcular compiladores correctos, superando así el esquema inicial basado en múltiples fases de generalización. Esta nueva metodología organiza el cálculo de compiladores a partir de especificaciones de corrección claras y semánticas de primer orden, permitiendo un proceso de derivación que es sistemático, modular y escalable. En términos generales, el proceso se desarrolla en tres grandes etapas:

Definir una semántica para el lenguaje fuente

Este objetivo se logra siguiendo tres subpasos:

- **Definir una función de evaluación:** Se formula una función *composicional*⁴¹ que asocia a cada expresión su significado, típicamente en términos de un valor final.
- **Defuncionalizar para producir una semántica de primer orden:** Con el fin de facilitar el cálculo posterior, cualquier aspecto funcional de alto orden, como continuaciones explícitas, se reexpresa mediante estructuras de datos de primer orden, asegurando así que la semántica permanezca operativamente concreta.
- **Reescribir la semántica en estilo relacional:** Finalmente, se traduce la *semántica funcional a un estilo relacional*⁴², que establece explícitamente cómo las configuraciones de ejecución (pila y demás estructuras necesarias) evolucionan paso a paso.

⁴⁰En este contexto, nos referimos a estas ecuaciones como especificaciones de corrección, ya que describen el comportamiento esperado de las funciones de compilaciones y ejecución.

⁴¹Se dice que una definición es *composicional* cuando el significado de una construcción sintáctica se determina exclusivamente a partir del significado de sus componentes inmediatos. En particular, la semántica de una operación como la adición se define únicamente en términos de la semántica de sus dos operandos [1].

⁴²Es decir, la transformación de una semántica funcional, basada en funciones de evaluación, a una semántica operacional que describe explícitamente las relaciones de transición entre configuraciones de ejecución en un sistema de transición.

Definir las especificaciones de corrección

Una vez establecida la semántica, el siguiente paso es formular las ecuaciones que servirán como especificaciones de corrección para el compilador y la máquina virtual. Estas especificaciones cumplen varios roles fundamentales:

- **Relación entre el compilador y la semántica:** Las especificaciones expresan que la ejecución del código compilado en la máquina virtual debe reflejar la evaluación semántica del programa fuente.
- **Componentes indefinidos:** Es habitual que la especificación incluya componentes adicionales aún indefinidos, tales como nuevas instrucciones de la máquina virtual o estructuras auxiliares, cuyo diseño se determinará durante el proceso de cálculo.
- **Configuraciones de máquina:** La máquina virtual opera sobre configuraciones que encapsulan el contexto de ejecución: una pila y cualquier otra estructura de datos pertinente, en función de los requisitos de la semántica.

Calcular las definiciones que satisfacen las especificaciones

Con la especificación de corrección en mano, el último paso consiste en calcular definiciones concretas para el compilador y la máquina virtual que satisfagan dichas especificaciones. Este cálculo sigue una estrategia de inducción estructural, guiada por principios bien establecidos:

- **Inducción sobre la sintaxis:** El cálculo procede por inducción estructural sobre las construcciones del lenguaje fuente, considerando cada forma sintáctica de manera sistemática.
- **Cálculo simultáneo de todos los componentes:** A medida que avanza la inducción, se calculan simultáneamente todas las partes desconocidas de la especificación: instrucciones, compilador y semántica de ejecución de la máquina.
- **Deseo de aplicar la hipótesis de inducción:** La fuerza impulsora del proceso es el *deseo de aplicar la hipótesis de inducción*, es decir, reescribir los términos en el lado derecho de las ecuaciones hasta que se ajusten a su lado izquierdo.

Este enfoque refinado no solo simplifica el proceso de cálculo, sino que también proporciona una fuerte garantía de corrección por construcción. Al derivar el compilador, el código objetivo y la máquina virtual directamente desde la semántica y las especificaciones, se elimina la necesidad de realizar pruebas de corrección posteriores, resultando en compiladores que son, en esencia, *correctos desde su cálculo* [1].

2.10.3. Aplicación práctica del método a MINIB

En esta sección comenzamos la aplicación del método a nuestro lenguaje MINIB. Como ya hemos formalizado su semántica operacional en la sección 2.5.1, nuestro primer paso consiste en trasladar esta semántica al entorno de trabajo en HASKELL, siguiendo el enfoque general

del método. Para ello, representamos los árboles de sintaxis abstracta y los valores semánticos como tipos de datos recursivos. A diferencia de los ejemplos iniciales en Bahr y Hutton, en nuestro caso no es necesario aplicar técnicas de defuncionalización ni reescritura semántica, dado que nuestra semántica ya ha sido establecida en términos de reglas operacionales. La semántica operacional de MINIB describe la evaluación de expresiones booleanas. En concordancia, definimos el tipo de dato que representa las expresiones de MINIB como sigue:

Listing 2.6: Definición del tipo de dato *Expr* (ASA) en MINIB

```
data Expr = Val Bool | Not Expr | And Expr Expr | Or Expr Expr
```

Ahora, procedemos a definir una función de evaluación *eval* para estas expresiones, respetando la estructura composicional del lenguaje:

Listing 2.7: Función de evaluación *eval* para expresiones booleanas en MINIB

```
eval :: Expr -> Bool
eval (Val b)      = b
eval (Not e)      = not (eval e)
eval (And e1 e2)  = eval e1 && eval e2
eval (Or e1 e2)   = eval e1 || eval e2
```

Esta definición refleja directamente la semántica de MINIB: cada construcción es evaluada en términos de las evaluaciones de sus componentes inmediatos, asegurando así la naturaleza composicional de *eval*.

Especificaciones de corrección y componentes del compilador

Una vez definida la función de evaluación para nuestro lenguaje MINIB, el siguiente paso consiste en establecer las especificaciones de corrección que guiarán el cálculo del compilador y de la máquina virtual. Las especificaciones de corrección capturan la relación entre la semántica del lenguaje fuente, el compilador y la ejecución del código en la máquina virtual. Siguiendo el enfoque general del método, las especificaciones de corrección para nuestro caso son análogas a las especificaciones 2.4 y 2.5.

Para dar soporte a estas especificaciones, introducimos los componentes básicos que definirán el compilador y la máquina virtual:

- Un tipo de dato *Stack* que representa la pila de valores booleanos:

Listing 2.8: Definición del sinónimo de tipo *Stack* (Pila de valores booleanos)

```
type Stack = [Bool]
```

- Un tipo de dato *Code* que representará el código para la máquina virtual. Este tipo se derivará a lo largo del proceso de cálculo, por lo que aún no fijamos su definición recursiva.
- Una función de compilación básica:

Listing 2.9: Función de compilación *comp* para convertir una expresión en código para la máquina virtual

```
comp :: Expr -> Code
```

que compila una expresión en código para la máquina virtual.

- Una función de compilación parametrizada:

Listing 2.10: Función de compilación auxiliar *comp'* con continuación

```
comp' :: Expr -> Code -> Code
```

que además recibe un código adicional como continuación.

- Una función de ejecución:

Listing 2.11: Función de ejecución *exec* para interpretar y ejecutar código sobre una pila de valores

```
exec :: Code -> Stack -> Stack
```

que interpreta y ejecuta un código sobre una pila de valores.

La metodología de cálculo procede definiendo estos componentes de manera que satisfagan las especificaciones de corrección anteriores. El objetivo es reescribir la ejecución del código compilado de una expresión hasta expresar la aplicación de la hipótesis de inducción, garantizando así la corrección por construcción.

Cálculo

Procedemos ahora a calcular las definiciones de los componentes *comp'*, *comp* y *exec* sobre la estructura de las expresiones del tipo *Expr*. Comenzamos analizando el caso base, correspondiente a las expresiones constantes de la forma *Val*.

El objetivo en cada caso será reescribir la expresión *exec (comp' x c) s* hasta obtener una expresión del tipo *exec c' s*, para algún código *c'* apropiadamente construido. A partir de esta forma, podremos derivar la definición correspondiente de *comp'*.

Val b

```
exec (comp' (Val b) c) s
= { especificacion 2.5 }
exec c (eval (Val b) : s)
= { definicion de eval }
exec c (b : s)
```

En este punto, el cálculo parece haberse estancado, ya que no es posible aplicar más definiciones conocidas directamente. Sin embargo, recordemos que nuestro objetivo es reescribir la expresión en la forma *exec c' s*, para algún nuevo código *c'*. En este caso, necesitamos encontrar un código *c'* tal que:

$$exec\ c'\ s = exec\ c\ (b : s)$$

No podemos simplemente definir la ejecución de c' como la de c con b en la pila, ya que b y c no están ligados en la definición de $exec$. La solución consiste en introducir un nuevo constructo en el tipo de dato *Code*, que encapsule tanto el valor b como el código c :

```
PUSH :: Bool -> Code -> Code
```

y extender la definición de la función de ejecución de la máquina virtual con una nueva ecuación:

```
exec (PUSH b c) s = exec c (b : s)
```

Es decir, ejecutar un código $PUSH\ b\ c$ equivale a insertar el valor b en la pila y continuar ejecutando el código c .

Con este nuevo constructo y su interpretación, completamos el cálculo de la siguiente forma:

```
exec c (b : s)
= { definicion de exec }
exec (PUSH b c) s
```

La expresión final ya tiene la forma deseada $exec\ c'\ s$, donde $c' = PUSH\ b\ c$. Por lo tanto, concluimos que la definición de $comp'$ en el caso de constantes es:

```
comp' (Val b) c = PUSH b c
```

Esta definición satisface la especificación de corrección en el caso base, asegurando así que el cálculo se mantiene correcto por construcción.

Not e

Continuamos el cálculo abordando el caso de la negación de una expresión, de la forma *Not e*. Procedemos de manera directa:

```
exec (comp' (Not e) c) s
= { especificacion 2.5 }
exec c (eval (Not e) : s)
= { definicion de eval }
exec c (not (eval e) : s)
= { definicion: exec (NOT c) (e:s) = exec c (not e:s) }
exec (NOT c) (eval e : s)
= { hipotesis de induccion para e }
exec (comp' e (NOT c)) s
```

En este cálculo, para poder reescribir $exec\ c\ (not\ (eval\ e) : s)$ en términos de una llamada recursiva sobre $exec$, necesitamos introducir un nuevo constructo en el tipo de dato *Code*:

```
NOT :: Code -> Code
```

y definir su ejecución en la máquina virtual con la siguiente ecuación:

```
exec (NOT c) (b : s) = exec c (not b : s)
```

Este nuevo constructo *NOT* representa la operación de aplicar la negación al valor en la cima de la pila, y luego continuar la ejecución con el código *c*. Con esta introducción, completamos el cálculo satisfactoriamente. Así, derivamos que la definición del compilador para el caso de *Not* es:

```
comp' (Not e) c = comp' e (NOT c)
```

And x y

Procedemos de manera sistemática:

```
exec (comp' (And x y) c) s
= { especificacion 2.5 }
exec c (eval (And x y) : s)
= { definicion de eval }
exec c (eval x && eval y : s)
```

En este punto, nuevamente nos encontramos en un estado en el cual no podemos aplicar más definiciones de forma directa. Sin embargo, como estamos realizando un cálculo inductivo, podemos utilizar las hipótesis de inducción disponibles para las subexpresiones *x* y *y*:

$$exec (comp' x c') s' = exec c' (eval x : s')$$

$$exec (comp' y c') s' = exec c' (eval y : s')$$

Para poder aplicar dichas hipótesis, es claro que necesitamos transformar la pila de manera que primero *eval x* y *eval y* sean empujados sobre *s*, es decir, lograr una expresión de la forma:

$$exec c' (eval y : eval x : s)$$

para algún código *c'*. En otras palabras, necesitamos resolver la ecuación:

$$exec c' (eval y : eval x : s) = exec c ((eval x \&\& eval y) : s)$$

Generalizando para valores arbitrarios *n* y *m*, obtenemos:

$$exec c' (m : n : s) = exec c ((n \&\& m) : s)$$

Sin embargo, no podemos simplemente usar esta ecuación como definición para *exec*, ya que *c* aparece libre en el cuerpo de la definición. La solución, como en los casos anteriores, es introducir un nuevo constructo en el tipo de dato *Code*:

$$AND :: Code \rightarrow Code$$

y definir su comportamiento en la máquina virtual de la siguiente manera:

```
exec (AND c) (m : n : s) = exec c (n \&\& m : s)
```

Este nuevo constructo *AND* modela la operación de conjunción lógica sobre los dos valores en la cima de la pila, continuando después con la ejecución del código *c*. Con este nuevo constructor, el resto del cálculo continúa de manera natural:

```

exec c (eval x && eval y : s)
= { definicion de exec }
exec (AND c) (eval y : eval x : s)
= { hipotesis de induccion para y }
exec (comp' y (AND c)) (eval x : s)
= { hipotesis de induccion para x }
exec (comp' x (comp' y (AND c))) s

```

Finalmente, el cálculo produce una expresión de la forma $exec\ c'\ s$, cumpliendo así con la especificación de corrección. Por tanto, derivamos que la definición del compilador para la conjunción es:

```

comp' (And x y) c = comp' x (comp' y (AND c))

```

Esta definición se obtiene de forma constructiva, asegurando la preservación de las propiedades de corrección establecidas inicialmente.

Or x y

Finalmente, procedemos con el cálculo para la disyunción $Or\ x\ y$. El procedimiento es completamente análogo al de la conjunción:

```

exec (comp' (Or x y) c) s
= { especificacion 2.5 }
exec c (eval (Or x y) : s)
= { definicion de eval }
exec c (eval x || eval y : s)

```

Nos encontramos nuevamente en la situación en la que deseamos aplicar las hipótesis de inducción para las subexpresiones x y y , es decir:

$$exec\ (comp'\ x\ c')\ s' = exec\ c'\ (eval\ x : s')$$

$$exec\ (comp'\ y\ c')\ s' = exec\ c'\ (eval\ y : s')$$

Para ello, buscamos transformar la pila al formato:

$$exec\ c'\ (eval\ y : eval\ x : s)$$

lo cual nos lleva a resolver la ecuación:

$$exec\ c'\ (m : n : s) = exec\ c\ ((n\ ||\ m) : s)$$

Como antes, para resolverlo introducimos un nuevo constructo en el tipo de datos *Code*:

$$OR :: Code \rightarrow Code$$

y definimos su semántica de ejecución de la siguiente manera:

```

exec (OR c) (m : n : s) = exec c (n || m : s)

```

Este constructo *OR* modela la operación de disyunción lógica sobre los dos valores superiores de la pila, continuando después con la ejecución del código *c*. Con este nuevo constructor, el cálculo se desarrolla como sigue:

```
exec c (eval x || eval y : s)
= { definicion de exec }
exec (OR c) (eval y : eval x : s)
= { hipotesis de induccion para y }
exec (comp' y (OR c)) (eval x : s)
= { hipotesis de induccion para x }
exec (comp' x (comp' y (OR c))) s
```

La expresión final tiene la forma *exec c' s*, cumpliendo así la especificación de corrección. De aquí, derivamos la definición del compilador para la disyunción:

```
comp' (Or x y) c = comp' x (comp' y (OR c))
```

Para concluir el desarrollo de nuestro compilador, abordamos ahora la función principal:

```
comp :: Expr -> Code
```

cuyo comportamiento correcto fue especificado previamente. En este caso, el cálculo es directo y no requiere inducción, sino simplemente a una reescritura basada en las especificaciones de corrección.

Comenzamos considerando el siguiente cálculo:

```
exec (comp e) s
= { especificacion 2.4 }
eval e : s
```

En este punto, el resultado *eval e : s* no tiene aún la forma *exec c s* que deseamos obtener. Para alcanzar dicha forma, introducimos un nuevo constructo en el tipo de dato *Code*:

```
HALT :: Code
```

y definimos su semántica de ejecución como:

```
exec HALT s = s
```

Este constructo *HALT* representa el final de la ejecución, sin realizar modificaciones adicionales sobre la pila.

Utilizando este nuevo constructor, podemos ahora transformar el cálculo:

```
eval e : s
= { definicion de exec }
exec HALT (eval e : s)
= { especificacion de comp' }
exec (comp' e HALT) s
```

Así, llegamos a la definición de *comp*:

```
comp e = comp' e HALT
```

Con esto, completamos de manera sistemática y correcta el desarrollo de nuestro compilador *comp*, fundamentado en cálculos inductivos sobre las especificaciones de corrección previamente establecidas.

Definiciones derivadas del compilador y máquina virtual

A continuación, agrupamos las definiciones de los componentes fundamentales derivados mediante el proceso de cálculo:

- El tipo de dato para el código de la máquina virtual es definido como:

Listing 2.12: Definición del tipo de dato *Code* para instrucciones de la máquina virtual

```
data Code = HALT
          | PUSH Bool Code
          | NOT Code
          | AND Code
          | OR Code
```

- El compilador principal *comp* se define como una especialización de *comp'* con la continuación *HALT*:

Listing 2.13: Definición principal del compilador *comp* usando *HALT* como continuación

```
comp :: Expr -> Code
comp e = comp' e HALT
```

- El compilador auxiliar *comp'* realiza la traducción estructural de las expresiones:

Listing 2.14: Definición estructural del compilador auxiliar *comp'* para MINIB

```
comp' :: Expr -> Code -> Code
comp' (Val n)      c = PUSH n c
comp' (Not e)      c = comp' e (NOT c)
comp' (And x y)    c = comp' x (comp' y (AND c))
comp' (Or x y)     c = comp' x (comp' y (OR c))
```

- Finalmente, la función *exec* proporciona la semántica operacional para la ejecución del código sobre una pila:

Listing 2.15: Definición de la función *exec* que interpreta y ejecuta código sobre una pila

```
exec :: Code -> Stack -> Stack
exec HALT      s          = s
exec (PUSH n c) s          = exec c (n : s)
exec (NOT c)   (b : s)     = exec c (not b : s)
exec (AND c)   (m : n : s) = exec c (n && m : s)
exec (OR c)    (m : n : s) = exec c (n || m : s)
```

Estas definiciones consolidan de manera completa el compilador correcto y la máquina virtual asociada para el lenguaje MINIB, cumpliendo con las especificaciones de corrección establecidas previamente.

Reflexión

Los resultados completos de la implementación del compilador correcto para el lenguaje MINIB, así como las definiciones de su máquina virtual, se encuentran disponibles en el material suplementario que acompaña a este trabajo (véase Apéndice A).

El proceso de cálculo presentado a lo largo de este capítulo refleja de manera precisa la filosofía que subyace en el enfoque moderno de cálculo de compiladores. A diferencia de métodos anteriores que dependían del uso de continuaciones y de técnicas de defuncionalización (conceptos tradicionalmente considerados avanzados y a menudo inaccesibles para una audiencia más amplia) el enfoque adoptado aquí se basa únicamente en técnicas elementales de razonamiento ecuacional, siguiendo un proceso de inducción estructural sobre la sintaxis del lenguaje fuente. Esta simplicidad metodológica permite un acceso más directo y claro al núcleo del problema, eliminando la necesidad de intermediaciones conceptuales que, si bien poderosas, aumentan la complejidad innecesariamente del proceso.

Más aún, el enfoque adoptado destaca por su carácter directo: no se parte de modificaciones de la semántica original del lenguaje para luego alcanzar indirectamente la corrección del compilador, sino que, desde el inicio, las especificaciones de corrección guían de forma explícita todo el proceso de derivación. Cada cálculo persigue satisfacer de manera inmediata las especificaciones establecidas, dotando al desarrollo de una orientación clara y unívoca hacia la obtención de un compilador correcto.

Sin embargo, es importante señalar que esta metodología, si bien elegante y efectiva, presenta desafíos para su mecanización formal. La estructura no estrictamente positiva de tipos como la pila (*Stack*) puede dificultar la formalización completa del desarrollo en asistentes de prueba como COQ o AGDA, una limitación que, aunque secundaria en el contexto del cálculo manual, se vuelve relevante en escenarios de verificación formal automatizada [1].

Finalmente, cabe destacar una reflexión fundamental: el proceso de cálculo seguido no solo permite derivar definiciones que satisfacen sus especificaciones, sino que, leído adecuadamente, constituye en sí mismo una prueba de corrección. Cada cálculo parte de la aplicación de una especificación y, si se interpreta adecuadamente, puede ser reorganizado como una prueba formal donde la definición obtenida es garantizada por la progresión lógica de los pasos dados. De esta forma, el enfoque adoptado reafirma la profunda conexión entre el acto de calcular y el de demostrar, cerrando así de manera sólida y fundamentada el marco teórico de este trabajo.

3 Caso de estudio 1: A λ ANIS

El lenguaje de programación que se explora en este caso de estudio ha sido diseñado para ser simple pero expresivo. A pesar de su simplicidad, permite representar construcciones fundamentales como operaciones aritméticas, pares ordenados con su proyección izquierda, estructuras condicionales, abstracciones lambda y aplicaciones de funciones. Las expresiones en este lenguaje se evalúan bajo un régimen *ansioso*, utilizando una estrategia de paso de parámetros por valor (*Call-by-Value*), en el cual los argumentos de una función se evalúan antes de su aplicación.

Sus construcciones son las siguientes:

- **Números enteros:** Representan valores numéricos utilizados en operaciones aritméticas básicas y comparaciones.
- **Valores booleanos:** Incluyen los valores lógicos “*True*” y “*False*”.
- **Variables:** Identificadores que asocian nombres a valores o expresiones.
- **Operaciones aritméticas:** Como la suma (+), la resta (-), la multiplicación (*) y la división (/).
- **Condicionales:** Expresiones del tipo *if* que permiten tomar decisiones basadas en evaluaciones lógicas.
- **Igualdad:** Comparaciones entre números enteros (=).
- **Pares ordenados:** Formados por dos elementos relacionados a partir de dos conjuntos en los que se establece un orden entre el primero y el segundo.
- **Proyección:** Operación para acceder al primer elemento del par ordenado (*fst*).
- **Funciones lambda:** Expresiones que representan funciones anónimas (*Lambda*).
- **Aplicaciones de función:** Evaluación de funciones con argumentos específicos.

Algunos ejemplos de expresiones sintácticamente válidas en este lenguaje, ilustrando su uso, observemos que nuestros operadores son prefijos¹:

¹La notación prefija elimina la necesidad de reglas de precedencia y asociatividad de operadores, reduciendo la complejidad del *parser* y, además se alinea con el paradigma funcional al reflejar la aplicación directa de funciones [7].

- $(+ \ 5 \ 3)$: Representa la suma de dos números enteros, 5 y 3.
- $(If \ True \ 1 \ 0)$: Una expresión condicional que devuelve 1 si la condición es verdadera y 0 en caso contrario.
- $(\lambda x \ (+ \ x \ 2))$: Una función lambda que toma un argumento x y devuelve el resultado de sumar x con 2.
- $((\lambda y \ (* \ y \ 4)) \ 3)$: Una aplicación de función donde se pasa el valor 3 a una función lambda que multiplica su argumento por 4.
- $(= \ 7 \ 7)$: Una comparación de igualdad que evalúa si 7 es igual a 7.
- $(fst \ (7, \ True))$: Una proyección del primer elemento del par ordenado.

3.1. Formalización de la sintaxis léxica

La sintaxis léxica de nuestro lenguaje se define comenzando por su alfabeto. Para esta formalización, incluimos números enteros, valores booleanos, identificadores, operadores aritméticos, delimitadores y palabras reservadas. El alfabeto queda especificado como:

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, =, (,), if, True, False, fst, \lambda\}$$

Recordemos que, los *tokens* del lenguaje corresponden a las unidades léxicas básicas que se pueden formar a partir de este alfabeto. En concreto, los tipos de *tokens* y sus correspondientes expresiones regulares:

- **Paréntesis y coma:** Delimitan las expresiones del lenguaje, incluyendo pares ordenados. Las únicas posibilidades son

$$(\ + \) \ + \ ,$$

- **Operadores aritméticos:** Incluyen las operaciones básicas de suma, resta, multiplicación y división:

$$+ \ + \ - \ + \ * \ + \ /$$

- **Igualdad:** La operación de comparación se realiza con el símbolo:

$$=$$

- **Números enteros:** Un número entero puede incluir un signo opcional seguido de una secuencia de dígitos. Definimos los conjuntos de dígitos como:

$$D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, \quad Z = D - \{0\}$$

Un número entero se describe como:

$$ZD^* \quad \text{o} \quad -ZD^*$$

Esto asegura que números como *01* no sean válidos.

- **Valores booleanos:** Representan los valores lógicos del lenguaje:

$True + False$

- **Identificadores:** Representan variables o nombres válidos, definidos como una letra inicial seguida de una combinación de letras:

$[a-zA-Z][a-zA-Z]^*$

- **Palabras reservadas:** Incluyen las construcciones propias del lenguaje:

$if + fst + Lambda$

- **Pares ordenados:** Representan expresiones del tipo (fst, snd) , delimitados por paréntesis y separados por una coma. Cada componente debe ser cualquier expresión válida del lenguaje. Usando *expr* como marcador de posición:

$(expr, expr)$

3.1.1. Sintaxis libre de contexto

En esta sección formalizamos la sintaxis libre de contexto de nuestro lenguaje, utilizando directamente una gramática en formato EBNF. Como se ha revisado previamente en la sección 2.4.1, una gramática libre de contexto permite describir con precisión la estructura sintáctica de un lenguaje. Sin embargo, para los fines de este estudio, optamos por emplear la notación EBNF, ya que ofrece ventajas prácticas sobre otras representaciones, como una mayor claridad y concisión gracias a sus construcciones explícitas para repetición, opción y agrupación.

La gramática que presentamos a continuación especifica cómo se combinan los *tokens* para formar expresiones válidas en nuestro lenguaje. Esta definición no solo delimita las estructuras sintácticas permitidas, sino que también evita ambigüedades, ya que cada expresión posee una única derivación posible y, por tanto, un único árbol sintáctico.

Sintaxis libre de contexto en EBNF de ALANIS

$$\langle S \rangle ::= \langle Expr \rangle$$

$$\begin{aligned} \langle Expr \rangle ::= & \langle Int \rangle \\ & | True \\ & | False \\ & | \langle Var \rangle \\ & | (fst \langle Expr \rangle) \\ & | (\langle Expr \rangle, \langle Expr \rangle) \\ & | (\langle Expr \rangle \langle Expr \rangle) \\ & | (if \langle Expr \rangle \langle Expr \rangle \langle Expr \rangle) \\ & | (+ \langle Expr \rangle \langle Expr \rangle) \\ & | (- \langle Expr \rangle \langle Expr \rangle) \\ & | (* \langle Expr \rangle \langle Expr \rangle) \\ & | (/ \langle Expr \rangle \langle Expr \rangle) \\ & | (= \langle Expr \rangle \langle Expr \rangle) \\ & | (Lambda \langle Var \rangle \langle Expr \rangle) \end{aligned}$$

$$\langle Int \rangle ::= \langle N \rangle \mid -\langle M \rangle$$

$$\langle D \rangle ::= 1 \mid \dots \mid 9$$

$$\langle N \rangle ::= 0 \mid \langle D \rangle \{ \langle N \rangle \}$$

$$\langle M \rangle ::= \langle D \rangle \{ \langle N \rangle \}$$

$$\langle Var \rangle ::= \langle Char \rangle \{ \langle Var \rangle \}$$

$$\langle Char \rangle ::= a \mid \dots \mid Z$$

En particular, se pone atención en la definición de operadores binarios, como la suma y la resta, que en esta gramática se estructuran de forma que se asocian siempre con exactamente dos operandos. Esta elección elimina derivaciones múltiples para una misma expresión y garantiza una interpretación unívoca de las operaciones.

Ejemplo de derivación

A continuación, se presenta un ejemplo que ilustra el proceso de derivación de expresiones válidas en nuestro lenguaje, conforme a las reglas de producción de la gramática.

Expresión: $(+ 5 -3)$

Derivación:

$$\begin{aligned}
 S &\rightarrow \\
 &< Expr > \rightarrow \\
 &(+ < Expr > < Expr >) \rightarrow \\
 &(+ < Int > < Expr >) \rightarrow \\
 &(+ < N > < Expr >) \rightarrow \\
 &(+ < D > \{ < N > \} < Expr >) \rightarrow \\
 &(+ 5 \{ < N > \} < Expr >) \rightarrow \\
 &(+ 5 < Int >) \rightarrow \\
 &(+ 5 - < M >) \rightarrow \\
 &(+ 5 - < D > \{ < N > \}) \rightarrow \\
 &(+ 5 - 3 \{ < N > \}) \rightarrow \\
 &(+ 5 - 3)
 \end{aligned}$$

3.2. Formalización de la sintaxis abstracta

En este apartado, se describe la transición de la *sintaxis concreta*, utilizada para representar programas tal como los escribiría un usuario, hacia la *sintaxis abstracta*, que organiza los elementos esenciales de las expresiones, eliminando ambigüedades y redundancias propias de su representación textual.

Ya se ha discutido cómo la sintaxis abstracta permite modelar las expresiones del lenguaje como estructuras jerárquicas que facilitan su análisis, manipulación y procesamiento dentro de un intérprete o compilador (véase la sección 2.4.2). Esta formalización se realiza mediante árboles de sintaxis abstracta, definidos en el marco teórico a través de reglas de inferencia que especifican cómo construir los nodos del árbol a partir de subexpresiones, de manera precisa y concisa.

Retomando el diseño de nuestro lenguaje en EBNF, cada alternativa en las reglas de producción de la gramática se corresponde con un tipo de expresión en el ASA. Estas expresiones pueden clasificarse en dos categorías principales: aquellas que representan valores atómicos² y aquellas que se construyen a partir de subexpresiones.

²En el contexto de los árboles de sintaxis abstracta, los valores atómicos son nodos terminales que no se componen de otras subestructuras, sino que representan directamente una constante, identificador o valor literal del lenguaje. Esta clasificación refleja que su interpretación semántica no depende de la evaluación de subexpresiones, sino que su significado está completamente determinado por su valor intrínseco [33].

Valores atómicos

Los valores atómicos son nodos con un único hijo, correspondiente al valor que encapsulan. En nuestro lenguaje, se representan mediante:

- **Num**: para valores enteros.
- **Bool**: para valores booleanos.
- **Var**: para variables.

Subexpresiones

Las subexpresiones son nodos que agrupan una o más expresiones como hijos. Entre ellas se encuentran:

- **Sumas** (*Add*): nodo con dos hijos, uno por cada sumando.
- **Restas** (*Sub*): dos hijos correspondientes a los operandos.
- **Multiplicaciones** (*Mul*): dos hijos para los factores.
- **Divisiones** (*Div*): estructura binaria análoga a las anteriores.
- **Igualdad** (*Eq*): dos hijos que representan las expresiones a comparar.
- **Proyección** (*Fst*): un solo hijo que representa el par del cual se extrae la primera componente.
- **Pares ordenados** (*Pair*): dos hijos que representan los elementos del par.
- **Condicionales** (*If*): tres hijos:
 1. Condición.
 2. Rama *then*.
 3. Rama *else*.
- **Funciones** (*Lambda*): contienen dos elementos:
 1. El nombre del parámetro (*p*).
 2. El cuerpo de la función (*c*), como expresión.
- **Aplicaciones de función** (*App*): dos hijos:
 1. La expresión que representa la función.
 2. La expresión que representa el argumento.

La formalización de las construcciones correspondientes a funciones (específicamente, variables, abstracciones y aplicaciones) se basa en las reglas sintácticas fundamentales del *Cálculo* λ , las cuales ya fueron introducidas en la sección 2.5.2. Estas reglas establecen que toda expresión válida puede ser construida inductivamente a partir de tres formas básicas: variables individuales, abstracciones $(\lambda x.M)$ y aplicaciones (MN) . En este enfoque, tales construcciones no se conciben como cadenas de texto, sino como árboles sintácticos abstractos que capturan la estructura jerárquica de las expresiones.

Dicha formulación puede entenderse como una instancia de lo que se conoce como *sintaxis abstracta de orden superior*, que permite representar funciones y sus componentes estructurales de forma precisa dentro del árbol. No obstante, para los fines de este trabajo no es necesario desarrollar un formalismo completo de dicha sintaxis. La representación a través de árboles de sintaxis abstracta que empleamos (con nodos etiquetados como *Var*, *Lambda* y *App*) resulta suficiente para el análisis sintáctico y semántico de estas expresiones dentro del lenguaje propuesto. Esta aproximación se alinea con el tratamiento estándar de la sintaxis abstracta en un estudio moderno de lenguajes de programación [19].

Para definir formalmente cuándo una expresión es un ASA, introducimos la relación:

$$a \text{ ASA}$$

que se lee como a es un ASA. A continuación, estableceremos las reglas que definen dicha relación, junto con su interpretación correspondiente.

Números

$Num(n)$ es un ASA si $n \in \mathbb{Z}$

$$\frac{n \in \mathbb{Z}}{Num(n) \text{ ASA}}$$

Booleanos

Aquí definimos el conjunto de valores booleanos $\mathbb{B} = \{0, 1\}$. Entonces, $Boolean(b)$ es un ASA si b es un valor booleano.

$$\frac{b \in \mathbb{B}}{Boolean(b) \text{ ASA}}$$

Variables

$Var(i)$ es un ASA si i es una cadena en el conjunto $\mathbb{S}$, de modo que $\mathbb{S} = \{s \mid s \in [a - zA - Z0 - 9]^*\}$.

$$\frac{i \in \mathbb{S}}{Var(i) \text{ ASA}}$$

Suma

$Add(i, d)$ es un ASA si tanto i como d son ASA también. Es decir, es un árbol con dos subárboles.

$$\frac{i \text{ ASA} \quad d \text{ ASA}}{Add(i, d) \text{ ASA}}$$

Resta

$Sub(i, d)$ es un ASA si tanto i como d son ASA también.

$$\frac{i \text{ ASA} \quad d \text{ ASA}}{Sub(i, d) \text{ ASA}}$$

La interpretación de las reglas restantes se extiende a las dos anteriores, entonces nos limitamos a exhibir únicamente la regla respectiva a cada etiqueta.

Multiplicación

$$\frac{i \text{ ASA} \quad d \text{ ASA}}{Mul(i, d) \text{ ASA}}$$

División

$$\frac{i \text{ ASA} \quad d \text{ ASA}}{Div(i, d) \text{ ASA}}$$

Igualdad

$$\frac{i \text{ ASA} \quad d \text{ ASA}}{Eq(i, d) \text{ ASA}}$$

Proyección

$Fst(p)$ es un ASA si la expresión que representa el par ordenado p también lo es. Este se trata entonces, de un árbol con un único subárbol.

$$\frac{p \text{ ASA}}{Fst(p) \text{ ASA}}$$

Pares ordenados

$Pair(f, s)$ es un ASA si tanto f y s son ASA. Es decir, es un árbol con dos subárboles.

$$\frac{f \text{ ASA} \quad s \text{ ASA}}{Pair(f, s) \text{ ASA}}$$

Condicionales

$If(c, t, e)$ es un ASA si c , t y e son ASA. Es decir, es un árbol con tres subárboles.

$$\frac{c \text{ ASA} \quad t \text{ ASA} \quad e \text{ ASA}}{If(c, t, e) \text{ ASA}}$$

Funciones

$Lambda(p, c)$ es un ASA si p es una cadena que representa el nombre de su parámetro y c un ASA que representa el cuerpo de la misma.

$$\frac{p \in \mathbb{S} \quad c \text{ ASA}}{Lambda(p, c) \text{ ASA}}$$

Aplicaciones de función

$App(f, a)$ es un ASA si la expresión f que representa la función que se aplicará es un ASA y la expresión a que representa el argumento con el cual se aplica dicha función también es un ASA .

$$\frac{f \text{ ASA} \quad a \text{ ASA}}{App(f, a) \text{ ASA}}$$

Esta formalización nos permite además, escribir de forma más compacta los árboles respecto a la representación gráfica. Usando la especificación de nuestra sintaxis concreta en EBNF, recordemos el ejemplo de la misma sección $(+ 5 -3)$. Obtengamos entonces la sintaxis abstracta de la expresión y su representación en formato de árbol, véase la figura 3.1.

Representación en formato plano

$$Add(Num(5), Num(-3))$$

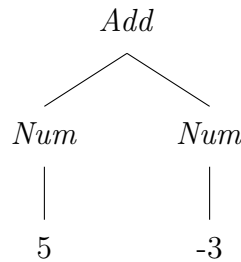


Figura 3.1: Árbol de sintaxis abstracta para $Add(Num(5), Num(-3))$.

3.2.1. Representación de la sintaxis abstracta

En esta sección se presenta la traducción de los árboles de sintaxis abstracta a una representación computacional en HASKELL. Este enfoque es consistente con el método propuesto por Bahr y Hutton, quienes emplean HASKELL debido a su capacidad para definir tipos recursivos de manera declarativa, lo que permite modelar directamente las construcciones de un lenguaje de programación en su sintaxis abstracta [1].

Cabe mencionar que, si bien en la sección 2.10 se revisó la aplicación del método al lenguaje MINIB, no se justificó de forma explícita la elección de HASKELL como lenguaje de implementación. Esta sección cumple con ese propósito general, válido para todas las fases de implementación consideradas a lo largo del trabajo, y refuerza la coherencia entre el modelo formal y su realización computacional.

El uso de HASKELL ofrece varias ventajas en el contexto de este trabajo:

- En primer lugar, permite implementar y probar directamente las reglas semánticas y los compiladores derivados como parte del proceso de desarrollo.
- En segundo lugar, los tipos algebraicos y la recursión estructural en HASKELL proporcionan una base clara y concisa para representar las construcciones del lenguaje, manteniendo una correspondencia directa con las definiciones formales introducidas en los ASA.

Para justificar la relación entre los árboles de sintaxis abstracta y un tipo inductivo como *Expr* en HASKELL, nos basamos en las propiedades estructurales que ambos comparten, así como en la correspondencia directa entre los elementos del ASA y los constructores de tipos algebraicos de datos. Esta relación permite representar de manera directa y natural la sintaxis abstracta de un lenguaje dentro de HASKELL, apoyándose en las características fundamentales del lenguaje funcional.

Justificación de la relación ASA y el tipo inductivo en HASKELL

- **Estructura recursiva:** Los ASA son estructuras jerárquicas y recursivas, donde cada nodo representa un operador o una construcción del lenguaje, y sus hijos corresponden a operandos o subexpresiones. HASKELL permite definir este tipo de estructuras mediante tipos algebraicos de datos recursivos, donde un constructo puede referirse a sí mismo. Esta propiedad es esencial para modelar expresiones compuestas de forma inductiva, como se muestra en la definición típica de tipos de datos para lenguajes embebidos en HASKELL [51].
- **Simetría estructural:** Existe una correspondencia directa entre la estructura de los nodos del ASA y los constructores de un tipo algebraico de datos. Esta simetría facilita tanto la construcción como el análisis de las expresiones, mediante técnicas como la recursión estructural (véase la sección 2.8). Por ejemplo:
 - Nodos con un único hijo, como $Fst(p)$, se representan en HASKELL mediante un constructo unario: $Fst\ Expr$.

- Nodos con múltiples hijos, como $If(c, t, e)$, se traducen a constructores con múltiples argumentos: $If\ Expr\ Expr\ Expr$.

Este diseño permite aprovechar herramientas del lenguaje como el *pattern matching* y las funciones recursivas para manipular y evaluar expresiones de forma declarativa y segura.

De lo anterior, la definición de un tipo de dato como *Expr* en HASKELL no solo refleja la estructura de un ASA, sino que proporciona una base natural para la implementación de nuestros intérpretes, compiladores y máquinas virtuales, tal como se propone en el desarrollo de lenguajes embebidos dentro del paradigma funcional [51].

Traducción de los árboles de sintaxis abstracta al tipo inductivo *Expr*

Definamos concretamente el tipo recursivo *Expr* en HASKELL, tal que, cada nodo de los ASA se corresponde con un constructo de *Expr*, manteniendo una correspondencia clara entre la representación formal y su implementación.

1. Expresiones literales

- **Números enteros** ($Num\ (n)$): Se representan mediante el constructo $Num\ Int$, que encapsula valores enteros. Esto permite modelar expresiones que contienen números literales en la sintaxis del lenguaje.
- **Valores booleanos** ($Boolean(b)$): Se representan mediante el constructo $Boolean\ Bool$ que encapsula valores booleanos.

2. **Variables:** Se representa con el constructo $Var\ String$, donde *String* almacena el nombre de la variable.

3. Operaciones aritméticas

- **Suma** ($Add(i, d)$): En el ASA, la suma toma dos subexpresiones (*i* y *d*). En HASKELL, esto se traduce directamente al constructo $Add\ Expr\ Expr$, donde ambos operados son expresiones del tipo *Expr*.
- **Resta, multiplicación y división** ($Sub(i, d)$, $Mul(i, d)$, $Div(i, d)$): Siguen el mismo patrón que la suma, utilizando los constructores *Sub*, *Mul* y *Div* que encapsulan dos subexpresiones.

4. Operadores relacionales

- **Igualdad** ($Eq(i, d)$): En el ASA, *Eq* representa la igualdad entre dos subexpresiones. En HASKELL, se presenta mediante el constructo $Eq\ Expr\ Expr$.

5. Estructuras de control

- **Condicionales** ($If(c, t, e)$): En el ASA, un condicional representa una condición (*c*) y selecciona entre dos ramas (*t* para verdadero y *e* para falso). En HASKELL esto se traduce al constructo $If\ Expr\ Expr\ Expr$.

6. Pares ordenados

- **Pares** ($Pair(f, s)$): Los pares constan de dos subexpresiones en el ASA. En HASKELL, el constructor $Pair\ Expr\ Expr$ modela esta estructura.
- **Proyección** ($Fst(p)$): Para obtener el primer elemento en el ASA, se utiliza el constructo Fst , que en HASKELL se traduce como $Fst\ Expr$.

7. Funciones

- **Abstracción** ($Lambda(i, c)$): En el ASA, una función anónima se define mediante un identificador (i) y el cuerpo (c). En HASKELL, se representa como $Lambda\ String\ Expr$, donde $String$ almacena el nombre del parámetro formal.

8. Aplicaciones de función

- **Aplicación** ($App(f, a)$): En el ASA, la aplicación toma una función (f) y un argumento (a). Esto se traduce a HASKELL como $App\ Expr\ Expr$.

Ahora, podemos agrupar estos constructores del tipo recursivo $Expr$ en sintaxis de HASKELL.

Listing 3.1: Definición del conjunto de expresiones $Expr$

```
data Expr = Num Int
          | Boolean Bool
          | Var String
          | Add Expr Expr
          | Sub Expr Expr
          | Mul Expr Expr
          | Div Expr Expr
          | Eq Expr Expr
          | If Expr Expr Expr
          | Pair Expr Expr
          | Fst Expr
          | Lambda String Expr
          | App Expr Expr
```

3.3. Formalización de la semántica operacional

En esta sección formalizamos la semántica operacional de nuestro lenguaje mediante un enfoque basado en la *semántica natural* o *de paso grande*. Este estilo, alineado con el tratamiento teórico previamente establecido, nos permite mantener la coherencia metodológica y avanzar hacia una caracterización clara y directa del comportamiento global de los programas en el lenguaje.

Optamos por este enfoque no sólo por su simplicidad, sino por su idoneidad para expresar evaluaciones completas como relaciones entre expresiones y sus resultados. Esta propiedad resulta especialmente ventajosa en nuestro caso de estudio, ya que facilita una exposición estructurada y comprensible de la semántica, sin comprometer la precisión formal.

La representación mediante reglas relacionales nos permite capturar el significado de cada construcción del lenguaje, manteniendo un marco formal robusto que sirve como base para razonamientos posteriores sobre propiedades del lenguaje o extensiones futuras.

Dado que los conceptos fundamentales del enfoque han sido establecidos en el marco teórico (véase la sección 2.5), en lo que sigue nos centraremos exclusivamente en la especificación formal del sistema de transición para ALANIS, siguiendo la estructura definida en la sintaxis abstracta del lenguaje.

Definición del sistema de transición

El sistema de transiciones de nuestro lenguaje se define a partir de los conceptos fundamentales de estados, estados iniciales, estados finales y una relación de transición. A continuación, se especifica cada uno de los componentes del sistema.

Estados

Los estados del sistema están definidos como las expresiones bien formadas del lenguaje en su representación de ASA. De este modo, el conjunto de estados E se formaliza como:

$$E = \{a \mid a \text{ es un ASA válido en la sintaxis abstracta del lenguaje}\}.$$

Este conjunto representa el puente entre la sintaxis formalizada previamente y la semántica operacional que aquí se describe.

Estados iniciales

Los estados iniciales I son todas las expresiones bien formadas del lenguaje. Por tanto, tenemos:

$$I = \{a \mid a \in E\}$$

En este caso, se cumple que $I = E$, ya que cualquier expresión bien formada del lenguaje puede ser utilizada como estado inicial.

Estados finales

Los estados finales F corresponden a los valores evaluados del lenguaje. Para este caso, los estados finales incluyen números enteros, valores booleanos y cerraduras de función. Formalmente, se definen como:

$$\begin{aligned} F = & \{Num_V(n) \mid n \in \mathbb{Z}\} \cup \{Boolean_V(b) \mid b \in \mathbb{B}\} \cup \{Pair_V(x, y) \mid x, y \in F\} \\ & \cup \{Closure(f, e) \mid f \text{ es una función y } e \text{ un ambiente}\} \\ & \cup \{Error \mid \text{la ejecución es interrumpida}\} \end{aligned}$$

En particular, el uso de cerraduras (*Closure*), estructuras que encapsulan una función con el ambiente donde fue definida, indica que el lenguaje emplea *alcance estático*. Esto significa que las variables libres de una función se resuelven con base en el contexto léxico en el que la función fue creada, y no en el ambiente desde el cual se aplica. La definición formal de cerradura puede consultarse en la sección 2.5.4.

Relación de transición

Para modelar el proceso de evaluación de expresiones en nuestro lenguaje, es necesario introducir formalmente el concepto de *ambiente de evaluación*. Un ambiente env es una estructura que representa el contexto léxico en el que se evalúan las expresiones, y contiene las asociaciones entre nombres de variables y valores evaluados. En nuestra formalización, cada nombre corresponde estrictamente a una variable del lenguaje, es decir, a una construcción de la forma $Var\ x_n$, donde $x_n \in \mathbb{S}$.

Sin embargo, en el contexto del ambiente, no se almacenan las construcciones Var explícitamente, sino únicamente los identificadores (los valores que éstas encapsulan). Esto se debe a que la construcción Var es principalmente útil dentro del programa fuente como representación de una variable, mientras que, en el ambiente de evaluación, basta con asociar directamente el identificador al valor correspondiente. Por tanto, un ambiente se define como una lista finita de pares de la forma (x_n, v) , donde x_n es un identificador y $v \in F$. Es decir:

$$env = [(x_1, v_1), (x_2, v_2), \dots, (x_n, v_n)]$$

El uso de ambientes es fundamental para evaluar expresiones con variables, particularmente en presencia de funciones. Esta estructura permite que nuestro lenguaje, en conjunto con el uso de cerraduras, mantenga el alcance estático (véase la sección 2.5.3).

Con esta base, definimos la relación de transición, denotada por $\Rightarrow$, que describe cómo una expresión se evalúa a un valor final bajo un ambiente dado. Esta relación constituye el núcleo de la semántica operacional del lenguaje y se especifica mediante un conjunto de reglas de inferencia.

Formalmente, se define como:

$$\Rightarrow \subseteq I \times Env \times F,$$

donde:

- I es el conjunto de expresiones bien formadas del lenguaje.
- Env es el conjunto de ambientes de evaluación definidos anteriormente.
- F es el conjunto de valores finales.

Así, una relación de la forma $(a, env, v) \in \Rightarrow$ indica que la expresión $a \in I$, evaluada en el ambiente $env \in Env$, produce el valor $v \in F$.

A partir de este punto empleamos la notación $\Rightarrow_\varepsilon$ que representa la relación de transición $\Rightarrow$ y permite resaltar el uso de ambientes en la evaluación (ε). Entonces, definimos formalmente las reglas de inferencia correspondientes a cada construcción del lenguaje, proporcionando una descripción detallada de su evaluación.

Números

Los ASA que representan números se reducen a sí mismos.

$$\frac{}{Num\ n \Rightarrow_\varepsilon Num_V\ n} \quad (NUM)$$

Booleanos

Similar con los números, los ASA que presentan valores booleanos se reducen a sí mismos.

$$\frac{}{Boolean\ b \Rightarrow_{\varepsilon} Boolean_V\ b} \quad (BOOL)$$

Variables

La regla para las variables establece que una expresión de la forma $Var\ x$ se evalúa al valor asociado a x dentro del ambiente de evaluación ε . Para expresar esta asociación, utilizamos una función auxiliar *lookup*, que toma como argumentos un nombre x y un ambiente ε , y devuelve el valor correspondiente si dicho nombre se encuentra mapeado en el ambiente.

En esta formalización adoptamos el juicio $lookup\ x\ \varepsilon\ existe$, siguiendo el enfoque presentado en [1]. Este juicio expresa de manera general que la función de búsqueda *lookup* tiene éxito al encontrar una asociación para el identificador x en el ambiente ε , sin requerir una definición concreta de la función ni del valor devuelto. Este nivel de abstracción permite modelar la semántica del lenguaje sin comprometerse con detalles de implementación específicos.

Bajo esta condición, la evaluación de variables se representa mediante la siguiente regla de inferencia:

$$\frac{lookup\ x\ \varepsilon\ existe}{Var\ x \Rightarrow_{\varepsilon} lookup\ x\ \varepsilon} \quad (VAR)$$

Cabe mencionar que no abordamos explícitamente en esta formalización el caso en que la función *lookup* no encuentra el identificador x en el ambiente, es decir, no modelamos aquí la semántica de fallo para esta construcción.

Suma

La regla de suma establece que, para evaluar la expresión $Add\ x\ y$, es necesario evaluar primero las subexpresiones x y y en el mismo ambiente ε . Si ambas subexpresiones se evalúan correctamente a valores numéricos, el resultado será la suma de estos valores en el conjunto de los números enteros ($\mathbb{Z}$).

$$\frac{x \Rightarrow_{\varepsilon} Num_V(n) \quad y \Rightarrow_{\varepsilon} Num_V(m)}{Add\ x\ y \Rightarrow_{\varepsilon} Num_V(n +_{\mathbb{Z}} m)} \quad (ADD)$$

Es importante mencionar que, al emplear valores del tipo $Num_V(n)$ con $n \in \mathbb{Z}$, estamos dando un significado denotativo a los números en nuestro lenguaje. Esta interpretación establece una correspondencia entre los programas y un dominio matemático bien definido: el conjunto de los números enteros. De este modo, se garantiza que las operaciones aritméticas dentro del lenguaje tengan un significado consistente y formalmente fundamentado.

División

La regla de división (*Div*) sigue un razonamiento similar al de la suma, evaluando primero las subexpresiones x y y . Si ambas se evalúan a valores numéricos, el resultado será la división entera de estos valores. Sin embargo, existe un caso excepcional que debe manejarse explícitamente: cuando el denominador es cero ($y \Rightarrow_\varepsilon \text{Num}_V 0$). En este caso, la evaluación debe producir un error en lugar de delegar la operación al lenguaje anfitrión. Existen dos razones fundamentales para este manejo explícito del error:

1. **Evitar la propagación del error al lenguaje anfitrión.** Al capturar el error dentro de nuestra semántica operacional, aseguramos que la máquina virtual y evaluador lo diagnostiquen correctamente sin depender de la implementación subyacente del lenguaje anfitrión.
2. **Preservar la definición de la división entera.** En algunos entornos de implementación, la división entre cero está indefinida o genera excepciones. Al representar explícitamente este caso en nuestra semántica, garantizamos un comportamiento bien definido dentro de nuestro sistema.

La siguiente regla refleja este caso particular:

$$\frac{x \Rightarrow_\varepsilon \text{Num}_V(n) \quad m = 0 \quad y \Rightarrow_\varepsilon \text{Num}_V(m)}{\text{Div } x \ y \Rightarrow_\varepsilon \text{Error}} \quad (\text{DIV-ZERO})$$

Dado que, en este caso no se puede obtener un resultado numérico válido, es natural que la evaluación produzca un error. Por esta razón, hemos incluido el estado final *Error* en el conjunto de valores finales F .

Por otro lado, en el caso general donde el denominador es distinto de cero ($m \neq 0$), podemos definir una regla análoga que exprese el comportamiento esperado de la división entera.

$$\frac{x \Rightarrow_\varepsilon \text{Num}_V(n) \quad m \neq 0 \quad y \Rightarrow_\varepsilon \text{Num}_V(m)}{\text{Div } x \ y \Rightarrow_\varepsilon \text{Num}_V(n/\mathbb{Z}m)} \quad (\text{DIV})$$

Las reglas del resto de operaciones aritméticas son análogas a la suma y división, pero introduciendo el operador correspondiente. Es decir, para la construcción *Sub* se usa el operador $-\mathbb{Z}$ y para la construcción *Mul* el operador $\times_{\mathbb{Z}}$.

Pares ordenados

La regla para la construcción de un par evalúa ambos componentes en el ambiente dado y los combina en un valor de tipo Pair_V . Esta evaluación previa refleja el régimen de evaluación ansioso empleado en nuestro lenguaje, donde los componentes de una construcción se evalúan de forma inmediata. Este comportamiento es coherente con el tratamiento de otras construcciones, como las operaciones aritméticas.

$$\frac{x \Rightarrow_\varepsilon f \quad y \Rightarrow_\varepsilon s}{\text{Pair } x \ y \Rightarrow_\varepsilon \text{Pair}_V f \ s} \quad (\text{PAIR})$$

Proyección

La regla para *Fst* indica que, si una expresión se evalúa a un valor par (*Pair f s*), la proyección del primer elemento del par resulta en el valor *f*.

$$\frac{x \Rightarrow_{\varepsilon} \text{Pair}_V f s}{\text{Fst } x \Rightarrow_{\varepsilon} f} \quad (\text{FST})$$

Condicional

Similar al caso de la división, la construcción condicional (*If*) requiere definir dos reglas, una para el caso en que la condición se evalúe como verdadera y otra para cuando se evalúe como falsa. Estas reglas establecen la relación entre el valor de la condición y la evaluación de la rama correspondiente.

Cuando la condición es verdadera, la evaluación debe continuar con la rama *then*, de modo que el resultado de la expresión condicional sea el mismo que el de esta rama. La siguiente regla formaliza este comportamiento:

$$\frac{x \Rightarrow_{\varepsilon} \text{Boolean}_V(\text{True}) \quad y \Rightarrow_{\varepsilon} v}{\text{If } x \text{ } y \text{ } z \Rightarrow_{\varepsilon} v} \quad (\text{IF-TRUE})$$

Análogamente, cuando la condición es falsa, la evaluación debe continuar con la rama *else*, asegurando que el resultado de la expresión condicional sea el mismo que el de esta rama. La regla correspondiente captura esta relación de manera explícita:

$$\frac{x \Rightarrow_{\varepsilon} \text{Boolean}_V(\text{False}) \quad z \Rightarrow_{\varepsilon} v}{\text{If } x \text{ } y \text{ } z \Rightarrow_{\varepsilon} v} \quad (\text{IF-FALSE})$$

Igualdad

La regla para la comparación de igualdad se define como sigue: si ambas subexpresiones son valores numéricos, el resultado será un valor booleano que indica si son iguales.

$$\frac{x \Rightarrow_{\varepsilon} \text{Num}_V(n) \quad y \Rightarrow_{\varepsilon} \text{Num}_V(m)}{\text{Eq } x \text{ } y \Rightarrow_{\varepsilon} \text{Boolean}_V(n =_{\mathbb{Z}} m)} \quad (\text{EQ})$$

Funciones

La regla de evaluación para una expresión *Lambda* produce una cerradura que incluye la función misma y el ambiente en el que se definió, preservando así el alcance estático.

$$\frac{}{\text{Lambda } x \text{ } y \Rightarrow_{\varepsilon} \text{Closure } (\text{Lambda } x \text{ } y) \varepsilon} \quad (\text{LAMBDA})$$

Aplicaciones de función

Para la aplicación de funciones (*App*), se evalúa la función y su argumento. Si la función es una cerradura, el cuerpo de la función se evalúa en un nuevo ambiente que incluye el argumento asociado al parámetro formal.

$$\frac{x \Rightarrow_{\varepsilon} \text{Closure}(\text{Lambda } a \ c) \ \varepsilon' \quad y \Rightarrow_{\varepsilon} v \quad c \Rightarrow_{\varepsilon'[a \leftarrow v]} cv}{\text{App } x \ y \Rightarrow_{\varepsilon} cv} \quad (\text{APP})$$

Ahora que hemos formalizado la semántica operacional de nuestro lenguaje, podemos continuar con la aplicación del método.

3.4. Introducción al método

En esta sección, iniciamos la formulación y definición de los componentes principales necesarios para aplicar el método de *Cálculo de Compiladores Correctos* [1]. Los elementos fundamentales incluyen: el dominio de valores (F), definido en la formalización semántica de nuestro lenguaje; el conjunto de expresiones iniciales admitidas por el lenguaje (I); y el ambiente de evaluación (Env). Estos componentes deben trasladarse adecuadamente a HASKELL, para proceder con la implementación del compilador.

3.4.1. Representación en HASKELL

Conjunto de expresiones (I)

El conjunto de expresiones admitidas por el lenguaje, I , ya se encuentra representado mediante la definición de tipo algebraico *Expr* en HASKELL definido en 3.1.

Conjunto de valores (F)

El dominio de valores F , que define los estados finales de nuestro lenguaje, se traduce a HASKELL como sigue:

Listing 3.2: Definición del tipo de dato *Value*, que modela el dominio de valores finales F en $\lambda\lambda$ ANIS

```
data Value = NumV Int
           | BooleanV Bool
           | Closure Expr Env
           | PairV Value Value
           | Error
```

Esta definición incluye los tipos de valores permitidos en nuestro lenguaje: números enteros, valores booleanos, cerraduras (*Closures*) para las funciones, pares ordenados y error en el caso de la división entre cero.

Ambiente de evaluación (*Env*)

Dado que nuestro lenguaje se basa en el *Cálculo* λ y estrategia de paso de parámetros *Call-by-Value*, los *bindings* producidos por las aplicaciones de función deben exhibir dicha ligadura. En *HASKELL*, esta relación puede modelarse utilizando una lista de pares ordenados, como se muestra a continuación:

Listing 3.3: Definición del tipo *Env* que representa el ambiente de evaluación

```
type Env = [(String, Value)]
```

En esta representación, cada par (x, v) en la lista asocia una variable x con su correspondiente valor v .

Relación de transición

Una vez representados los conjuntos E , F y Env en el lenguaje anfitrión, el siguiente paso consiste en definir la relación de transición que permita modelar el proceso de evaluación. Basándonos en el enfoque de los autores, establecemos una función de evaluación, *eval*, que toma como entrada una expresión (*Expr*) perteneciente al conjunto de estados iniciales I , junto con un estado del ambiente de evaluación (*Env*), para producir un valor o estado final (F) [1]. A continuación, presentamos la definición de la función *eval*:

Listing 3.4: Definición de la función *eval* para modelar la relación de transición en λ ANIS

```

1 eval :: Expr -> Env -> Value
2 eval (Num n) env      = NumV n
3 eval (Boolean b) env = BooleanV b
4 eval (Var id) env     = lookup id env
5 eval (Add x y) env = case eval x env of
6     NumV n -> case eval y env of
7         NumV m -> NumV (n + m)
8 eval (Sub x y) env = case eval x env of
9     NumV n -> case eval y env of
10    NumV m -> NumV (n - m)
11 eval (Mul x y) env = case eval x env of
12    NumV n -> case eval y env of
13    NumV m -> NumV (n * m)
14 eval (Div x y) env = case eval x env of
15    NumV n -> case eval y env of
16        NumV 0 -> Error
17        NumV m -> NumV (n `div` m)
18 eval (Eq x y) env = case eval x env of
19    NumV n -> case eval y env of
20        NumV m -> BooleanV (n == m)
21 eval (Pair x y) env = PairV (eval x env, eval y env)
22 eval (Fst x) env = case eval x env of
23     | PairV f s -> f
24 eval (If c t e) env = case eval c env of
25     b -> if b == BooleanV True

```

```

26                                     then eval t env
27                                     else eval e env
28 eval (Lambda x y) env = Closure (Lambda x y) env
29 eval (App x y) env = case eval x env of
30     Closure (Lambda a c) env' -> eval c ((a, eval y env):env')

```

Esta implementación refleja la semántica operacional del lenguaje ALANIS asegurando una correspondencia entre la especificación formal y su implementación práctica. Es importante mencionar que:

- En el caso de valores literales (*Num* y *Boolean*), la evaluación simplemente retorna el mismo valor, pero encapsulado en un constructo del tipo *Value* (líneas 2–3).
- Para variables (*Var*), la función realiza una búsqueda en el ambiente de evaluación utilizando *lookup* (línea 4).
- Las construcciones aritméticas, como *Add*, evalúan recursivamente ambos operandos y aplican la operación correspondiente (líneas 5–7). Las demás operaciones (*Sub*, *Mul* y *Div*) siguen un patrón similar (líneas 8–17), garantizando la correcta propagación de errores en caso de valores no numéricos, como se observa explícitamente en el caso de división entre cero (línea 16).
- La comparación por igualdad entre enteros (*Eq*) sigue el mismo patrón que las operaciones aritméticas: primero evalúa ambos operandos y luego compara sus valores numéricos resultantes (líneas 18–20).
- Las construcciones de pares ordenados (*Pair*) generan un valor que combina las evaluaciones de sus elementos (línea 21), mientras que *Fst* extrae el primer elemento del par (líneas 22–23).
- En las expresiones condicionales (*If*), se evalúa la condición y, dependiendo de su valor booleano, se evalúa la rama correspondiente (líneas 24–27), delegando este manejo de ejecución a HASKELL.
- Las abstracciones lambda (*Lambda*) se traducen en cerraduras (*Closures*), capturando el ambiente en el que se definen (línea 28).
- Las aplicaciones de funciones (*App*) desempacan la cerradura y evalúan el cuerpo de la función con su ambiente léxico extendido, donde la variable vinculada apunta al valor del argumento real evaluado (líneas 29–30).

Hasta este punto, hemos hecho referencia a la función auxiliar *lookup*, pero no la hemos definido formalmente. Intuitivamente, su propósito es realizar la búsqueda de un valor asociado a un identificador de variable dentro del ambiente de evaluación. Como vimos, tanto en las reglas de la semántica natural como en la definición de la función de evaluación (*eval*), *lookup* toma dos argumentos: el identificador de la variable (*s*) y el ambiente de evaluación. A continuación, definimos el cuerpo de esta función en HASKELL:

Listing 3.5: Función auxiliar *lookup* para búsqueda de identificadores en el ambiente

```

lookup :: String -> Env -> Value
lookup s [] = error ("Variable no encontrada:" ++ s)
lookup s ((k, v) : xs)
    | s == k = v
    | otherwise = lookup s xs

```

La función *lookup* es recursiva y está diseñada para recorrer el ambiente de evaluación, representado como una lista de pares ordenados, buscando el identificador especificado. Su comportamiento puede describirse de la siguiente manera:

- **Caso base:** Si la lista del ambiente está vacía (`[]`), significa que no se encontró ninguna variable que coincida con el identificador buscado. En este caso, se genera un error con el mensaje *"Variable no encontrada"* seguido del nombre del identificador.
- **Caso recursivo:** Si el ambiente no está vacío, se toma el primer par (k, v) de la lista. Si el identificador buscado s coincide con la clave k , se devuelve el valor asociado v . De lo contrario, se llama recursivamente a *lookup* con el resto de la lista (xs).

Al tratarse de una función recursiva, es obligatorio establecer un caso base que funcione como cláusula de escape. En este contexto, el caso base se define para la lista vacía (`[]`), lo que asegura que la función sea total y que cualquier búsqueda que no tenga un resultado válido se maneje explícitamente mediante la generación de un error [51].

Con esta implementación, garantizamos que *lookup* cumple su propósito como una herramienta esencial para la evaluación de expresiones en nuestro lenguaje, permitiendo la correcta resolución de variables dentro del ambiente de evaluación.

3.4.2. Especificación

Para los propósitos del cálculo, es necesario recordar los tipos de las funciones de compilación y de la máquina virtual. Sin embargo, se requiere un ajuste: nuestra formalización semántica incluye un ambiente de evaluación, por lo que la configuración de la máquina debe incorporar todos los elementos relevantes. Por ello, se introduce un nuevo tipo generalizado, *Conf* (véase el *listing* 3.6), que refleja con mayor precisión la estructura necesaria para la evaluación semántica.

```

comp' :: Expr -> Code -> Code
exec  :: Code -> Conf -> Conf

```

Estas funciones ya fueron descritas en la revisión general del método (véase la sección 2.10). En particular:

- *comp'* recibe como entrada un elemento del conjunto de estados iniciales (I), que en *HASKELL* corresponde al tipo *Expr*, junto con una continuación de código.
- *exec* que modela el comportamiento de la máquina virtual de pila, toma como entrada un código y una configuración inicial para generar una configuración final.

Dado que la formalización semántica de nuestro lenguaje ALANIS incorpora explícitamente un ambiente de evaluación, es fundamental reflejar esta decisión en la configuración de la máquina virtual. Recordemos que la configuración de una máquina virtual representa el estado completo del sistema durante la evaluación, e incluye componentes esenciales como la pila y el ambiente. En nuestro caso, definimos la configuración como:

Listing 3.6: Definición de la configuración de la máquina virtual

```
type Conf = (Stack, Env)
```

En esta definición, el componente *Stack* es fundamental, pues optamos por una máquina virtual de pila. Este modelo nos permite hacer explícito el procesamiento de valores, ya que la pila almacena de forma ordenada los valores calculados, facilitando su recuperación y manipulación durante la evaluación de las expresiones [1]. Siguiendo la metodología de definición parcial y extensible de los componentes, en lugar de definir *Stack* directamente como una lista del tipo *Value*, empleamos una definición más general que posibilita su extensión a lo largo del cálculo. Por ello, definimos:

Listing 3.7: Definición del tipo de pila *Stack* y su componente elemental *Elem*

```
data Elem = VAL Value
type Stack = [Elem]
```

Esta abstracción nos proporciona la flexibilidad necesaria para incorporar elementos adicionales en la pila durante el proceso de cálculo, sin comprometer la claridad ni la consistencia del diseño inicial. Además, esta generalización permite que el método pueda adaptarse fácilmente a lenguajes más complejos o con características adicionales que requieran una estructura de pila más sofisticada.

Especificación de corrección

La especificación para *comp'* es análoga a la propuesta original de los autores [1], por lo que resulta esencial definir previamente los componentes relevantes de la máquina virtual. La corrección de esta función se expresa mediante la siguiente ecuación:

$$exec (comp' x c) (s, e) = exec c (VAL v : s, e) \quad \text{si} \quad x \Rightarrow_{\varepsilon} v \quad (3.1)$$

Esta se interpreta del siguiente modo:

- **Precondición:** $x \Rightarrow_{\varepsilon} v$ indica que la expresión x , evaluada en el ambiente ε , produce correctamente el valor v . Esta condición es indispensable, pues la corrección solo se garantiza para expresiones cuya evaluación termina.
- **Lado izquierdo:** $exec (comp' x c) (s, e)$ representa la ejecución del código generado por *comp'* para x , seguido de la continuación c , partiendo de una configuración con pila s y ambiente e .
- **Lado derecho:** $exec c (VAL v : s, e)$ indica que, una vez evaluada la expresión x y obtenido el valor v , este se coloca en la cima de la pila antes de continuar con la ejecución del código c , manteniendo intacto el ambiente e .

En concreto, la especificación³ afirma que compilar y ejecutar x , seguido de c , es semánticamente equivalente a evaluar x , colocar su resultado en la pila, y continuar con c . Esta formulación sigue el enfoque empleado en el lenguaje MINIB, cuyos cálculos se presentan en la sección 2.10.3.

El tipo de valores producidos por el evaluador incluye cerraduras de función y pares ordenados. Las cerraduras encapsulan tanto el cuerpo de la función como el ambiente en el que fueron definidas; los pares agrupan valores. Para que la máquina virtual opere correctamente, es necesario que ambos constructos contengan representaciones compiladas del programa. En el evaluador, las expresiones se procesan en su forma sintáctica I , mientras que la máquina virtual espera expresiones ya traducidas al tipo *Code*.

Para asegurar esta correspondencia, los autores proponen una transformación que sustituye la componente *Expr* de las cerraduras y los pares por su versión compilada [1]. Esto da lugar a una variante del tipo de valores, denominada *Value'*:

Listing 3.8: Definición del tipo de valores *Value'* adaptado para la máquina virtual

```
data Value' = NumV' Int
           | BooleanV' Bool
           | Closure' Code Env'
           | PairV' Value' Value'
           | Error'
type Env' = [Value']
```

Con esta definición, tanto las cerraduras como los pares se almacenan en la pila y el ambiente con sus componentes internos ya compilados. En consecuencia, también se adapta la configuración de la máquina virtual:

Listing 3.9: Configuración y pila de la máquina virtual adaptadas al dominio *Value'*

```
data Elem = VAL Value'
type Stack = [Elem]
type Conf = (Stack, Env')
```

Sin embargo, esta adaptación genera una incompatibilidad con la especificación 3.1, la cual fue formulada sobre valores del tipo *Value*. Esta discrepancia surge porque *eval* produce valores cuyas cerraduras y pares contienen expresiones sin compilar, mientras que *exec* opera sobre valores donde dichas expresiones ya han sido compiladas. Por tanto, la especificación no resulta correctamente tipificada, ya que el valor v producido por *eval* no es directamente compatible con lo que espera *exec*.

Para resolver esta inconsistencia, es necesario definir una función de conversión *conv* que mapee valores del tipo *Value* al tipo *Value'* [1]. Esta función debe ser total y cubrir exhaustivamente todos los constructos de *Value*. Las traducciones para *Num_V*, *Boolean_V* y *Error* son directas, pero los casos correspondientes a *Pair_V* y *Closure* requieren una conversión

³En esta especificación, el símbolo ε denota el ambiente utilizado por la semántica formal. Por otro lado, el símbolo e representa el ambiente léxico de la máquina virtual, el cual forma parte de su configuración en tiempo de ejecución. Dado que este ambiente se incluye explícitamente en la configuración (s, e) , su rol es reflejar en la máquina virtual la información contextual necesaria para interpretar variables, análogamente a como lo hace ε en la semántica formal.

recursiva y una compilación explícita de sus componentes, por lo que sus definiciones se postergan para una etapa posterior del desarrollo.

Listing 3.10: Definición parcial de la función *conv* para transformar valores evaluados en valores ejecutables

```
conv :: Value -> Value'
conv (NumV n)           = NumV' n
conv (BooleanV b)       = BooleanV' b
conv Error              = Error'
conv (PairV x y)         = ??? -- Por definir
conv (Closure (Lambda x y) e) = ??? -- Por definir
```

Este enfoque asegura una transición consistente entre los dos tipos de valores, aunque aún debemos extender *conv* a los ambientes de evaluación, mapeando la función sobre los valores de la lista de pares ordenados.

Listing 3.11: Definición de la función de conversión *conv_E* para transformar ambientes de evaluación

```
convE :: Env -> Env'
convE = map (\(x, y) -> (x, conv y))
```

Para hacer explícita la conversión del tipo *Value*, podemos modificar la especificación de corrección de *comp'* de forma directa, introduciendo la conversión de valores tanto al ambiente como al valor asociado a una expresión [1].

$$exec (comp' x c) (s, conv_E e) = exec c (VAL (conv v) : s, conv_E e) \text{ si } x \Rightarrow_\epsilon v \quad (3.2)$$

3.4.3. Cálculo

Con base en la especificación de corrección, procedemos a derivar las definiciones del compilador y la máquina virtual mediante inducción estructural sobre la relación de evaluación $x \Rightarrow_\epsilon v$. Nuestro objetivo en cada caso será transformar la expresión:

$$exec (comp' x c) (s, conv_E e)$$

en la forma:

$$exec c' (s, conv_E e)$$

para algún código c' , lo cual nos permitirá concluir que $comp' x c = c'$ satisface la especificación [1].

Este proceso sigue el enfoque ilustrado en la sección 2.10, y a lo largo del cálculo introduciremos nuevos constructos en los tipos de datos *Code* y *Stack*, así como definiciones adicionales para las funciones *exec* y *comp'*. También completaremos la definición de la función de conversión *conv*.

Los casos base (*Num*, *Boolean* y *Var*) resultan directos, y establecen los fundamentos sobre los cuales se construyen los casos inductivos más complejos.

Num n

```

exec (comp' (Num n) c) (s, convE e)
= { especificacion 3.2 }
exec c (VAL (conv (NumV n)) : s, convE e)
= { definicion de conv }
exec c (VAL (NumV' n) : s, convE e)
= { definicion:
exec (PUSHN n c) (s, e) = exec c (VAL (NumV' n) : s, e) }
exec (PUSHN n c) (s, convE e)

```

El razonamiento anterior es conciso y se sigue directamente de los cálculos realizados por los autores en casos similares [1] y los presentados para el lenguaje MINIB. No obstante, podemos desglosarlo aún más para detallar cada paso del proceso.

Partimos de la especificación de corrección 3.2, sustituyendo la expresión inicial en:

$$exec (comp' x c) (s, conv_E e)$$

con la correspondiente conclusión de la regla NUM en nuestra semántica operacional (véase la sección 3.3) en la cima de la pila, lo cual modela el comportamiento de la máquina virtual para hacer explícita la manipulación de una expresión numérica.

En este punto, aplicamos la función de conversión *conv* para transformar el valor resultante del lenguaje fuente al formato interno manejado por la máquina virtual.

A continuación, ajustamos la definición de *exec* para reflejar el nuevo comportamiento requerido: apilar el valor numérico *n* en la pila y continuar con la ejecución del código restante. Esta modificación se expresa mediante la introducción de un nuevo patrón de código:

$$c' = PUSHN n c,$$

donde *PUSHN* le indica a la máquina cómo insertar el valor *n* en la cima de la pila y luego ejecutar la continuación *c*.

Así, logramos reescribir el estado inicial en la forma deseada:

$$exec c' (s, conv_E e),$$

completando el caso base de forma satisfactoria y alineada con la especificación.

Boolean b

```

exec (comp' (Boolean b) c) (s, convE e)
= { especificacion 3.2 }
exec c (VAL (conv (BooleanV b)) : s, convE e)
= { definicion de conv }
exec c (VAL (BooleanV' b) : s, convE e)
= { definicion:
exec (PUSHB b c) (s, convE e) = exec c (VAL (BooleanV' b):s, convE e)
}
exec (PUSHB b c) (s, convE e)

```

El razonamiento para *Boolean* b sigue una estructura análoga al caso numérico. Sin embargo, en lugar de insertar un entero, se requiere introducir un valor booleano en la pila. Para ello, definimos un nuevo constructo de código:

$$c' = PUSHB\ b\ c,$$

el cual instruye a la máquina virtual colocar el valor booleano b en la cima de la pila y continuar con la ejecución de la continuación c .

Var x

```
exec (comp' (Var x) c) (s, convE e)
= { especificacion 3.2 }
exec c (VAL (conv (lookup x e)) : s, convE e)
= { relacion de convE y lookup }
exec c (VAL (lookup x (convE e)) : s, convE e)
= { definicion:
exec (LOOKUP x c) (s, e) = exec c (VAL (lookup x e):s, e) }
exec (LOOKUP x c) (s, convE e)
```

El caso *Var* introduce un elemento adicional respecto a los anteriores: la interacción entre el ambiente original e y su versión transformada $conv_E e$. Para abordar correctamente este caso, establecemos la siguiente propiedad funcional:

$$lookup\ x\ (conv_E\ e) = conv\ (lookup\ x\ e)$$

Esta igualdad garantiza que buscar una variable x en el ambiente transformado equivale a buscarla en el ambiente original y luego aplicar la conversión. Esto se debe a que la función $conv_E$ transforma cada par (k, v) en el ambiente original en $(k, conv\ v)$, preservando las claves y aplicando sistemáticamente la conversión solo a los valores.

La validez de esta propiedad se fundamenta en el contexto de lenguajes funcionales puros como HASKELL, donde las estructuras inmutables y las funciones puras permiten razonar algebraicamente sobre la equivalencia de evaluaciones. En particular, esta igualdad refleja un principio más general observado en estructuras funcionales como listas, donde una consulta sobre una estructura transformada equivale a transformar el resultado de la consulta original. Este principio está ampliamente documentado en la literatura de programación funcional. Por ejemplo, Bird y Wadler lo discuten en [57], y un tratamiento más técnico puede encontrarse en [58], donde se estudia la interacción entre transformaciones y consultas en estructuras de búsqueda.

Este tipo de propiedades también son posibles gracias al polimorfismo del sistema de tipos, ya que permite definir funciones que operan sobre estructuras parametrizadas sin necesidad de conocer el tipo específico de sus valores.

Una instancia concreta de este principio aparece en el trabajo de Bahr y Hutton [1], donde introducen el llamado *indexing lemma*:

$$f\ (xs\ !!\ i) = (map\ f\ xs)\ !!\ i$$

para cualquier función estricta f , lista xs e índice i . Este lema, identificado como el *teorema libre* correspondiente al tipo del operador $!!$, según Wadler [1], respalda propiedades como la nuestra cuando se trabaja con estructuras iterables transformadas. En nuestro caso, la evaluación estricta de la función $conv$, definida por correspondencia de patrones, garantiza que la propiedad funcional propuesta se mantenga.

Cabe señalar que esta propiedad puede demostrarse formalmente mediante *inducción estructural* sobre las definiciones de $conv$, $conv_E$ y el tipo $Value$. Sin embargo, dado que la verificación de cada caso es mecánica y directa, esta demostración se omite en el presente trabajo.

Ahora bien, al aplicar esta propiedad en el cálculo, surge un problema de tipos. La función $lookup$, tal como fue definida originalmente, posee el tipo:

```
lookup :: String -> Env -> Value
```

Esto impide aplicarla directamente sobre $conv_E e$, ya que este último tiene tipo Env' y no Env . Para resolver esta incompatibilidad, generalizamos la definición de $lookup$ de forma que pueda operar sobre listas asociativas con valores de tipo arbitrario:

Listing 3.12: Definición polimorfa de la función $lookup$

```
lookup :: String -> [(String, a)] -> a
lookup s [] = error ("Variable no encontrada: " ++ s)
lookup s ((k, v) : xs)
    | s == k      = v
    | otherwise   = lookup s xs
```

Esta versión polimórfica de $lookup$ permite su uso tanto en ambientes del tipo Env como en los del tipo Env' , según lo requiera el contexto. De esta manera, se evita la necesidad de definir múltiples funciones de búsqueda o de recurrir a conversiones explícitas en la implementación, manteniendo la generalidad sin comprometer la claridad del razonamiento.

Add x y

```
exec (comp' (Add x y) c) (s, convE e)
= { especificacion 3.2 }
exec c (VAL (conv (NumV (n+m))) : s, convE e)
= { definicion de conv }
exec c (VAL (NumV' (n+m)) : s, convE e)
= { definicion:
exec (ADD c) (VAL (NumV' m):VAL (NumV' n):s, e)
          = exec c (VAL (NumV' (n + m)):s, e) }
exec (ADD c) (VAL (NumV' m):VAL (NumV' n):s, convE e)
= { definicion de conv }
exec (ADD c) (VAL (conv (NumV m)):VAL (conv (NumV n)):s, convE e)
= { H.I. para y }
exec (comp' y (ADD c)) (VAL (conv (NumV n)):s, convE e)
  { H.I. para x }
exec (comp' x (comp' y (ADD c))) (s, convE e)
```

Las primeras tres derivaciones no requieren el uso de la *hipótesis de inducción*, ya que sus construcciones se corresponden directamente con valores canónicos. En contraste, en el caso de la construcción *Add*, la hipótesis de inducción se aplica en los dos últimos pasos del razonamiento, tanto a la expresión x como a la expresión y . Este enfoque es análogo a los cálculos realizados para los conectores *And* y *Or*, discutidos previamente en la sección 2.10. La definición derivada para la máquina virtual (*exec*) correspondiente a esta construcción es la siguiente:

```
exec (ADD c) (VAL (NumV' m):VAL (NumV' n) : s, e)
              = exec c (VAL (NumV' (n + m)):s, e)
```

Aquí, $c' = ADD\ c$, y el constructo de código *ADD* encapsula la operación de suma de los dos valores numéricos ubicados en la cima de la pila. Esta relación justifica la elección del nombre *ADD*.

Asimismo, la definición correspondiente de la función *comp'* para esta construcción se expresa como:

```
comp' (Add x y) c = (comp' x (comp' y (ADD c)))
```

Esta definición adopta un patrón estructural recurrente, que se repetirá en otras construcciones del compilador. En esencia, especifica cómo la compilación de la expresión *Add x y* se propaga de forma recursiva hacia ambas subexpresiones, asociando el constructo *Add* con el constructo de código *ADD* en la máquina destino.

Las derivaciones correspondientes a las construcciones (*Sub x y*) y (*Mul x y*) siguen un razonamiento análogo al de *Add*. Por esta razón, se omiten aquí sus desarrollos detallados; no obstante, pueden consultarse en su totalidad en el material suplementario que acompaña este trabajo (véase Apéndice A). Como se observa en las reglas de semántica natural, estos constructos difieren únicamente en el operador semántico que denotan: *Sub* denota el operador de resta sobre enteros ($-\mathbb{Z}$), mientras que *Mul* denota el operador de producto ($\times_{\mathbb{Z}}$). Esta asociación entre constructos y operadores matemáticos refleja una interpretación denotativa del lenguaje, donde las expresiones se interpretan directamente en un dominio como $\mathbb{Z}$ [24]. A continuación, se presentan las definiciones correspondientes para cada uno de estos constructos.

Definiciones de *Sub*

La derivación para la construcción *Sub* difiere de la de *Add* únicamente en el constructo de código introducido por el compilador, el cual refleja el operador aritmético de resta:

```
comp' (Sub x y) c = comp' x (comp' y (SUB c))
```

Por su parte, la definición correspondiente en la máquina virtual incorpora el operador de resta sobre enteros, ejecutando la operación directamente sobre los dos valores en la cima de la pila:

```
exec (SUB c) (VAL (NumV' m):VAL (NumV' n):s, e)
              = exec c (VAL (NumV' (n-m)):s, e)
```

Definiciones de *Mul*

De manera análoga, las definiciones asociadas a la construcción *Mul* introducen únicamente el constructo de código específico y el operador correspondiente al producto entero:

```
comp' (Mul x y) c = (comp' x (comp' y (MUL c)))
```

La máquina virtual ejecuta esta operación multiplicando los dos valores extraídos de la pila:

```
exec (MUL c) (VAL (NumV' m):VAL (NumV' n) : s , e )
              = exec c (VAL (NumV' (n * m)):s , e)
```

Div x y

Siguiendo el esquema de las operaciones aritméticas anteriores, la definición del compilador para la división mantiene la misma estructura general. No obstante, la definición correspondiente en la máquina virtual requiere distinguir explícitamente entre dos casos: el caso particular en que el denominador es cero ($m = 0$) y el caso general en que $m \neq 0$.

Caso 1: Denominador $m = 0$

```
exec (comp' (Div x y) c) (s, convE e)
= { especificacion 3.2 }
exec c (VAL (conv Error) : s, convE e)
= { definicion de conv }
exec c (VAL Error' : s, convE e)
= { definicion:
exec (DIV c) (VAL (NumV' 0): VAL (NumV' n):s,convE e)
              = exec c (VAL Error' : s, convE e) }
exec (DIV c) (VAL (NumV' 0): VAL (NumV' n):s, convE e)
= { definicion de conv }
exec (DIV c) (VAL (conv (NumV 0)):VAL (conv(NumV n)):s,convE e)
= { H.I. para y }
exec (comp' y (DIV c)) (VAL (conv (NumV n)):s,convE e)
= { H.I. para x }
exec (comp' x (comp' y (DIV c))) (s,convE e)
```

Aunque la compilación de la expresión aritmética conserva el patrón general previamente establecido, la definición de la máquina virtual introduce aquí un mecanismo explícito para el manejo de errores. Esta decisión permite capturar internamente situaciones excepcionales como la división por cero, sin depender del control de errores proporcionado por el lenguaje anfitrión:

```
exec (DIV c) (VAL (NumV' 0): VAL (NumV' n):s,convE e)
              = exec c (VAL Error' : s, convE e)
```

Caso 2: Denominador $m \neq 0$

```

exec (comp' (Div x y) c) (s, convE e)
= { Especificacion 3.2 }
exec c (VAL (conv (NumV (n 'div' m))) : s, convE e)
= { definicion conv }
exec c (VAL (NumV' (n 'div' m)) : s, convE e)
= { definicion:
  exec (DIV c) (VAL (NumV' m): VAL (NumV' n):s,convE e)
    = exec c (VAL NumV'(n 'div' m) : s, convE e) }
exec (DIV c) (VAL (NumV' m): VAL (NumV' n):s,convE e)
= { definicion de conv }
exec (DIV c) (VAL (conv(NumV m)):VAL (conv(NumV n)):s, convE e)
= { H.I. para y }
exec (comp' y (DIV c)) (VAL (conv (NumV n)):s,convE e)
= { H.I. para x }
exec (comp' x (comp' y (DIV c))) (s,convE e)

```

Este caso representa la ejecución normal de la división y puede entenderse como una extensión natural del caso especial anterior. Es fundamental que el manejo del caso excepcional ($m = 0$) se defina antes que el caso general; de lo contrario, una división por cero sería diagnosticada directamente por el lenguaje anfitrión, en lugar de ser manejada por la máquina virtual. Esto implicaría que el error escapa al control de nuestra semántica formalizada, invalidando el modelo de ejecución que se busca preservar, donde todos los comportamientos, incluidos los errores, son gestionados dentro del propio lenguaje.

Para implementar la operación aritmética, se utiliza el operador infijo *div* del lenguaje anfitrión, el cual realiza la división entera. Al igual que en los casos anteriores, se delega a HASKELL la evaluación del operador, lo que simplifica la implementación y evita redundancias en la definición de los operadores aritméticos.

```

exec (DIV c) (VAL (NumV' m):VAL (NumV' n):s,convE e)
    = exec c (VAL (NumV'(n 'div' m)) : s, convE e)

```

Pair x y

```

exec (comp' (Pair x y) c) (s, convE e)
= { Especificacion 3.2 }
exec c (VAL (conv (PairV u v)) : s, convE e)
={ definicion: conv (PairV u v) = PairV' (conv u) (conv v) }
exec c (VAL (PairV' (conv u) (conv v)) : s, convE e)
= { definicion:
  exec (PAIR c) (VAL v : VAL u : s, e)
    = exec c (VAL (PairV' u v) : s,e) }
exec (PAIR c) (VAL (conv v) : VAL (conv u) :s,convE e)
= { H.I. para y }
exec (comp' y (PAIR c)) (VAL (conv u) :s,convE e)
= { H.I. para x }

```



```
exec (comp' x (comp' y (PAIR c))) (s, convE e)
```

La construcción *Pair* extiende el patrón de evaluación y compilación recursiva observado en las expresiones aritméticas, incorporando además una definición adicional para la función de conversión *conv*, derivada directamente del razonamiento sobre esta construcción:

```
conv (PairV u v) = PairV' (conv u) (conv v)
```

Esta definición transforma el constructo original y aplica recursivamente la conversión a sus componentes. Por su parte, la función del compilador se deriva de forma análoga a los casos anteriores:

```
comp' (Pair x y) c = (comp' x (comp' y (PAIR c)))
```

Finalmente, la definición correspondiente en la máquina virtual indica que el constructo de código *PAIR* encapsula los dos valores en la cima de la pila dentro de un valor *PairV'*, respetando el orden de evaluación esperado:

```
exec (PAIR c) (VAL v:VAL u:s,e) = exec c (VAL (PairV' u v):s,e)
```

Fst x

```
exec (comp' (Fst x) c) (s, convE e)
= { Especificacion 3.2 }
exec c (VAL (conv u) : s, convE e)
= { definicion:
exec (FST c) (VAL (PairV' u v):s,e) = exec c (VAL u:s,e) }
exec (FST c) (VAL (PairV' (conv u) (conv v) : s, convE e)
= { definicion de conv }
exec (FST c) (VAL (conv (PairV u v)):s,convE e)
= { H.I. para x }
exec (comp' x (FST c)) (s, convE e)
```

La construcción *Fst* reutiliza la definición de *conv* obtenida previamente para valores de tipo *PairV*, asegurando consistencia en la interpretación de estructuras compuestas. La derivación correspondiente para el compilador mantiene el patrón composicional, compilando la subexpresión y agregando el constructo de código específico:

```
comp' (Fst x ) c = comp' x (FST c)
```

En la máquina virtual, el constructo de código *FST* extrae el primer componente del valor de tipo *PairV'* ubicado en la cima de la pila y lo reemplaza, preservando así el flujo correcto de evaluación:

```
exec (FST c) (VAL (PairV' u v):s, e) = exec c (VAL u:s, e)
```

If x y z

El razonamiento detrás de esta construcción puede parecer inicialmente poco intuitivo, ya que la semántica de la expresión condicional se formaliza mediante dos reglas de inferencia distintas (reglas IF-TRUE e IF-FALSE). A diferencia del caso de la construcción *Div*, donde realizamos derivaciones separadas, aquí aprovechamos que ambas reglas siguen una estructura análoga. La diferencia fundamental radica en la necesidad de vincular explícitamente el valor de la condición con la rama que debe ejecutarse.

```

exec (comp' (If x y z ) c) (s, convE e)
= { Especificacion 3.2 }
exec c ((if (conv b) == (BooleanV' True)
          then (VAL (conv v))
          else (VAL (conv w))) : s, convE e)
= { distribuir la pila y ambiente sobre las ramas then y else }
exec c (if (conv b) == (BooleanV' True)
          then (VAL (conv v):s, convE e)
          else (VAL (conv w):s, convE e))
= { distribuir exec c sobre ramas }
if (conv b) == (BooleanV' True)
  then exec c (VAL (conv v):s, convE e)
  else exec c (VAL (conv w):s, convE e)
= { H.I. para z }
if (conv b) == (BooleanV' True)
  then exec c (VAL (conv v):s, convE e)
  else exec (comp' z c) (s, convE e)
= { H.I. para y }
if (conv b) == (BooleanV' True)
  then exec (comp' y c) (s, convE e)
  else exec (comp' z c) (s, convE e)
= { definicion:
exec (IF c c') (VAL b:s,e) =
  if b == (BooleanV' True) then exec c (s,e) else exec c' (s,e) }
exec (IF (comp' y c) (comp' z c)) (VAL (conv b):s,convE e)
= { H.I. para x }
exec (comp' x (IF (comp' y c) (comp' z c))) (s, convE e)

```

En este punto, es pertinente destacar una característica esencial del enfoque metodológico adoptado. Siguiendo la propuesta de Bahr y Hutton, nuestra formalización semántica es intrínsecamente parcial: modela de forma correcta únicamente aquellas ejecuciones derivadas de expresiones fuente bien formadas. Esto implica que el compilador puede generar código que produzca cualquier resultado permitido por la semántica de las construcciones, una propiedad conocida como *completitud*, pero no asegura la propiedad dual de *solidez*⁴. Esta propiedad asegura que cualquier resultado producido por la ejecución del código compilado corresponde a un comportamiento capturado por la semántica del lenguaje fuente.

⁴Las nociones de *completitud* y *solidez* son traducciones directas de los términos *completeness* y *soundness*, respectivamente, utilizados por los autores en su especificación original [1].

Esta decisión es deliberada y coherente con el objetivo de centrarse en configuraciones válidas, evitando así el tratamiento explícito de casos mal formados (véase la sección 3.5 para una discusión más profunda).

Por ello, adoptamos un compromiso pragmático: en lugar de extender la semántica para capturar la solidez en general, nos concentramos únicamente en el comportamiento de la función *exec* sobre código bien tipificado generado por el compilador, conforme al enfoque de Bahr y Hutton [1].

A partir de esto, el razonamiento inicia con la aplicación de la especificación de corrección, que introduce la estructura condicional en la cima de la pila. Aquí conviene subrayar la seguridad de tipos: el tipo de la expresión condicional queda determinado por el tipo común de sus dos ramas, como establecen los sistemas de tipos simples [19], y en particular el sistema de tipos de HASKELL, que requiere que ambas ramas de una expresión condicional produzcan un valor del mismo tipo [59].

Siguiendo el enfoque de [60], distribuimos la configuración de la máquina virtual a ambas ramas de la condición. Esto podría parecer un punto muerto en el razonamiento, pero aquí se vuelve relevante una propiedad conocida en lenguajes funcionales: la distributividad de la aplicación de funciones sobre condicionales. Específicamente, la siguiente propiedad:

$$f \text{ (if } b \text{ then } x \text{ else } y) = \text{if } b \text{ then } f x \text{ else } f y$$

En el trabajo de Pickard y Hutton, esta propiedad se garantiza plenamente porque el lenguaje anfitrión es AGDA, un lenguaje total y con tipos dependientes. Gracias a su seguridad semántica, AGDA no solo valida esta propiedad, sino que, al integrarse tipos dependientes en el lenguaje objeto de estudio, esta propiedad se hereda también para las expresiones del lenguaje modelado.

Sin embargo, nuestro contexto es distinto. En este trabajo, el lenguaje anfitrión es HASKELL, un lenguaje funcional puro y perezoso con semántica basada en dominios (CPO), lo cual introduce ciertas sutilezas. Según Gibbons [61], la propiedad de distributividad solo es válida cuando el valor booleano *b* está definido y cuando la función *f* es estricta en su argumento; de lo contrario, puede fallar.

No obstante, cuando nos limitamos al análisis de expresiones bien tipificadas, es decir, aquellas que cumplen con las condiciones de tipo esperadas, la propiedad sí se mantiene en HASKELL. En consecuencia, en nuestro lenguaje ALANIS adoptamos el mismo compromiso metodológico que Bahr y Hutton: restringimos el análisis a programas bien formados y terminantes. Este enfoque justifica válidamente la asunción de propiedades como la distributividad, siempre que se opere dentro del dominio de expresiones correctamente tipificadas. De este modo, obtenemos una estructura casi idéntica a la función de evaluación *eval*. Una vez establecida la correspondencia entre ramas y valores, queda tratar la condición. Para ello, transformamos la expresión condicional a la forma esperada:

$$c' = \text{comp}' x c$$

Introducimos entonces la definición correspondiente en la máquina virtual:

```
exec (IF c c') (VAL b :s , e) = if b == BooleanV ' True
                                then exec c (s, e)
                                else exec c' (s, e)
```

A partir de aquí, solo es necesario hacer explícita la relación entre el valor evaluado de la condición y su expresión asociada. Es natural observar que el código *IF* espera que el valor booleano correspondiente a la condición se encuentre en la cima de la pila. Por tanto, basta con aplicar la hipótesis de inducción sobre la subexpresión correspondiente, lo cual finalmente nos lleva a derivar la definición del compilador:

```
comp' (If x y z) c = comp' x (IF (comp' y c) (comp' z c))
```

Eq x y

```
exec (comp' (Eq x y) c) (s, convE e)
= { Especificacion 3.2 }
exec c (VAL (conv (BooleanV (n == m))) : s, convE e)
= { definicion de conv }
exec c (VAL (BooleanV' (n == m)) : s, convE e)
= { definicion: exec (EQ c) (VAL (NumV' m):VAL (NumV' n), e) = exec c
  (VAL (BooleanV' (n == m)) : s, e) }
exec (EQ c) (VAL (NumV' m): VAL (NumV' n):s, convE e)
= { definicion de conv }
exec (EQ c) (VAL (conv(NumV m)): VAL (conv(NumV n)):s, convE e)
= { H.I. para y }
exec (comp' y (EQ c)) (VAL(conv(NumV n)):s,convE e)
= { H.I. para x }
exec (comp' x (comp' y (EQ c))) (s, convE e)
```

La construcción *Eq* extiende el patrón de evaluación de las operaciones aritméticas, diferenciándose únicamente en el tipo de operación aplicada sobre los operandos. En lugar de producir un valor numérico, el resultado es un valor booleano que representa el resultado de la comparación entre los valores asociados a *x* y *y*.

La derivación para el compilador mantiene la forma composicional habitual, aplicando *comp'* a cada subexpresión y agregando el constructo de código *EQ*:

```
comp' (Eq x y) c = comp' x (comp' y (EQ c))
```

Por su parte, la máquina virtual evalúa los operandos en la pila, asegurando que ambos sean valores numéricos, y luego aplica la operación de igualdad. El resultado es encapsulado en un valor booleano y colocado en la cima de la pila:

```
exec (EQ c) (VAL (NumV' m) : VAL (NumV' n) : s, e)
= exec c (VAL (BooleanV' (n == m)) : s, e)
```

Lambda x y

```
exec (comp' (Lambda x y) c) (s, convE e)
= { Especificacion 3.2 }
exec c (VAL(conv(Closure (Lambda x y) (convE e))):s,convE e)
```

```
= { definicion: conv Closure (Lambda x y) e = Closure' (LAMBDA x (
  comp' y RET)) (convE e) }
exec c (VAL (Closure' (LAMBDA x (comp' y RET)) (convE e)):s, convE e)
= { definicion:
  exec (ABS c' c) (s, e) = exec c (VAL (Closure' c' e):s, e) }
exec (ABS (LAMBDA x (comp' y RET)) c) (s, e)
```

A partir del razonamiento previo, se deriva la definición de la función de conversión *conv* para la construcción *Lambda x y*. En este caso, la variable *a* queda ligada al cuerpo *c*, y es fundamental que esta vinculación se conserve durante la conversión del cuerpo. Para reflejar esta relación, se introduce el constructo de código *LAMBDA*, que mantiene el contexto de la vinculación. Aunque aún no se asocia con una definición específica en la máquina virtual, su inclusión es esencial para asegurar la correcta representación semántica de la construcción, particularmente en lo que respecta a la noción de cierre. La definición de *conv* para el valor *Closure* es la siguiente:

```
conv Closure (Lambda x y) e
  = Closure' (LAMBDA x (comp' y RET)) (convE e)
```

Además, se introduce el constructo *RET*, que actúa como un marcador del final de una función. Este componente es esencial en la evaluación de aplicaciones de función, ya que permite realizar un cambio de contexto: primero se evalúa el cuerpo de la función en el ambiente donde fue definida, y luego se restaura el ambiente original. El uso de *RET* facilita a la máquina virtual la correcta gestión de esta transición.

En cuanto a la máquina virtual, el constructo *ABS* se encarga de la creación del cierre. Cuando se encuentra en el código, este encapsula tanto la definición de la función como la continuación *c*, colocando la cerradura en la pila y retomando la ejecución desde el punto indicado por *c*:

```
exec (ABS c' c) (s, e) = exec c (VAL (Closure' c' e):s, e)
```

A diferencia de las definiciones anteriores, la derivación para el compilador en este caso emplea varios constructos de código. Aunque existen alternativas (como modificar la aridad de los constructos), la estrategia adoptada preserva una relación clara y directa entre la variable ligada y el cuerpo de la función. La definición resultante es:

```
comp' (Lambda x y) c = ABS (LAMBDA x (comp' y RET)) c
```

Al igual que el constructo *IF*, el constructo *ABS* introducido en esta etapa recibe dos códigos como argumentos. En ambos casos, esta estructura refleja explícitamente una bifurcación en el control de flujo de evaluación: en el caso de *IF*, en función del valor booleano que determina la rama a seguir, y en el caso de *Lambda*, al separar la función del resto de la ejecución. Esta representación implica que el código generado por el compilador no es completamente lineal, sino que contiene estructuras anidadas que capturan alternativas de ejecución.

Sin embargo, esta no es una limitación del enfoque ni del compilador. La existencia de constructos que admiten múltiples bloques de código no impide la transformación del código hacia una representación lineal si así se desea. Tal transformación puede lograrse sistemáticamente, por ejemplo, reemplazando los argumentos de tipo *Code* por referencias o apuntadores

a código, sin que esto requiera modificaciones sustanciales en la lógica del compilador ni comprometa su corrección [1].

App x y

```

exec (comp' (App x y) c) (s, convE e)
= { Especificacion 3.2 }
exec c (VAL (conv cv) : s, convE e)
= { definicion:
exec RET (VAL u:CLO c d:s, _) = exec c (VAL u : s, d) }
exec RET (VAL (conv cv):CLO c (convE e):s,(a, conv u):convE e')
= { H.I. para x' }
exec (comp' x' RET) (CLO c (convE e) : s, (a, conv u):convE e')
= { definicion:
exec (APP c) (VAL u:VAL (Closure' (LAMBDA a c') e'):s, e)
= exec c' (CLO c e:s,(a, u):e') }
exec (APP c) (VAL (conv u): VAL (Closure' (LAMBDA a (comp' x' RET)) (
convE e')) : s, convE e)
= { H.I. para y }
exec (comp' y (APP c)) (VAL (Closure' (LAMBDA a (comp' x' RET)) (
convE e')) : s, convE e)
= { definicion de conv }
exec (APP c) (VAL (conv (Closure (Lambda a x') e')):s, convE e)
= { H.I. para x }
exec (comp' x (comp' y (APP c))) (s, convE e)

```

Este razonamiento presenta una estructura más robusta que los anteriores, ya que permite derivar no solo la definición del compilador para esta construcción, sino también múltiples definiciones correspondientes de la máquina virtual. La profundidad del análisis se debe a la complejidad inherente de la aplicación de funciones, que implica manipulación explícita de (*Closures*) y ambientes de evaluación.

El proceso inicia aplicando la especificación de corrección 3.2, lo cual nos conduce a una configuración donde la pila contiene el resultado de aplicar una función a un argumento. Este valor se ubica en la cima de la pila, y la evaluación continúa con el contexto original, lo cual es coherente con el régimen de evaluación ansioso adoptado.

A continuación, introducimos la definición del constructo *RET*. Aunque esta construcción fue considerada previamente durante el razonamiento para las abstracciones, su definición concreta en la máquina virtual se establece aquí. El propósito de *RET* es restaurar el contexto de ejecución anterior tras la evaluación de una función. Específicamente, si en la pila el tope es un valor y el segundo elemento es una construcción *CLO*, entonces *RET* extrae la continuación original *c* encapsulada en la cerradura y restablece el ambiente de evaluación correspondiente, preservando el valor resultante de la aplicación en la cima de la pila.

Un aspecto que merece atención en este razonamiento es el uso de la hipótesis de inducción sobre una subexpresión que no está explícitamente presente en la expresión fuente *App x y*, a saber, x' , el cuerpo de la función evaluada. Esta situación se justifica formalmente por la regla semántica APP, que establece que, tras evaluar x , se obtiene una abstracción cuyo cuerpo

x' se evalúa en el ambiente extendido con el argumento formal. Así, aunque x' no figura de forma textual en la expresión original, su inclusión en el razonamiento es válida. Más aún, este paso no constituye una excepción al esquema inductivo, sino que se deriva directamente de su aplicación: suponer que x cumple la propiedad permite concluir que también la cumple cualquier cuerpo de función x' contenido en ella, conforme al mismo principio inductivo. Esta estructura refleja una simplificación natural del proceso inductivo sobre construcciones anidadas.

Este paso no solo permite estructurar adecuadamente la cerradura en la pila, sino que además nos conduce a completar la definición pendiente de la función *conv* para cerraduras, cuya importancia sustancial para el razonamiento había sido introducida en el análisis de las abstracciones. En este sentido, la semántica guía naturalmente la derivación de la definición deseada, al proporcionarnos los elementos necesarios para construir la cerradura correctamente.

Por otro lado, la expresión y , correspondiente al argumento de la aplicación, puede tratarse de forma más directa. El valor u en la cima de la pila, justo antes de aplicar la hipótesis de inducción a y , proviene de su evaluación, lo cual permite aplicar sin obstáculos el razonamiento inductivo habitual.

A partir del análisis anterior podemos obtener la definición de la función de compilación *comp'* para aplicaciones. Esta definición muestra una estructura anidada, donde primero se compila el argumento, luego la función, y finalmente se inserta la instrucción *APP c*, lo cual concuerda con las formas derivadas para otras construcciones previamente analizadas:

```
comp' (App x y) c = comp' x (comp' y (APP c))
```

Por su parte, las definiciones correspondientes de la máquina virtual que permiten ejecutar este tipo de construcciones son las siguientes:

```
exec (APP c) (VAL u : VAL (Closure' (LAMBDA a c') e') : s, e)
    = exec c' (CLO c e : s, (a, u) : e')
```

```
exec RET (VAL u : CLO c d : s, _) = exec c (VAL u : s, d)
```

Estas definiciones reflejan con precisión el comportamiento operativo de la aplicación de funciones: evaluación del argumento, activación de la cerradura con el nuevo ambiente, y eventual retorno al contexto original mediante *RET*.

Función de compilación general *comp*

El desarrollo del compilador concluye con la definición de la función principal:

```
comp :: Expr -> Code
```

La corrección de esta función ya fue establecida formalmente en la sección 2.10, donde se mostró que su justificación no requiere un razonamiento inductivo, sino que puede derivarse mediante razonamiento ecuacional directo a partir de las especificaciones de corrección. En particular, en dicha sección se introdujo el constructo *HALT* como instrucción terminal necesaria para completar esta derivación.

Dado que la especificación de corrección 2.4 para *comp* permanece inalterada, no es necesario reconsiderar el razonamiento en este punto. En consecuencia, se conserva la misma definición:

```
comp x = comp' x HALT
```

Esta definición es plenamente coherente con la semántica establecida, y como se señala en [1], refleja un patrón recurrente: la función *comp* se obtiene aplicando *comp'* con una continuación que marca el fin de la ejecución, en este caso *HALT*, garantizando así una finalización correcta sin requerir transformaciones adicionales en la configuración.

3.4.4. Resultados

Una vez completado el cálculo de las definiciones para el compilador y la máquina virtual, así como de los componentes auxiliares (como la función de conversión (*conv*) y el tipo de dato *Code*) en la sección anterior, procedemos ahora a organizar dichas definiciones agrupándolas según cada uno de los componentes involucrados.

Lenguaje destino

Nuestro lenguaje destino es descrito por el tipo de dato *Code*, por lo que es suficiente agrupar todos los constructos derivados en la definición de este tipo en *HASKELL*.

Listing 3.13: Definición del tipo *Code* para el lenguaje destino

```
data Code = HALT | PUSHN Int Code | PUSHB Bool Code
          | LOOKUP String Code | ADD Code | SUB Code | MUL Code
          | DIV Code | PAIR Code Code | FST Code | IF Code Code
          | EQ Code | LAMBDA String Code | ABS Code Code
          | RET | APP Code Code
```

Compilador

La función *comp* se encarga únicamente de invocar a *comp'* utilizando el código *HALT* para indicar el término de la ejecución, de modo que la traducción completa de una expresión se realiza de la siguiente forma

Listing 3.14: Definición de la función de compilación general *comp*

```
comp :: Expr -> Code
comp e = comp' e HALT
```

La función auxiliar *comp'* efectúa la compilación recursiva de las expresiones, atendiendo a cada constructor del lenguaje y generando el código correspondiente.

Listing 3.15: Definición de función de compilación auxiliar *comp'*

```
comp' :: Expr -> Code -> Code
comp' (Num n) c      = PUSHN n c
comp' (Boolean b) c  = PUSHB b c
comp' (Var x) c       = LOOKUP x c
comp' (Add x y) c     = comp' x (comp' y (ADD c))
comp' (Sub x y) c     = comp' x (comp' y (SUB c))
```



```

comp' (Mul x y) c      = comp' x (comp' y (MUL c))
comp' (Div x y) c      = comp' x (comp' y (DIV c))
comp' (If x y z) c     = comp' x (IF (comp' y c) (comp' z c))
comp' (Eq x y) c       = comp' x (comp' y (EQ c))
comp' (Pair x y) c     = comp' x (comp' y (PAIR c))
comp' (Fst x) c        = comp' x (FST c)
comp' (Lambda x y) c   = ABS (LAMBDA x (comp' y RET)) c
comp' (App x y) c      = comp' x (comp' y (APP c))

```

Máquina virtual

Aquí basta con agrupar las definiciones derivadas en el proceso de cálculo.

Listing 3.16: Definición de la máquina virtual para la ejecución de código y manejo de errores

```

exec :: Code -> Conf -> Conf
exec _ (VAL Error:s,e) = error ("Division por cero.")
exec HALT (s, e)       = (s, e)
exec (PUSHN n c) (s, e) = exec c ((VAL (NumV' n)) : s, e)
exec (PUSHB b c) (s, e) = exec c ((VAL (BooleanV' b)) : s, e)
exec (LOOKUP x c) (s, e) = exec c (VAL (lookup x e):s, e)
exec (ADD c) ((VAL (NumV' m)):(VAL (NumV' n)):s, e)
    = exec c ((VAL (NumV' (n+m))):s, e)
exec (SUB c) ((VAL (NumV' m)):(VAL (NumV' n)):s, e)
    = exec c ((VAL (NumV' (n-m))):s, e)
exec (MUL c) ((VAL (NumV' m)):(VAL (NumV' n)):s, e)
    = exec c ((VAL (NumV' (n*m))):s, e)
exec (DIV c) ((VAL (NumV' 0)):(VAL (NumV' n1)):s, e)
    = exec c (VAL Error:s, e)
exec (DIV c) ((VAL (NumV' m)):(VAL (NumV' n)):s, e)
    = exec c ((VAL (NumV' (n `div` m))):s, e)
exec (IF c c') ((VAL b) : s, e) = if b == BooleanV True
    then exec c (s, e)
    else exec c' (s, e)
exec (EQ c) ((VAL (NumV' m)):(VAL (NumV' n)):s, e)
    = exec c (VAL (BooleanV' (n == m)):s, e)
exec (PAIR c) ((VAL v):(VAL u):s, e)
    = exec c (VAL (PairV' u v):s, e)
exec (FST c) ((VAL (PairV' u v)):s, e) = exec c (VAL u:s, e)
exec (ABS c' c) (s, e) = exec c (VAL (Closure' c' e):s, e)
exec RET (VAL u : CLO c d : s, _) = exec c (VAL u : s, d)
exec (APP c) (VAL u:VAL (Closure' (LAMBDA a c') e'):s, e)
    = exec c' (CLO c e:s, (a,u):e')

```

En los cálculos anteriores, específicamente en el caso de la división por cero, se introdujo un constructo para representar errores. Sin embargo, en ese mismo razonamiento no se derivó la definición correspondiente en la máquina virtual. Esto puede parecer una omisión menor,

pero resulta importante si queremos asegurar que nuestra máquina maneje correctamente todos los casos posibles de ejecución; en concreto, véase la definición:

```
exec _ (VAL Error:s, e) = error ("Division por cero.")
```

Agregar una definición explícita para el caso de error no compromete la corrección de la máquina virtual. Al contrario, permite que se gestione el error de forma clara y controlada. Si no lo hacemos, la máquina simplemente se detendría al evaluar una división por cero, no porque hayamos modelado ese comportamiento, sino porque no existe una regla definida para esa situación. El error seguiría ocurriendo, pero no por diseño, sino por falta de cobertura en la implementación.

En lugar de eso, al incluir una definición sobre el constructo *Error* y delegar la señalización del fallo al lenguaje anfitrión mediante sus propios mecanismos, estamos siendo explícitos sobre cómo manejar ese caso. Esto no significa que el error desaparezca, sino que lo propagamos de manera controlada usando los mecanismos ya provistos por HASKELL. Este enfoque es natural dentro del lenguaje anfitrión y sigue la práctica común descrita en el reporte del lenguaje [59].

Así, al introducir esta definición en la máquina virtual, no alteramos ni comprometemos la corrección de los componentes derivados. Más bien, completamos el diseño de la máquina virtual alineando su comportamiento con la implementación concreta en HASKELL. Al mismo tiempo, este caso ilustra cómo algunas situaciones que no son abordadas directamente por la teoría, como la propagación operativa de errores, encuentran un tratamiento adecuado en la práctica, mostrando así la complementariedad entre el enfoque teórico y las exigencias del entorno de implementación.

Función de conversión

Los cálculos realizados nos han permitido derivar las definiciones restantes de la función de conversión. En un principio, se estableció una definición parcial para *conv*, ya que se optó por no hacer suposiciones sobre su comportamiento en los casos de *Pair* y *Closure*. Una vez obtenidas las definiciones faltantes, es posible formalizar completamente la función de conversión, agregando los casos restantes.

Listing 3.17: Definición completa de la función de conversión *conv* entre *Value* y *Value'*

```
conv :: Value -> Value'
conv (NumV n)           = NumV' n
conv (BooleanV b)       = BooleanV' b
conv ErrorV             = ErrorV'
conv PairV u v          = PairV' (conv u) (conv v)
conv Closure (Lambda x y) e = Closure' (LAMBDA x (comp' y RET))
                                (convE e)
```

En la definición para cerraduras de función, se sigue utilizando la función auxiliar *conv_E*, que mapea la conversión sobre los segundos elementos de cada tupla del ambiente. Cabe destacar que la definición de *conv_E* no ha sufrido modificaciones y puede consultarse en la sección 3.4.2.

3.5. Reflexión

En esta sección se presenta una reflexión centrada en la aplicación del método a nuestro lenguaje ALANIS, resaltando los contrastes con la aproximación original expuesta en el trabajo de los autores [1] (sección 5). Aunque ALANIS conserva algunos aspectos de la formalización sintáctica base, introduce diferencias claramente observables, tales como el uso de ambientes con *bindings* en lugar de índices de *De Bruijn*, y la incorporación de nuevas construcciones como los condicionales y los pares ordenados con la proyección *fst*, que en conjunto enriquecieron la semántica del lenguaje para permitirnos contrastar entre los casos de estudio. Así, el análisis aquí presentado se focaliza en extender los aspectos reflexivos observados en la propuesta original de los autores a los objetivos y decisiones adoptadas en nuestro trabajo. Esta reflexión se orienta a subrayar las diferencias y nuevas perspectivas surgidas en este caso de estudio, estableciendo un puente hacia el análisis comparativo que se seguirá en secciones posteriores.

Parcialidad

Con parcialidad nos referimos a que la semántica del sistema, y en particular, la función que ejecuta el código generado (*exec*), está definida únicamente sobre configuraciones que satisfacen ciertas precondiciones mínimas. Este enfoque implica que la ejecución puede fallar o simplemente no estar especificada cuando dichas condiciones no se cumplen, por ejemplo, si una instrucción como *ADD* se aplica a una pila con menos de dos elementos válidos [1]. Esta decisión metodológica, tomada conscientemente, se manifiesta tanto en el diseño de la máquina virtual como en la especificación del compilador. Como se ha destacado en el enfoque original de Bahr y Hutton, la parcialidad es una característica inherente al modelado de máquinas abstractas en lenguajes de estilo *ML*⁵, donde funciones como *exec* están definidas parcialmente sobre estructuras bien formadas [1]. Por ello, nuestra formalización, al igual que la propuesta de los autores, se restringe a capturar el comportamiento de la máquina bajo condiciones válidas, excluyendo explícitamente los casos en los que el código fuente o el estado de ejecución no son adecuados.

Cabe destacar que, aunque es posible transformar una función parcial como *exec* en una total, ello requiere introducir un sistema de tipos. Por ejemplo, en un lenguaje con tipos dependientes, se puede expresar de forma estática el comportamiento esperado de cada instrucción, incluyendo las exigencias sobre la pila o los registros de ser el caso [60]. Esto permitiría garantizar que el código generado siempre opera sobre estados válidos, evitando fallos de ejecución como el desbordamiento de pila o el acceso a registros vacíos. De este modo, la totalización de la semántica no se logra añadiendo casos especiales en la implementación, sino especificando de antemano las condiciones necesarias mediante el sistema de tipos.

En contraste, cuando se parte de un lenguaje fuente no tipificado, como es el caso de ALANIS, la parcialidad es difícil de evitar. Incluso en lenguajes muy reducidos (como aquellos que solo

⁵ML (*MetaLanguage*) es una familia de lenguajes de programación funcionales y tipificados estáticamente, inicialmente creado como lenguaje de metaprogramación para el sistema de pruebas LCF (*Logic for Computable Functions*). Se distingue por su inferencia de tipos polimórfica y su evaluación por valor [41]. Ejemplos notables incluyen STANDARD ML, OCAML y F#.

permiten enteros y operaciones aritméticas) la semántica de la máquina destino puede volverse parcial: basta con que una instrucción acceda a un registro no inicializado o intente consumir más elementos de los que hay disponibles en la pila para que surjan errores en tiempo de ejecución [60]. Esto refleja una limitación inherente del método, más que una falla de su formulación.

Por ello, en lugar de imponer desde el inicio una semántica total, el enfoque de Bahr y Hutton adopta especificaciones y definiciones parciales tanto para la semántica del lenguaje como para el compilador. Esto permite derivar de manera calculada los componentes faltantes y evita comprometer prematuramente decisiones de implementación [1]. Nuestra formalización sigue esta misma estrategia: se concentra en el comportamiento correcto bajo configuraciones bien formadas, asumiendo explícitamente la parcialidad como un compromiso pragmático.

Finalmente, si bien la incorporación de un sistema de tipos dependientes permitiría eliminar dicha parcialidad al garantizar estáticamente la validez de cada construcción, esta línea de trabajo excede los objetivos del presente estudio. Retomando a Bahr y Hutton, nos enfocamos en modelar y derivar el comportamiento esencial de la máquina virtual en condiciones de validez, posponiendo la exploración de mecanismos más expresivos para futuras investigaciones.

Escalabilidad

La escalabilidad del método se evidencia en nuestro caso de estudio, ya que la formulación original presentada en [1] permitió trasladar el enfoque a un lenguaje con construcciones semánticas adicionales, como es el caso de AΛANIS. En este contexto, entendemos por escalabilidad la facilidad que ofrece el método para incorporar nuevos constructores en el lenguaje fuente sin comprometer la estructura general del proceso de cálculo. A pesar de la inclusión de nuevos elementos en el lenguaje, la estrategia de trabajar con definiciones parciales ha facilitado la adaptación del método sin imponer restricciones prematuras en las decisiones de implementación. Un ingrediente esencial para la escalabilidad de este enfoque es, precisamente, el uso de especificaciones parciales que se completan progresivamente durante el proceso de cálculo.

En nuestro trabajo se destaca esta característica mediante la derivación gradual de definiciones, tales como la función de conversión *conv* y el tipo de dato que modela la pila (*CLO*). La capacidad de establecer estas definiciones de manera parcial y luego completarlas en fases posteriores del cálculo no solo permitió gestionar la complejidad añadida de AΛANIS, sino que también confirmó la adaptabilidad y generalidad del método propuesto originalmente por los autores.

Cabe señalar que la escalabilidad, en este sentido, no se refiere únicamente a la extensión del método a lenguajes más expresivos, sino a su capacidad para ajustarse, con o sin modificaciones, a diferentes decisiones de diseño y a características semánticas. Los propios autores abordan esta cuestión de forma reflexiva, mostrando cómo variantes del enfoque, como el uso de continuaciones múltiples o funciones indefinidas como *fail*, en presencia de excepciones, permiten que el método escale adecuadamente incluso a lenguajes de orden superior como lo es AΛANIS [1].

Esta experiencia demuestra que, al mantener una aproximación flexible y modular, es posi-

ble extender el método a lenguajes con características semánticas más complejas sin perder la claridad y consistencia del proceso de derivación, abriendo así el camino para futuras aplicaciones en contextos aún más sofisticados.

Recursión

Si bien esta integración ya parece viable a partir de los resultados obtenidos, su análisis se vuelve pertinente en este punto, dado que ofrece evidencia adicional sobre la robustez del enfoque. En particular, la posibilidad de incorporar recursión en el lenguaje ALANIS surge como una consecuencia natural de las propiedades estructurales del *Cálculo λ* , particularmente en su variante *Call-by-Value*. En este marco, la recursión puede modelarse directamente mediante la aplicación de un operador de punto fijo, lo cual sugiere que dicha funcionalidad no exige una modificación sustancial de la semántica del lenguaje ni la introducción de mecanismos semánticos adicionales [19].

Esta observación se apoya en el hecho de que nuestras definiciones actuales (basadas en árboles de sintaxis abstracta ya formalizados) se mantienen válidas aun cuando se introduce una construcción concreta para la definición de funciones recursivas. En otras palabras, la recursión puede tratarse como una extensión sintáctica del lenguaje fuente, la cual es descompuesta en una fase de eliminación de azúcar sintáctica hacia expresiones que ya pertenecen a la sintaxis abstracta, para las cuales el evaluador y compilador han sido presentados.

Este enfoque sugiere que el soporte para recursión no requiere alterar la semántica ya establecida, sino que puede integrarse a través de una transformación bien definida, preservando así la consistencia del marco semántico original. La capacidad de representar funciones recursivas mediante operadores de punto fijo consolida esta visión, en la medida en que dichas expresiones pueden ser reescritas en términos del núcleo funcional del *Cálculo λ* , sin añadir nuevas reglas semánticas al sistema de transición.

La implementación concreta de esta construcción se aborda de manera detallada en el capítulo 6, donde se formaliza su traducción a la sintaxis abstracta del lenguaje.

Configuraciones

La incorporación del ambiente en la máquina virtual constituye una manifestación de la escalabilidad del enfoque adoptado, ya que nuestra formalización semántica se construye sobre una relación de transición que incluye explícitamente un ambiente léxico. Aunque el método original se centraba en derivar máquinas virtuales basadas exclusivamente en pila [1], en nuestro caso se evidencia que extender la especificación para incluir un ambiente de evaluación no solo es posible, sino natural dentro del marco general del método.

Esta extensión no representa una desviación, sino más bien una aplicación directa del principio metodológico de empaquetar, junto a la pila, todas las estructuras de datos adicionales requeridas por la semántica del lenguaje. En este sentido, el ambiente pasa a ser un componente esencial de las configuraciones de la máquina virtual en nuestro desarrollo, lo cual facilita el tratamiento de otras estrategias como *Call-by-Name* o *Call-by-Need* al permitir un control explícito del contexto de evaluación.

Cabe aclarar que estos estilos semánticos también pueden modelarse mediante mecanismos

alternativos, como la sustitución directa, sin requerir necesariamente un ambiente. Sin embargo, el uso de ambientes en nuestra formalización ha resultado especialmente conveniente para expresar de manera clara y modular la dinámica de evaluación, en coherencia con los principios del método.

De este modo, la inclusión del ambiente en nuestro caso de estudio no solo se alinea con los principios propuestos por Bahr y Hutton, sino que también enfatiza la adaptabilidad y robustez del método, al demostrar que es capaz de integrar estructuras adicionales sin comprometer la claridad ni la sistematicidad del proceso de cálculo.

Semántica relacional

En el desarrollo de nuestros lenguajes, tanto MINIB como ALANIS, la formalización sintáctica y semántica fue establecida de manera explícita antes de aplicar el método de cálculo. En este marco, al haber definido previamente la semántica en estilo relacional⁶, la transformación mediante defuncionalización (proceso comúnmente utilizado para reescribir funciones de orden superior en términos de estructuras de primer orden) no resultó necesaria. La función *eval*, en efecto, se integró de forma natural con la semántica operacional ya especificada para el lenguaje, permitiendo una transición directa hacia la máquina virtual [1].

Este enfoque contrasta con la estrategia adoptada por Bahr y Hutton, quienes, para simplificar la exposición, introducen directamente la sintaxis abstracta y definen sobre ella una función de evaluación sin una formalización semántica previa. En nuestro caso, se optó por mantener una formalización completa desde el inicio, incluyendo tanto la sintaxis concreta y abstracta, como la semántica relacional, lo cual permitió aplicar el método sobre una base rigurosamente definida y justificable.

Si bien podría parecer que ambos procedimientos difieren, en la práctica conducen a resultados semánticamente equivalentes: ya sea mediante la defuncionalización de una función de evaluación, o bien a través de una especificación relacional previa. Esta equivalencia subraya no solo la solidez teórica del enfoque, sino también su capacidad de adaptación a distintos estilos formales de semántica.

Cabe destacar, sin embargo, que en muchos contextos prácticos no se cuenta con una formalización semántica tan detallada como la adoptada en nuestro trabajo. Aun así, el método demuestra ser aplicable, ya que puede operar sobre funciones de evaluación definidas directamente, como en el enfoque original, sin requerir una especificación relacional previa. En este sentido, la versatilidad del método permite su uso tanto en desarrollos rigurosamente

⁶En este trabajo, el término *semántica relacional* se refiere a una especificación del significado de los programas mediante un sistema de transición definido por reglas de inferencia, en lugar de funciones deterministas. Esta formulación se adopta tanto en la semántica operacional estructural del lenguaje MINIB como en la semántica natural utilizada para ALANIS, en donde se describe el proceso de evaluación como una relación entre expresiones y valores. Esta perspectiva permite capturar de forma natural la parcialidad de las evaluaciones, una propiedad fundamental cuando se trabaja con construcciones como la recursión o la pereza. Tal como señalan Bahr y Hutton [1], el uso de una semántica relacional surge como una generalización natural del enfoque funcional, motivada por el cambio de la inducción estructural sobre la sintaxis hacia una inducción estructural sobre reglas de evaluación. Aunque el mismo compilador puede derivarse a partir de una semántica funcional (posiblemente en estilo monádico con la mónada *Maybe* para representar la parcialidad, véase la sección 4.2.1), el uso de una semántica relacional simplifica el cálculo y permite mantener una metodología uniforme en todo el proceso.

formales como en escenarios más pragmáticos, manteniendo en ambos casos la claridad del proceso de cálculo y la fidelidad al comportamiento del lenguaje.

Corrección

La especificación de corrección adoptada en nuestro trabajo fue adecuada para alcanzar el objetivo principal: derivar el compilador a partir de la semántica del lenguaje. No obstante, debido a la parcialidad inherente de la semántica relacional empleada, dicha especificación sólo permite establecer de manera explícita una de las dos propiedades clásicas de corrección de compiladores: la *completitud*.

Esta propiedad garantiza que todo resultado que puede obtenerse mediante la evaluación de una expresión en el lenguaje fuente también puede obtenerse al ejecutar su versión compilada en la máquina virtual. Es decir, el compilador no omite ningún comportamiento permitido por la semántica del lenguaje.

La propiedad dual, conocida como *solidez*, asegura lo contrario: que toda ejecución posible del código compilado corresponde a una evaluación válida en el lenguaje fuente [1]. En otras palabras, la máquina virtual no produce comportamientos que el lenguaje fuente no permite. Esta propiedad es igualmente importante, pero no queda reflejada directamente en nuestra formalización actual.

La razón principal de esta omisión radica en la parcialidad de la semántica relacional y de la máquina virtual derivada, las cuales no definen un comportamiento para todas las configuraciones posibles. Como resultado, no es posible derivar la solidez directamente a partir de la especificación adoptada, a diferencia de los casos en los que la semántica es total y determinista.

Este contraste ha sido ampliamente discutido en la literatura. Como se explica en [19], cuando se trabaja con lenguajes cuya semántica es total, como ocurre en MINIB, es posible establecer ambas propiedades (completitud y solidez) a partir de una única especificación. Incluso en el caso del *Cálculo* λ , si se restringe a fragmentos como los términos simplemente tipificados donde la evaluación es total, se puede obtener la solidez como consecuencia directa de la completitud [1].

En cambio, en semánticas relacionales genuinamente parciales o no deterministas, resulta necesario abordar ambas propiedades por separado para asegurar una noción completa de corrección. Esta necesidad ha motivado propuestas más expresivas, como la de Pickard y Hutton [60], quienes utilizan tipos dependientes para garantizar, de forma estática, que el compilador sólo genera términos bien formados y que toda ejecución respeta las restricciones semánticas definidas por el lenguaje fuente. Dicho enfoque permite formalizar ambas propiedades de manera unificada y rigurosa.

Sin embargo, dado que el objetivo de este trabajo no es reformular el método de cálculo, sino aplicarlo en un nuevo contexto, no exploramos tales extensiones. Desde esta perspectiva, nuestro análisis pone en evidencia tanto la capacidad del método para derivar compiladores correctos bajo especificaciones parciales, como sus limitaciones al tratar de capturar formalmente una noción más fuerte de corrección. Esto sugiere posibles líneas de trabajo futuro orientadas a extender el método para cubrir explícitamente tanto la completitud como la solidez.

Aplicabilidad y decisiones de diseño

La experiencia obtenida en nuestro caso de estudio reafirma la aplicabilidad y flexibilidad del método de cálculo propuesto por Bahr y Hutton, incluso en contextos donde el lenguaje objetivo posee una semántica enriquecida y construcciones adicionales. La estrategia de trabajar con definiciones parciales se mantuvo como un principio guía fundamental: permitió posponer decisiones de implementación sin perder el hilo conductor del cálculo, facilitando así la adaptación del método a nuevas exigencias sintácticas y semánticas.

Un ejemplo destacado de esta dinámica fue la definición inicial de la función de conversión (*conv*), la cual se formuló de forma incompleta, dejando abiertos casos como los pares ordenados y las cerraduras de función. Esta omisión no implicó una debilidad en la especificación, sino todo lo contrario: al diferir estas decisiones, fue posible completarlas de manera coherente una vez que los razonamientos lo exigían, asegurando que las definiciones finales se alinearan de manera natural con las estructuras derivadas del compilador y máquina virtual. Este enfoque confirma una de las virtudes esenciales del método: permitir que las propias necesidades inductivas y semánticas guíen la evolución del proceso de cálculo, en lugar de imponer restricciones arbitrarias desde el inicio. Así, la modularidad y la progresividad no solo facilitan la implementación, sino que además ofrecen una estrategia robusta para escalar el método a lenguajes más complejos sin comprometer su claridad ni consistencia.

Sin embargo, también es importante reconocer los límites de esta aproximación. Tal como se discutió previamente, la corrección garantizada por la especificación adoptada (centrada en la completitud) no cubre de forma explícita la propiedad dual de solidez. Este límite no surge de un error metodológico, sino de una consecuencia directa de la parcialidad asumida en la semántica relacional y en la máquina virtual que hemos considerado. Aunque el método se aplica exitosamente en contextos más expresivos y modulares, su alcance en términos de corrección total permanece condicionado por las propiedades de la semántica subyacente. En conjunto, estos hallazgos refuerzan el valor del método como una herramienta efectiva para derivar compiladores correctos de manera sistemática y extensible. Al mismo tiempo, abren una línea de trabajo prometedora: explorar cómo fortalecer las garantías de corrección (particularmente la solidez) en entornos con semánticas parciales o no deterministas, ya sea mediante tipos dependientes, técnicas adicionales de verificación formal, o extensiones del propio marco de cálculo. Este equilibrio entre potencia práctica y precisión formal constituye uno de los desafíos más estimulantes para el desarrollo futuro del enfoque.

4 Caso de estudio 2: IRIS

En este segundo caso de estudio se extienden las características semánticas presentadas en el primer caso de estudio, ALANIS, incorporando un régimen de evaluación perezoso y la adopción de la estrategia de paso de parámetros *Call-by-Name*. A pesar de estos cambios semánticos significativos, se mantiene la *sintaxis léxica*, la *sintaxis abstracta* y la representación de los árboles de sintaxis abstracta en HASKELL, tal como fueron formalizados en el primer caso de estudio (ver secciones 3.1, 3.2 y 3.2.1).

La decisión de conservar esta base sintáctica común se fundamenta en tres razones fundamentales:

- **Contraste de aplicabilidad:** El uso de una sintaxis compartida facilita la comparación directa entre diferentes enfoques semánticos, permitiendo analizar con mayor precisión las adaptaciones necesarias del método ante la transición de un régimen de evaluación ansioso a uno perezoso.
- **Reutilización del trabajo previo:** Aprovechar la formalización existente evita la redundancia y permite enfocar el esfuerzo en el rediseño semántico, aportando coherencia al proceso de evolución del lenguaje.
- **Eficiencia y extensibilidad:** La continuidad sintáctica favorece la expansión progresiva del lenguaje y permite incorporar futuras extensiones semánticas de manera más sistemática.

En consecuencia, la siguiente sección se centra exclusivamente en la reformulación semántica del lenguaje para adaptarlo al nuevo régimen de evaluación perezoso, sin alterar la base sintáctica establecida en ALANIS.

4.1. Formalización de la semántica operacional

Dando continuidad al segundo caso de estudio, adoptamos en esta sección un enfoque basado en la *semántica operacional de paso pequeño* (o *semántica estructural*, véase la sección 2.5.1) con el fin de capturar con precisión la dinámica de evaluación bajo un régimen perezoso. A diferencia de la semántica de paso grande empleada en el primer caso, donde se modelaban únicamente las evaluaciones completas desde expresiones iniciales hasta valores finales, el enfoque de paso pequeño permite descomponer el proceso en una secuencia detallada de transiciones entre estados intermedios.

Esta decisión metodológica resulta especialmente adecuada en presencia de constructos como

los *thunks*, cuya evaluación diferida requiere una descripción explícita de cuándo y cómo son forzados a evaluar. En este contexto, la granularidad del paso pequeño no solo facilita el modelado fino del control de evaluación, sino que también proporciona una base sólida para el análisis y la verificación formal del comportamiento del lenguaje.

Los principales motivos que justifican el uso de semántica de paso pequeño son los siguientes:

- **Representación explícita de transiciones intermedias:** La semántica de paso pequeño modela la evaluación mediante reglas que describen transformaciones locales entre configuraciones. Esta capacidad es fundamental en ambientes perezosos, donde las expresiones pueden permanecer en forma de *think* hasta que su evaluación sea requerida. Al detallar cada transición, se obtiene una trazabilidad precisa del momento en que se produce el forzamiento.
- **Control sobre el diferimiento y forzamiento:** En lenguajes con evaluación perezosa, la introducción de puntos estrictos exige un control fino sobre el momento en que ciertas subexpresiones deben evaluarse. Este control se logra de forma natural mediante reglas específicas en el sistema de transición, lo que permite definir con claridad las condiciones bajo las cuales se debe abandonar el diferimiento y proceder con la evaluación.
- **Soporte riguroso para el razonamiento formal:** La descomposición del proceso de evaluación en pasos elementales alinea esta semántica con el enfoque del método de cálculo adoptado. Esta característica nos permite estructurar pruebas por inducción más detalladas, facilita el razonamiento ecuacional, y habilita la verificación de propiedades mediante transiciones más moduladas y transparentes.

Definición del sistema de transición

Como parte del nuevo enfoque semántico adoptado en este segundo caso de estudio, se introduce una representación intermedia denominada *runtime expressions*¹. A diferencia de la semántica de paso grande, esta representación permite describir de manera explícita los estados intermedios que atraviesa el programa durante su evaluación.

Este cambio resulta esencial en el contexto de evaluación perezosa, ya que permite reflejar con precisión cuándo una expresión permanece suspendida en forma de *think*, y cuándo es forzada a evaluarse. Para ello, la noción original de estado se extiende incorporando nuevos constructos que enriquecen el modelado del proceso de evaluación. En particular, se introduce la construcción *Think*, acompañada de su correspondiente ambiente y de la expresión diferida que será evaluada bajo demanda.

El sistema de transición resultante se articula en torno a tres componentes esenciales: los estados iniciales, los estados intermedios y los estados finales. A través de una relación de transición, se describe la evolución paso a paso de estos estados, estableciendo así las reglas que rigen el comportamiento del lenguaje bajo el nuevo régimen de evaluación.

¹Estas expresiones en tiempo de ejecución permiten modelar la evolución paso a paso de un programa, capturando fenómenos dinámicos como excepciones o transferencias de control, que interrumpen o alteran la evaluación antes de alcanzar un estado final [62].

Estados

En este sistema, los estados corresponden a expresiones bien formadas del lenguaje en su representación mediante ASA. Formalmente, el conjunto de estados E se define análogamente como:

$$E = \{a \mid a \text{ es un ASA válido en la sintaxis abstracta del lenguaje}\}.$$

Estados iniciales

Los estados iniciales I están constituidos por todas las expresiones bien formadas del lenguaje, es decir, por el conjunto completo de ASA definidos en E :

$$I = \{a \mid a \in E\}.$$

Estados intermedios

A diferencia del enfoque de paso grande, en el presente modelo se introducen estados intermedios para capturar la evaluación paso a paso del programa. Estos estados, denotados por M , representan expresiones en tiempo de ejecución (*runtime expressions*), y extienden los constructos del lenguaje original con elementos que permiten gestionar la evaluación diferida.

En particular, se incorporan tres formas principales:

- **Closure:** Representa una función junto con su ambiente de evaluación. Se denota por $Closure(\lambda x.e, \varepsilon)$, donde $\lambda x.e$ es una abstracción lambda (como aparece en los ASA) y ε es el ambiente correspondiente.
- **Thunk:** Representa una expresión diferida, acompañada del ambiente (ε) bajo el cual será evaluada cuando se requiera. Se expresa como $Thunk(e, \varepsilon)$, donde $e \in I$.
- **Error:** Representa el único error diagnosticado explícitamente por nuestra semántica, la división entre cero.

El conjunto de estados intermedios queda formalizado como:

$$\begin{aligned} M = I \cup \{ & Closure(f, \varepsilon) \mid f \text{ es una función en } I \text{ y } \varepsilon \text{ es un ambiente} \} \\ & \cup \{ Thunk(e, \varepsilon) \mid e \in I \text{ y } \varepsilon \text{ es un ambiente} \} \\ & \cup \{ Error \mid \text{la ejecución es interrumpida} \}. \end{aligned}$$

Estados finales

Los estados finales F representan valores evaluados, es decir, resultados de la ejecución completa de una expresión. Aunque son similares a los del caso anterior, en este sistema se incorpora la capacidad de mantener expresiones diferidas como valores válidos, reflejando el carácter perezoso de la evaluación.

Formalmente, se definen como:

$$\begin{aligned} F = & \{Num_V(n) \mid n \in \mathbb{Z}\} \cup \{Boolean_V(b) \mid b \in \mathbb{B}\} \cup \{Pair_V(x, y) \mid x, y \in F\} \\ & \cup \{Closure_V(f, \varepsilon) \mid f \text{ es una función y } \varepsilon \text{ es un ambiente}\} \\ & \cup \{Error_V \mid \text{la ejecución es interrumpida}\} \cup \{Thunk_V(e, \varepsilon) \mid e \in I \text{ y } \varepsilon \text{ es un ambiente}\}. \end{aligned}$$

La distinción entre *Closure* y *Thunk* en nuestra especificación responde tanto a fundamentos semánticos como a convenciones ampliamente reconocidas en la literatura especializada [34]. En este contexto, reservamos el constructo *Closure* para funciones, esto es, abstracciones lambda acompañadas del ambiente en el que deben evaluarse, mientras que *Thunk* representa cualquier expresión cuya evaluación ha sido diferida.

Ambos constructos permiten encapsular una expresión junto a su ambiente de evaluación, lo cual resulta esencial bajo un régimen de evaluación perezoso con estrategia *Call-by-Name*. Esta aproximación, bien fundamentada en trabajos como los de Peyton Jones [34] y Krishnamurthi [20], facilita el modelado de los distintos momentos en que una evaluación debe ser activada: mientras que los *closures* capturan funciones que se activan al ser aplicadas, los *thunks* permiten posponer la evaluación hasta que una expresión se vuelve estrictamente necesaria.

Desde una perspectiva semántica, esta separación aporta claridad a las reglas de transición del lenguaje, permitiendo especificar con precisión cuándo debe evaluarse una expresión y cuándo debe preservarse en forma suspendida. Esta organización no sólo alinea nuestro enfoque con modelos semánticos clásicos como el de Abelson y Sussman [31], sino que también sienta las bases para posibles extensiones futuras, tales como la incorporación de estrategias de memoización o el uso compartido de resultados (*sharing*), ampliamente exploradas en otros marcos teóricos [35] pero que no forman parte del presente estudio.

En pocas palabras, el uso diferenciado de *thunks* y *closures* en nuestra semántica no representa una decisión de implementación, sino una herramienta de especificación que permite modelar el comportamiento esperado de un lenguaje funcional perezoso. Esta diferenciación habilita una descripción más expresiva de los estados intermedios, manteniendo la elegancia de una semántica formal y respetando el principio de evaluación diferida que guía nuestro segundo caso de estudio.

Relación de transición

A continuación, definimos la relación de transición elemental, $\rightarrow$, que opera sobre configuraciones de la forma $\langle e, \varepsilon \rangle$, donde $e \in M$ representa una expresión en tiempo de ejecución y ε es el ambiente léxico.

Aunque las expresiones iniciales del lenguaje pertenecen al subconjunto sintáctico $I \subseteq M$, es necesario definir la relación $\rightarrow$ sobre el conjunto extendido M , ya que la evaluación perezosa introduce constructos intermedios (como *thunks*) que no forman parte de la sintaxis original. De este modo, toda configuración alcanzable a lo largo de la evaluación permanece en M , asegurando la cerradura semántica del sistema bajo $\rightarrow$. La justificación detallada de esta elección se expone en la sección 4.1.

Esta notación hace explícito el estado de evaluación en cada paso y facilita la descripción formal del proceso de reducción.

La relación se establece como sigue:

1. R-GENERAL

$$\langle e, \varepsilon \rangle \rightarrow \langle e', \varepsilon' \rangle$$

indica que la expresión e se reduce a e' en un solo paso de evaluación, y el ambiente puede cambiar o permanecer igual. Aunque en muchas reglas se mantiene $\varepsilon = \varepsilon'$, escribir el nuevo ambiente de forma explícita permite capturar los casos en los que hay cambios.

Esta práctica es común en la especificación de semánticas operacionales, donde se prefiere una notación general para no restringir innecesariamente las reglas [63]. Es especialmente útil en construcciones que requieren extender el ambiente, como ocurre con la aplicación de funciones, donde el cuerpo se evalúa en un ambiente que incluye la nueva asociación del argumento real.

Esta forma también se ajusta al comportamiento esperado en semánticas naturales y estructurales, donde se busca mantener consistencia en el ambiente a lo largo de la evaluación de expresiones compuestas [3]. Por ello, cuando se produce una modificación del ambiente, esta debe propagarse para que la evaluación siga siendo coherente.

En este sentido, permitir ambientes distintos en las transiciones es una decisión notacional que hace la semántica más flexible. Aunque en muchos casos concretos el ambiente no cambia, mantener ε' en la notación evita ambigüedades y facilita la extensión a situaciones donde sí hay efectos sobre el ambiente.

2. R-THUNK

$$\frac{\langle e, \varepsilon' \rangle \rightarrow \langle e', \varepsilon'' \rangle}{\langle \theta(e, \varepsilon'), \varepsilon \rangle \rightarrow \langle \theta(e', \varepsilon''), \varepsilon \rangle}$$

Esta regla muestra que, al evaluar una configuración que contiene un *thunk* $\theta(e, \varepsilon')$, la reducción se realiza dentro del cuerpo diferido, sin modificar el ambiente exterior ε . Esto refleja el comportamiento propio de los *thunks*, que retienen el ambiente donde fueron creados.

Incluso, vale notar que esta formulación se alinea con el tratamiento clásico de ciertas construcciones en semánticas operacionales estructurales. Por ejemplo, en [19] y [64], se describe cómo expresiones del tipo *Let* se evalúan extendiendo el ambiente antes de continuar con el cuerpo. Esta evaluación se comporta igual que una aplicación de función, lo cual respalda el uso de ambientes explícitamente distintos $\varepsilon \rightarrow \varepsilon'$ incluso en expresiones primitivas [33].

Introducción de la relación auxiliar *force*

Cuando una configuración contiene un *thunk*, es necesario evaluar su cuerpo para obtener un valor canónico². Para este propósito, introducimos la relación auxiliar *force*, que abstraer el comportamiento esperado de los puntos estrictos: fuerza la evaluación del contenido de un *thunk* aplicando recursivamente la relación $\rightarrow$ hasta alcanzar una configuración canónica. A diferencia de $\rightarrow$, que opera sobre configuraciones completas, *force* actúa únicamente sobre *thunks*, permitiendo distinguir entre aquellos que deben evaluarse y los que ya representan un valor. Esta relación es utilizada exclusivamente en las construcciones del lenguaje que integran puntos estrictos, es decir, aquellas que requieren forzosamente conocer el valor evaluado de una subexpresión para continuar. De este modo, *force* garantiza que la evaluación avance únicamente donde el contexto semántico impone una necesidad estricta de cómputo. La relación *force* se define formalmente, como:

$$force(\theta(e, \varepsilon)) \Longrightarrow \langle v, \varepsilon'' \rangle \iff \langle e, \varepsilon \rangle \rightarrow^* \langle v, \varepsilon'' \rangle \wedge \neg \exists \langle v', \varepsilon''' \rangle \mid \langle v, \varepsilon'' \rangle \rightarrow \langle v', \varepsilon''' \rangle,$$

donde $\rightarrow^*$ denota el cierre reflexivo-transitivo de la relación $\rightarrow$ (véase la definición 2.5.2), y $v \in M$ es un valor canónico.

En el caso de que una subexpresión sea un *thunk*, la regla R-THUNK garantiza que la evaluación se realiza internamente, respetando su ambiente original sin afectar el ambiente exterior. Sin embargo, cuando se requiere un valor explícito para continuar, *force* asegura su completa evaluación. Este mecanismo permite separar claramente los contextos en los que una evaluación diferida puede mantenerse de aquellos donde debe materializarse.

Por ejemplo, al evaluar una expresión condicional de la forma (*if* x *y* x), es indispensable conocer el valor lógico de la condición x para determinar el camino de ejecución correspondiente. Si x resulta ser un *thunk*, entonces debe forzarse mediante *force* hasta obtener un valor booleano canónico (*True* o *False*) que permita aplicar la regla correspondiente del evaluador. En este contexto, la subexpresión x actúa como un punto estricto, ya que su evaluación no puede diferirse sin comprometer la semántica del programa.

La relación se formaliza mediante las siguientes reglas:

1. F-THUNK-STEP

$$\frac{\langle t, \varepsilon \rangle \rightarrow \langle t', \varepsilon' \rangle}{force(\theta(t, \varepsilon)) \Longrightarrow force(\theta(t', \varepsilon'))}$$

Si el cuerpo del *thunk* puede dar un paso de evaluación, se continúa forzando la evaluación del nuevo *thunk* resultante.

2. F-THUNK-VALUE

$$\frac{t \text{ es canónico}}{force(\theta(t, \varepsilon)) \Longrightarrow \langle t, \varepsilon \rangle}$$

²En este trabajo, un *valor canónico* es una expresión que ya no puede ser evaluada más bajo la relación de transición definida; es decir, se trata de una forma irreducible que representa el resultado final de una evaluación.

Cuando el cuerpo ya no puede reducirse, *force* devuelve el valor final junto con su ambiente.

La relación auxiliar *force* cumple un papel fundamental en la semántica de reducción paso a paso, al introducir un mecanismo explícito para modelar los contextos estrictos del lenguaje. Esta separación de responsabilidades permite preservar la generalidad y simplicidad de la relación principal $\rightarrow$, delegando en *force* el control preciso de la evaluación en los puntos donde se requiere forzar la evaluación de un *thunk*.

Este diseño evidencia una de las ventajas clave del enfoque de semántica de paso pequeño adoptado: su capacidad para describir de manera modular y controlada el comportamiento dinámico de construcciones que interactúan con evaluación diferida. En particular, permite especificar con exactitud cuándo y cómo debe activarse la evaluación en posiciones estrictas, sin imponer una estrategia de evaluación global, lo que preserva la modularidad y expresividad del sistema.

Justificación de la coherencia de tipos y de la normalización

La semántica operacional definida, junto con la relación auxiliar *force*, se apoya en la estructura del conjunto M , el cual extiende a I mediante la incorporación de tres formas adicionales: *closures*, *thunks* y un estado de *error*. Esta extensión asegura que cualquier estado alcanzado durante la evaluación de una expresión inicial $e \in I$ permanezca dentro del dominio M , preservando así la coherencia del sistema.

1. **Coherencia de las construcciones:** Las construcciones *thunk* $\theta(t, \varepsilon)$ y *closure* se generan exclusivamente como resultado de la evaluación. Por lo tanto, si partimos de una expresión $t \in I$, cada paso de evaluación produce una expresión en M , garantizando que todo estado intermedio se mantenga dentro del dominio semánticamente válido.

2. **Cerradura y Normalización en M :**

- Por construcción, $I \subseteq M$; es decir, cualquier expresión bien formada del lenguaje pertenece también al conjunto de estados intermedios.
- Además, las reglas de evaluación están definidas de manera que, si un estado $m \in M$ se reduce a otro m' mediante un paso de evaluación $m \rightarrow m'$, entonces $m' \in M$. Esto garantiza que M es cerrado bajo la relación de evaluación: toda evolución semántica permanece dentro del dominio previsto.
- Como consecuencia, si partimos de una configuración inicial $\langle e, \varepsilon \rangle$, con $e \in I$, toda la secuencia de evaluación $\langle e, \varepsilon \rangle \rightarrow^* \langle v, \varepsilon' \rangle$ transcurre en M . Y dado que v es un valor canónico de acuerdo con la semántica perezosa, se sigue que $v \in M$, aunque no necesariamente en I . Esto refleja que los valores finales pueden incluir construcciones de error y evaluación diferida (como *thunks*) o *closures*, propias del dominio extendido.

La relación auxiliar *force* complementa esta estructura al forzar la evaluación de aquellas construcciones diferidas (*thunks*) cuando la semántica lo requiere. Como *force* actúa únicamente sobre elementos en M , y sus resultados también están contenidos en M , se garantiza

que la evaluación respeta los límites del dominio. En conjunto, estas propiedades aseguran tanto la coherencia de tipos como la robustez de la evaluación dentro del marco definido por M .

Llegados a este punto, podemos definir formalmente las reglas de inferencia correspondientes a cada construcción del lenguaje.

Num n

$$\frac{n \in \mathbb{Z}}{\langle \text{Num } n, \varepsilon \rangle \rightarrow \langle \text{Num } n, \varepsilon \rangle} \quad (\text{NUM})$$

Esta regla exhibe que un número n bajo la relación $\rightarrow$ es canónico.

Boolean b

$$\frac{b \in \mathbb{B}}{\langle \text{Boolean } b, \varepsilon \rangle \rightarrow \langle \text{Boolean } b, \varepsilon \rangle} \quad (\text{BOOLEAN})$$

De forma análoga al primer caso de estudio, esta construcción recibe como único parámetro un valor booleano b .

Var x

$$\frac{\text{lookup } x \ \varepsilon \text{ existe}}{\langle \text{Var } x, \varepsilon \rangle \rightarrow \langle \text{lookup } x \ \varepsilon, \varepsilon \rangle} \quad (\text{VAR})$$

Análogo a la semántica formalizada para $\Lambda\lambda\text{ANIS}$, esta construcción hace uso de la función de búsqueda *lookup* que recupera el valor en el ambiente asociado al identificador x .

Add $x \ y$

Tenemos que, la suma no puede ser efectuada hasta que todos sus argumentos sean reducidos. Por lo tanto, en los operandos de las operaciones aritméticas encontramos un punto estricto. Así, para esta construcción debemos hacer explícita la aplicación de las relaciones de transición, a través de los siguientes cinco casos:

- **Caso 1:** Ninguno de los argumentos de la suma ha sido reducido a un valor. El paso siguiente consiste aplicar una reducción elemental ($\rightarrow$) a el lado izquierdo.

$$\frac{\langle x, \varepsilon \rangle \rightarrow \langle x', \varepsilon' \rangle}{\langle \text{Add } x \ y, \varepsilon \rangle \rightarrow \langle \text{Add } x' \ y, \varepsilon' \rangle} \quad (\text{IADD})$$

- **Caso 2:** Si el primer argumento resulta ser un *thunk*, aplicamos la relación *force* para obtener su valor reducido. Obsérvese que, en este caso el ambiente léxico resultante en la configuración final, continua siendo el mismo donde se evalúo el constructo.

$$\frac{force(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle Add(\theta(t, \varepsilon')) y, \varepsilon \rangle \rightarrow \langle Add v y, \varepsilon \rangle} \quad (\text{F-IADD})$$

- **Caso 3:** El argumento izquierdo de la suma ha sido reducido a un número, pero el derecho no, el paso siguiente es proceder con el lado derecho.

$$\frac{\langle y, \varepsilon \rangle \rightarrow \langle y', \varepsilon' \rangle}{\langle Add (Num n) y, \varepsilon \rangle \rightarrow \langle Add (Num n) y', \varepsilon' \rangle} \quad (\text{DADD})$$

- **Caso 4:** En caso de que el segundo argumento resulte en un *thunk*, el procedimiento es análogo a la regla F-IADD.

$$\frac{force(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle Add (Num n) (\theta(t, \varepsilon')), \varepsilon \rangle \rightarrow \langle Add (Num n) v, \varepsilon \rangle} \quad (\text{F-DADD})$$

- **Caso 5:** Los dos argumentos de la suma han sido reducidos a un número. El resto es realizar la suma directamente.

$$\frac{}{\langle Add (Num n) (Num m), \varepsilon \rangle \rightarrow \langle Num (n +_{\mathbb{Z}} m), \varepsilon \rangle} \quad (\text{ADD})$$

De manera análoga, las reglas para las construcciones *Sub*, *Mul*, *Div* y *Eq* siguen la estructura postergada de su evaluación y forzamiento de subexpresiones. Pues ciertamente, estas construcciones solo pueden ser evaluadas al reducir a valores canónicos sus subexpresiones, difiriendo entre ellas tan solo en cómo son operados estos valores.

Las reglas F-IADD y F-DADD exhiben un comportamiento fundamental de la evaluación en nuestro lenguaje: antes de poder efectuar una suma, es necesario reducir completamente sus operandos. En estos casos, si alguno de los argumentos se encuentra encapsulado en un *thunk*, se invoca explícitamente la relación auxiliar *force* para forzar su evaluación hasta alcanzar un valor canónico. Esta exigencia de evaluación previa refleja la presencia de *puntos estrictos*.

Este patrón de evaluación estricta no se limita al caso de la suma: se replica de forma análoga en el resto de las operaciones binarias como *Sub*, *Mul*, *Div* y *Eq*. En todos estos casos, la semántica exige que ambos argumentos sean evaluados (y forzados si es necesario) antes de aplicar la operación correspondiente.

A continuación, detallamos los puntos estrictos identificados en otras construcciones del lenguaje:

- **If:** La condición debe evaluarse de manera estricta para determinar cuál rama (*then* o *else*) debe ejecutarse. Esto se debe a que no se puede seleccionar una alternativa sin conocer el valor booleano de la condición.
- **Fst:** El argumento de esta construcción debe ser forzado hasta obtener un valor par, ya que no es posible acceder a su primera componente si la estructura no ha sido completamente evaluada.

- **App**: En la aplicación de funciones, la expresión que representa la función se evalúa estrictamente hasta alcanzar una *Closure*; mientras que el argumento se encapsula como un *Thunk* y se evalúa solo si el cuerpo de la función lo requiere, lo cual es coherente con el régimen de evaluación perezoso adoptado.
- **Pair**: Los componentes del par no se evalúan de inmediato. Su evaluación depende del contexto: por ejemplo, si una proyección como *Fst* accede a uno de ellos, entonces se fuerza la evaluación de ese componente.
- **Lambda**: Las funciones lambda no se evalúan durante su declaración, sino que se consideran valores por sí mismas (encapsuladas como *Closure*) y solo se evalúan cuando son aplicadas.

Esta caracterización de los puntos estrictos en IRIS permite una implementación precisa del modelo de evaluación perezosa mediante semántica estructural de paso pequeño. Al distinguir claramente qué subexpresiones deben evaluarse en cada contexto, se garantiza un control granular del proceso de reducción.

Dicho lo anterior, establecemos una generalización de los primeros cuatro casos revisados para el constructo *Add*.

Sea $C \in \{Sub, Mul, Div, Eq\}$ en las siguientes reglas genéricas:

- **Caso 1**: Si ninguno de los argumentos del constructo ha sido reducido a un valor, aplicamos una reducción elemental a la subexpresión izquierda.

$$\frac{\langle x, \varepsilon \rangle \rightarrow \langle x', \varepsilon' \rangle}{\langle C x y, \varepsilon \rangle \rightarrow \langle C x' y, \varepsilon' \rangle} \quad (\text{IC})$$

- **Caso 2**: Si la subexpresión izquierda resulta ser un *think*, aplicamos la relación *force*.

$$\frac{force(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle C (\theta(t, \varepsilon')) y, \varepsilon \rangle \rightarrow \langle C v y, \varepsilon \rangle} \quad (\text{F-IC})$$

- **Caso 3**: El argumento izquierdo de C ha sido reducido a un número pero el derecho no. Entonces, continuamos con la subexpresión derecha.

$$\frac{\langle y, \varepsilon \rangle \rightarrow \langle y', \varepsilon' \rangle}{\langle C (Num n) y, \varepsilon \rangle \rightarrow \langle C (Num n) y', \varepsilon' \rangle} \quad (\text{DC})$$

- **Caso 4**: En caso de que el segundo argumento resulte en un *think*, el procedimiento es análogo a la regla a F-IC.

$$\frac{force(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle C (Num n) (\theta(t, \varepsilon')), \varepsilon \rangle \rightarrow \langle C (Num n) v, \varepsilon \rangle} \quad (\text{F-DC})$$

Aún debemos definir también el quinto caso, es decir, cuando tenemos $\langle C (Num n) (Num m), \varepsilon \rangle$ para cada construcción anterior, así que los abordamos de forma autónoma a partir de ahora.

Sub $x y$

Para el caso de la resta, continuando desde la generalización anterior de las reglas de evaluación y a diferencia de la suma, solo es necesario introducir el operador aritmético correspondiente.

$$\frac{}{\langle \text{Sub } (\text{Num } n) (\text{Num } m), \varepsilon \rangle \rightarrow \langle \text{Num } (n -_{\mathbb{Z}} m), \varepsilon \rangle} \quad (\text{SUB})$$

Mul $x y$

La regla faltante para esta construcción es similar; basta con introducir el operador $\times_{\mathbb{Z}}$.

$$\frac{}{\langle \text{Mul } (\text{Num } n) (\text{Num } m), \varepsilon \rangle \rightarrow \langle \text{Num } (n \times_{\mathbb{Z}} m), \varepsilon \rangle} \quad (\text{MUL})$$

Div $x y$

Establezcamos de forma análoga las reglas diferenciadas por el manejo explícito del error de división por cero, introduciendo la expresión de tiempo de ejecución *Error*, para evitar la propagación del error directamente al lenguaje anfitrión.

- **Caso 1:** Denominador igual a cero.

$$\frac{m =_{\mathbb{Z}} 0}{\langle \text{Div } (\text{Num } n) (\text{Num } m), \varepsilon \rangle \rightarrow \langle \text{Error}, \varepsilon \rangle} \quad \text{DIV-CERO}$$

- **Caso 2:** Denominador distinto de cero.

$$\frac{m \neq_{\mathbb{Z}} 0}{\langle \text{Div } (\text{Num } n) (\text{Num } m), \varepsilon \rangle \rightarrow \langle \text{Num } (n /_{\mathbb{Z}} m), \varepsilon \rangle} \quad \text{DIV}$$

Eq $x y$

A diferencia de las reglas anteriores, este constructo realiza una conversión de tipo en la expresión de la configuración resultante, es decir, pasa de números a booleanos.

$$\frac{}{\langle \text{Eq } (\text{Num } n) (\text{Num } m), \varepsilon \rangle \rightarrow \langle \text{Boolean } (n =_{\mathbb{Z}} m), \varepsilon \rangle} \quad \text{EQ}$$

If $x y z$

La evaluación de esta estructura condicional se muestra en los siguientes cuatro casos:

- **Caso 1:** La condición x no ha sido reducida a un valor, entonces aplicamos una reducción elemental.

$$\frac{\langle x, \varepsilon \rangle \rightarrow \langle x', \varepsilon' \rangle}{\langle \text{If } x y z, \varepsilon \rangle \rightarrow \langle \text{If } x' y z, \varepsilon' \rangle} \quad \text{IF}$$

- **Caso 2:** Si la condición resulta en un *thunk*, aplicamos la relación *force* para reducir hasta un valor.

$$\frac{force(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle If (\theta(t, \varepsilon')) y z, \varepsilon \rangle \rightarrow \langle If v y z, \varepsilon \rangle} \quad \text{F-IF}$$

- **Caso 3:** Cuando la condición x es ya un valor booleano, basta vincular el valor de verdad con la subexpresión que continuara a ser evaluada. En el caso de que el valor encapsulado sea verdadero, se procede a evaluar la rama y .

$$\frac{b = True}{\langle If (Boolean b) y z, \varepsilon \rangle \rightarrow \langle y, \varepsilon \rangle} \quad \text{IF-TRUE}$$

- **Caso 4:** Similar a la regla anterior, pero vinculando el valor de verdad de la condición a la rama z .

$$\frac{b = False}{\langle If (Boolean b) y z, \varepsilon \rangle \rightarrow \langle z, \varepsilon \rangle} \quad \text{IF-FALSE}$$

Pair x y

Los pares ordenados, a diferencia de las demás reglas expuestas, son la primera construcción que introduce *thunks* en las configuraciones. Por ello, su evaluación y proyección son inherentemente perezosas, como se formaliza a continuación.

$$\frac{}{\langle Pair x y, \varepsilon \rangle \rightarrow \langle Pair (\theta(x, \varepsilon)) (\theta(y, \varepsilon)), \varepsilon \rangle} \quad \text{PAIR}$$

Fst x

Recordando que, para el constructo que abstrae la proyección del primer elemento de un par ordenado, debemos hacer explícito el forzamiento de su única subexpresión x , hasta que este represente visiblemente un par. Los siguientes casos describen el proceso de evaluación de este constructo.

- **Caso 1:** Si la subexpresión x no ha sido reducida, aplicamos una reducción elemental.

$$\frac{\langle x, \varepsilon \rangle \rightarrow \langle x', \varepsilon' \rangle}{\langle Fst x, \varepsilon \rangle \rightarrow \langle Fst x', \varepsilon' \rangle} \quad \text{FST1}$$

- **Caso 2:** En caso de que, la subexpresión derive en un *thunk*, se aplica la relación *force* para reducir hasta un valor.

$$\frac{force(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle Fst (\theta(t, \varepsilon')), \varepsilon \rangle \rightarrow \langle Fst v, \varepsilon \rangle} \quad \text{F-FST}$$

- **Caso 3:** La subexpresión del constructo es un par ordenado, se procede a proyectar el primer *thunk* encapsulado.

$$\frac{}{\langle Fst (Pair f s), \varepsilon \rangle \rightarrow \langle f, \varepsilon \rangle} \quad \text{FST}$$

Lambda x y

En el caso de la abstracción de funciones, introducimos nuestro constructo restante del conjunto M , el *Closure* nos permitirá capturar el ambiente léxico donde fue definida la función, preservando así el alcance estático de nuestro lenguaje. Véase el único paso de evaluación de este constructo.

$$\frac{}{\langle \text{Lambda } x y, \varepsilon \rangle \rightarrow \langle \text{Closure } (\text{Lambda } x y, \varepsilon), \varepsilon \rangle} \quad \text{ABS}$$

App x y

Para la construcción restante de nuestro lenguaje, debemos explicitar que (al igual que en constructos anteriores) la primera subexpresión x es un punto estricto. Esto implica que la aplicación de función solo puede proceder una vez x se haya reducido a una cerradura de función. Las siguientes reglas formalizan este comportamiento:

- **Caso 1:** La subexpresión x no ha sido reducida, entonces le aplicamos una reducción elemental.

$$\frac{\langle x, \varepsilon \rangle \rightarrow \langle x', \varepsilon' \rangle}{\langle \text{App } x y, \varepsilon \rangle \rightarrow \langle \text{App } x' y, \varepsilon' \rangle} \quad \text{APP1}$$

- **Caso 2:** Si la primera subexpresión resulta ser un *thunk*, aplicamos la relación *force* para reducir a un valor.

$$\frac{\text{force}(\theta(t, \varepsilon')) \implies \langle v, \varepsilon'' \rangle}{\langle \text{App } (\theta(t, \varepsilon')) y, \varepsilon \rangle \rightarrow \langle \text{App } v y, \varepsilon \rangle} \quad \text{F-APP}$$

- **Caso 3:** Una vez que la primera subexpresión x se reduce a una cerradura de función, la aplicación se formaliza como:
 - El cuerpo de la función c se evalúa en el ambiente extendido $\varepsilon'[p \leftarrow \theta(y, \varepsilon)]$
 - donde $\theta(y, \varepsilon)$ es el *thunk* que encapsula el argumento real y y el ambiente actual.

$$\frac{f = (\text{Closure } (\text{Lambda } p c) \varepsilon')}{\langle \text{App } x y, \varepsilon \rangle \rightarrow \langle c, \varepsilon'[p \leftarrow \theta(y, \varepsilon)] \rangle} \quad \text{APP}$$

Transformación de M a F

Hasta ahora, hemos definido las reglas de paso pequeño para describir cómo se evalúan las expresiones y se fuerzan los *thunks* mediante la relación auxiliar *force*. Sin embargo, para cumplir con la metodología de Bahr y Hutton, es necesario disponer de una función de evaluación general. Esta función corresponde a una nueva relación que será la encargada de aplicar iterativamente las reglas de transición previamente definidas hasta obtener un valor canónico, el cual posteriormente se mapeará del conjunto intermedio M al conjunto de estados finales F .

Formalmente, definimos la relación global de evaluación, denotada por $\Downarrow$, de la siguiente manera:

$$\langle s, \varepsilon \rangle \Downarrow v \iff \langle s, \varepsilon \rangle \rightarrow^* \langle m, \varepsilon' \rangle, \text{ } m \text{ es canónico, y } v = \varphi(m),$$

donde:

- $s \in I$ es la expresión inicial.
- ε es el ambiente de evaluación.
- $\rightarrow^*$ denota la aplicación iterativa de la relación de transición $\rightarrow$ (la cual, en determinados pasos, utiliza *force* para evaluar *thunks*).
- $m \in M$ es el estado canónico alcanzado.
- $\varphi : M \rightarrow F$ es el mapeo que transforma los valores canónicos de M en la representación final en F .

El proceso de evaluación se describe de la siguiente manera:

1. Se parte de una configuración inicial $\langle s, \varepsilon \rangle$, donde $s \in I$.
2. La relación $\rightarrow$ se aplica de forma iterativa para transformar la configuración, llevando la evaluación dentro del conjunto M .
3. Una vez que se alcanza una configuración canónica $\langle m, \varepsilon' \rangle$, se aplica el mapeo φ para obtener el valor final $v \in F$.

Esta formulación integra de forma coherente las reglas de paso pequeño y la evaluación forzada, asegurando que toda entrada en I se procese dentro del dominio intermedio M y, al final, se transforme en un estado final en F .

Sobre la notación utilizada. Aunque esta relación abstraer el mismo proceso de evaluación que en los lenguajes MINIB y ALANIS, es decir, una relación que asocia expresiones con valores, en este caso adoptamos la notación $\Downarrow$ en lugar de $\Rightarrow$ (o $\Rightarrow_\varepsilon$, utilizada anteriormente para hacer explícito el uso de ambientes).

Esta decisión notacional tiene como propósito distinguir claramente entre los distintos estilos de semántica empleados. En el caso de ALANIS, la notación $\Rightarrow_\varepsilon$ enfatiza el papel del ambiente en la semántica natural, mientras que aquí, al modelar de manera explícita los ambientes como parte de las configuraciones $\langle e, \varepsilon \rangle$, resulta más apropiado emplear una notación diferente que refleje este enfoque más estructurado.

Así, el uso de $\Downarrow$ permite evitar ambigüedades y sobrecarga notacional, al tiempo que señala el cambio metodológico hacia una semántica de paso pequeño con transiciones intermedias. Esta distinción favorece la claridad conceptual entre los diferentes modelos, incluso cuando comparten una base semántica común.

Función de mapeo φ : Esta función transforma un valor canónico en M a su correspondiente estado final en F (i.e., $\varphi : M \rightarrow F$). Se define por casos de la siguiente manera:

$$\varphi(m) = \begin{cases} Num_V(n) & \text{si } m = Num(n), \\ Boolean_V(b) & \text{si } m = Boolean(b), \\ Pair_V(Thunk_V(f, \varepsilon), Thunk_V(s, \varepsilon)) & \text{si } m = Pair(Thunk(f, \varepsilon), Thunk(s, \varepsilon)), \\ Closure_V(e, \varepsilon) & \text{si } m = Closure(e, \varepsilon), \\ Thunk_V(e, \varepsilon) & \text{si } m = Thunk(e, \varepsilon), \\ Error_V & \text{si } m = Error. \end{cases} \quad (4.1)$$

4.2. Introducción al método

Para alinear nuestro estudio con el enfoque de los autores, debemos formular y definir los componentes principales necesarios para aplicar el método a nuestro lenguaje. Los elementos fundamentales incluyen: el conjunto de expresiones admitidas por el lenguaje E , el conjunto de valores F , definido en la formalización semántica de nuestro lenguaje; el conjunto de expresiones en tiempo de ejecución M y el ambiente de evaluación.

4.2.1. Representación en HASKELL

Conjunto de expresiones (E)

El conjunto de expresiones admitidas por el lenguaje, E , se encuentra representado mediante la definición del tipo algebraico *Expr* en HASKELL (véase la sección 3.1). Esta representación formaliza la sintaxis del lenguaje y establece la base sobre la cual se define la función de evaluación.

Conjunto de expresiones de tiempo de ejecución (M)

Para modelar adecuadamente el proceso de evaluación en nuestro sistema, es necesario definir un conjunto extendido de expresiones, previamente denotado como M . Este conjunto no solo incluye las expresiones originales del lenguaje, representadas en I , sino también los constructos adicionales que emergen durante la evaluación, como los *thunks* o los *closures*, introducidos por la semántica operacional.

Sin embargo, en HASKELL existen restricciones en la reutilización de nombres dentro de los constructores de tipos [59]. Es decir, no es posible declarar dos tipos de datos distintos que utilicen exactamente los mismos nombres de constructores. Esta limitación impide que el conjunto M se defina simplemente extendiendo I con nuevos casos dentro de la misma declaración de tipo.

Adicionalmente, incluso si fuese técnicamente viable, incorporar explícitamente cada constructor de I dentro de M resultaría en una duplicación innecesaria de la estructura sintáctica original, lo que aumentaría la complejidad y disminuiría la claridad del modelo.

Para resolver este problema de manera elegante y modular, se adopta una estrategia en la que las expresiones de I son encapsuladas dentro de un nuevo constructor en la definición de M . Esto permite preservar la distinción explícita entre las expresiones originales del lenguaje y los elementos añadidos por la semántica de ejecución, manteniendo al mismo tiempo una definición más limpia, flexible y extensible del conjunto de estados de tiempo de ejecución.

En consecuencia, definimos el siguiente tipo de dato:

Listing 4.1: Definición del tipo *RTEpr* para representar expresiones en tiempo de ejecución

```
data RTEpr = S Expr
           | C Expr Env
           | T Expr Env
           | F RTEpr
           | E
```

La construcción *RTEpr* cumple con los siguientes objetivos:

- Permitir que las configuraciones de evaluación operen de manera uniforme sobre el conjunto M , el cual extiende a I de forma cerrada y normalizada (véase la sección 4.1).
- Facilitar una distinción explícita entre los elementos originales del lenguaje y aquellos introducidos por la semántica operacional, evitando al mismo tiempo la duplicación de la representación interna de I . Esta decisión, como se argumenta en la sección 4.1, garantiza tanto la claridad estructural como la robustez del modelo formal.

Cada constructo de *RTEpr* cumple un papel específico:

- ***S Expr***: Este constructo encapsula una expresión del conjunto E . Su uso garantiza que la expresión inicial, al ser pasada a una configuración, se encuentre correctamente encapsulada en el conjunto extendido M .
- ***C Expr Env***: Representa la construcción de una cerradura de función que es introducida en las configuraciones durante el proceso de evaluación según las reglas semánticas definidas.
- ***T Expr Env***: Modela la evaluación diferida, o *thunk*, encapsulando tanto la expresión como su ambiente para permitir su posterior evaluación bajo demanda.
- ***F RTEpr***: Facilita la aplicación iterativa de la relación de transición elemental ($\rightarrow$) en aquellos *thunks* que requieren ser evaluados a fondo mediante la relación auxiliar *force*. De esta manera, se distingue entre *thunks* que pueden considerarse valores finales y aquellos que deben ser reducidos.
- ***E***: Representa un valor de error explícitamente manejado dentro de la especificación semántica, en lugar de delegarlo al sistema de errores de HASKELL. Este constructo, en general, permitiría modelar diversos errores semánticos del lenguaje; sin embargo, en el contexto de nuestro estudio, se introduce únicamente para capturar y diagnosticar el error asociado a la división entre cero.

Esta representación permite pasar de una expresión original en I a un elemento encapsulado en M mediante el constructo S . De este modo, se establece un puente entre la sintaxis inicial del lenguaje y el conjunto extendido en el que se modela la evaluación. Al encapsular las expresiones originales, se garantiza que todas las configuraciones de evaluación trabajen exclusivamente con elementos de M , lo que simplifica la implementación del proceso de evaluación.

Conjunto de valores (F)

El conjunto de valores F extiende el conjunto definido en 3.4 al incorporar los constructos correspondientes a los elementos irreducibles de la representación intermedia, como se muestra a continuación:

Listing 4.2: Definición del conjunto de valores *Value* con soporte para evaluación diferida

```
data Value = NumV Int
           | BooleanV Bool
           | Closure Expr Env
           | Thunk Expr Env
           | PairV Value Value
           | ErrorV
```

En este contexto, se incluye el constructo *Thunk*, el cual representa una expresión cuyo cálculo ha sido diferido junto con su ambiente léxico. Aunque un *think* puede considerarse irreducible en tanto que no se expande espontáneamente, es decir, permanece suspendido hasta ser forzado, no necesariamente corresponde a un valor canónico del lenguaje. Su presencia dentro del conjunto de estados finales refleja que, bajo el régimen de evaluación perezoso, es válido terminar la evaluación con expresiones que aún contienen cómputos diferidos.

Ambiente de evaluación (Env)

El ambiente léxico se mantiene tal como fue definido en el primer caso de estudio (véase la sección 3.4). Esto se debe a que, a pesar de emplear una estrategia de paso de parámetros *Call-by-Name*, es necesario preservar una asociación adecuada de las expresiones con sus respectivos identificadores. Esta preservación asegura la coherencia y consistencia en la representación de las relaciones entre variables y sus valores, incluso en un contexto donde la evaluación de los parámetros se realiza de manera diferida.

Listing 4.3: Definición del tipo *Env* para representar ambientes de evaluación en IRIS

```
type Env = [(String, Value)]
```

Relación de transición

Para trasladar nuestro sistema de transición al lenguaje HASKELL, es necesario representar cada una de las relaciones de forma coherente a su formalización. Por ello, comenzamos traduciendo las reglas en el mismo orden en que aparecen en la especificación semántica.

A partir de este punto, nos referiremos a la relación de paso pequeño como *step*, identificador que también asignamos a la función correspondiente en nuestra implementación. Para asociar cada regla semántica a un patrón específico de la función *step*, es esencial capturar dichos patrones en un orden concreto. Esto se debe a que HASKELL utiliza comodines³ que generalizan los patrones, por lo que es importante establecer patrones específicos que reflejen con precisión el estado de la expresión en la configuración evaluada [59].

Utilizaremos la estructura monádica *Maybe*⁴ para indicar cuándo una expresión se encuentra en un estado irreducible.

Antes de definir la función *step* de forma concreta, recordemos que, según la formalización semántica, la relación opera sobre configuraciones de la forma $\langle e, \varepsilon \rangle$, donde e es una expresión del conjunto M y ε es el ambiente léxico. En HASKELL representamos estas configuraciones como un par ordenado, manteniendo la congruencia con la formalización original:

Listing 4.4: Representación de configuraciones del evaluador como pares de expresión en M y ambiente

```
type Config = (RTEExpr, Env)
```

Con esta representación, podemos definir la función *step* para aquellas configuraciones que contengan una expresión en forma canónica:

Listing 4.5: Casos para valores canónicos en la función *step*

```
step :: Config -> Maybe Config
step (S (Num n), env)      = Nothing
step (S (Boolean b), env) = Nothing
step (S (Pair f s), env)   = Nothing
```

Los casos correspondientes a números, booleanos y pares ordenados reflejan que, según la formulación semántica, estas expresiones no son susceptibles de seguir siendo evaluadas. En consecuencia, el valor *Nothing* se utiliza para indicar que la expresión ha alcanzado un estado irreducible dentro del sistema de transición.

Continuamos con las definiciones para la suma. En primer lugar, es suficiente capturar los patrones de las reglas ADD, DADD e IADD en ese mismo orden:

Listing 4.6: Patrones iniciales para la evaluación de la suma

```
step (S (Add (Num n) (Num m)), env) = Just (S (Num (n+m)), env)
step (S (Add (Num n) e2), env)      = ? -- por definir
step (S (Add e1 e2), env)           = ? -- por definir
```

³En HASKELL, los comodines se representan mediante el carácter de guion bajo `_`, que indica que cualquier valor es aceptado en esa posición, pero sin vincularlo a un nombre. Alternativamente, es posible utilizar una letra o un identificador cualquiera (como x , e o env) para capturar y referirse al valor que hace coincidir el patrón. Mientras que el comodín `_` permite omitir el valor, el uso de un identificador permite operar o inspeccionar dicho valor dentro del cuerpo de la función.

⁴La estructura monádica *Maybe* en HASKELL representa cálculos que pueden fallar. Consiste en un tipo constructo con dos operaciones fundamentales: *return*, que encapsula un valor exitoso como *Just x*, y el operador de encadenamiento `>>=`, que propaga el fallo representado por *Nothing* o aplica una función a un valor exitoso. Estas operaciones respetan las leyes monádicas de identidad y asociatividad, lo que garantiza que los cálculos puedan componerse de manera estructurada y segura [51].

Es importante notar que los dos últimos patrones deben capturar el comportamiento estrictamente evaluado del constructo *Add*. Esto se debe a que, como se justificó en la formalización semántica, la suma presenta puntos estrictos en ambos operandos: no es posible realizar la operación hasta que tanto el operando izquierdo como el derecho hayan sido completamente evaluados (véase la sección 4.1).

Sin embargo, dado que el tipo *RTErr* encapsula una *Expr* y que el constructo *T* (que representa un *Thunk*) también es de tipo *RTErr*, no es factible capturar de forma explícita este patrón dentro de la definición de la función *step*. Por lo tanto, es necesario que cada patrón maneje consistentemente el estado de la configuración durante la evaluación, permitiendo forzar los *thunks* cuando se encuentran en posiciones estrictas.

Considerando las reglas DADD y F-DADD, observamos que el patrón singular correspondiente a cada una ya ha sido identificado. En la regla DADD, el segundo argumento aparece como comodín, lo que permite capturar tanto su caso habitual como el caso correspondiente a F-DADD, donde este operando es un *Thunk*. De acuerdo con la definición semántica, y en concordancia con el punto estricto de la operación, el patrón genérico descrito por DADD permite dar un paso elemental en la evaluación del segundo argumento. Una vez realizado este paso, se verifica la forma resultante del operando para determinar la regla que debe aplicarse a continuación.

Listing 4.7: Desarrollo del caso DADD con manejo del segundo argumento

```
step (S (Add (Num n) e2), env) =
  case step (S e2, env) of
    Just (S e2', env')    -> Just (S (Add (Num n) e2'), env')
    Just (T e2' d, env')  ->
      case step (F (T e2' d), env') of
        Just (S v, env'') -> Just (S (Add (Num n) v), env'')
    Nothing                -> Nothing
```

En esta definición se utiliza la expresión *case* para capturar el resultado de un paso en la evaluación del segundo argumento de la suma. El primer patrón:

```
Just (S e2', env') -> Just (S (Add (Num n) e2'), env')
```

indica que, si la evaluación de e_2 produce una configuración con una expresión que no está siendo evaluada de forma diferida, se inserta directamente la expresión e_2' en la suma.

Por otro lado, el siguiente patrón:

```
Just (T e2' d, env') -> case step (F (T e2' d), env') of
  Just (S v, env'') -> Just (S (Add (Num n) v), env'')
```

captura el caso en el que e_2 es un *Thunk* (representado por $T e_2' d$). De acuerdo con la regla F-DADD, es necesario forzar el *Thunk* antes de poder continuar con la evaluación de la suma. Este paso no es opcional, sino una consecuencia directa del punto estricto que caracteriza a las operaciones aritméticas: la operación no puede proceder mientras alguno de sus operandos permanezca como un cálculo diferido.

En este contexto, encapsulamos el *Thunk* dentro del constructo *F* para señalar explícitamente que debe ser forzado. Posteriormente, aplicamos la función *step* sobre esta configuración, lo cual equivale operativamente a realizar un paso de la relación auxiliar *force* sobre el *Thunk*.

El resultado es un valor canónico v junto con un ambiente actualizado env'' , el cual se inserta en la construcción de la suma Add . De este modo, se garantiza que la evaluación continúe sobre operandos completamente reducidos, respetando la semántica estricta de la operación. Para definir el comportamiento de $step$ para el constructo F , es necesario reflejar la semántica de las reglas F-THUNK-STEP y F-THUNK-VALUE. Con la regla F-THUNK-STEP se indica que se debe aplicar iterativamente la relación $step$ hasta que la regla F-THUNK-VALUE capture el estado en el que la expresión no puede ser reducida.

De este modo, la relación auxiliar $force$ bloquea la evaluación global, propagando el resultado de una evaluación elemental como una nueva evaluación diferida hasta obtener el valor irreducible del $thunk$.

Listing 4.8: Definición del comportamiento de $step$ para el constructo F , reflejando las reglas F - $Thunk$ - $Step$ y F - $Thunk$ - $Value$

```
step (F (T expr env), oEnv) = case step (S expr, env) of
  Just (S expr', env')    -> step (F (T expr' env'), oEnv)
  Just (T expr' envT, env') -> step (F (T expr' envT), oEnv)
  Just (C expr' envC, env') -> Just (C expr' envC, oEnv)
  Nothing                 -> Just (S expr, oEnv)
```

La definición anterior establece que dar un paso en la evaluación de la expresión diferida $expr$ puede producir cuatro posibles casos, cada uno correspondiente a una regla de la relación $force$:

- F-THUNK-STEP: En el primer patrón, si tras dar un paso en la evaluación de $expr$ se obtiene una nueva expresión, se continúa la evaluación encapsulando nuevamente el $thunk$ con el constructo F . De manera similar, el segundo patrón captura el caso en que el paso en la evaluación produce otro $thunk$; en ambos casos se propaga el punto estricto F y se preserva el ambiente léxico original, garantizando que la evaluación se realice en el ambiente adecuado.
- F-THUNK-VALUE: El cuarto patrón, representado por *Nothing*, indica que no es posible realizar más pasos de evaluación, es decir, que la expresión $expr$ ya es un valor canónico. En este caso, se retorna una configuración que contiene la expresión irreducible y su ambiente, cumpliendo con la regla que determina el fin del proceso de forzado. De forma similar, el tercer patrón captura el estado en el cual la reducción elemental de $Expr$ produce una cerradura y dado que este es ya un valor irreducible, se retorna la misma configuración.

De lo anterior, tenemos entonces que la definición de $step$ para las reglas IADD y F-IADD se sigue de lo anterior, pero reduciendo al primer argumento e_1 .

Listing 4.9: Definición de $step$ para las reglas IADD y F -IADD, enfocándose en la evaluación del primer argumento

```
step (S (Add e1 e2), env) = case step (S e1, env) of
  Just (S e1', env')    -> Just (S (Add e1' e2), env')
  Just (T e1' d, env') -> case step (F (T e1' d), env') of
    Just (S v, env'') -> Just (S (Add v e2), env'')
  Nothing               -> Nothing
```

Esta definición de *step* se alinea con la formalización semántica de la relación *force*, la cual es empleada por diversas construcciones del lenguaje que incorporan puntos estrictos. En consecuencia, en la traducción de las reglas para otros constructos que requieran el uso de *force*, se seguirá un patrón similar al definido para la suma.

Las definiciones de *step* para el resto de las operaciones aritméticas y para la igualdad de enteros siguen una estructura similar. Para mejorar la legibilidad, se generaliza la definición, siguiendo las reglas IC, F-IC, DC y F-DC. La implementación completa puede consultarse en el material suplementario que acompaña este trabajo (véase Apéndice A). Sea $Op \in \{Sub, Mul, Div, Eq\}$; la definición genérica es la siguiente:

Listing 4.10: Patrones genéricos de *step* para operaciones binarias (evaluación de operandos)

```
step (S (Op (Num n) e2), env) = case step (S e2, env) of
  Just (S e2', env') -> Just (S (Op (Num n) e2'), env')
  Just (T e2' d, env') -> case step (F (T e2' d), env') of
    Just (S v, env'') -> Just (S (Op (Num n) v), env'')
  Nothing -> Nothing

step (S (Op e1 e2), env) = case step (S e1, env) of
  Just (S e1', env') -> Just (S (Op e1' e2), env')
  Just (T e1' d, env') -> case step (F (T e1' d), env') of
    Just (S v, env'') -> Just (S (Op v e2), env'')
  Nothing -> Nothing
```

Queda pendiente definir explícitamente el patrón:

Listing 4.11: Patrón genérico de *step* para operaciones binarias (evaluación del operador)

```
step (S (Op (Num n) (Num m)), env)
```

puesto que es el único punto en el que la evaluación difiere de acuerdo con la semántica particular de cada operador. Se definen, entonces, los siguientes casos por construcción, siguiendo la diferenciación expuesta en 4.1:

- **Sub**: Se inserta el operador de resta entre ambos operandos.

```
step (S (Sub (Num n) (Num m)), env) = Just (S (Num (n-m)), env)
```

- **Mul**: Similar a suma y resta, pero utilizando el operador de multiplicación.

```
step (S (Mul (Num n) (Num m)), env) = Just (S (Num (n*m)), env)
```

- **Div**: Se distinguen dos casos:

1. Cuando $m = 0$:

```
step (S (Div (Num n) (Num 0)), env) = Just (E, env)
```

2. Cuando $m \neq 0$:

```
step (S (Div (Num n) (Num m)), env)
      = Just (S (Num (n 'div' m)), env)
```

- ***Eq***: Este caso cambia el tipo de dato de la configuración.

```
step (S (Eq (Num n) (Num m)), env)
      = Just (S (Boolean (n == m)), env)
```

Con este entendimiento y al identificar la correspondencia entre cada definición y el conjunto de reglas semánticas asociadas a cada constructo, se exhibe el resto de la definición de la función *step* de manera coherente y estructurada.

Listing 4.12: Definiciones restantes de función *step*

```
step (S (Var x), env) = case lookup x env of
                          Thunk e env'   -> Just (T e env', env)
step (S (Fst p), env) = case p of
  Pair f s -> Just (T f env, env)
  -         -> case step (S p, env) of
    Just (S p', env') -> Just (S (Fst p'), env')
    Just (T p' d, env') -> case step (F (T p' d), env') of
      Just (S v, env'') -> Just (S (Fst v), env'')
      Nothing           -> Nothing
step (S (If (Boolean True) t e), env) = Just (S t, env)
step (S (If (Boolean False) t e), env) = Just (S e, env)
step (S (If c t e), env) = case step (S c, env) of
  Just (S c', env') -> Just (S (If c' t e), env')
  Just (T c' d, env') -> case step (F (T c' d), env') of
    Just (S v, env'') -> Just (S (If v t e), env'')
    Nothing           -> Nothing
step (S (Lambda a c), env) = Just (C (Lambda a c) env, env)
step (S (App f a), env) = case step (S f, env) of
  Just (C (Lambda x y) d, env') -> Just (S y, (x, Thunk a env') : d)
  Just (S f', env')             -> Just (S (App f' a), env')
  Just (T f' d, env')           -> case step (F (T f' d), env') of
    Just (S v, env'') -> Just (S (App v a), env'')
    Just (C (Lambda x y) d, env'') -> Just (S y, (x, Thunk a env'') : d)
  Nothing               -> Nothing
```

Cabe señalar que, en el caso de las variables, este es el único caso capturado que requiere atención en nuestra semántica. Esto se debe a que, dado que únicamente la aplicación de funciones introduce nuevas asociaciones en el ambiente, y estas se realizan siempre a través de *thunks*, toda consulta realizada mediante la función *lookup* necesariamente retorna el *thunk* correspondiente al identificador buscado.

En este punto ya hemos trasladado a nuestro lenguaje anfitrión las relaciones *step* y *force*. Sin embargo, falta implementar la relación global de evaluación, *eval*, que aplica de forma iterativa la relación *step* hasta obtener un valor canónico, el cual es posteriormente mapeado

por la función φ al conjunto de estados finales F .

La función *eval* tiene el mismo encabezado que en el primer caso de estudio. Es decir, se encarga de transformar inicialmente una entrada del conjunto E a su correspondiente representación en M , aplicar recursivamente *step* hasta obtener un valor irreducible, y finalmente efectuar el mapeo mediante φ . La definición es la siguiente:

Listing 4.13: Definición de función de evaluación global *eval* para IRIS

```
eval :: Expr -> Env -> Value
eval expr env = case step (S expr, env) of
  Nothing          -> case expr of
    Num n          -> NumV n
    Boolean b       -> BooleanV b
    Pair f s        -> PairV (Thunk f env) (Thunk s env)
  Just (E, env')    -> ErrorV
  Just (T s d, env') -> Thunk s d
  Just (C f d, env') -> Closure f d
  Just (S expr', env') -> eval expr' env'
```

Esta definición de la función global de evaluación se corresponde con su formalización semántica, ajustándola al lenguaje HASKELL. En ella, el caso *Nothing* representa la no reducción de la expresión, lo que conduce a la aplicación del mapeo φ para transformar la expresión irreducible de M en un valor de F . Por otro lado, al obtener un *Just* se realizan las siguientes comprobaciones:

- El caso *Just* (E, env') corresponde al manejo del error de la división entre cero.
- El caso *Just* ($T s d, env'$) se asocia al retorno de una evaluación diferida, es decir, un *thunk*.
- El caso *Just* ($C f d, env'$) se relaciona con el retorno de una cerradura de función.
- El caso recursivo *Just* ($S expr', env'$) indica la necesidad de continuar evaluando la expresión.

De este modo, la función *eval* encapsula el proceso de evaluación, transformando una expresión inicial en su representación intermedia. Para ello, genera una configuración inicial, aplica iterativamente los pasos de evaluación mediante la función *step* y, al alcanzar un valor canónico, lo mapea, análogo a la función φ , para obtener el estado final en F . Este enfoque no solo se alinea con la formalización semántica propuesta, sino que también satisface los requisitos de la metodología de los autores.

Por último, cabe recordar que previamente se hizo referencia a la función *lookup* en las definiciones de *eval* y *step*. Dado que, su único propósito es realizar la búsqueda del valor asociado a un identificador en el ambiente, y considerando que nuestro tipo de ambiente no ha sufrido modificaciones en este contexto, se empleará la misma versión de la función utilizada en el caso de estudio anterior (véase la sección 3.4.1).

4.2.2. Especificación

Al igual que en el primer caso de estudio, en esta sección enfocamos nuestra atención en el compilador y la máquina virtual correspondiente, sobre los cuales se aplicará el proceso de cálculo para derivar las definiciones de las funciones de compilación y ejecución, siguiendo la metodología presentada en la sección 3.4.2. Las firmas de las funciones que guían dicho proceso son las siguientes:

```
comp'  :: Expr -> Code -> Code
exec   :: Code -> Conf -> Conf
```

De manera similar al estudio previo, la máquina virtual se definirá sobre configuraciones del tipo:

Listing 4.14: Definición de la configuración de la máquina virtual

```
type Conf = (Stack, Env)
```

La inclusión explícita del ambiente (*Env*) en la configuración ya fue justificada anteriormente, dado que permite mantener el contexto de evaluación durante la ejecución de forma clara y controlada.

En cuanto a la pila, se representa en HASKELL como una lista de elementos del tipo *Elem*, donde cada elemento encapsula un valor final mediante el constructo *VAL*:

Listing 4.15: Definición del tipo de pila *Stack* y su componente elemental *Elem*

```
type Stack = [Elem]
data Elem = VAL Value
```

Este diseño se alinea con los principios de la metodología adoptada, y ofrece una base flexible que facilita futuras extensiones en el proceso de cálculo. El tipo *Value* agrupa las posibles formas de valores evaluados que pueden almacenarse en la pila o manipularse durante la ejecución; su definición es presentada en la sección 4.2.1.

Especificación de Corrección

En este segundo caso de estudio, mantenemos la esencia de la especificación anterior, pero adaptamos la representación de valores y ambientes para que tanto las cerraduras de función como nuestro nuevo constructo *Thunk* se carguen con sus componentes *Expr* ya compilados. Recordemos que el tipo de dato *Code* es derivado durante el proceso de cálculo. La definición extendida del nuevo tipo de *Value'* es la siguiente:

Listing 4.16: Definición del tipo de valores *Value'* adaptado para la máquina virtual

```
data Value' = NumV' Int
            | BooleanV' Bool
            | Closure' Code Env'
            | Thunk' Code Env'
            | PairV' Value' Value'
            | ErrorV'
type Env' = [(String, Value')]
```


Por su parte, reciclamos la configuración de la máquina virtual para operar sobre el mismo tipo de dato:

Listing 4.17: Configuración y pila de la máquina virtual adaptadas al conjunto *Value'*

```
data Elem = VAL Value '
type Stack = [Elem]
type Conf = (Stack, Env ')
```

Recordemos que, la introducción de *Value'* implica que las funciones de evaluación y ejecución deben operar sobre versiones bien tipadas de los valores, lo que requiere definir una función de conversión, *conv*, que mapea los valores del evaluador original a la nueva representación de la máquina virtual. Siguiendo el enfoque de los autores [1], y aprovechando la libertad para establecer definiciones parciales que serán refinadas durante el proceso de cálculo, se propone la siguiente definición:

Listing 4.18: Definición parcial de la función *conv* para transformar valores evaluados en valores ejecutables

```
conv :: Value -> Value '
conv (NumV n)           = NumV ' n
conv (BooleanV b)      = BooleanV ' b
conv ErrorV            = ErrorV '
conv (PairV x y)        = ??? -- Por definir
conv (Closure (Lambda x y) e) = ??? -- Por definir
conv (Thunk e d)       = ??? -- Por definir
```

Del mismo modo, evoquemos la conversión del ambiente de evaluación mediante la función *conv_E*, que preserva la estructura original:

Listing 4.19: Definición de la función de conversión *conv_E* para transformar ambientes de evaluación

```
convE :: Env -> Env '
convE = map (\(x, y) -> (x, conv y))
```

Finalmente, la especificación de corrección para la función *comp'* se ajusta a nuestra nueva relación de evaluación, conservando ambos lados de la igualdad propuesta para el primer caso de estudio.

$$\text{exec } (comp' x c) (s, conv_E e) = \text{exec } c (VAL (conv v) : s, conv_E e) \quad \text{si } \langle x, \varepsilon \rangle \Downarrow v \quad (4.2)$$

Esta recapitulación ilustra cómo, sin alterar la esencia de la metodología empleada en el primer caso de estudio, se han adaptado las representaciones internas para adecuarlas a las características semánticas de nuestro nuevo lenguaje perezoso. En este contexto, la maquinaria reformulada se basa en la semántica estructural, lo que garantiza una transición consistente y correcta entre la evaluación directa de expresiones en un ambiente y la compilación y ejecución del código generado en la máquina virtual.

4.2.3. Cálculo

Como recordatorio, en esta sección se procede a derivar las definiciones del compilador y de la máquina virtual para nuestro nuevo lenguaje perezoso, basándonos en la especificación formalizada previamente. Cabe destacar que, para los casos de las expresiones *Num*, *Boolean* y *Var*, los cálculos son análogos a los presentados en el primer caso de estudio, ya que la semántica para estos constructos es idéntica.

A lo largo del proceso, reescribiremos la ecuación de corrección

$$exec (comp' x c) (s, conv_E e)$$

en la forma

$$exec c' (s, conv_E e)$$

para algún código c' . Esto nos permitirá concluir que la definición $comp' x c = c'$ cumple con la especificación en cada caso particular. En este procedimiento, se introducirán nuevos constructores en los tipos *Code* y *Stack*, junto con nuevas cláusulas en la definición de *exec*, y se completará la definición de la función de conversión *conv*.

Los detalles para cada constructo serán presentados a continuación, comenzando por los casos directos de *Num*, *Boolean* y *Var* (para los cuales se remite a las definiciones ya derivadas), y continuando con los casos correspondientes a los nuevos constructos que caracterizan nuestro lenguaje.

Num n

A partir del proceso de cálculo realizado en el primer caso de estudio (véase la sección 3.4.3), se obtiene la siguiente definición para el compilador:

```
comp' (Num n) c = PUSHN n c
```

Para la máquina virtual, la operación consiste en introducir el número en la cima de la pila, de acuerdo con el tipo de la máquina virtual:

```
exec (PUSHN n c) (s, e) = exec c (VAL (NumV' n) : s, e)
```

Boolean b

Para los booleanos se recuperan las definiciones del primer cálculo. En el caso del compilador:

```
comp' (Boolean b) c = PUSHB b c
```

y para la máquina virtual:

```
exec (PUSHB b c) (s, e) = exec c ((VAL (BooleanV' b)) : s, e)
```

Var x

En este caso, las definiciones del compilador y de la máquina virtual se derivan directamente de las especificaciones iniciales. Para el compilador, recuperamos la definición:

```
comp' (Var x) c = LOOKUP x c
```

y para la máquina virtual tenemos:

```
exec (LOOKUP x c) (s, e) = exec c (VAL (lookup x e):s, e)
```

Se recuerda además la siguiente propiedad fundamental:

$$\text{lookup } x (\text{conv}_E e) = \text{conv} (\text{lookup } x e)$$

la cual garantiza que, al buscar un identificador x en el ambiente transformado $\text{conv}_E e$, se obtiene directamente el valor convertido. Esto implica que no es necesario aplicar la función conv de manera explícita durante la ejecución. Asimismo, la versión extendida de la función de búsqueda lookup ha sido adaptada de forma congruente con las definiciones del compilador y la máquina virtual en el contexto del lenguaje IRIS.

Add $x y$

Tal como se hizo en el primer caso de estudio, utilizaremos el constructo de suma para ilustrar, paso a paso, el razonamiento aplicado en el contexto de nuestro lenguaje perezoso. El desarrollo inicial del cálculo sigue los mismos lineamientos anteriores, diferenciándose únicamente por la introducción de puntos estrictos en los operandos.

```
exec (comp' (Add x y) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv (NumV (n+m))):s, convE e)
= { definicion de conv }
exec c (VAL (NumV' (n+m)):s, convE e)
= { def: exec (ADD c) (VAL (NumV' m):VAL (NumV' n):s, e)
      = exec c (VAL (NumV' (n + m)):s, e) }
exec (ADD c) (VAL (NumV' m):VAL (NumV' n):s, convE e)
```

En esta etapa se alcanza una configuración equivalente a la regla ADD de la semántica operacional, donde ambos argumentos son valores numéricos. Sin embargo, para modelar adecuadamente la evaluación perezosa, es necesario hacer explícitos los puntos estrictos. Esto se logra mediante un mecanismo que fuerza el valor en la cima de la pila utilizando un constructo de código especializado.

De acuerdo con nuestras reglas semánticas, podemos suponer que cualquier valor numérico ubicado en la cima de la pila es el resultado de haber forzado una expresión, ya sea una forma normal o una expresión diferida. Para representar este proceso, introducimos la deducción correspondiente a la regla F-DADD:

```
= { def: exec CEV (VAL v:CLO c e:s, _) = exec c (VAL v:s, e) }
exec CEV (VAL (NumV' m):CLO (ADD c)(convE e):VAL (NumV' n):s, convE e')
= { definicion de conv }
```

```

exec CEV (VAL(conv(NumV m)):CLO (ADD c) (convE e):VAL(NumV' n):s,
          convE e')
= { H.I. para expresion del Thunk }
exec (comp' ct CEV) (CLO (ADD c) (convE e):VAL (NumV' n):s, convE e')

```

En este contexto, se introduce el constructo de código *CEV* (abreviatura de *Canonical Evaluation*), que tiene como propósito hacer explícito el forzado de un *Thunk* hasta obtener un valor canónico. Este constructo implementa de manera operativa la regla F-DADD, integrando de forma explícita el control de la evaluación estricta.

Adicionalmente, es necesario considerar el caso en que al forzar un *think* se obtiene una expresión que aún no ha alcanzado su forma normal. Para manejar esta situación, se hace explícita la regla F-THUNK-STEP, la cual permite continuar con la evaluación de dicha subexpresión hasta su resolución final:

```

= { def: exec CEV (VAL(Thunk' c' d):CLO c e:s, _)
      = exec c' (CLO c e:s, d) }
exec CEV (VAL (Thunk' (comp' ct CEV) (convE e')): CLO (ADD c) (convE
e): VAL (NumV' n):s, convE d)
= { def: conv (Thunk e d) = Thunk' (comp' e CEV) (convE d) }
exec CEV (VAL(conv(Thunk ct e')):CLO (ADD c) (convE e): VAL (NumV' n)
:s, convE d)
= { H.I. a expresion del Thunk }
exec (comp' ct' CEV) (CLO (ADD c) (convE e): VAL (NumV' n):s, convE d
)

```

Con base en este análisis, se derivan tres definiciones fundamentales para los mecanismos de compilación y ejecución:

1. La primera corresponde al constructo *CEV*, que al obtener un valor canónico en la cima de la pila, restituye el contexto original de ejecución y reanuda la ejecución con la continuación de código previa al forzado.
2. La segunda definición contempla el caso en que el tope de la pila es un *Thunk*. En este escenario, la evaluación debe continuar en el contexto de dicha subexpresión, manteniendo almacenado el entorno original para su posterior reanudación.
3. Finalmente, se formaliza el mecanismo que garantiza que cada valor utilizado en una operación estricta ha sido previamente forzado, ya sea hasta una forma normal (canónica) o hasta un nuevo paso evaluativo, según corresponda.

Por último, se deriva una definición adicional dentro de la función de conversión, correspondiente específicamente al constructo *Thunk*:

```
conv (Thunk e d) = Thunk' (comp' e CEV) (convE d)
```

Con esto, se introduce un nuevo constructo de código destinado a iniciar el proceso de forzado del valor en la cima de la pila. Este constructo representa de forma abstracta el inicio de un punto de evaluación estricto:

```

= { def: exec (FORCE c) (VAL (Thunk' c' d):s,e)
    = exec c' (CLO c e:s,d) }
exec (FORCE (ADD c)) (VAL (Thunk' (comp' ct' CEV) (convE d)):VAL (
    NumV' n):s, convE e)
= {def conv }
exec (FORCE (ADD c)) (VAL(conv(Thunk ct' d)):VAL(NumV' n):s, convE e)
= { H.I. para y }
exec (comp' y (FORCE (ADD c))) (VAL(NumV' n):s, convE e)

```

El propósito de *FORCE* es activar la evaluación del elemento en el tope de la pila, en particular cuando dicho elemento es un *Thunk*. Este mecanismo permite que la máquina virtual reconozca expresiones diferidas y las reduzca hasta obtener un valor canónico, siguiendo las reglas de evaluación estricta definidas en la semántica operacional.

Siguiendo el patrón observado en la evaluación de aplicaciones (*App*) en el primer caso de estudio, se recurre nuevamente al constructo de pila *CLO*. Esta elección resulta natural dentro de un régimen de evaluación perezoso, dado que la postergación de expresiones requiere preservar el ambiente léxico en el que dichas expresiones fueron definidas. De este modo, *CLO* actúa como un encapsulador del contexto de ejecución, almacenando tanto la continuación del código como el ambiente actual. Esto permite que, una vez concluido el proceso de evaluación forzada, se pueda restaurar correctamente el entorno léxico original, garantizando así la correcta resolución de las referencias a variables libres dentro del alcance correspondiente. Este razonamiento conduce a una de las definiciones centrales del comportamiento de *FORCE*. Además, el tratamiento del primer argumento en la regla F-IADD adopta una estructura análoga, reutilizando definiciones previamente obtenidas y aplicando el mismo enfoque sistemático para el manejo de evaluación perezosa en operaciones binarias.

```

= { definicion de exec: CEV }
exec CEV (VAL(NumV' n):CLO (comp' y (FORCE (ADD c))) (convE e):s,
    convE e')
= {definicion de conv }
exec CEV (VAL(conv(NumV n)):CLO (comp' y (FORCE (ADD c))) (convE e):s,
    convE e')
= { H.I. para expresion del Thunk }
exec (comp' ct CEV) (CLO (comp' y (FORCE (ADD c))) (convE e):s,convE
    e')
= { definicion de exec: CEV }
exec CEV (VAL(Thunk' (comp' ct CEV) (convE e')):CLO (comp' y (FORCE (
    ADD c))) (convE e):s,convE d)
= { definicion de conv }
exec CEV (VAL(conv(Thunk ct e')):CLO (comp' y (FORCE (ADD c))) (convE
    e):s,convE d)
= { H.I. para expresion del Thunk }
exec (comp' ct' CEV) (CLO (comp' y (FORCE (ADD c)) (convE e)):s,convE
    d)
= { definicion de exec: FORCE }
exec (FORCE (comp' y (FORCE (ADD c)))) (VAL(Thunk' (comp' ct' CEV) (
    convE d)):s,convE e)

```

```

= { definicion de conv }
exec (FORCE (comp' y (FORCE (ADD c)))) (VAL(conv(Thunk ct' d)):s,
    convE e)
= { H.I. para x }
exec (comp' x (FORCE (comp' y (FORCE (ADD c))))) (s, convE e)

```

En el desarrollo anterior, se asumió que los valores numéricos presentes en la pila, correspondientes a las subexpresiones del constructo *Add*, resultan de forzar la evaluación de un *Thunk* mediante la relación semántica *force*, la cual se implementa mediante el constructo de código *CEV*.

No obstante, dichas definiciones no capturan de forma exhaustiva el comportamiento especificado por la relación *force*.

En particular, es fundamental extender la definición del constructo *FORCE* para que presente completamente la semántica asociada, tal como se expresa en la regla F-THUNK-VALUE. Esta regla contempla un escenario distinto: cuando el elemento en la cima de la pila ya es un valor canónico y, por lo tanto, no requiere evaluación adicional. En este caso, la máquina virtual debe simplemente preservar dicho valor y continuar con la ejecución.

Este comportamiento se puede ilustrar claramente dentro del contexto del constructo *Add*:

```

exec (comp' (Add x y) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv (NumV (n+m))) : s, convE e)
= { definicion de conv }
exec c (VAL (NumV' (n+m)) : s, convE e)
= { definicion de exec }
exec (ADD c) (VAL (NumV' m):VAL (NumV' n):s, e)
= { def: exec (FORCE c) (VAL(NumV' n):s,e)
    = exec c (VAL(NumV' n):s,e) }
exec (FORCE (ADD c)) (VAL(NumV' m):VAL(NumV' n):s,e)
= { definicion de conv }
exec (FORCE (ADD c)) (VAL(conv(NumV m)):VAL(NumV' n):s,e)
= { H.I. para y }
exec (comp' y (FORCE (ADD c))) (VAL (NumV' n):s, e)
= { definicion de exec: FORCE }
exec (FORCE (comp' y (FORCE (ADD c)))) (VAL (NumV' n):s, e)
= { definicion de conv }
exec (FORCE (comp' y (FORCE (ADD c)))) (VAL(conv(NumV n)):s,e)
= { H.I. para x }
exec (comp' x (FORCE (comp' y (FORCE (ADD c))))) (s, e)

```

A partir de este razonamiento, se ha derivado una definición complementaria para la máquina virtual que trata específicamente el caso en que el valor en la cima de la pila es un número. De acuerdo con la regla F-THUNK-VALUE, basta con continuar la ejecución utilizando el resto del código, sin alterar el valor ya forzado.

Cabe subrayar que ambos casos, el que evalúa un *Thunk* hasta obtener un valor canónico, y el que encuentra un valor ya evaluado, son tratados de manera congruente en nuestra máquina virtual. Las definiciones derivadas reflejan la semántica de la relación *force*, y permiten definir, de manera unificada, el comportamiento del compilador en presencia de puntos estrictos

dentro de constructos como *Add* y sus subexpresiones.

Concluido el caso de la suma, el mismo razonamiento se puede aplicar de forma directa a los demás operadores aritméticos y relacionales. Dado que el proceso de forzado se comporta de manera uniforme en todos estos casos, se reutiliza la estructura del cálculo ecuacional, modificando únicamente el código específico asociado a cada operación.

En particular, puede verificarse que la definición derivada para el constructo *Add* en este segundo caso de estudio coincide con la obtenida previamente para *ALANIS*, lo cual también se cumple para los demás operadores. Esta consistencia no es casual, sino que refleja la solidez y generalidad del método seguido: a partir de una semántica estructural bien definida, es posible derivar de forma sistemática la definición de una máquina virtual correcta, incluso al cambiar el régimen de evaluación. De este modo, la coincidencia en la construcción de los operadores pone de manifiesto que la metodología aplicada es lo suficientemente robusta como para mantenerse estable a través de distintos modelos semánticos, conservando la estructura esencial de la máquina virtual a pesar de las diferencias en los regímenes de evaluación.

Por razones de claridad y para evitar redundancias innecesarias, se omite en esta sección el desarrollo detallado del cálculo para cada operación aritmética y relacional. No obstante, dichos cálculos completos pueden consultarse en el material suplementario que acompaña este trabajo (véase Apéndice A). En su lugar, se presentan aquí únicamente las definiciones finales correspondientes a cada operador, lo que permite resaltar la coherencia estructural del sistema derivado.

Sub x y

La definición derivada para esta construcción difiere únicamente en el constructo de código introducido por el compilador.

```
comp' (Sub x y) = comp' x (FORCE (comp' y (FORCE (SUB c))))
```

La máquina virtual, por su parte, incorpora el operador correspondiente, en línea con el comportamiento establecido en el primer caso de estudio.

```
exec (SUB c) (VAL (NumV' m):VAL (NumV' n) : s , e )
              = exec c (VAL (NumV' (n-m)):s , e)
```

Mul x y

El constructo *Mul* sigue el mismo esquema: el compilador introduce el constructo de código *MUL*, y la máquina virtual aplica el operador de multiplicación sobre los valores forzados.

```
comp' (Mul x y) = comp' x (FORCE (comp' y (FORCE (MUL c))))
```

```
exec (MUL c) (VAL (NumV' m):VAL (NumV' n) : s , e )
              = exec c (VAL (NumV' (n*m)):s , e)
```

Div x y

En el caso de la división, se distingue entre dos escenarios según el valor del denominador. La definición del compilador permanece constante, mientras que la máquina virtual contempla ambas posibilidades:

- Si $m = 0$, se define el tratamiento del error:

```
exec (DIV c) (VAL (NumV' 0) : VAL (NumV' n) : s, convE e)
      = exec c (VAL ErrorV' : s, convE e)
```

- Cuando $m \neq 0$, la máquina virtual aplica la división estándar:

```
exec (DIV c) (VAL (NumV' m) : VAL (NumV' n) : s, e)
      = exec c (VAL (NumV' (n 'div' m)) : s, e)
```

Pair x y

```
exec (comp' (Pair x y) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv (PairV (Thunk x e) (Thunk y e))) : s, convE e)
= { def: conv (PairV v u) = PairV' (conv v) (conv u) }
exec c (VAL (PairV' (conv (Thunk x e)) (conv (Thunk y e))) : s, convE e)
= { definicion de conv }
exec c (VAL (PairV' (Thunk' (comp' x CEV) (convE e)) (Thunk' (comp' s
    CEV) (convE e))) : s, convE e)
= { def: exec (PAIR c' c'' c) (s, e) =
    exec c (VAL (PairV' (Thunk' c' e) (Thunk' c'' e))) (s, e) }
exec (PAIR (comp' x CEV) (comp' s CEV) c) (e, convE e)
```

A diferencia de los pares ordenados considerados en el primer caso de estudio, donde ambos elementos eran evaluados secuencialmente antes de ser encapsulados mediante el constructo *PAIR*, en el presente enfoque los pares se construyen siguiendo una evaluación perezosa, en concordancia con la semántica establecida.

En este nuevo esquema, el constructo *PAIR* simplemente coloca en la pila el código compilado de ambas subexpresiones, sin evaluarlas de inmediato. De este modo, el par es tratado como un valor final desde el punto de vista de la máquina virtual, postergando la evaluación de sus componentes hasta que sea necesario.

Como resultado del análisis anterior, se ha derivado una versión extendida de la función de conversión *conv*:

```
conv (PairV v u) = PairV' (conv v) (conv u)
```

Dado que los pares en este sistema pueden ser heterogéneos, es decir, contener construcciones arbitrarias del lenguaje, no es posible definir una conversión directa y uniforme para sus elementos. Sin embargo, gracias a que *conv* es una función total, es posible aplicarla recursivamente a cada componente del par, garantizando así la conversión estructural completa.

Fst x

```

exec (comp' (Fst x) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL(conv(Thunk u d)):s, convE e)
= { definicion de conv }
exec c (VAL (Thunk' (comp' u CEV) (convE d)):s, convE e)
= { def: exec (FST c) (VAL (PairV' u _):s,e)
      = exec c (VAL u :s,e) }
exec (FST c) (VAL (PairV' (Thunk' (comp' u CEV) (convE d)) (Thunk' (
  comp' v CEV) (convE d))) : s, convE e)
= { def: exec (FORCE c) (VAL (PairV' u v):s, e)
      = exec c (VAL (PairV' u v):s, e) }
exec (FORCE (FST c)) (VAL (PairV' (Thunk' (comp' u CEV) (convE d)) (
  Thunk' (comp' v CEV) (convE d))):s,convE e)
= { definicion de conv }
exec (FORCE (FST c)) (VAL (conv(PairV (Thunk u d) (Thunk v d)):s,
  convE e)
= { H.I. para x }
exec (comp' x (FORCE (FST c))) (s,convE e)

```

El tratamiento del operador *fst* sigue una lógica similar a la del primer caso de estudio, pero incorpora explícitamente los nuevos elementos de la semántica operacional.

En este contexto, la proyección del primer componente no desencadena inmediatamente la evaluación del par, sino que accede directamente a su representación estructural mediante el constructo *PairV'*. No obstante, este acceso implica de manera explícita un punto estricto, ya que se requiere que la expresión proporcionada haya sido evaluada hasta obtener un valor canónico (en este caso, un par) antes de poder extraer el primer elemento.

If x y z

```

exec (comp' (IF x y z) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv u):s, convE e)
= { H.I. para y }
exec (comp' y c) (s, convE e)
= { def: (VAL (BooleanV' True):s,e) = exec c (s, e) }
exec (IF (comp' y c) (comp' z c)) (VAL (BooleanV' True):s, convE e)
= { definicion de exec: FORCE }
exec (FORCE (IF (comp' y c) (comp' z c))) (VAL (BooleanV' True):s,
  convE e)
= { H.I. para x }
exec (comp' x (FORCE (IF (comp' y c) (comp' z c)))) (s,convE e)

```

```

exec (comp' (IF x y z) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv u):s, convE e)

```

```

= { H.I. para z }
exec (comp' z c) (s, convE e)
= { def: exec (IF _ c) (VAL (BooleanV' False):s,e) = exec c (s, e)}
exec (IF (comp' y c) (comp' z c)) (VAL (BooleanV' False):s, convE e)
= { definicion de exec: FORCE }
exec (FORCE (IF (comp' y c) (comp' z c))) (VAL (BooleanV' False):s,
      convE e)
= { H.I. para x }
exec (comp' x (FORCE (IF (comp' y c) (comp' z c)))) (s, convE e)

```

En el primer caso de estudio, aprovechamos que el lenguaje anfitrión es total para distribuir la compilación y ejecución de ambas ramas (*then* y *else*) de la estructura condicional. Aunque podríamos haber seguido el mismo enfoque en este análisis, optamos por una alternativa diferente con el propósito de explorar otra vía de razonamiento. En particular, al trabajar con una semántica estructural y modular, buscamos mostrar que, partiendo por separado de cada uno de los posibles casos de evaluación, es posible alcanzar una definición convergente del compilador y su correspondiente interpretación en la máquina virtual, sin depender del comportamiento condicional propio de HASKELL. Esta elección permite seguir con claridad el flujo de evaluación y resalta la autonomía de la semántica operacional definida.

En contraste, el uso de una semántica operacional de paso pequeño permite establecer reglas precisas para determinar el comportamiento de cada subexpresión condicional. En particular, se derivan directamente las definiciones de la máquina virtual que determinan qué rama se evalúa, según el valor de la condición.

Para el caso verdadero, se obtuvo la siguiente definición:

```
exec (IF c _) (VAL (BooleanV' True):s,e) = exec c (s,e)
```

la cual garantiza la evaluación de la rama *then*, compilada como *comp' y c*.

De manera análoga, el caso falso queda representado por:

```
exec (IF _ c) (VAL (BooleanV' False):s,e) = exec c (s,e)
```

permitiendo así la ejecución de la rama *else*, correspondiente a *comp' z c*.

A partir de estos casos se obtiene la siguiente definición del compilador:

```
comp' (If x y z) c = comp' x (FORCE (IF (comp' y c) (comp' z c)))
```

Aunque esta definición incluye la compilación de ambas ramas, solo se evalúa aquella que corresponde al resultado de la condición. Este comportamiento, en apariencia redundante, se justifica por las reglas IF-TRUE e IF-FALSE, que exigen que el constructo condicional contenga la información completa desde el momento de su construcción.

Lo decisivo radica en que la inclusión de ambas ramas no afecta el comportamiento observable del programa, ya que la semántica operacional asegura que únicamente se ejecuta la rama seleccionada, mientras que la otra permanece sin evaluarse.

En este contexto, si bien los razonamientos para los casos verdadero y falso se desarrollan por separado, ambos comparten los mismos supuestos sobre las subexpresiones *x*, *y* y *z*, y convergen en una definición única y coherente del compilador. Esta estrategia respeta la especificación semántica, a la vez que refuerza la consistencia de la implementación, asegurando una ejecución condicional correcta y conforme a la semántica operacional definida.

Eq x y

```

exec (comp' (Eq x y) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv (BooleanV (n == m))) : s, convE e)
= { definicion de conv }
exec c (VAL (BooleanV' (n == m)) : s, convE e)
= { def: exec (EQ c) (VAL(NumV' m):VAL(NumV' n):s,e)
      = exec c (VAL(BooleanV' (n == m)):s,e) }
exec (EQ c) (VAL (NumV' m):VAL (NumV' n):s, e)
= { def: exec (FORCE c) (VAL(NumV' n):s,e)
      = exec c (VAL(NumV' n):s,e) }
exec (FORCE (EQ c)) (VAL(NumV' m):VAL(NumV' n):s,e)
= { definicion de conv }
exec (FORCE (EQ c)) (VAL(conv(NumV m)):VAL(NumV' n):s,e)
= { H.I. para y }
exec (comp' y (FORCE (EQ c))) (VAL (NumV' n):s, e)
= { definicion de exec }
exec (FORCE (comp' y (FORCE (EQ c)))) (VAL (NumV' n):s, e)
= { definicion de conv }
exec (FORCE (comp' y (FORCE (EQ c)))) (VAL(conv(NumV n)):s,e)
= { H.I. para x }
exec (comp' x (FORCE (comp' y (FORCE (EQ c))))) (s, e)

```

El razonamiento para este constructo permitió derivar la misma definición para la máquina virtual que se presentó en el primer caso de estudio (véase la sección 3.4.3), en coherencia con lo observado en otros constructos. En este caso, al trabajar con números, podría pensarse inicialmente que sería necesario distinguir múltiples casos, como ocurrió en la operación de suma.

Sin embargo, como se evidencia tanto en las operaciones aritméticas como en la formalización semántica, el constructo *Eq* no introduce diferencias sustanciales en el tratamiento de los puntos estrictos, dado que también requiere que ambos operandos sean evaluados hasta obtener valores numéricos.

Por esta razón, el razonamiento correspondiente al caso en que los valores numéricos de las subexpresiones *Eq x y* se obtienen mediante la evaluación forzada de *thunks* resulta completamente análogo al empleado en el constructo de la suma. Esto se debe a que, al igual que en las operaciones aritméticas, la comparación por igualdad introduce un punto estricto sobre sus operandos: la evaluación no puede proceder sin conocer previamente sus valores canónicos. Esta necesidad de forzar los *thunks* para obtener dichos valores se encuentra formalizada explícitamente en las reglas semánticas correspondientes (F-IC y F-DC). Los detalles específicos de este razonamiento están disponibles en el material suplementario que acompaña esta tesis (véase Apéndice A).

Lambda x y

```

exec (comp' (Lambda x y) c) (s, convE e)

```

```

= { Especificacion 4.2 }
exec c (VAL (conv (Closure (Lambda p c) e)):s, convE e)
{ def: conv (Closure (Lambda x y) e)
      = Closure' (LAMBDA x (comp' y RET)) (convE e) }
exec c (VAL (Closure' (LAMBDA x (comp' y RET)) (convE e)):s, convE e)
={ def: exec (ABS c' c) (s, e)
      = exec c (VAL (Closure' c' e):s, e) }
exec (ABS (LAMBDA x (comp' y RET)) c) (s, e)

```

Para el constructo *Lambda* se obtiene una definición análoga en ambos regímenes de evaluación: tanto en el régimen perezoso adoptado en el presente estudio, como en el ansioso en el caso de ALANIS. Esta equivalencia se deduce de manera intuitiva a partir de la formalización semántica y de la especificación de corrección, lo que permitió derivar definiciones fundamentales coincidentes.

En el primer caso de estudio, se estableció que al construir la expresión *Lambda x y* se preservaba la asociación de la variable *x* con el cuerpo *y* mediante el constructo de código *LAMBDA*. Esto se refleja en la función de conversión para el constructo *Closure* del tipo *Value*, cuya definición es:

```

conv Closure (Lambda x y) e = Closure' (LAMBDA x (comp' y RET)) (
  convE e)

```

Además, se introdujo el constructo *RET* para marcar explícitamente el final de la evaluación de una función y facilitar el cambio de contexto durante la evaluación de una aplicación. Esto se formaliza en la máquina virtual mediante la siguiente definición:

```

exec (ABS c' c) (s, e) = exec c (VAL (Closure' c' e):s, e)

```

En el presente caso de estudio, se mantiene la misma lógica de preservación de la asociación y continuidad de la evaluación. En consecuencia, el razonamiento sobre la conversión del constructo *Lambda* y la generación del código *ABS* resulta análogo al del primer enfoque, como queda plasmado en la definición:

```

comp' (Lambda x y) c = ABS (LAMBDA x (comp' y RET)) c

```

App x y

```

exec (comp' (App x y) c) (s, convE e)
= { Especificacion 4.2 }
exec c (VAL (conv cv) : s, convE e)
= { def: exec RET (VAL u:CLO c d:s,_) = exec c (VAL u:s,d) }
exec RET (VAL (conv cv):CLO c (convE e):s, (p, conv (Thunk y e)):convE e')
= { H.I. para b }
exec (comp' b RET) (CLO c (convE e):s, (p, conv (Thunk y e)):convE e')
= { definicion de conv }
exec (comp' b RET) (CLO c (convE e):s, (p, Thunk' (comp' y CEV) (
  convE e)):convE e')
= { def: exec (APP y c) (VAL (Closure' (LAMBDA p c') e'):s, e)

```

```

                                = exec c' (CLO c e:s, (p, Thunk' y e):e') }
exec (APP (comp' y CEV) c) (VAL (Closure' (LAMBDA p (comp' b RET)) (
    convE e')):s, convE e)
= { def: exec (FORCE c) (VAL (Closure' (LAMBDA y b) d):s, e))
    = exec c (VAL (Closure' (LAMBDA y b) d):s, e) }
exec (FORCE (APP (comp' y CEV) c)) (VAL (Closure' (LAMBDA p (comp' b
    RET)) (convE e'))):s, convE e)
= { definicion de conv }
exec (FORCE (APP (comp' y CEV) c)) (VAL (conv (Closure (Lambda p b) e')
    ):s, convE e)
= { H.I. para x }
exec (comp' x (FORCE (APP (comp' y CEV) c))) (s, convE e)

```

En el razonamiento correspondiente a este constructo se evidencian las características semánticas distintivas de nuestro lenguaje, particularmente su régimen de evaluación perezoso y una estrategia de paso de parámetros *Call-by-Name*. Este razonamiento se diferencia notablemente del caso ansioso, debido a la especificación semántica distinta de la construcción *App* bajo cada régimen.

En el presente análisis se abordaron aspectos menos intuitivos, comenzando con la representación explícita del punto estricto que fuerza la subexpresión correspondiente a la función. De manera similar a lo ocurrido en otros constructos, sería natural revisar los dos casos derivados de la relación *force*; sin embargo, dado que la evaluación forzada de la subexpresión que representa la función sigue de manera recursiva el mismo patrón razonado en el caso de la suma, optamos por incluir únicamente el razonamiento esencial que conduce a la definición final de la máquina virtual para el constructo *FORCE*.

En particular, esta definición muestra que forzar una cerradura de función no altera la ejecución, ya que una cerradura es ya un valor canónico. Así, la máquina virtual simplemente continúa ejecutando el código *c*, tal como se expresa en la definición:

```

exec (FORCE c) (VAL (Closure' (LAMBDA a b) d):s, e)
    = exec c (VAL (Closure' (LAMBDA a b) d):s, e)

```

De forma análoga al razonamiento desarrollado en el primer caso de estudio, la introducción del constructo *RET* marca el término de la aplicación de función. En este proceso, se extiende el ambiente de la cerradura de la función incorporando el argumento real, pero difiriendo su evaluación de acuerdo con el estilo *Call-by-Name*:

```

exec RET (VAL cv: CLO c e:s, (p, Thunk a e):e')

```

Siguiendo el enfoque propuesto por los autores, se aplica la función de conversión al argumento real para mantener la consistencia de los tipos manejados por la máquina virtual:

```

exec (comp' b RET) (CLO c (convE e):s, (p, Thunk' (comp' a CEV) (
    convE e))): convE e')

```

Finalmente, se introduce la definición de la máquina virtual para el constructo *APP* en estricta correspondencia con la formalización semántica adoptada:

```

exec (APP a c) (VAL (Closure' (LAMBDA p c') e'): s, e)
    = exec c' (CLO c e: s, (p, Thunk' a e): e')

```

Esta definición evidencia que, al encontrar en la pila una cerradura de función, la máquina virtual procede a evaluar el cuerpo de la función (c') en un nuevo contexto, extendido a partir del ambiente de la cerradura mediante la asociación del parámetro formal p al *thunk* correspondiente al argumento real.

Así, hemos derivado las definiciones finales de la máquina virtual tanto para el constructo *APP* como para *FORCE*. Aunque este razonamiento introduce diferencias notables respecto al enfoque ansioso, los pasos fundamentales se derivan de los principios establecidos en el primer caso de estudio, adaptándose de forma precisa a la formalización semántica propia del régimen de evaluación perezoso.

Por último, la definición para la función de compilación principal *comp* se mantiene análoga al primer caso de estudio. Esto se debe a que la especificación de corrección 2.4 no fue modificada, por lo que no resulta necesario repetir el razonamiento. La definición se construye simplemente aplicando *comp'* al constructo *HALT*, el cual marca la terminación de la ejecución:

```
comp :: Expr -> Code
comp e = comp' e HALT
```

Dado que la semántica de *HALT* y la especificación de la función *comp* se preservan en el ambiente perezoso, esta definición continúa satisfaciendo los requisitos de corrección establecidos.

4.2.4. Resultados

Finalmente, agrupamos las definiciones derivadas en los razonamientos anteriores correspondientes a cada componente del compilador y de la máquina virtual.

Lenguaje destino

El lenguaje destino se describe mediante el tipo de dato *Code*, como puede observarse en la definición, la mayoría de los constructores resultan análogos a los del lenguaje destino del primer caso de estudio. Sin embargo, algunos constructos han sido adaptados para reflejar las particularidades del régimen de evaluación perezoso.

Por ejemplo, el constructo de aplicación *APP* incorpora ahora una continuación de código adicional, que representa el código compilado del argumento formal de la función. Esta modificación responde directamente a los requisitos establecidos en el cálculo asociado y permite modelar adecuadamente la semántica de evaluación perezosa. De forma análoga, el constructo *PAIR* (que en el primer caso de estudio manejaba una única continuación de código) ha sido extendido a tres continuaciones en el presente caso, reflejando la mayor complejidad necesaria para representar correctamente la interacción entre la evaluación perezosa y la construcción de pares ordenados.

Estas adaptaciones subrayan la coherencia de las definiciones obtenidas y la alineación de nuestro compilador y máquina virtual respecto a la formalización semántica.

Listing 4.20: Definición del tipo *Code* para el lenguaje destino

```
data Code = HALT | PUSHN Int Code | PUSHB Bool Code
```

LOOKUP String Code	ADD Code	SUB Code
MUL Code	DIV Code	EQ Code
IF Code	FST Code	PAIR Code Code Code
FORCE Code	CEV	RET
ABS Code Code	LAMBDA String Code	
APP Code Code		

Adicionalmente, se han derivado nuevos constructos, como *FORCE* y *CEV*. Aunque su número es reducido, estos constructos son fundamentales para representar de manera precisa el proceso de forzado de expresiones y asegurar la consistencia del régimen de evaluación en nuestro nuevo lenguaje destino.

A partir de la definición del tipo de dato *Code*, se observa claramente la relación estructural entre los tipos *Expr* y *Code*. Esta correspondencia se justifica mediante la función de compilación *comp'*, la cual se define inductivamente sobre la estructura de las expresiones (*Expr*). Es decir, a cada constructo de *Expr* se le asigna sistemáticamente una secuencia de instrucciones en *Code*, garantizando así una correspondencia directa entre la sintaxis del lenguaje fuente y el código de la máquina virtual.

Sin embargo, los constructores *CEV* y *FORCE* no corresponden directamente a ningún constructo de *Expr*. En este sentido, más que una relación sintáctica, se establece una correspondencia semántica entre *Expr* y *Code*. De este modo, aunque no todos los constructores de *Code* se originan directamente de la sintaxis de *Expr*, cada uno de ellos cumple un rol esencial en la preservación de la semántica del lenguaje fuente durante el proceso de compilación y ejecución.

Compilador

La función de compilación global *comp* se mantiene idéntica a la utilizada en el primer caso de estudio. Su propósito es invocar a la función auxiliar *comp'* con el código *HALT* como continuación, marcando así el término de la ejecución del programa:

Listing 4.21: Definición de la función de compilación general *comp*

```
comp :: Expr -> Code
comp e = comp' e HALT
```

Por otro lado, la función auxiliar *comp'* presenta diferencias notables con respecto a su versión anterior. En particular, incorpora explícitamente los nuevos constructos *FORCE* y *CEV* en las definiciones asociadas a diversos constructores del lenguaje.

Listing 4.22: Definición de función de compilación auxiliar *comp'*

```
comp' :: Expr -> Code -> Code
comp' (Num n) c      = PUSHN n c
comp' (Boolean b) c  = PUSHB b c
comp' (Var x) c       = LOOKUP x c
comp' (Add x y) c     = comp' x (FORCE (comp' y (FORCE (ADD c))))
comp' (Div x y) c     = comp' x (FORCE (comp' y (FORCE (DIV c))))
comp' (Sub x y) c     = comp' x (FORCE (comp' y (FORCE (SUB c))))
comp' (Eq x y) c      = comp' x (FORCE (comp' y (FORCE (EQ c))))
comp' (If c t e) c'   = comp' c (FORCE (IF (comp' t c') (comp' e c')))
```

```

comp' (Pair x y) c    = PAIR (comp' x CEV) (comp' y CEV) c
comp' (Fst x) c       = comp' x (FORCE (FST c))
comp' (Lambda x y) c  = ABS (LAMBDA x (comp' y RET)) c
comp' (App x y) c     = comp' x (FORCE (APP (comp' y CEV) c))

```

La incorporación de *FORCE* permite representar explícitamente el mecanismo de forzado de una subexpresión en la máquina virtual, mientras que *CEV* actúa como marcador que señala el término del procesamiento de un punto estricto. Ambos constructos son esenciales para alinear la implementación del compilador con la semántica operacional formalizada, garantizando así la correcta evaluación de las expresiones bajo el régimen perezoso.

En consecuencia, la función *comp'* traduce las expresiones bien formadas del lenguaje fuente en secuencias de código que no solo son ejecutables por la máquina virtual, sino que también preservan las reglas semánticas establecidas. Esta alineación explícita entre semántica y compilación refuerza la corrección del compilador respecto a su especificación formal.

Máquina virtual

En la máquina virtual se ha derivado el constructo de pila *CLO*, el cual encapsula el contexto original de ejecución. Si bien este constructo ya se encontraba presente en el primer caso de estudio, en el contexto actual adquiere una mayor relevancia, dado que se emplea activamente en las definiciones de los nuevos constructos *FORCE* y *CEV*.

Listing 4.23: Definición de la máquina virtual para la ejecución de código y manejo de errores

```

exec :: Code -> Conf -> Conf
exec _ (VAL ErrorV':s,e) = error ("Division por cero")
exec HALT (s, e)         = (s, e)
exec (FORCE c) (VAL (Thunk' c' d):s,e)
    = exec c' (CLO c e:s,d)
exec (FORCE c) (VAL (NumV' n):s,e)
    = exec c (VAL (NumV' n):s,e)
exec (FORCE c) (VAL (BooleanV' b):s,e)
    = exec c (VAL (BooleanV' b):s,e)
exec (FORCE c) (VAL (PairV' u v):s,e)
    = exec c (VAL (PairV' u v):s,e)
exec (FORCE c) (VAL (Closure' (LAMBDA a b) d):s,e)
    = exec c (VAL (Closure' (LAMBDA a b) d):s,e)
exec CEV (VAL (Thunk' c' d): CLO c e:s,_)
    = exec c' (CLO c e:s,d)
exec CEV (VAL v: CLO c e:s,_) = exec c (VAL v:s,e)
exec (PUSHN n c) (s,e) = exec c (VAL (NumV' n):s,e)
exec (PUSHB b c) (s,e) = exec c (VAL (BooleanV' b):s,e)
exec (LOOKUP x c) (s,e) = exec c (VAL (lookupEnv x e):s,e)
exec (ADD c) (VAL (NumV' m):VAL (NumV' n):s,e)
    = exec c (VAL (NumV' (n+m)):s,e)
exec (SUB c) (VAL (NumV' m):VAL (NumV' n):s, e)
    = exec c (VAL (NumV' (n-m)):s,e)
exec (MUL c) (VAL (NumV' m):VAL (NumV' n):s, e)

```



```

                                = exec c (VAL (NumV' (n*m)):s,e)
exec (DIV c) (VAL (NumV' 0):VAL (NumV' n):s,e)
                                = exec c (VAL ErrorV':s,e)
exec (DIV c) (VAL (NumV' m):VAL (NumV' n):s,e)
                                = exec c (VAL (NumV' (n 'div' m)):s,e)
exec (PAIR x y c) (s, e)
    = exec c (VAL (PairV' (Thunk' x e) (Thunk' y e)):s,e)
exec (FST c) (VAL (PairV' x y):s, e) = exec c (VAL x:s,e)
exec (IF c _) (VAL (BooleanV' True):s,e) = exec c (s,e)
exec (IF _ c) (VAL (BooleanV' False):s,e) = exec c (s,e)
exec (EQ c) (VAL (NumV' n):VAL (NumV' m):s, e)
    = exec c (VAL (BooleanV' (n==m)):s, e)
exec RET (VAL v: CLO c e:s, _) = exec c (VAL v:s,e)
exec (ABS c' c) (s,e) = exec c (VAL (Closure' c' e):s,e)
exec (APP a c) (VAL (Closure' (LAMBDA p c') e'): s, e)
    = exec c' (CLO c e:s, (p,Thunk' a e):e')

```

La mayoría de las definiciones asociadas a los constructos tradicionales, como números, booleanos, variables, operaciones aritméticas, igualdad, condicionales, proyección del primer elemento de un par y abstracción de funciones, se conservan sin cambios respecto al primer caso de estudio. Sin embargo, las definiciones correspondientes a los pares ordenados, la aplicación de funciones y, especialmente, los nuevos constructos *FORCE* y *CEV*, presentan diferencias naturales, necesarias para capturar correctamente la semántica del régimen perezoso y puntos estrictos.

Las definiciones del constructo *FORCE* reflejan directamente la relación de transición semántica homónima, encargándose de ejecutar el mecanismo de forzado dentro de la máquina virtual. Por su parte, *CEV* funciona como un marcador auxiliar que, de forma análoga al constructo *RET* utilizado en la aplicación de funciones, permite diferir la finalización del forzado de una expresión, así como señalar el término de la ejecución, ya sea exitoso (*HALT*) o erróneo (*ERROR*).

La siguiente definición muestra cómo *CEV* gestiona el procesamiento de un *thunk*, aplicando recursivamente la relación *FORCE*, en concordancia con la regla F-THUNK-STEP:

```

exec CEV (VAL (Thunk' c' d):CLO c e:s, _) = exec c' (CLO c e:s, d)

```

Por otro lado, una vez que el *thunk* ha sido forzado y se obtiene un valor canónico, la ejecución continúa eliminando el contexto diferido, de acuerdo con la regla F-THUNK-VALUE:

```

exec CEV (VAL v: CLO c e : s, _) = exec c (VAL v : s, e)

```

Respecto a las definiciones del constructo *FORCE*, se derivan dos casos principales. La primera definición realiza el cambio de contexto, similar al mecanismo observado en la aplicación de funciones, iniciando así la evaluación de la expresión encapsulada en el *thunk* dentro de su ambiente original. Las definiciones restantes simplemente eliminan el constructo *FORCE* cuando el valor en la cima de la pila ya es un valor canónico, en conformidad con la regla F-THUNK-VALUE:

```

exec (FORCE c) (VAL (Thunk' c' d) : s, e)
    = exec c' (CLO c e : s, d)

```

```

exec (FORCE c) (VAL (NumV' n) : s, e)
    = exec c (VAL (NumV' n) : s, e)
exec (FORCE c) (VAL (BooleanV' b) : s, e)
    = exec c (VAL (BooleanV' b) : s, e)
exec (FORCE c) (VAL (PairV' u v) : s, e)
    = exec c (VAL (PairV' u v) : s, e)
exec (FORCE c) (VAL (Closure' (LAMBDA a b) d) : s, e)
    = exec c (VAL (Closure' (LAMBDA a b) d) : s, e)

```

En el contexto de nuestro segundo caso de estudio, la inclusión explícita de la siguiente definición en la máquina virtual, sigue la misma lógica y justificación que se adoptó en el primer estudio:

```

exec _ (VAL' ErrorV':s, e) = error ("Division por cero")

```

Durante los cálculos previos, el constructo $Error_V$ fue introducido como un valor canónico que modela el fallo controlado asociado a la división entre cero. Sin embargo, en ese mismo análisis no se derivó explícitamente una definición correspondiente para el manejo del error en la máquina virtual.

Conviene recordar que, si no se especifica cómo debe actuar la máquina virtual ante la presencia de un valor de error en la pila, cualquier intento de ejecución en dicho contexto provocará un fallo de emparejamiento en la definición de la función *exec*. Es decir, el error no se manifestaría como una situación semánticamente controlada, sino como un error de ejecución no manejado en el lenguaje anfitrión. Este comportamiento es claramente indeseable desde la perspectiva del diseño, ya que traslada el manejo del error a un nivel externo al modelo semántico, comprometiendo así su robustez.

En cambio, al introducir explícitamente una regla para este caso, se garantiza que la máquina virtual reconoce y gestiona de manera deliberada y coherente la aparición de un error. Es importante subrayar que esta incorporación no modifica ni extiende la semántica formal previamente establecida, dado que el valor $Error_V$ ya forma parte de los estados finales definidos por nuestra semántica operacional. En otras palabras, no se altera el comportamiento del modelo, sino que se hace explícita y concreta su implementación en HASKELL.

Además, como se argumentó en el caso anterior, este enfoque se alinea con las prácticas aceptadas en HASKELL y está respaldado por lo establecido en el reporte del lenguaje [59], el cual indica que los errores, una vez evaluados, causan una terminación inmediata del programa y no son distinguibles de una no terminación. Al modelar el error explícitamente, no solo evitamos que el sistema falle de manera inesperada, sino que también conservamos la capacidad de diagnosticar y comunicar la causa del error de forma clara y controlada.

Por lo tanto, la incorporación de esta definición en la máquina virtual no es solo coherente con nuestro enfoque semántico, sino necesaria para asegurar la claridad del modelo de ejecución aquí desarrollado.

Función de conversión

En lo que respecta a la función de conversión, esta fue inicialmente definida de manera parcial, siguiendo un enfoque similar al adoptado en el primer caso de estudio. En ese sentido, se optó por no anticipar el comportamiento de los constructos *Pair*, *Closure* y *Thunk* antes

de contar con el desarrollo completo del cálculo correspondiente.

Una vez concluido el razonamiento, se obtuvo una definición para el constructo *Pair* que resulta análoga a la presentada en el primer caso. De igual manera, la definición para la cerradura de función (*Closure*) conserva la estructura previamente establecida, asegurando así la coherencia entre ambos regímenes de evaluación.

Listing 4.24: Definición completa de la función de conversión *conv* entre *Value* y *Value'*

```
conv :: Value -> Value'
conv (NumV n)           = NumV' n
conv (BooleanV b)       = BooleanV' b
conv ErrorV             = ErrorV'
conv (PairV x y) = PairV' (conv x) (conv y)
conv (Closure (Lambda x y) e)
    = Closure' (LAMBDA x (comp' y RET)) (convE e)
conv (Thunk c d) = Thunk' (comp' c CEV) (convE d)
```

La definición del constructo *Thunk* en esta función de conversión es una de las principales novedades del presente caso. Dicha definición:

```
conv (Thunk c d) = Thunk' (comp' c CEV) (convE d)
```

permite marcar de forma sistemática el código compilado de una expresión con el constructo *CEV*, señalando así que se trata de un *thunk* que ya incluye su mecanismo de finalización. Esto permite distinguir claramente entre las construcciones de terminación local (*RET* y *CEV*) y la terminación global representada por *HALT*.

Además, al igual que en el caso de las cerraduras de función, se propaga la conversión del ambiente léxico capturado por el *thunk*, lo cual garantiza la consistencia en la representación del tipo de dato *Value'* manejado por la máquina virtual.

4.3. Reflexión

Concluimos este caso de estudio con un análisis reflexivo sobre el enfoque adoptado, que se fundamenta en la formalización del lenguaje IRIS. Este lenguaje incorpora características semánticas inéditas en trabajos previos relacionados con el método, ya que, aunque los autores establecen la aplicabilidad del método a estrategias de paso de parámetros como *Call-by-Name* y *Call-by-Need*, no se profundiza en un estudio detallado de estas variantes [1]. Aprovechando estas suposiciones, en IRIS se integró el régimen de evaluación perezoso, entendiendo la formulación de la especificación de corrección utilizada en el primer caso de estudio.

Cabe recordar que IRIS mantiene la misma formalización sintáctica que el lenguaje ALANIS, lo que proporciona un punto de referencia fundamental para un análisis comparativo. La reflexión se centra en los aspectos menos intuitivos y en las diferencias respecto a la aplicación original del método. En IRIS, la formalización de una semántica estructural que se expresa mediante reglas de transición elementales y el uso de una relación auxiliar *force*, contrasta directamente con el estilo semántico y el régimen de evaluación del primer caso, sin implicar cambios en la especificación de corrección y la parcialidad de la máquina virtual derivada.

Para evitar duplicidades en la discusión, en este análisis se destacan únicamente los aspectos análogos y las diferencias esenciales respecto al estudio anterior. Recordemos que el primer caso se presentó como una extensión y reformulación del estudio original de los autores, lo cual permitió abordar de forma exhaustiva cada aspecto reflexivo señalado en ese trabajo. De manera similar, el presente caso de estudio sigue la línea de extender y reformular el análisis del lenguaje ALANIS, evidenciando una relación transitiva en las reflexiones expuestas. Así, las propiedades revisadas en el primer caso se heredan metodológicamente en este nuevo estudio, manteniéndose la misma especificación de corrección y la parcialidad inherente a la máquina virtual, al capturar únicamente la propiedad de completitud sin alcanzar una corrección total, una limitación prevista por los propios autores.

Otro aspecto relevante es el aprovechamiento de las ventajas que ofrece la definición parcial de componentes. Esta aproximación evita imponer suposiciones previas y permite que el cálculo derive las definiciones restantes, guiado por la especificación de corrección. Si bien este enfoque se mantuvo en el primer caso, en IRIS adquiere nuevas implicaciones debido a la necesidad de gestionar evaluaciones diferidas, puntos estrictos, lo que es natural al considerar la complejidad añadida de manejar ambientes con paso de parámetros *Call-by-Name*.

Finalmente, la reflexión sobre IRIS se orienta a explorar en detalle cómo el cambio en el estilo semántico y el régimen de evaluación impacta tanto en la formulación como en la aplicación del método. Este análisis revela los desafíos y oportunidades derivados de la integración de características semánticas avanzadas, estableciendo las bases para un análisis comparativo profundo que, en etapas posteriores, permitirá contrastar de forma sistemática los resultados e implicaciones de ambos enfoques en la derivación del compilador y la máquina virtual.

Escalabilidad

La escalabilidad del método se evidencia en cómo el enfoque propuesto por los autores se extendió de forma natural a nuestro lenguaje IRIS, sin exigir la garantía de una corrección total. En nuestro estudio, adoptamos nuevamente un compromiso pragmático: asumimos la parcialidad inherente de las máquinas virtuales derivadas, lo que permite que el método preserve, de manera explícita, únicamente la propiedad de *completitud* de la corrección de nuestro compilador. En el caso de IRIS, concebido como una extensión del lenguaje ALANIS, se formalizó previamente a la aplicación del método, la semántica utilizando un sistema de transición basado en relaciones de paso pequeño, complementado por la relación *force*. Esta formulación estableció una correspondencia directa entre una expresión de entrada y su valor resultante, lo cual es fundamental para la metodología de los autores, pues es esencial definir una función de evaluación que permita establecer la especificación de corrección. La alineación entre la formalización del lenguaje y el método es, por tanto, indispensable para asegurar que, aunque la corrección se limite a la *completitud*, la transformación de nuestros programas preserve la semántica subyacente.

El hecho de que la misma formulación del método utilizada en el primer caso de estudio permitiera derivar, de forma directa, el compilador y la máquina virtual para IRIS, demuestra que el enfoque escala sin mayores complicaciones a nuevas características semánticas. La integración de un régimen de evaluación perezoso y una estrategia de paso de parámetros *Call-by-Name* no supuso un reto insuperable, ya que la estructura genérica del método facilitó la adaptación al incorporar nuevos constructos de manera coherente y sin exigir un rediseño

en las ecuaciones de corrección del método.

En este sentido, el análisis de escalabilidad revela que las derivaciones obtenidas a través de la aplicación del método, basadas en la preservación semántica, dependen en gran medida de la estrecha relación entre la formalización del lenguaje y la metodología de los autores.

Recursión

La incorporación de la recursión en el lenguaje IRIS se fundamenta en la misma intuición teórica que guió el análisis en el caso de estudio de ALANIS. En IRIS se exhibe que la recursión puede presentarse como un efecto natural del *Cálculo* λ . Análogo al primer caso de estudio, la aplicación sistemática de un operador de punto fijo, junto con una leve reformalización sintáctica, permite introducir la recursión sin necesidad de construir mecanismos semánticos adicionales, sino simplemente mediante la eliminación del azúcar sintáctica.

Esta visión teórica sostiene que la recursión, en lugar de requerir constructos propios y exclusivos, emerge de manera inherente al marco del *Cálculo* λ , tal como se ha referido en las derivaciones obtenidas para la versión *Call-by-Value*. La extensión para admitir recursión en IRIS implicaría la inserción de una fase intermedia en la formalización del lenguaje, destinada a la eliminación del azúcar sintáctica asociada a la recursión. Dado que este proceso conlleva modificaciones en la formalización sintáctica, la semántica subyacente del *Cálculo* λ en IRIS se mantiene invariable, reafirmando así que la integración de la recursión es un proceso natural esperado dentro del propio marco teórico.

Es importante subrayar que, aunque la incorporación de la recursión en IRIS abre interesantes líneas de discusión sobre el impacto de la modificación sintáctica en la aplicabilidad del método, el alcance de esta reflexión se limita a la evaluación teórica del caso de estudio actual. La formalización y validación detallada de esta extensión se abordarán en el capítulo 6 posteriores del trabajo, donde se profundizará en las implicaciones y en la integración de la recursión en nuestro estudio general.

Aplicabilidad

En el contexto del lenguaje IRIS, la decisión de adoptar una *semántica estructural* en lugar de la *semántica natural* empleada en trabajos previos responde a la necesidad de capturar de manera rigurosa las particularidades semánticas del lenguaje, especialmente aquellas asociadas a la evaluación diferida y la gestión de puntos estrictos. Si bien la *semántica natural* ofrece una descripción directa e intuitiva del resultado final de las expresiones, resulta limitada cuando se requiere modelar con precisión los procesos intermedios de evaluación, indispensables para el desarrollo y análisis de una máquina virtual.

Por el contrario, la *semántica estructural* permite descomponer el flujo de evaluación en pasos elementales, lo que no solo preserva la corrección del modelo, sino que además proporciona un marco más transparente para el razonamiento formal. Este enfoque facilita un seguimiento detallado de cómo se propagan los efectos de la evaluación y cómo estos se integran coherentemente en la arquitectura de la máquina virtual. Este nivel de granularidad es esencial para asegurar la coherencia del modelo semántico y la corrección de los componentes derivados mediante el método.

Cabe subrayar que esta elección metodológica no implica una pérdida de generalidad ni un cambio en la interpretación del lenguaje. Tal como se estableció en la sección 2.5.1, ambas aproximaciones (natural y estructural) son formalmente equivalentes en cuanto al comportamiento que describen. Las diferencias radican exclusivamente en la forma en que ese comportamiento se modela: mientras que la *semántica natural* expresa la evaluación en términos de relaciones directas entre expresiones y sus resultados finales, la *semántica estructural* explicita cada paso intermedio del proceso de evaluación.

Adicionalmente, la estructura modular propia de la semántica estructural permite extender el lenguaje de manera más controlada y sistemática, por ejemplo, mediante la incorporación de relaciones auxiliares como la evaluación forzada de expresiones diferidas. Esta capacidad de extender y analizar detalladamente el comportamiento semántico valida la pertinencia del enfoque adoptado y proporciona la base conceptual para abordar, en las secciones siguientes, aspectos críticos como la interacción entre evaluación perezosa y puntos estrictos, así como la correcta derivación de la máquina virtual.

Semántica estructural

La formalización de una semántica estructural en IRIS fue cuidadosamente ajustada para alinearse con la metodología de los autores, sin sacrificar la capacidad de capturar las particularidades semánticas del lenguaje, como las evaluaciones diferidas y los puntos estrictos. Esta elección, lejos de ser una desviación, se integró de forma modular en el marco del método, permitiendo que la definición de la relación auxiliar *force* se exprese en términos de la relación elemental *step* y se incorpore orgánicamente al sistema global de transición.

Esta modularidad no solo facilitó el ajuste a la especificación original, mediante la incorporación de la función de mapeo φ para transformar la representación intermedia en el valor final, sino que también impactó de manera ventajosa en el proceso de cálculo. Al estructurar las reglas semánticas de forma modular, el razonamiento se vio guiado no únicamente por la intuición, sino por la propia formalización. Esto permitió verificar de manera rigurosa la preservación semántica en cada paso de las derivaciones, reforzando la integridad del sistema de evaluación.

Así, la adopción del estilo estructural ha resultado ser una estrategia importante: ha permitido adaptar el método a las exigencias específicas de IRIS y, al mismo tiempo, proporcionar un marco de trabajo en el que cada componente del proceso de evaluación, desde la relación elemental hasta la integración de *force* y la función φ se verifica formalmente, garantizando la corrección y coherencia del sistema de transición con la metodología de los autores.

Integración de la relación *force*

Uno de los elementos distintivos de la semántica estructural adoptada en IRIS es la integración directa de la relación auxiliar *force* dentro del sistema de transición, evitando la necesidad de definirla como un componente independiente. En lugar de depender de la función global de evaluación para gestionar la normalización de valores canónicos, la relación *force* fue formulada en términos de la semántica relacional de paso pequeño, permitiendo una integración más natural en el esquema de evaluación. Esto representa una diferencia fundamental con el primer caso de estudio, donde la evaluación asociaba directamente expresiones

a valores. Así, la semántica estructural de IRIS no solo preserva la expresividad del lenguaje, sino que también permite alinear la modularidad de la semántica con la formulación del método, al definir mecanismos auxiliares dentro del propio sistema de transición que son directamente derivables a través del proceso de cálculo.

La adopción de una semántica estructural en lugar de una semántica natural representa un cambio significativo en la formalización semántica del lenguaje, particularmente al abordar la necesidad de forzar evaluaciones diferidas en puntos específicos del programa. Este enfoque, que ya había sido sugerido por los autores como una opción de formalización alineada de manera directa con el método, resulta especialmente beneficioso para delinear el flujo subyacente de la evaluación de manera explícita.

En este contexto, se vuelve imprescindible la introducción de una relación *force*, que se encarga de hacer explícita la evaluación forzada de las expresiones diferidas en los puntos estrictos de cada constructo. La elección de no optar por una semántica natural se fundamenta en que, aunque esta última generalizaría de forma implícita el proceso de forzar las evaluaciones, no permitiría hacer explícitos los puntos estrictos ni modular el proceso de evaluación de forma tan detallada. En cambio, al trabajar con una semántica estructural basada en la relación elemental de transición, se puede integrar directamente la función auxiliar *force* en el marco de la evaluación paso a paso, sin tratarla como un componente independiente, lo que permite derivar una máquina virtual similar a la del primer caso de estudio.

Esta integración directa tiene varias ventajas. En primer lugar, el proceso de derivación se vuelve más natural y se puede determinar con precisión cuándo, cómo y hasta qué punto se debe forzar una expresión en el flujo de evaluación.

Además, al integrar *force* de forma inherente en la relación de transición, se logra reutilizar la especificación de corrección desarrollada en el primer caso de estudio. Esto significa que la mayor parte de las definiciones de la máquina virtual obtenida previamente se preservan, afectando únicamente la compilación de aquellas construcciones que introducen puntos estrictos en su formulación semántica. De esta manera, se mantiene la coherencia con el enfoque adoptado, permitiendo que el lenguaje IRIS sea equiparable con nuestro estudio previo.

Especificación de corrección

En un principio, la evidencia experimental parecía señalar la necesidad de establecer una nueva especificación de corrección para abordar las particularidades de IRIS.

Sin embargo, al formalizar la semántica del lenguaje, se constató que las suposiciones de extensiones o modificaciones de las especificaciones de corrección resultaron innecesarias. Tanto la intuición como los experimentos demostraron que las especificaciones de corrección utilizadas en estudios anteriores eran lo suficientemente generales como para garantizar la corrección de nuestro cálculo, incluso en un contexto de creciente complejidad.

Esta preservación de la especificación original para la función *comp'* es un aspecto esencial, ya que permite exhibir la escalabilidad del método. Es decir, el hecho de que la formulación adoptada en los trabajos previos se mantenga inalterada en IRIS evidencia que el enfoque inicial es capaz de adaptarse y extenderse a contextos semánticos más complejos. Aunque es cierto que el método presenta ciertas desventajas en cuanto a su parcialidad, su definición de forma general se ha comprobado en entornos de evaluación más exigentes, lo que refuerza la validez del enfoque.

Si bien los autores originales sugieren que el método puede extenderse a lenguajes con las estrategias de paso de parámetros *Call-by-Name* y *Call-by-Need* [1], es importante señalar que dicha extensión no se desarrolla formalmente en el cuerpo principal del artículo. En su material suplementario (véase ApéndiceA) se incluyen implementaciones y pruebas mecanizadas en el asistente COQ, pero no se presenta una formulación explícita de las derivaciones semánticas y de la corrección del método aplicada a estas estrategias, como sí ocurre con los casos de evaluación estricta dentro del texto principal.

En este contexto, los resultados obtenidos en nuestro trabajo no solo se alinean con las suposiciones planteadas por los autores, sino que también aportan una verificación formal de que el marco de corrección propuesto se mantiene sólido cuando se aplica a lenguajes con régimen de evaluación perezoso bajo la estrategia *Call-by-Name*. Esto refuerza la generalidad del método y amplía su aplicabilidad a escenarios que, aunque mencionados, no habían sido desarrollados formalmente en los términos del propio enfoque.

Compilador

La formulación del método ha supuesto una evolución notable en la arquitectura del compilador en comparación con el primer caso de estudio, introduciendo modificaciones que, si bien son sutiles, resultan esenciales para reflejar con precisión las particularidades semánticas de IRIS. En este contexto, la incorporación de nuevos constructos de código (específicamente *FORCE* y *CEV*) no se trata simplemente de una adición superflua, sino de la materialización de la representación intermedia que emergió en la formalización de nuestro sistema de transición, especialmente en la integración de la relación *force*. Estos constructos actúan como marcadores fundamentales que permiten a la máquina virtual distinguir de manera explícita entre los puntos estrictos y las evaluaciones diferidas, capturando con exactitud la compleja interacción que se produce durante el proceso de evaluación.

El constructo *FORCE*, derivado conceptualmente de la aplicación de funciones (*APP*), ha sido refinado para indicar el inicio de la evaluación forzada de una expresión. Por otro lado, *CEV*, análogo al retorno de valores (*RET*), se encarga de señalar la transición hacia el término de un punto estricto, reflejando la aplicación recursiva de la evaluación hasta alcanzar un valor canónico. Estas modificaciones han permitido derivar un compilador que introduce dichos constructos como indicadores para la máquina virtual, aportando claridad y precisión en el control del flujo de evaluación. Al explicitar estos marcadores de características semánticas directamente en la compilación, se preserva la coherencia de la especificación de corrección y se alinea el compilador con la estructura definitoria de la semántica.

El impacto en el compilador, aunque conceptualmente significativo, se manifiesta en cambios mínimos que resultan fundamentales para adaptar el sistema a las complejidades adicionales de IRIS. La decisión de sacrificar la invariabilidad total del compilador fue deliberada, ya que preservar la especificación de corrección exigía introducir un mecanismo explícito para diferenciar con precisión entre evaluaciones forzadas y diferidas. De no haberlo hecho, la formulación del método habría requerido la incorporación de un mecanismo implícito, menos transparente, que dificultaría asegurar una transición coherente entre estos estados. Esta adaptación, por tanto, permitió un razonamiento más directo sobre cada constructo del lenguaje.

Máquina virtual

La máquina virtual se ha extendido de manera mínima, adaptándose exclusivamente a los dos nuevos constructos introducidos por el compilador: *FORCE* y *CEV*. Esta modificación puntual permitió mantener inalterada la estructura general de la máquina virtual obtenida en el primer caso de estudio, lo que resulta esencial para la comparación entre enfoques. Al preservar la configuración de la máquina virtual, el contraste entre ambos estudios permite identificar con claridad cómo la integración de nuevas características semánticas puede realizarse de manera natural y sin comprometer la arquitectura preexistente.

Esta extensión mínima, junto con la redefinición del constructo *APP*, permitió reflejar directamente la formalización semántica de IRIS. En particular, la relación *force* se integró como parte fundamental del sistema de transición de la máquina virtual, en lugar de ser tratada como un componente externo o adicional. Esta decisión de diseño es significativa, ya que evita la fragmentación de la semántica y permite que la máquina virtual interprete las evaluaciones forzadas de manera coherente con el resto del sistema. La alineación directa entre la semántica formal y la arquitectura de la máquina virtual refuerza la robustez del método, asegurando que las extensiones expresivas del lenguaje puedan incorporarse sin introducir complejidad innecesaria en la implementación.

En este contexto, cabe destacar que para el constructo *FORCE* se derivaron definiciones que capturan configuraciones particulares de ejecución, distinguiendo de manera explícita el comportamiento del sistema según el valor encapsulado en el constructo *VAL* que se encuentra en la cima de la pila. Esta decisión hace visible, de forma modular, cada posible caso cubierto por la relación *force*, lo cual facilita el análisis detallado del proceso de evaluación. En contraste, en el caso del constructo *CEV*, se optó por derivar únicamente dos definiciones generales: una para cuando el valor en la cima de la pila ya está en forma normal, y otra para cuando se trata de un *think* que requiere evaluación forzada. Esta diferencia no responde a una limitación del método, sino que es una decisión deliberada. De hecho, podríamos haber seguido el mismo enfoque detallado que con *FORCE*, derivando definiciones particulares para cada valor posible. Sin embargo, la formulación adoptada para *CEV* puede verse como una leve generalización, que incluso podría interpretarse como una optimización legítima en términos del diseño de la máquina virtual.

En este sentido, las definiciones restantes para *CEV* serían análogas a las de *FORCE*, por lo que no hay pérdida de generalidad en esta simplificación. Más bien, esta elección evidencia la flexibilidad del método y la libertad que otorga en el diseño de las definiciones operacionales. Cualquiera de las alternativas conduciría a un comportamiento semántico equivalente en la ejecución del código generado, lo cual refuerza la idea de que el método permite distintas estrategias de razonamiento ecuacional sin comprometer la corrección del sistema.

Asimismo, esta adaptación exhibe que las decisiones de desarrollo no solo responden a las necesidades específicas de IRIS, sino que también se ajustan a los objetivos comparativos del trabajo. La capacidad de generalizar extensiones expresivas dentro de un marco de implementación estable permite que la estructura genérica de los componentes derivados se mantenga, lo que, a su vez, facilita el análisis comparativo sin introducir sesgos derivados de modificaciones estructurales profundas. Más aún, esta alineación entre la semántica y el método revela una ventaja no explorada hasta ahora: la influencia que tiene la integración semántica en los efectos prácticos del sistema. La manera en que la relación *force* se fusiona

con la arquitectura de la máquina virtual ilustra cómo una alineación precisa entre la teoría y la implementación puede derivar en un sistema más elegante y manejable, permitiendo una transición fluida entre distintas extensiones del lenguaje sin comprometer la coherencia global del método.

5 Análisis comparativo

En esta sección se presenta un análisis comparativo que sintetiza las reflexiones derivadas de ambos casos de estudio, permitiendo identificar sus convergencias y divergencias en la aplicación del método. En el caso de ALANIS, se desarrolla una extensión del estudio original que preserva la especificación de corrección mediante un régimen de evaluación ansioso y la estrategia de paso de parámetros *Call-by-Value*. Por otro lado, en IRIS, se integra una semántica estructural que aborda evaluaciones diferidas y la estrategia *Call-by-Name*. A través de esta comparación, se observa cómo el método conserva ciertos fundamentos esenciales mientras se adapta para afrontar nuevos desafíos semánticos.

El análisis se organiza en torno a aspectos esenciales abordados en cada estudio. En ALANIS, destacan la parcialidad, la escalabilidad, las configuraciones, la corrección y las decisiones de diseño basadas en la especificación parcial, junto con la integración de la recursión y su aplicabilidad. En IRIS, se enfatiza la escalabilidad en relación con la preservación del enfoque base, la viabilidad de la recursión mediante una leve reformulación sintáctica y la aplicación de una semántica estructural que incorpora la relación *force* y ajusta tanto el compilador como la máquina virtual para mantener la especificación de corrección.

A continuación, se desglosan estos aspectos, resaltando las similitudes inherentes al método original y las diferencias que surgen al extenderlo a contextos semánticos más complejos.

Parcialidad

La parcialidad en ambos estudios se preserva debido a la naturaleza del propio método, el cual asume explícitamente que la máquina virtual ejecuta solo configuraciones bien formadas. En ambos casos de estudio, la parcialidad se presenta en la función *exec*, que solo es válida para expresiones fuente correctamente formadas, sin garantizar una definición total para casos fuera de este dominio. Así, toda adaptación del método mantiene esta característica, enfocándose en la propiedad de *completitud* y evitando requerimientos de corrección total. El trabajo de Bahr y Hutton reconoce que la parcialidad es una propiedad inherente al diseño de máquinas virtuales en lenguajes funcionales, ya que ciertas configuraciones incorrectas pueden generar errores. Esta idea se refleja en ambas formalizaciones: en ALANIS, se considera la posibilidad de tipos dependientes como una vía para eliminar la parcialidad, aunque sin implementarla; en IRIS, la parcialidad también se acepta como un compromiso práctico, alineándose con la intención original del método.

Escalabilidad

Ambos casos de estudio evidencian que el método se adapta de forma natural a la extensión a lenguajes con características semánticas adicionales, gracias a un enfoque modular basado en especificaciones parciales. En *ALANIS*, la escalabilidad se refleja en la capacidad de derivar progresivamente definiciones fundamentales (como la función de conversión y el modelado de la pila) permitiendo incorporar nuevas construcciones sin comprometer la coherencia del proceso de cálculo. La estrategia de completar estas definiciones en fases posteriores demuestra que el método original es lo suficientemente flexible para gestionar la complejidad creciente.

Por otro lado, en *IRIS* se incorpora un sistema de transición basado en relaciones de paso pequeño, complementado con la relación *force*, lo cual refuerza la alineación entre la formalización semántica y la metodología de corrección. La misma formulación del método, que permitió la extensión en el caso de *ALANIS*, se aplica en *IRIS* sin mayores complicaciones, evidenciando que la integración de evaluaciones perezosas, control fino de puntos estrictos y una estrategia de paso de parámetros *Call-by-Name* se acomoda de manera coherente en la estructura genérica del enfoque. En ambos contextos, la escalabilidad se sustenta en la preservación de la especificación de corrección (enfocada en la completitud) y en el manejo modular de las definiciones, lo que facilita la incorporación de nuevas características semánticas sin la necesidad de rediseñar las ecuaciones fundamentales del método.

Recursión

Ambos lenguajes admiten, de manera aparentemente natural, la recursión basada en operadores de punto fijo, y se evidencia que esta integración puede lograrse mediante una extensión en la sintaxis del lenguaje, considerándose como azúcar sintáctica de la aplicación del operador de punto fijo. Aunque esta no es la única estrategia posible, es la opción más común y efectiva para incorporar la recursión sin introducir complejidades semánticas adicionales. En ambos casos de estudio, los resultados obtenidos en nuestras versiones del *Cálculo* λ parecen ser suficientes para evitar la necesidad de incorporar extensiones semánticas que obliguen a reformular y aplicar nuevamente el método. Esta fortaleza subraya la intuición de que la integración de la recursión puede efectuarse en una fase previa a la evaluación, específicamente durante la formalización sintáctica. La introducción y eliminación del azúcar sintáctica parece ser la opción promisorio de permitir que nuestras derivaciones computen la recursión sin mayores complicaciones, manteniendo inalterada la semántica subyacente del sistema.

Aplicabilidad

Ambos casos demuestran la aplicabilidad del método, aunque se diferencian en el grado de detalle y en el enfoque adoptado para reflejar la complejidad del lenguaje. En *ALANIS*, la aplicación del método se realiza de forma directa y natural, siguiendo de cerca el enfoque original de los autores. La formulación basada en la inducción sobre reglas y la utilización de especificaciones parciales permiten derivar el compilador de manera modular y escalable. Por un lado, en el primer caso de estudio, el enfoque se mantuvo fiel a la propuesta origi-

nal de los autores, utilizando una semántica natural que permitió alinear el estudio con el marco teórico inicial. La elección de este estilo facilitó una representación compacta de las definiciones y, para propósitos prácticos, demostró ser ventajosa en cuanto a la eficiencia de la derivación del método. No obstante, se reconoce que, en contextos que involucren características semánticas más avanzadas, una semántica natural podría limitar el razonamiento exhaustivo sobre la derivación, puesto que su simplicidad puede dificultar la comprensión detallada del proceso de evaluación.

Por el contrario, en el segundo caso de estudio, la adopción de una semántica estructural en IRIS se implementó de forma cuidadosa para adaptarse a las exigencias adicionales del lenguaje, como las evaluaciones diferidas y los puntos estrictos. Esta formalización permitió descomponer el flujo evaluativo en componentes claramente definidos, lo que facilitó la integración modular de la relación auxiliar *force* en términos de la relación elemental *step*. La incorporación de la función de mapeo φ para transformar la representación intermedia en el valor final se convirtió en un elemento central que reforzó la corrección y coherencia del sistema de transición.

La modularidad conseguida mediante el estilo estructural ha tenido un impacto notable en el proceso de cálculo. Al estructurar las reglas semánticas de manera que cada componente se verifique formalmente, se permitió que el razonamiento se guiara no solo por la intuición, sino por una formalización rigurosa, lo cual facilitó la verificación de la preservación semántica en cada paso de las derivaciones. Esta estrategia, en esencia, refuerza la idea de que la adaptación del método a contextos semánticos más complejos no requiere una reinención de los fundamentos, sino más bien una extensión modular que integra nuevos elementos sin sacrificar la base original.

Así, la comparación entre ambos casos destaca que, mientras en ALANIS la semántica natural ofrece una aproximación directa y práctica, en IRIS la formalización estructural permite una descomposición más detallada y un control riguroso de los procesos de evaluación. Ambos enfoques comparten la intención de preservar la especificación de corrección y mantener la integridad del método, pero se diferencian en la profundidad y el nivel de detalle con que se abordan las particularidades semánticas del lenguaje.

Por otro lado, en el primer caso de estudio, el compilador y la máquina virtual se diseñaron siguiendo de manera casi idéntica el enfoque original de los autores, donde cada construcción del lenguaje se traduce de forma directa en código, y el flujo de ejecución se refleja de manera intuitiva según la semántica formalizada. En ALANIS, esta aproximación permitió derivar el sistema sin necesidad de introducir mecanismos adicionales, manteniendo la estructura base intacta.

Mientras que, en IRIS se introdujeron dos nuevos constructos (*FORCE* y *CEV*) junto con la redefinición de *APP*. Estos elementos se integraron de forma modular para reflejar con precisión las nuevas características semánticas, como las evaluaciones forzadas y diferidas, sin alterar la estructura del compilador y la configuración de la máquina virtual. Esta extensión mínima evidencia cómo el método, fundamentado en una formalización general, se adaptó de manera coherente y robusta a las exigencias de IRIS.

En conjunto, el hecho de que el mismo enfoque metodológico se aplicara de manera casi directa en ambos casos refuerza las ventajas de este método. Se abre así la cuestión sobre hasta qué rango de complejidad puede ser aplicable, pero para los propósitos del trabajo, el método demostró ser lo suficientemente robusto y adaptable para gestionar las variaciones

semánticas presentadas en cada estudio.

Aspecto	ΑΛΑΝΙΣ	ΙΡΙΣ
Parcialidad	Mantiene el compromiso original de operar solo sobre configuraciones bien formadas, garantizando la completitud sin buscar corrección total.	Adopta la misma premisa, operando sobre configuraciones válidas y preservando la completitud, en línea con el enfoque original.
Escalabilidad	Se logra mediante especificaciones parciales que permiten extender el método sin modificar la estructura base, facilitando la incorporación progresiva de definiciones.	Se evidencia en la integración de evaluaciones diferidas y puntos estrictos mediante una semántica estructural modular, sin alterar los fundamentos del método.
Recursión	Se incorpora de forma natural mediante un operador de punto fijo, con una leve eliminación del azúcar sintáctica que preserva la semántica subyacente.	Se introduce de manera similar como una extensión sintáctica (azúcar del operador de punto fijo) que permite computarla sin reformular la semántica.
Aplicabilidad	La aplicación es directa y compacta gracias a una semántica natural, ideal para entornos menos complejos, aunque con ciertas limitaciones en el análisis profundo.	La aplicación se enriquece mediante una semántica estructural que permite un análisis detallado y modular, integrando nuevas características sin comprometer la base del método.
Compilador y máquina virtual	El compilador y la máquina virtual siguen el esquema original, traduciendo directamente las construcciones del lenguaje y manteniendo la configuración preexistente.	Se realizan mínimas extensiones en el compilador (introduciendo <i>FORCE</i> , <i>CEV</i> , redefinición de <i>APP</i> y función φ) y en la máquina virtual para integrar las nuevas complejidades semánticas, sin alterar la estructura base.

Tabla 5.1: Análisis comparativo entre ΑΛΑΝΙΣ e ΙΡΙΣ

6 Incorporación de la recursión

Como se argumentó en el análisis previo, la incorporación de recursión en los casos de estudio no requiere una reformatización completa del lenguaje ni altera su semántica fundamental. En lugar de ello, basta con una extensión léxica y gramatical que permita automatizar la inserción del operador de punto fijo en las definiciones recursivas. De este modo, se habilita al programador para escribir expresiones recursivas de manera directa, sin necesidad de invocar explícitamente dicho operador, trasladando esta responsabilidad a una etapa intermedia del procesamiento del lenguaje. Esta decisión permite preservar la claridad del diseño semántico y mantener inalterada la implementación de la máquina virtual.

Para ello, se extiende la sintaxis concreta del lenguaje mediante una fase de eliminación de azúcar sintáctica. Esta fase transforma expresiones recursivas en su forma explícita utilizando el combinador de punto fijo, justo en el paso de *srcASA* a *dstASA*. En consecuencia, la recursión no se introduce como un nuevo constructo semántico, sino como una transformación sistemática previa a la interpretación formal, expresada en términos de operadores ya definidos.

Dado que esta transformación es externa al núcleo semántico del estudio, su implementación se apoya en herramientas específicas para el análisis léxico y sintáctico. En particular, se utiliza HAPPY, un generador de analizadores sintácticos para HASKELL, con el cual se definen las fases anteriores al análisis semántico: la construcción del *lexer* y el *parser*. Esta infraestructura permite validar empíricamente que los casos de estudio se mantienen correctos una vez que las expresiones han sido transformadas automáticamente por la fase de desazucaramiento.

Gracias a esta integración, la recursión se incorpora de forma transparente para el usuario, sin modificar la estructura base del lenguaje ni comprometer su modelo de evaluación. La eliminación del azúcar sintáctica garantiza que toda definición recursiva sea traducida correctamente al marco formal establecido, asegurando una evaluación conforme a la semántica ya definida.

Extensión de sintaxis léxica

El primer paso para habilitar expresiones recursivas es ampliar el alfabeto del lenguaje con una nueva palabra reservada: *Rec*. Dado que nuestro propósito no es redefinir el lenguaje por completo, sino ampliar su capacidad expresiva, esta adición se realiza directamente sobre la sintaxis léxica.

Partimos del alfabeto original:

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, =, (,), if, True, False, fst, Lambda\}$$

Al agregar la palabra reservada *Rec*, obtenemos el alfabeto extendido:

$$\Sigma' = \Sigma \cup \{Rec\}$$

Esta ampliación léxica, sin embargo, no basta por sí sola: es necesario extender también la gramática libre de contexto del lenguaje para integrar la nueva construcción.

Extensión de la gramática libre de contexto

Para incorporar expresiones recursivas, añadimos la siguiente producción a la gramática:

$$(Rec \ < Var \ > \ < Expr \ > \ < Expr \ >)$$

donde:

- $\< Var \ >$ representa el identificador de la función recursiva.
- El primer $\< Expr \ >$ define el cuerpo de la función.
- El segundo $\< Expr \ >$ corresponde a la aplicación de la función.

Introduciendo producción a nuestra gramática en EBNF, tenemos:

Sintaxis libre de contexto extendida

```
<S> ::= <Expr>
```

```
<Expr> ::= <Int>  
         | True  
         | False  
         | <Var>  
         | (fst <Expr>)  
         | (<Expr>, <Expr>)  
         | (<Expr> <Expr>)  
         | (if <Expr> <Expr> <Expr>)  
         | (+ <Expr> <Expr>)  
         | (- <Expr> <Expr>)  
         | (* <Expr> <Expr>)  
         | (/ <Expr> <Expr>)  
         | (= <Expr> <Expr>)  
         | (Lambda <Var> <Expr>)  
         | (Rec <Var> <Expr> <Expr>)
```

```
<Int> ::= <N> | -<M>
```

```
<D> ::= 1 | ... | 9
```

```
<N> ::= 0  
       | <D>{<N>}
```

```
<M> ::= <D>{<N>}
```

```
<Var> ::= <Char>{<Var>}
```

```
<Char> ::= a | ... | Z
```

Para ilustrar mejor el funcionamiento de esta extensión, derivamos la función factorial aplicada a un número arbitrario (m) utilizando nuestra nueva gramática:

```

S → (E)
E → (Rec < Var > < Expr > < Expr >)
  → (Rec factorial < Expr > < Expr >)
  → (Rec factorial (Lambda n < Expr >) < Expr >)
.
.
.
  → (Rec factorial (Lambda n (if (= n 0) 1 (* n (factorial (-n 1))))) < Expr >)
  → (Rec factorial (Lambda n (if (= n 0) 1 (* n (factorial (-n 1))))) (factorial m))

```

Transición de la gramática a HAPPY

La gramática formal definida en EBNF se traduce directamente a un conjunto de reglas en HAPPY, donde cada producción se asocia con una acción semántica que construye nodos del *srcASA* utilizando constructores de HASKELL (como *NumS*, *BooleanS*, *VarS*, etc.). Por ejemplo, la producción:

$$Expr : int \quad \{ NumS \$1 \}$$

indica que al reconocer un entero, se construye un nodo *NumS* con ese valor. De forma análoga, la regla:

$$Expr : '(' "Rec" var Expr Expr ')' \quad \{ RecS \$3 \$4 \$5 \}$$

define cómo se representa una expresión recursiva, donde $\$3$ es el identificador de la función, $\$4$ es el cuerpo, y $\$5$ la aplicación, creando un nodo *RecS*.

La implementación en HAPPY permite definir de forma directa tanto el *parser* como el *lexer*, facilitando la construcción del *srcASA* en términos de estructuras de datos de HASKELL. Esta aproximación mantiene una correspondencia clara y sistemática entre la gramática formal y su representación computacional.

A continuación se muestra la implementación de la gramática en HAPPY:

Listing 6.1: Implementación de la gramática del lenguaje en HAPPY

```

S  : Expr { $1 }

Expr : int                { NumS $1 }
     | boolT              { BooleanS True }
     | boolF              { BooleanS False }
     | var                { VarS $1 }
     | '(' "fst" Expr ')' { FstS $3 }
     | '(' Expr ',' Expr ')' { PairS $2 $4 }

```

```

| '(' Expr Expr ')' { AppS $2 $3 }
| '(' "if" Expr Expr Expr ')' { IfS $3 $4 $5 }
| '(' '+' Expr Expr ')' { AddS $3 $4 }
| '(' '-' Expr Expr ')' { SubS $3 $4 }
| '(' '*' Expr Expr ')' { MulS $3 $4 }
| '(' '/' Expr Expr ')' { DivS $3 $4 }
| '(' '=' Expr Expr ')' { EqS $3 $4 }
| '(' "Lambda" var Expr ')' { LambdaS $3 $4 }
| '(' "Rec" var Expr Expr ')' { RecS $3 $4 $5 }

```

En esta implementación, cada acción semántica (la sección entre llaves) transforma la producción reconocida en un nodo correspondiente del árbol. Los elementos sintácticos (como paréntesis, operadores o palabras reservadas) se combinan con las subexpresiones usando las posiciones indicadas por $\$n$, generando así una estructura interna coherente con la gramática extendida.

La traducción de la gramática a HAPPY refleja su estructura original, garantizando que cada producción en EBNF tenga una contraparte bien definida en el analizador sintáctico. Este paralelismo no solo facilita la implementación, sino que refuerza la integridad entre la especificación formal del lenguaje y su realización práctica.

Definición de *tokens* y *lexer*

Además de la gramática, la definición de los *tokens* y del *lexer* es esencial para transformar un programa escrito en sintaxis concreta en una lista de *tokens* que el *parser* pueda procesar. La siguiente definición de *tokens* ilustra cómo se mapean los distintos símbolos y palabras reservadas:

Listing 6.2: Definición de los *tokens* del lenguaje

```

%token
var      { TokenId $$ }
int      { TokenNum $$ }
boolT    { TokenTrue }
boolF    { TokenFalse }
"fst"    { TokenFst }
'('      { TokenPA }
')'      { TokenPC }
', '     { TokenComma }
'+'      { TokenAdd }
'-'      { TokenSub }
'*'      { TokenMul }
'/'      { TokenDiv }
'='      { TokenEq }
"if"     { TokenIf }
"Lambda" { TokenLambda }
"Rec"    { TokenRec }

```

El *lexer* se encarga de leer el programa como una cadena de caracteres y producir la secuencia de *tokens* correspondiente. A continuación se muestra la implementación del *lexer* en HASKELL:

Listing 6.3: Implementación del *lexer* en HASKELL

```
lexer :: String -> [Token]
lexer [] = []
lexer (' ' : xs) = lexer xs
lexer '(' : xs) = TokenPA : lexer xs
lexer ')' : xs) = TokenPC : lexer xs
lexer (',' : xs) = TokenComma : lexer xs
lexer ('+' : xs) = TokenAdd : lexer xs
lexer ('-' : xs) = TokenSub : lexer xs
lexer ('*' : xs) = TokenMul : lexer xs
lexer ('/' : xs) = TokenDiv : lexer xs
lexer ('=' : xs) = TokenEq : lexer xs
lexer ('f':'s':'t':xs) = TokenFst : lexer xs
lexer ('i':'f':xs) = TokenIf : lexer xs
lexer ('L':'a':'m':'b':'d':'a':xs) = TokenLambda : lexer xs
lexer ('R':'e':'c':xs) = TokenRec : lexer xs
lexer ('T':'r':'u':'e':xs) = TokenTrue : lexer xs
lexer ('F':'a':'l':'s':'e':xs) = TokenFalse : lexer xs
lexer (x:xs)
  | isDigit x = lexNum (x:xs)
  | isAlpha x = lexAlph (x:xs)

lexNum :: String -> [Token]
lexNum cs = TokenNum (read num) : lexer rest
  where (num,rest) = span isDigit cs

lexAlph :: String -> [Token]
lexAlph cs = TokenId var : lexer rest
  where (var,rest) = span isAlpha cs
```

La implementación del *lexer* asigna cada carácter o secuencia de caracteres a su correspondiente *token*. Por ejemplo, cuando encuentra la cadena “Rec”, la convierte en el *token* *TokenRec*. Del mismo modo, los números y los identificadores se agrupan mediante las funciones *lexNum* y *lexAlph*, generando los *tokens* *TokenNum* y *TokenId*, respectivamente.

Este mecanismo es esencial para transformar el código fuente en una lista estructurada de *tokens*, que luego será procesada por el *parser* para construir el *srcASA*. Así, el *lexer* actúa como un puente entre el texto del programa y la representación formal que permitirá la eliminación del azúcar sintáctica, la evaluación por los intérpretes y la posterior transformación a código destino para su ejecución en la máquina virtual.

Generación del *parser*

A partir de la lista de *tokens* generada por el *lexer*, HAPPY construye un árbol siguiendo la estructura establecida en nuestra gramática. En este contexto, nuestros *srcASA* están definidos en HASKELL mediante el siguiente tipo de datos.

Listing 6.4: Definición del árbol de sintaxis abstracta azucarado *srcASA*

```
data ASAS = NumS Int
          | BooleanS Bool
          | VarS String
          | FstS ASAS
          | PairS ASAS ASAS
          | AppS ASAS ASAS
          | IfS ASAS ASAS ASAS
          | AddS ASAS ASAS
          | SubS ASAS ASAS
          | MulS ASAS ASAS
          | DivS ASAS ASAS
          | EqS ASAS ASAS
          | LambdaS String ASAS
          | RecS String ASAS ASAS
```

Cada constructo en la definición de ASAS representa una estructura sintáctica reconocida en el lenguaje. Dado que los detalles técnicos sobre la generación del *parser* con HAPPY se encuentran en el material suplementario (véase Apéndice A), aquí nos limitamos a destacar su papel en la conversión de la lista de *tokens* en una estructura sintáctica bien definida.

Eliminación del azúcar sintáctica

El proceso de eliminación de azúcar sintáctica se centra exclusivamente en la transformación de la construcción *Rec*, ya que el resto de las expresiones presentes en el ASAS se corresponden directamente con las del ASA y, por tanto, pueden ser trasladadas sin modificaciones estructurales.

El propósito de esta fase es garantizar la preservación semántica al traducir expresiones desde el ASAS hacia el ASA. Esto permite confirmar que los resultados obtenidos en estudios anteriores, donde la recursión no se trataba de forma explícita, siguen siendo válidos al incorporar esta capacidad mediante una extensión sintáctica no intrusiva.

Para formalizar dicha transformación, definimos la función Δ , la cual traduce cada expresión del ASAS a su correspondiente representación en el ASA. Esta función se especifica de la siguiente forma:

$$\Delta = ASAS \rightarrow ASA$$

donde cada construcción sintáctica azucarada se mapea a su forma abstracta en el núcleo del lenguaje, respetando la estructura y significado definidos previamente en los lenguajes IRIS y ALANIS. Esta correspondencia asegura que la semántica de base se conserve de manera rigurosa, aun tras la introducción de nuevas formas sintácticas.

En el caso particular del lenguaje MINIB, este proceso de eliminación de azúcar sintáctica no se formaliza en esta sección, pero su implementación completa se encuentra disponible en el material suplementario (véase Apéndice A), derivada directamente de la formalización general presentada en la sección 2.4.2 y siguiendo rigurosamente los principios establecidos a lo largo de esta sección.

A continuación, se presenta la definición estructurada de la función Δ para todas las construcciones que no involucren azúcar sintáctica:

$$\Delta : ASAS \rightarrow ASA, \quad \Delta(e) = \begin{cases} Num(n) & \text{si } e = NumS(n), \\ Boolean(b) & \text{si } e = BoolS(b), \\ Var(x) & \text{si } e = VarS(x), \\ Add(\Delta(e_1), \Delta(e_2)) & \text{si } e = AddS(e_1, e_2), \\ Sub(\Delta(e_1), \Delta(e_2)) & \text{si } e = SubS(e_1, e_2), \\ Mul(\Delta(e_1), \Delta(e_2)) & \text{si } e = MulS(e_1, e_2), \\ Div(\Delta(e_1), \Delta(e_2)) & \text{si } e = DivS(e_1, e_2), \\ If(\Delta(e_1), \Delta(e_2), \Delta(e_3)) & \text{si } e = IfS(e_1, e_2, e_3), \\ Eq(\Delta(e_1), \Delta(e_2)) & \text{si } e = EqS(e_1, e_2), \\ Pair(\Delta(e_1), \Delta(e_2)) & \text{si } e = PairS(e_1, e_2), \\ Fst(\Delta(e)) & \text{si } e = FstS(e), \\ Lambda(x, \Delta(e)) & \text{si } e = LambdaS(x, e), \\ App(\Delta(e_1), \Delta(e_2)) & \text{si } e = AppS(e_1, e_2). \end{cases} \quad (6.1)$$

Sin embargo, la transformación de la construcción *RecS* no puede definirse de forma uniforme, ya que depende de la elección del operador de punto fijo, el cual varía en función del régimen de evaluación adoptado por el lenguaje (véase la sección 2.5.2).

Para resolver esta dependencia, introducimos un operador de punto fijo τ , que se incorpora explícitamente en la traducción. Aunque la sintaxis endulzada permite expresar recursión mediante la construcción *RecS*, esta no forma parte del ASA formalizado previamente. En cambio, la recursión se representa composicionalmente a través de la aplicación del operador τ , el cual encapsula el comportamiento recursivo deseado de acuerdo con la semántica establecida.

Formalmente, la transformación de una expresión de la forma *RecS* se especifica como:

$$App((Lambda\ x\ \Delta(e_2)), (App\ \tau\ (Lambda\ x\ \Delta(e_1)))) \quad \text{si } e = RecS(x, e_1, e_2).$$

Recordando que en esta transformación:

- x representa el identificador de la función recursiva.
- e_1 representa la definición de la función.
- e_2 es la expresión a la que se aplica la función recursiva.
- τ es el operador de punto fijo que permite que x se refiera a sí misma dentro de e_1 .

La elección del operador de punto fijo adecuado está determinada por el régimen de evaluación del lenguaje, en concordancia con lo establecido en la sección 2.5.2. En nuestro estudio, consideramos los dos combinadores clásicos ampliamente documentados en la literatura:

- **El combinador Z** : empleado en lenguajes con evaluación estricta, donde los argumentos son evaluados antes de ser pasados a la función. Este combinador es adecuado para lenguajes que implementan la estrategia de paso de parámetros *Call-by-Value*, como es el caso de $\Lambda\lambda\text{ANIS}$. Su diseño garantiza que la recursión se adapte correctamente al régimen ansioso, donde las expresiones se evalúan tan pronto como son asociadas a un parámetro.
- **El combinador Y** : utilizado en lenguajes con evaluación perezosa, donde la evaluación de los argumentos se difiere hasta que su valor es estrictamente necesario. Este combinador es compatible con las estrategias de paso de parámetros *Call-by-Name* y *Call-by-Need*, como en el lenguaje IRIS . Permite la definición de funciones recursivas sin forzar la evaluación inmediata de los argumentos, preservando así la semántica diferida característica de este régimen.

Para formalizar esta adaptación, definimos dos variantes especializadas de la función Δ : $\Delta_{\Lambda\lambda\text{ANIS}}$ y Δ_{IRIS} . Ambas son extensiones de la función global Δ , diferenciándose únicamente en la transformación del constructo *RecS*, mediante la elección del operador de punto fijo correspondiente. Formalmente, establecemos:

$$\Delta_{\Lambda\lambda\text{ANIS}}(e) = \begin{cases} \text{App} (\text{Lambda } x \Delta_{\Lambda\lambda\text{ANIS}}(e_2)) \\ \quad (\text{App } Z (\text{Lambda } x \Delta_{\Lambda\lambda\text{ANIS}}(e_1))) & \text{si } e = (\text{RecS } x \ e_1 \ e_2) \\ \Delta(e) & \text{en otro caso.} \end{cases}$$

$$\Delta_{\text{IRIS}}(e) = \begin{cases} \text{App} (\text{Lambda } x \Delta_{\text{IRIS}}(e_2)) \\ \quad (\text{App } Y (\text{Lambda } x \Delta_{\text{IRIS}}(e_1))) & \text{si } e = (\text{RecS } x \ e_1 \ e_2) \\ \Delta(e) & \text{en otro caso.} \end{cases}$$

Esta definición en términos de Δ se presenta únicamente para mejorar la comprensión de la transformación de *RecS*. En la implementación en HASKELL , se definirán las funciones $\Delta_{\Lambda\lambda\text{ANIS}}$ y Δ_{IRIS} de forma completamente independiente.

Representación en HASKELL

Una vez formalizadas las funciones de transformación $\Delta_{\Lambda\lambda\text{ANIS}}$ y Δ_{IRIS} como especializaciones de la función global Δ , procedemos a su implementación en HASKELL . Para ello, definimos una función *desugar* específica para cada lenguaje, siguiendo las estructuras propuestas en la formalización anterior.

Previo a la definición de estas funciones, es necesario representar explícitamente los operadores de punto fijo en el nivel de los ASA. En el caso de $\Lambda\lambda\text{ANIS}$, el operador Z se introduce como una variable local, consistente con su uso dentro del entorno del lenguaje:

Listing 6.5: Operador de punto fijo Z para λANIS

```

z :: Expr
z = Lambda "f" (
  App
    (Lambda "x" (App (Var "f") (Lambda "y" (App (App (Var "x") (
      Var "x")) (Var "y")))))
    (Lambda "x" (App (Var "f") (Lambda "y" (App (App (Var "x") (
      Var "x")) (Var "y")))))
  )

```

Para IRIS, definimos el operador Y :

Listing 6.6: Operador de punto fijo Y para IRIS

```

y :: Expr
y = Lambda "f" (
  App
    (Lambda "x" (App (Var "f") (App (Var "x") (Var "x"))))
    (Lambda "x" (App (Var "f") (App (Var "x") (Var "x"))))
  )

```

Las implementaciones concretas pueden ser revisadas directamente en el material suplementario (véase Apéndice A); aquí presentamos un esbozo de cada una de las implementaciones.

Listing 6.7: Implementación de desugar para λANIS

```

desugarA :: ASAS -> Expr
desugarA (NumS n)      = Num n
desugarA (BoolS b)     = Boolean b
desugarA (VarS x)      = Var x
desugarA (AddS e1 e2)  = Add (desugarA e1) (desugarA e2)
-- otros casos estandar
(RecS x e1 e2) = App (Lambda x (desugarA e2))
                  (App z (Lambda x (desugarA e1)))

```

Y de manera similar, la implementación para IRIS sería:

Listing 6.8: Implementación de desugar para IRIS

```

desugarI :: ASAS -> Expr
desugarI (NumS n)      = Num n
desugarI (BoolS b)     = Boolean b
desugarI (VarS x)      = Var x
desugarI (AddS e1 e2)  = Add (desugarI e1) (desugarI e2)
-- otros casos estandar
desugarI (RecS x e1 e2) = App (Lambda x (desugarI e2))
                  (App y (Lambda x (desugarI e1)))

```

De este modo, trasladamos formalmente nuestras funciones de eliminación del azúcar sintáctica en `HASKELL`, definiéndolas por separado para cada lenguaje según su régimen de

evaluación. Esta estrategia permite validar empíricamente que los resultados obtenidos previamente siguen siendo válidos tras la incorporación de la recursión.

La decisión de introducir el operador de punto fijo directamente en la transformación de la construcción *RecS* responde a la necesidad de preservar la integridad tanto de los lenguajes como de los marcos semánticos previamente establecidos. Si bien sería posible tratar dicho operador como una variable ligada y extender el entorno léxico de los intérpretes y máquinas virtuales para incluir este nuevo *binding*, tal alternativa implicaría modificar los fundamentos semánticos de los trabajos anteriores, comprometiendo potencialmente la validez de sus derivaciones.

En cambio, al incorporar el operador de punto fijo directamente como parte de la transformación sintáctica, se evita intervenir en los entornos de evaluación originales. Esta decisión mantiene intacta la configuración inicial de los intérpretes y máquinas virtuales de ambos lenguajes, asegurando así la estabilidad y reproducibilidad de los resultados previos. La transformación del *ASAS* al *ASA* se mantiene aislada, sin introducir nuevos *bindings* ni afectar el comportamiento global del sistema.

Por estas razones, esta segunda opción resulta preferible, pues permite extender los lenguajes para soportar recursión de manera natural, sin alterar su semántica base ni comprometer la consistencia del enfoque metodológico adoptado.

Validación empírica de la extensión

Para validar la incorporación de recursión a través de la extensión sintáctica y su correspondiente eliminación, se integró una fase de validación empírica en los intérpretes, compiladores y máquinas virtuales desarrollados. Esta validación se realizó dentro del flujo completo de procesamiento, que incluye análisis léxico, análisis sintáctico, desazucarado y evaluación. En la práctica, el usuario puede escribir programas en el lenguaje fuente extendido, los cuales se procesan automáticamente mediante la siguiente secuencia funcional:

$$\text{Evaluador} > (\text{desugar } (\text{parse } (\text{lexer } \text{str})) \text{ }) \text{ } []$$

$$\text{Máquina virtual} > (\text{comp } (\text{desugar } (\text{parse } (\text{lexer } \text{str}))) \text{ }) ([], [])$$

Este *pipeline*¹ garantiza que todas las etapas intermedias (incluida la introducción explícita del operador de punto fijo) se integran sin necesidad de modificar los ambientes de evaluación existentes. Así, la extensión propuesta se valida de forma natural en los sistemas previamente establecidos, manteniendo su comportamiento semántico.

La implementación en HASKELL y su integración en las herramientas construidas muestran que el enfoque adoptado es compatible con los lenguajes originales y suficientemente general para incorporar recursión. Además, este proceso sienta las bases para extender el análisis a futuras ampliaciones del lenguaje.

¹Un *pipeline* es una secuencia de etapas de procesamiento automatizado en la que la salida de cada etapa se convierte en la entrada de la siguiente. Esta estructura es común en el diseño de compiladores e intérpretes [7].

7 Conclusiones

Recapitulación y síntesis contextual

Uno de los principales desafíos en el área de lenguajes de programación es el diseño de compiladores correctos por construcción, es decir, compiladores cuya formulación garantice desde su origen la preservación exacta de la semántica del lenguaje fuente, sin requerir procesos de verificación posteriores [65]. Esta necesidad se acentúa conforme los lenguajes evolucionan hacia modelos más expresivos y estructuralmente complejos, en los que las decisiones sobre cuándo y cómo evaluar una expresión tienen un impacto determinante sobre su significado. En este contexto, el método de *Cálculo de Compiladores Correctos* se presenta como una vía metodológica sólida, al ofrecer un marco basado en razonamiento ecuacional e inducción estructural que permite derivar compiladores a partir de una especificación semántica precisa del lenguaje fuente, asegurando así su corrección formal de manera inherente [1]. La presente tesis toma como punto de partida dicha problemática y se propone explorar la aplicabilidad del método en escenarios donde las propiedades semánticas del lenguaje fuente no responden a un único patrón de evaluación ni a una semántica operacional homogénea. Para ello, se diseñaron dos lenguajes que comparten una base sintáctica común, pero se diferencian por el régimen de evaluación y por otras características semánticas fundamentales. Esta construcción paralela permitió mantener una coherencia estructural entre ambos lenguajes, al tiempo que habilitó una comparación detallada de los efectos del régimen de evaluación sobre la derivación del compilador y la preservación de la semántica.

Es importante destacar que las propiedades semánticas asociadas a cada lenguaje no se superponen ni coexisten dentro de una misma formalización. Por el contrario, cada lenguaje fue tratado como un sistema independiente, con su propio conjunto de reglas semánticas operacionales, máquina virtual y compilador derivado. Esta separación metodológica respondió a la necesidad de analizar con precisión el comportamiento del método frente a estrategias de paso de parámetros divergentes. De este modo, el estudio se enfocó en observar hasta qué punto el método, en su formulación original, es capaz de adaptarse a distintas propiedades semánticas de lenguajes sin que su estructura general se vea alterada.

La derivación de compiladores correctos para ALANIS e IRIS se llevó a cabo a partir de la formalización explícita de sus respectivas semánticas operacionales. Esta etapa implicó capturar de forma rigurosa las condiciones bajo las cuales una expresión es evaluada, así como los mecanismos mediante los cuales dicha evaluación se refleja en el comportamiento de la máquina virtual. Lejos de imponer modificaciones sustanciales al método, esta tarea consistió en trasladar con fidelidad la semántica operacional a través de las fases de transformación del lenguaje, siguiendo el principio fundamental del cálculo: que la traducción preserve la

semántica a través de las derivaciones, y no como resultado de una verificación posterior. A lo largo de este proceso, se adoptó una estrategia metodológica que evitó depender de las características del lenguaje de implementación. Aunque la validación práctica se realizó en el lenguaje HASKELL, la semántica de ambos lenguajes fue definida de forma autónoma, sin importar construcciones o convenciones propias del lenguaje anfitrión. Esta elección permitió analizar los compiladores derivados como instancias genuinas del método de cálculo correcto, evaluadas exclusivamente en función de la coherencia con que traducen la semántica del lenguaje fuente hacia su correspondiente representación en lenguaje destino ejecutable por la máquina virtual.

Aunque las propiedades semánticas tratadas en este estudio no habían sido exploradas previamente en los trabajos relacionados con el método de cálculo, su inclusión no debe interpretarse como una complicación adicional frente a otras líneas de investigación. De hecho, existen trabajos previos que abordan aspectos formales de considerable sofisticación, como tipos dependientes [52], no terminación [54], concurrencia [55] o la adaptación del método a máquinas basadas en registros [53]. Sin embargo, lo que distingue a estos enfoques no es tanto la complejidad inherente de las características tratadas, sino el hecho de que muchos de ellos modifican o extienden el marco original del método, conservando únicamente el núcleo metodológico basado en la inducción estructural y el razonamiento ecuacional.

En contraste, el presente estudio adopta una estrategia diferente: en lugar de redefinir o adaptar la estructura general del enfoque, se optó por mantener intactos los principios del cálculo propuestos originalmente por Bahr y Hutton, aplicándolos de forma rigurosa a un contexto semántico distinto. De este modo, se preservó completamente la arquitectura conceptual del método (incluyendo la progresión de derivación desde la semántica operacional hacia la máquina virtual) sin introducir extensiones sustanciales en su formulación. Esta decisión metodológica permitió evaluar con mayor claridad la robustez del enfoque en su forma original, y observar su comportamiento frente a lenguajes que, si bien no incorporan tipos dependientes ni concurrencia, sí presentan desafíos semánticos que no habían sido considerados previamente en la literatura, como lo son los regímenes de evaluación diferenciados, la inserción sistemática de puntos estrictos y el tratamiento formal de la recursión en este marco.

Así, el estudio se alinea con la tradición de trabajos que expanden el método hacia nuevos dominios, pero lo hace con una fidelidad estructural singular. Esta aproximación no solo contribuye a consolidar la aplicabilidad del método más allá de su formulación base, sino que permite identificar con mayor precisión los límites y alcances del enfoque cuando se somete a semánticas no triviales, sin depender de transformaciones *ad hoc* ni alteraciones a su lógica fundamental.

Sobre la introducción de recursión

Un aspecto central en la construcción de los lenguajes desarrollados fue la incorporación de la recursión, no como una extensión al núcleo metodológico, sino como una parte integrante de la formalización sintáctica y semántica de los lenguajes objeto de estudio. Aunque inicialmente se contempló derivar una semántica explícita para la recursión como un constructo independiente, una vez completada la derivación de los componentes fundamentales

del *Cálculo* λ en ambos regímenes de evaluación, se constató que las expresiones recursivas podían representarse directamente mediante construcciones estándar del *Cálculo* λ puro, sin necesidad de introducir nuevas reglas ni alterar el marco formal previamente establecido.

Este hallazgo redefinió el rol de la recursión dentro del proyecto: en lugar de ser tratada como un mecanismo adicional, fue integrada en el plano sintáctico del lenguaje como una abstracción de conveniencia sobre las construcciones ya derivadas. Aunque no estrictamente necesaria desde el punto de vista semántico, esta decisión resultó fundamental para habilitar fases fundamentales del diseño, tales como la extensión gramatical, el desarrollo del análisis léxico y sintáctico, y la posterior eliminación del azúcar sintáctica. Esto permitió consolidar una especificación global del lenguaje que facilitó la validación de su comportamiento desde la representación concreta hasta su interpretación formal.

La integración de la recursión demostró que es posible representar y evaluar definiciones recursivas con precisión, sin comprometer la estructura del método ni la cadena de derivaciones. Las reglas semánticas derivadas, construidas únicamente sobre variables, abstracciones y aplicaciones, resultaron suficientes para capturar este comportamiento mediante el tradicional mecanismo de autorreferencia [31], en coherencia con los principios del *Cálculo* λ .

De este modo, la recursión se incorporó de forma natural al sistema, utilizando exclusivamente la infraestructura ya establecida. El hecho de que esta incorporación no exigiera modificaciones al método, sino que pudiera implementarse como una instancia más de su aplicación sistemática, constituye un indicio de su capacidad para adaptarse a expresiones semánticas de mayor complejidad, siempre que estas respeten las estructuras fundamentales del enfoque.

Si bien no se derivó formalmente una semántica particular para la recursión dentro del régimen inductivo del método, esta ausencia no debe interpretarse como una omisión metodológica. La posibilidad de expresar comportamientos recursivos mediante la inserción explícita de un operador de punto fijo sobre las construcciones ya definidas ofrece una validación indirecta de la expresividad lograda. Desde una perspectiva teórica, esta práctica está respaldada por enfoques consolidados, lo que justifica no haber extendido los ambientes de evaluación ni modificado la semántica base de los lenguajes.

Aunque otros marcos semánticos abordan la recursión mediante ambientes autorreferenciales [66], variables ligadas especiales o modelos con referencias explícitas, la estrategia adoptada en este trabajo (introducir el operador de punto fijo como parte de la transformación sintáctica del constructo *Rec*) permitió preservar la integridad de las definiciones semánticas derivadas. Si bien sería viable considerar una alternativa en la que dicho operador se tratara como una variable ligada, ello implicaría extender los ambientes léxicos de los intérpretes y máquinas virtuales, alterando así el marco semántico existente y comprometiendo el aislamiento de las fases ya derivadas.

Modificar los ambientes de evaluación requeriría reformular las reglas semánticas, las estructuras intermedias y los procedimientos de compilación y evaluación, introduciendo una complejidad adicional que no resulta necesaria para representar el comportamiento recursivo. En contraste, la solución implementada (insertar directamente la representación del operador de punto fijo en el árbol de sintaxis abstracta durante la transformación del ASC al ASA) evita toda reconfiguración de los entornos semánticos, garantizando la consistencia con los resultados previos.

Cabe señalar, sin embargo, que esta elección no es habitual en el diseño de lenguajes de

programación. Aunque no afecta a la corrección formal del sistema, permite preservar íntegramente los resultados ya obtenidos, lo cual sugiere una equivalencia semántica relevante entre introducir el operador de punto fijo como expresión directa o como *binding*. Esta observación abre una posible línea de estudio sobre cómo distintas formulaciones sintácticas pueden capturar propiedades semánticas equivalentes sin comprometer la estabilidad del método ni alterar su estructura derivativa.

Lenguaje anfitrión

En el primer caso de estudio, se adoptó un enfoque basado en trabajos previos como [52], que propone transportar la semántica de la estructura condicional mediante la distribución de las funciones de compilación y evaluación sobre ambas ramas de la expresión. Este principio, que descansa en la totalidad del lenguaje anfitrión, permitió reutilizar directamente la construcción condicional de HASKELL en el proceso de derivación. A pesar de que HASKELL es un lenguaje perezoso, esta construcción, al ser una estructura de control y no una función, se comporta de forma estricta [19], evaluando únicamente la rama seleccionada tras la evaluación de la condición. Este comportamiento es coherente con la semántica de evaluación ansiosa deseada para el lenguaje ALANIS.

La compatibilidad semántica entre la construcción condicional de HASKELL y la semántica objetivo de ALANIS justificó su reutilización durante la derivación de la máquina virtual. De esta forma, fue posible aprovechar propiedades del lenguaje anfitrión sin comprometer la integridad formal del método, ni alterar la semántica del lenguaje derivado. En contraste, el desarrollo de IRIS (un lenguaje con régimen de evaluación perezoso) exigió una definición estructural más cuidadosa, que reflejara con fidelidad la no estricta de su semántica. Para ello, se derivaron de forma explícita ambas versiones del condicional, una por cada valor posible de la condición, eliminando cualquier dependencia del comportamiento del lenguaje anfitrión.

Esta diferencia metodológica entre ambos casos no debe interpretarse como una inconsistencia, sino como una decisión deliberada, fundada en las características semánticas de cada lenguaje y en las propiedades del lenguaje anfitrión. Ambas estrategias, reutilizar la estructura existente o derivar manualmente las ramas condicionales, resultan funcionalmente equivalentes en términos de flujo de control y evaluación, y permiten describir correctamente la semántica operacional del condicional. La elección entre una u otra radica en el nivel de autonomía requerido respecto del anfitrión: ALANIS aprovecha una coincidencia semántica para simplificar el desarrollo, mientras que IRIS opta por una construcción independiente que garantiza total control sobre el comportamiento semántico.

Cabe subrayar que la elección de HASKELL como lenguaje anfitrión no responde únicamente a consideraciones técnicas, sino también a un principio metodológico: preservar la línea de trabajo propuesta por los autores del método. Al igual que en trabajos anteriores, se asume que los programas son bien formados y terminantes, lo que permite aplicar válidamente propiedades como la distributividad de funciones sobre condicionales, dentro del dominio de expresiones bien tipadas. En este marco, la implementación en HASKELL no representa una fuente de sesgo, sino un alineamiento con el enfoque original, en el que las propiedades semánticas del lenguaje derivado se mantienen independientes del lenguaje anfitrión y son

verificables internamente mediante el propio sistema formal del método.

Validación de resultados

En el marco del presente estudio, se desarrollaron implementaciones funcionales completas que permiten una validación empírica de los resultados obtenidos. Si bien no se integraron pruebas unitarias concretas en esta etapa del desarrollo, es importante destacar que el método empleado garantiza, por su propia naturaleza, la corrección parcial de los componentes derivados. Al basarse en un razonamiento inductivo estructural y un razonamiento ecuacional riguroso, cada componente semántico se deriva de manera que respeta la semántica del lenguaje fuente, asegurando propiedades fundamentales sin necesidad de validación posterior.

En este sentido, la incorporación de un conjunto de pruebas empíricas en el lenguaje de implementación (por ejemplo, mediante herramientas como *QuickCheck* en HASKELL) no refuerza por sí misma la corrección de los resultados, sino que complementa su validez en la práctica. De hecho, dado que el trabajo exhibe explícitamente la derivación formal de cada resultado esencial, se considera más representativo este enfoque como método de verificación que la evaluación de casos de prueba en el lenguaje anfitrión.

No obstante, para fortalecer la validación empírica de manera sistemática, especialmente en contextos futuros que requieran mayor robustez práctica, puede considerarse la integración de estas herramientas. *QuickCheck* permitiría automatizar la generación de casos de prueba y evaluar propiedades observables del comportamiento del sistema. Por otro lado, un paso adicional significativo consistiría en la formalización completa de las pruebas dentro de un asistente como COQ, herramienta ya utilizada por los autores del método, lo cual permitiría verificar con garantías mecánicas la corrección de los desarrollos.

Ambos enfoques, ya sea desde una perspectiva práctica con pruebas automatizadas, o desde una validación formal mediante un asistente de pruebas, son caminos perfectamente integrables como extensiones del trabajo realizado, y ofrecen vías complementarias para reforzar su confiabilidad.

Uso de herramientas del lenguaje HASKELL

Una herramienta fundamental incorporada durante el desarrollo de este estudio fue HAPPY, generador de analizadores sintácticos para HASKELL, cuya integración permitió transitar con precisión desde la gramática abstracta de los lenguajes diseñados hacia una implementación ejecutable. En particular, su uso estuvo motivado por la necesidad de hacer explícita la fase de eliminación del azúcar sintáctica, lo cual resultó fundamental para habilitar la interacción completa desde la sintaxis concreta hasta la evaluación de los programas en nuestros lenguajes.

Aunque en términos estrictos no era imprescindible recurrir a esta herramienta, pues la descomposición del azúcar sintáctica podría haberse especificado directamente mediante transformaciones sobre el ASA, la decisión de emplearla trajo consigo beneficios significativos. En primer lugar, permitió validar la formalización de la gramática libre de contexto defi-

nida para los lenguajes desarrollados, conectando directamente nuestra descripción formal con una práctica común en el desarrollo de lenguajes de programación. En este sentido, la incorporación de HAPPY alineó nuestras decisiones de implementación con la metodología académica en la que se enmarca este trabajo, donde la automatización de las fases de análisis léxico y sintáctico constituye un aspecto esencial en el diseño riguroso de compiladores.

Desde una perspectiva práctica, HAPPY facilitó la generación de analizadores léxicos y sintácticos a partir de nuestras especificaciones formales, aportando una capa de automatización que refuerza el carácter sistemático del *pipeline* de implementación. A su vez, esto contribuyó a que las implementaciones derivadas no quedaran como especificaciones aisladas, sino que cobraran forma como sistemas funcionales y operativos, con una interfaz de interacción amigable incluso para usuarios con experiencia limitada en HASKELL.

En efecto, las versiones implementadas incluidas en el material suplementario (véase Apéndice A) permiten que un usuario interactúe directamente con la sintaxis concreta de los lenguajes, mientras que internamente se ejecutan de manera transparente las fases de análisis léxico y sintáctico. Así, aunque originalmente no se consideró explícitamente la inclusión de estas fases en el enfoque formal, su incorporación final aporta claridad y rigor al proceso, fortaleciendo la correspondencia entre las formalizaciones abstractas y su implementación concreta. Esta integración refuerza el carácter replicable y verificable del trabajo, extendiendo su aplicabilidad más allá del marco puramente teórico.

Robustez del método y generalización

La trayectoria metodológica seguida a lo largo de este trabajo permite sostener que el enfoque aplicado, basado en el método de *Cálculo de Compiladores Correctos* propuesto por Bahr y Hutton, exhibe una solidez significativa tanto en su formulación original como en su capacidad de adaptación a contextos no contemplados explícitamente por sus autores. En particular, la derivación de compiladores correctos por construcción para lenguajes con propiedades inéditas (como regímenes de evaluación contrastantes, manejo fino de puntos estrictos, o extensiones sintácticas como la recursión) confirma que el método ofrece una plataforma confiable y adaptable para el diseño formal de compiladores. La decisión deliberada de respetar íntegramente la estructura del enfoque original, sin alterar su base metodológica, ha sido esencial para validar su portabilidad semántica y demostrar que puede escalar a lenguajes con semánticas operacionales más ricas sin comprometer los principios fundamentales de corrección.

Este carácter de “portabilidad” semántica no solo se reflejó en el diseño modular de los evaluadores, compiladores y máquinas virtuales, sino también en la posibilidad de extender naturalmente el lenguaje con nuevas construcciones sin perturbar su semántica base. La introducción de la recursión, por ejemplo, ilustra este punto: aunque no fue integrada en el núcleo del método, su formalización se apoyó en los componentes ya derivados, permitiendo que surgiera como una extensión semánticamente alineada al marco del *Cálculo λ* puro. Esta capacidad de absorber nuevas construcciones mediante transformaciones sintácticas y conservando el entorno semántico original exhibe el poder expresivo del enfoque y sugiere que puede emplearse como base para futuros trabajos que involucren mecanismos similares, como estructuras de repetición, listas o memoización.

El método no solo permitió mantener la estructura de razonamiento formal sin necesidad de reformas significativas, sino que también se mostró capaz de producir resultados técnicamente viables y ejecutables en entornos prácticos. La implementación en HASKELL de los resultados derivados refuerza la idea de que el enfoque permite una transición fluida de la especificación formal al prototipo ejecutable. Además, la validación empírica basada en los ejecutables funcionales, junto con la posibilidad de incorporar herramientas como QUICK-CHECK o asistentes de prueba como COQ para extender la verificación, consolida su potencial como una técnica científicamente sustentada.

Este potencial se fundamenta en los pilares del enfoque: la inducción estructural y el razonamiento ecuacional. Ambos principios no solo son rigurosos desde un punto de vista formal, sino también ampliamente aceptados y utilizados en el campo de la semántica de lenguajes y el diseño de compiladores. Su integración como base metodológica le otorga al enfoque una solidez conceptual que podría posicionarlo como un marco estándar dentro de la formación académica y la investigación en estas áreas, independientemente de si se adopta como herramienta de desarrollo para lenguajes modernos.

De hecho, aunque su aplicabilidad directa a lenguajes contemporáneos con infraestructuras complejas, como compilación *just-in-time*, optimizaciones agresivas o gestión automática de memoria avanzada, no sea inmediata, el valor del enfoque radica en su claridad, su capacidad para guiar derivaciones correctas paso a paso y su potencial para estructurar estudios teóricos sobre semántica y diseño de lenguajes de programación. Como se ha evidenciado en este trabajo, su aplicación rigurosa permitió replicar y extender formulaciones previas sin comprometer su corrección, lo que sugiere que podría consolidarse como una técnica de referencia académica, susceptible de ser enriquecida por la comunidad científica con nuevas extensiones, análisis comparativos y formalizaciones adicionales.

En este contexto, el enfoque también demuestra ser intuitivamente extensible a nuevas construcciones semánticas. La derivación de estructuras como listas, condicionales o mecanismos de control podría seguir principios similares a los ya aplicados en este trabajo, apoyándose en la claridad composicional de las definiciones semánticas previamente desarrolladas. Sin embargo, algunas extensiones, como la introducción de memoización, propia de estrategias de paso de parámetros como *Call-by-Need*, implicarían modificaciones sustanciales en el diseño del entorno de evaluación y en la configuración de las máquinas virtuales, al requerir estructuras adicionales como *heaps* o estados compartidos. Si bien los propios autores del método afirman que este puede escalar para manejar efectos como estado o excepciones, también advierten que tal reformulación exige un tratamiento cuidadoso [1]. En nuestro caso, optar por dicha vía habría implicado desviar el enfoque del presente trabajo, que busca mantenerse fiel a la formulación original del método. Por tanto, más que una limitación del método, se trata de una decisión metodológica consciente: preservar la alineación con el marco teórico propuesto inicialmente, asegurando así la coherencia, claridad y comparabilidad de los resultados obtenidos.

Proyecciones y líneas futuras

Por un lado, una proyección natural de este estudio consiste en explorar enfoques orientados a garantizar formalmente la solidez de las máquinas virtuales derivadas. En particular,

la incorporación de un sistema de tipos más expresivo (como los tipos dependientes) permitiría garantizar la terminación de los programas y, con ello, asegurar la corrección total de los resultados obtenidos mediante el método. Este enfoque habilitaría un análisis estático riguroso capaz de verificar, de forma previa a la ejecución, invariantes semánticos fundamentales como la finalización del programa o la normalización de las evaluaciones. Si bien esta línea promete una mayor robustez formal y una capa adicional de garantías semánticas, su adopción implicaría un compromiso metodológico distinto al que guía este trabajo.

En efecto, nuestra elección ha sido pragmática: restringimos el análisis a programas bien formados y terminantes, tal como se ha hecho en los desarrollos fundacionales del método. Esta decisión permite mantener la claridad y accesibilidad del enfoque, sin introducir la complejidad adicional que supone el diseño e integración de un sistema de tipos dependientes, aunque sin renunciar a la posibilidad de adoptar este enfoque en trabajos futuros.

Por otro lado, desde una perspectiva de eficiencia operacional, se identifica como una línea de trabajo complementaria la exploración de optimizaciones clásicas, como la transformación de recursión en recursión de cola. Esta optimización requeriría una reformulación cuidadosa de las definiciones derivadas, a fin de permitir la reutilización del entorno de ejecución sin alterar la semántica original. Asimismo, se considera de interés investigar mecanismos de evaluación más refinados, como la introducción explícita de estructuras perezosas o anotaciones de *strictness*, que habiliten un mayor control sobre el orden de evaluación y el uso de recursos. Este tipo de extensiones no solo enriquecerían el modelo semántico actual, sino que también ofrecerían un marco propicio para evaluar la adaptabilidad del método ante estrategias de evaluación más complejas, siempre bajo el principio rector de preservar la integridad teórica del enfoque original.

A modo de cierre, es pertinente valorar el proceso completo de desarrollo de este trabajo desde la perspectiva metodológica adoptada. Haber retomado un método presentado hace más de una década, el cual, a pesar de su potencial, no ha alcanzado una difusión amplia ni ha permeado de forma significativa los marcos estándar de estudio en el ámbito de los lenguajes de programación y compiladores, representa no solo un ejercicio de recuperación académica, sino también una apuesta por explorar caminos rigurosos, aunque poco transitados, dentro del estudio formal de la semántica operacional.

El enfoque aplicado se fundamenta en principios sólidos de las ciencias de la computación: la inducción estructural, el razonamiento ecuacional, y un diseño de máquinas abstractas. A través de estos pilares, el método permite derivar componentes correctos por construcción, habilitando la verificación formal de propiedades semánticas esenciales en intérpretes, compiladores y máquinas virtuales. Aunque su corrección es parcial en términos de completitud, la claridad con la que permite rastrear el comportamiento de los programas desde la sintaxis hasta su ejecución efectiva refuerza su valor académico y su vigencia como marco de estudio.

El proceso de aplicar el método ha sido exigente en cuanto al dominio conceptual requerido. La necesidad de entender profundamente la semántica operacional, la relación entre

la sintaxis abstracta y concreta, y la interacción entre las distintas estrategias de paso de parámetros, no solo representa un reto técnico, sino una oportunidad para profundizar en el núcleo mismo del diseño de lenguajes. En este sentido, la metodología demuestra ser una herramienta poderosa para formalizar y sistematizar procesos de diseño e implementación, alineados con los estándares más rigurosos de la disciplina.

Esperamos que este trabajo contribuya a enriquecer el estado del arte, sirviendo como punto de partida para futuras investigaciones que busquen aprovechar o extender este enfoque original. En última instancia, queda claro que el diseño y desarrollo de compiladores no es solo un proceso riguroso y exigente, sino también, inevitablemente, un ejercicio profundamente creativo.

“La semántica operacional no es solo una herramienta para describir el comportamiento de los programas, sino un lenguaje mediante el cual podemos razonarlos, transformarlos y garantizar su corrección.”

— *(Paráfrasis de Gordon Plotkin y Matthias Felleisen)*

A Material suplementario

Todos los recursos desarrollados durante este trabajo han sido organizados y documentados en un repositorio público, el cual constituye el material suplementario que acompaña esta tesis. Este repositorio está disponible en la siguiente dirección:

<https://github.com/Erickarroyo18/calc-comp.git>

El repositorio contiene las implementaciones completas de los lenguajes MINIB, ALANIS, IRIS y su núcleo común ASA, junto con los evaluadores, compiladores, máquinas virtuales y sus respectivas extensiones para el análisis léxico, el análisis sintáctico y el proceso de eliminación del azúcar sintáctica (*desugar*).

La estructura del repositorio ha sido diseñada para facilitar tanto la validación técnica de los resultados como su exploración interactiva. Se incluyen instrucciones detalladas para compilar y ejecutar cada uno de los componentes, lo cual permite a los lectores interesados comprobar empíricamente la correspondencia entre las formalizaciones presentadas y sus implementaciones. Esta organización también favorece la reutilización y extensión de los artefactos desarrollados, promoviendo así una continuidad natural hacia futuras líneas de investigación.

Bibliografía

- [1] P. BAHR and G. HUTTON, “Calculating correct compilers,” *Journal of Functional Programming*, vol. 25, p. e14, 2015.
- [2] *Métodos Formales, Ingeniería de Requisitos y Pruebas del Software*. Alicante : Biblioteca Virtual Miguel de Cervantes, 2022, 2022.
- [3] G. Winskel, *The formal semantics of programming languages: an introduction*. Cambridge, MA, USA: MIT Press, 1993.
- [4] R. Harper, *Practical Foundations for Programming Languages*. Cambridge University Press, 2 ed., 2016.
- [5] Institute of Electrical and Electronics Engineers, “IEEE Standard Glossary of Software Engineering Terminology,” 1990.
- [6] M. L. Scott, *Programming language pragmatics / Michael L. Scott*. San Francisco: Morgan Kaufmann, 2nd ed. ed., 2006.
- [7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compiladores: Principios, técnicas y herramientas*. Pearson, 2 ed., 2008. ISBN ebook: 9786074428742. Traducción al español.
- [8] D. Watson, *A Practical Approach to Compiler Construction*. Springer Publishing Company, Incorporated, 1st ed., 2017.
- [9] R. W. Sebesta, *Concepts of Programming Languages*. USA: Addison-Wesley Longman Publishing Co., Inc., 6 ed., 2003.
- [10] M. S. Romero, “Notas de clase de lenguajes de programación.” Notas de Clase. Lenguajes de Programación.
- [11] K. D. Lee, *Foundations of Programming Languages*. Undergraduate Topics in Computer Science, Cham: Springer International Publishing, 2017.
- [12] International Organization for Standardization and International Electrotechnical Commission, “Syntactic metalanguage,” ISO/IEC 14977, 1996.
- [13] G. R. Grice, “The emergence of norms by edna ullmann-margalit clarendon press: Oxford university press, 1977, xiii + 206 pp., £8.00,” *Philosophy*, vol. 54, no. 209, p. 420–421, 1979.

- [14] J. C. Mitchell, *Concepts in Programming Languages*. Cambridge University Press, 2002.
- [15] M. Felleisen, “On the expressive power of programming languages,” *Science of Computer Programming*, vol. 17, no. 1, pp. 35–75, 1991.
- [16] J. Pombrio and S. Krishnamurthi, “Resugaring: lifting evaluation sequences through syntactic sugar,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’14, (New York, NY, USA), p. 361–371, Association for Computing Machinery, 2014.
- [17] J. Inala, X. Qiu, B. Lerner, and A. Solar-Lezama, “Type assisted synthesis of recursive transformers on algebraic data types,” 07 2015.
- [18] J. Pombrio, *Resugaring: Lifting Languages through Syntactic Sugar*. Ph.d. thesis, Brown University, Providence, RI, USA, 2018. Advisor: Shriram Krishnamurthi.
- [19] B. C. Pierce, *Types and Programming Languages*. The MIT Press, 1st ed., 2002.
- [20] S. Krishnamurthi, *Programming Languages: Application and Interpretation*. Brown Univ., 2007.
- [21] G. Keller, L. O’Connor, and T. Smeding, *Concepts of Programming Language Design*. Utrecht University, 2012.
- [22] M. Gabbrielli and S. Martini, *Programming Languages: Principles and Paradigms*. Springer, 2023.
- [23] L. Aceto, W. Fokkink, and C. Verhoef, “Structural operational semantics,” *Handbook of Process Algebra*, vol. 6, 10 1999.
- [24] H. Nielson and F. Nielson, *Semantics with Applications: A Formal Introduction*. Wiley, 1992.
- [25] H. Barendregt, W. Dekkers, and R. Statman, *Lambda Calculus with Types*. Perspectives in Logic, Cambridge University Press, 2013.
- [26] H. P. Barendregt, *The Lambda Calculus: Its Syntax and Semantics*. New York, N.Y.: Sole distributors for the U.S.A. and Canada, Elsevier Science Pub. Co., 1984.
- [27] M. Felleisen and M. Flatt, “Programming languages and lambda calculi,” 01 2007.
- [28] A. A. R. Hernández, “Una implementación de máquinas de reducción,” 2023.
- [29] A. D. Ker, “Lambda calculus and types.” <https://www.cs.ox.ac.uk/andrew.ker/docs/lambdacalculus-lecture-notes-ht2009.pdf>, 2009. 16 Lectures, Hilary Term 2009, Oxford University Computing Laboratory.
- [30] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd ed., 2009.

- [31] H. Abelson and G. J. S. with Julie Sussman, *Structure and Interpretation of Computer Programs*. The MIT Press, 1996. Unofficial Texinfo Format 2.andresraba6.6.
- [32] D. Van Horn and M. Might, “Systematic abstraction of abstract machines,” *Journal of Functional Programming*, vol. 22, 07 2011.
- [33] M. Felleisen, R. Findler, M. Flatt, and S. Krishnamurthi, *How to Design Programs: An Introduction to Programming and Computing*. MIT Press, 2001.
- [34] S. L. P. Jones, “Implementing lazy functional languages on stock hardware: the spineless tagless g-machine,” *Journal of Functional Programming*, vol. 2, no. 2, p. 127–202, 1992.
- [35] J.-W. Maessen and Arvind, *Hybrid eager and lazy evaluation for efficient compilation of haskell*. PhD thesis, USA, 2002.
- [36] P. Sestoft, “Deriving a lazy abstract machine,” *J. Funct. Program.*, vol. 7, pp. 231–264, 05 1997.
- [37] A. Church, *The Calculi of Lambda Conversion*. Princeton University Press, 1941.
- [38] J. Hatchliff and O. Danvy, “Thunks and the λ -calculus,” *J. Funct. Program.*, vol. 7, p. 303–319, May 1997.
- [39] T. Uustalu and V. Vene, “Comonadic notions of computation,” *Electr. Notes Theor. Comput. Sci.*, vol. 203, pp. 263–284, 06 2008.
- [40] F. Pottier, A. Guéneau, J.-H. Jourdan, and G. Mével, “Thunks and debits in separation logic with time credits,” *Proc. ACM Program. Lang.*, vol. 8, Jan. 2024.
- [41] R. Harper, “Programming in standard ml,” 07 2001.
- [42] L. D. C. G. Huesca, “Programación dinámica puramente funcional: el caso de la memoización,” tesis de maestría, Universidad Nacional Autónoma de México, Posgrado en Ciencia e Ingeniería de la Computación, México D.F., 2010. Directores de tesis: Dr. Francisco Hernández Quiroz y Dr. Favio Ezequiel Miranda Perea.
- [43] S. B. Lassen, “Relational reasoning about contexts,” *BRICS Report Series*, vol. 4, Jan. 1997.
- [44] B. Davis, A. Beatty, K. Casey, D. Gregg, and J. Waldron, “The case for virtual register machines,” in *Proceedings of the 2003 Workshop on Interpreters, Virtual Machines and Emulators*, IVME ’03, (New York, NY, USA), p. 41–49, Association for Computing Machinery, 2003.
- [45] R. Fang and S. Liu, “A performance survey on stack-based and register-based virtual machines,” 11 2016.
- [46] M. Schröder, “Optimizing lua using run-time type specialization,” 2012.

- [47] P. Hudak, J. Hughes, S. Peyton Jones, and P. Wadler, “A history of haskell: being lazy with class,” in *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, HOPL III, (New York, NY, USA), p. 12–1–12–55, Association for Computing Machinery, 2007.
- [48] K. Doets and J. van Eijck, *The Haskell Road to Logic, Math and Programming*. College Publications, Mar. 2004.
- [49] C. Kirchner and H. Kirchner, “Equational logic and rewriting,” in *Handbook of the History of Logic, Vol. 9: History of Logic and Computation in the 20th Century* (D. M. Gabbay, J. H. Siekmann, and J. Woods, eds.), Handbook of the History of Logic, pp. 255–282, Elsevier, 2014.
- [50] C. Kirchner and H. Kirchner, “Constrained equational reasoning,” in *Proceedings of the ACM-SIGSAM 1989 International Symposium on Symbolic and Algebraic Computation*, ISSAC ’89, (New York, NY, USA), p. 382–389, Association for Computing Machinery, 1989.
- [51] G. Hutton, *Programming in Haskell*. USA: Cambridge University Press, 2nd ed., 2016.
- [52] P. Bahr and G. Hutton, “Beyond trees: Calculating graph-based compilers (functional pearl),” *Proc. ACM Program. Lang.*, vol. 8, Aug. 2024.
- [53] P. BAHR and G. HUTTON, “Calculating correct compilers ii: Return of the register machines,” *Journal of Functional Programming*, vol. 30, p. e25, 2020.
- [54] P. Bahr and G. Hutton, “Monadic compiler calculation (functional pearl),” *Proc. ACM Program. Lang.*, vol. 6, Aug. 2022.
- [55] P. Bahr and G. Hutton, “Calculating compilers for concurrency,” *Proc. ACM Program. Lang.*, vol. 7, Aug. 2023.
- [56] A. W. Appel, *Compiling with Continuations*. USA: Cambridge University Press, 2007.
- [57] R. Bird and P. Wadler, *An introduction to functional programming*. GBR: Prentice Hall International (UK) Ltd., 1988.
- [58] S. P. Jones and S. Graf, “Triemaps that match,” 2024.
- [59] e. Simon Marlow, “Haskell 2010 language report.” <https://www.haskell.org/onlinereport/haskell2010/>, 2010. Accessed: 2025-05-04.
- [60] M. Pickard and G. Hutton, “Calculating dependently-typed compilers (functional pearl),” *Proc. ACM Program. Lang.*, vol. 5, Aug. 2021.
- [61] J. Gibbons, “Unifying theories of programming with monads,” vol. 7681, 01 2013.
- [62] K. D. Cooper and L. Torczon, *Engineering a Compiler*. Waltham, MA: Morgan Kaufmann, 2 ed., 2011.

- [63] S. Chong, “Cs152: Programming languages, spring 2010.” <https://groups.seas.harvard.edu/courses/cs152/2010sp/lectures.html>, January 2010. Harvard School of Engineering and Applied Sciences.
- [64] G. Plotkin, “A structural approach to operational semantics,” *J. Log. Algebr. Program.*, vol. 60-61, pp. 17–139, 07 2004.
- [65] N. Shah, “Program construction: Calculating implementations from specifications by r.c. backhouse, john wiley sons, 2004,” *J. Funct. Program.*, vol. 14, pp. 598–600, 09 2004.
- [66] D. A. Schmidt, *Denotational semantics: a methodology for language development*. USA: William C. Brown Publishers, 1986.

